## Removing duplicates with ROW_NUMBER

SQL

```sql
WITH d AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id, order_date, product_id ORDER BY
item_id) AS rn
    FROM order_items_raw
)
DELETE FROM d WHERE rn > 1;            -- SQL Server lets you DELETE through a CTE
```

Rows with the same business key get numbers 1, 2, 3…; keeping `rn = 1` keeps one copy. Always check with a `SELECT … WHERE rn > 1` first.

Let's count how many lines in our data repeat the same product in the same order (what a de-duplication would touch):

SQL

```sql
WITH d AS (
    SELECT order_id, product_id,
           ROW_NUMBER() OVER (PARTITION BY order_id, product_id ORDER BY row_id) AS rn
    FROM superstore
)
SELECT COUNT(*) AS repeated_lines FROM d WHERE rn > 1;
```

RESULT

| repeated_lines |
|---|
| 70 |

1 rows × 1 columns

(These are legitimate repeat lines, not errors; the point is the pattern.)

## NTILE: buckets and percentiles

SQL

```sql
WITH cust AS (
    SELECT customer_id, SUM(sales) AS sales FROM superstore GROUP BY customer_id
), q AS (
    SELECT customer_id, sales, NTILE(4) OVER (ORDER BY sales DESC) AS quartile FROM cust
)
SELECT quartile,
       COUNT(*)              AS customers,
       ROUND(SUM(sales), 0)   AS sales,
       ROUND(100.0 * SUM(sales) / SUM(SUM(sales)) OVER (), 1) AS pct_of_total_sales
FROM q
GROUP BY quartile
ORDER BY quartile;
```

RESULT

| quartile | customers | sales | pct_of_total_sales |
|---|---|---|---|
| 1 | 197 | 111,100,409 | 55.2 |
| 2 | 196 | 54,386,341 | 27.0 |
| 3 | 196 | 28,080,117 | 13.9 |
| 4 | 196 | 7,846,704 | 3.9 |

4 rows × 4 columns

Quartile 1 (the top 25% of customers) generates a disproportionate share of sales: the **Pareto principle** from the Excel charts lesson, now computed in SQL. NTILE is also how RFM scoring is done (ML track).

## Aggregate window functions

Any aggregate (`SUM`, `AVG`, `COUNT`, `MIN`, `MAX`) becomes a window function with `OVER`. Without `ORDER BY` in the window, it uses the whole partition: perfect for totals, shares and comparisons with the group average.

SQL

```sql
SELECT department, emp_id, monthly_income,
       ROUND(AVG(monthly_income) OVER (PARTITION BY department), 0)          AS dept_avg,
       monthly_income - ROUND(AVG(monthly_income) OVER (PARTITION BY department), 0) AS
vs_dept_avg,
       COUNT(*) OVER (PARTITION BY department)                               AS dept_headcount,
       MAX(monthly_income) OVER (PARTITION BY department)                    AS dept_max
FROM employees
ORDER BY department, monthly_income DESC;
```

RESULT

| department | emp_id | monthly_income | dept_avg | vs_dept_avg | dept_headcount | dept_max |
|---|---|---|---|---|---|---|
| Finance | E00519 | 209833 | 83,681 | 126,152 | 154 | 209833 |
| Finance | E00145 | 185500 | 83,681 | 101,819 | 154 | 209833 |
| Finance | E00336 | 174417 | 83,681 | 90,736 | 154 | 209833 |
| Finance | E00201 | 169500 | 83,681 | 85,819 | 154 | 209833 |
| Finance | E01172 | 168167 | 83,681 | 84,486 | 154 | 209833 |
| Finance | E01151 | 155167 | 83,681 | 71,486 | 154 | 209833 |
| Finance | E00759 | 152083 | 83,681 | 68,402 | 154 | 209833 |
| Finance | E01004 | 144667 | 83,681 | 60,986 | 154 | 209833 |

1,500 rows × 7 columns (first 8 shown)

This replaces the correlated subquery from Lesson 10 ("earns more than the department average") with one pass over the data.

### GROUP BY vs window functions

| | GROUP BY | Window function |
|---|---|---|
| Rows returned | One per group | Same as input |
| Detail columns | Only grouped columns | All columns |
| Use for | Summary reports | Rankings, running totals, shares, comparisons with group values |
| Filter the result | `HAVING` | Wrap in CTE/subquery and `WHERE` |
| Can combine | Yes: group first, then window over the groups | |

## Summary

KEY TAKEAWAYS

- `OFFSET n ROWS FETCH NEXT m ROWS ONLY` (with `ORDER BY`) paginates; it can skip rows, unlike `TOP`.
- A **window function** adds a calculation over a set of related rows to each row, without collapsing them. `OVER (PARTITION BY … ORDER BY …)` defines the window.
- `ROW_NUMBER` (unique), `RANK` (ties, gaps), `DENSE_RANK` (ties, no gaps), `NTILE(n)` (equal buckets).
- To filter on a window result, compute it in a CTE or derived table, then use `WHERE`.
- Top-N per group and Nth highest per group: `DENSE_RANK`/`ROW_NUMBER` partitioned by group; de-duplicate with `ROW_NUMBER … WHERE rn > 1`.
- Aggregate windows (`SUM() OVER (PARTITION BY …)`) show totals, shares and group averages beside each row.

## Quick quiz

6 questions · tap an answer to check it

**1. Scores 100, 90, 90, 80. What does DENSE_RANK() OVER (ORDER BY score DESC) return?**

A 1, 2, 3, 4

B 1, 2, 2, 4

C 1, 2, 2, 3

D 1, 1, 2, 3

**Answer: C.** DENSE_RANK gives tied rows the same rank and continues without gaps.

**2. Why can't you write `WHERE ROW_NUMBER() OVER (...) = 1`?**

A ROW_NUMBER needs GROUP BY

B Window functions are evaluated after WHERE, so you must filter in an outer query or CTE

C OVER cannot contain ORDER BY

D It is allowed in SQL Server

**Answer: B.** Logical processing order: WHERE runs before SELECT, where windows are computed.

**3. What is the difference between PARTITION BY and GROUP BY?**

A None

B PARTITION BY defines the window groups but keeps every row; GROUP BY collapses rows

C PARTITION BY only works with RANK

D GROUP BY keeps all rows

**Answer: B.** Partitions restart the calculation for each group without reducing the number of rows.

**4. Which returns the 3rd highest distinct salary overall?**

A DENSE_RANK() OVER (ORDER BY salary DESC) = 3

B ROW_NUMBER() OVER (ORDER BY salary DESC) = 3

C TOP 3 salary

D RANK() OVER (ORDER BY salary) = 3

**Answer: A.** DENSE_RANK counts distinct values; ROW_NUMBER would count rows even if values repeat.

**5. `SUM(sales) OVER (PARTITION BY region)` returns:**

A One row per region

B On every row, the total sales of that row's region

C The overall total on every row

D A running total

**Answer: B.** Without ORDER BY in the window, it sums the whole partition.

**6.** To fetch rows 21–30 of a sorted result you write:

A `TOP 30`

B `ORDER BY … OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`

C `LIMIT 10, 20` in T-SQL

D `WHERE ROW 21 TO 30`

**Answer: B.** Skip 20, take 10.

EXERCISE **Window practice**

(1) For each customer segment, rank customers by total sales and show the top 3. (2) For each order line, show its sales as a percentage of its order's total. (3) For each department, list employees whose income is within the top 5% of their department (use NTILE(20)). (4) Show products whose total sales rank in the top 10% of their category.

**Show solution (worked answer)**

SQL

```sql
-- (1)
WITH t AS (SELECT c.segment, c.customer_name, SUM(oi.sales) AS sales
           FROM order_items oi JOIN orders o ON o.order_id = oi.order_id JOIN customers c
ON c.customer_id = o.customer_id
           GROUP BY c.segment, c.customer_id, c.customer_name),
     r AS (SELECT *, DENSE_RANK() OVER (PARTITION BY segment ORDER BY sales DESC) AS rnk
FROM t)
SELECT * FROM r WHERE rnk ⩽ 3 ORDER BY segment, rnk;
-- (2)
SELECT item_id, order_id, sales, ROUND(100.0 * sales / SUM(sales) OVER (PARTITION BY
order_id), 1) AS pct_of_order FROM order_items;
-- (3)
WITH x AS (SELECT department, emp_id, monthly_income, NTILE(20) OVER (PARTITION BY
department ORDER BY monthly_income DESC) AS bucket FROM employees)
SELECT * FROM x WHERE bucket = 1;
-- (4)
WITH p AS (SELECT pr.category, pr.product_id, SUM(oi.sales) AS sales FROM order_items oi
JOIN products pr ON pr.product_id = oi.product_id GROUP BY pr.category, pr.product_id),
     n AS (SELECT *, NTILE(10) OVER (PARTITION BY category ORDER BY sales DESC) AS decile
FROM p)
SELECT * FROM n WHERE decile = 1;
```

INTERVIEW CORNER

### Questions you may be asked

**Q: RANK vs DENSE_RANK vs ROW_NUMBER?** ROW_NUMBER numbers rows uniquely; RANK gives ties the same rank and skips following ranks; DENSE_RANK gives ties the same rank without gaps.

**Q: Find the second-highest salary in each department.** CTE with `DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC)`, filter `= 2`.

**Q: How do you remove duplicate rows keeping one?** `ROW_NUMBER() OVER (PARTITION BY business_key ORDER BY …)` and delete rows where it is greater than 1.

**Q: Window function vs GROUP BY?** Windows keep detail rows and add group-level values; GROUP BY returns one row per group.

SQL FOR DATA ANALYSIS · LESSON 12

# Window Functions II: Running Totals, LEAD, LAG & Frames

Build time-series analytics in SQL. Use window frames (ROWS BETWEEN) for running totals and moving averages, LAG and LEAD for month-on-month and year-on-year growth, FIRST_VALUE and LAST_VALUE, cumulative percentages, PERCENT_RANK, CUME_DIST and PERCENTILE_CONT, and gaps-and-islands patterns.

SQL class 5–6 · SQL Server 2022, 2026 · Advanced · 130 min

**By the end of this lesson you can**

- Define a window frame with ROWS BETWEEN and explain the default frame when ORDER BY is present
- Calculate running totals, moving averages and cumulative shares
- Compare each row with the previous or next row using LAG and LEAD (MoM, YoY, gaps)
- Use FIRST_VALUE, LAST_VALUE and NTH_VALUE, PERCENT_RANK, CUME_DIST and PERCENTILE_CONT
- Solve streak and gap problems with ROW_NUMBER differences

## Window frames

Inside `OVER`, `ORDER BY` defines the order and an optional **frame** defines *which rows around the current row* the function uses.

```sql
SUM(x) OVER (PARTITION BY p ORDER BY d
             ROWS BETWEEN <start> AND <end>)
```

| Frame boundary | Meaning |
| --- | --- |
| `UNBOUNDED PRECEDING` | From the first row of the partition |
| `n PRECEDING` | n rows before the current row |
| `CURRENT ROW` | The current row |
| `n FOLLOWING` | n rows after |
| `UNBOUNDED FOLLOWING` | To the last row of the partition |




Frames relative to the current row (highlighted). A running total uses everything up to the current row; a 3-row moving average uses the previous two rows plus the current one.

**COMMON MISTAKES** **The default frame trap**

When `ORDER BY` is present and no frame is written, SQL Server uses `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. `RANGE` treats rows with the **same ORDER BY value** as peers and includes them all, which gives surprising results with ties, and `LAST_VALUE` then returns the current row's value. For running totals always write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` explicitly; it is also much faster.

## Running totals

First build the monthly sales table, then add a running (cumulative) total and a share:

SQL

```sql
WITH monthly AS (
    SELECT YEAR(order_date) AS yr, MONTH(order_date) AS mth, SUM(sales) AS sales
    FROM superstore
    GROUP BY YEAR(order_date), MONTH(order_date)
)
SELECT yr, mth,
       ROUND(sales, 0) AS sales,
       ROUND(SUM(sales) OVER (PARTITION BY yr ORDER BY mth
                              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 0) AS
ytd_sales,
       ROUND(100.0 * SUM(sales) OVER (PARTITION BY yr ORDER BY mth
                              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
                  / SUM(sales) OVER (PARTITION BY yr), 1)                          AS
ytd_pct_of_year
FROM monthly
ORDER BY yr, mth;
```

RESULT

| yr | mth | sales | ytd_sales | ytd_pct_of_year |
|---|---|---|---|---|
| 2023 | 1 | 2,447,151 | 2,447,151 | 4.2 |
| 2023 | 2 | 3,861,852 | 6,309,002 | 10.9 |
| 2023 | 3 | 3,441,306 | 9,750,308 | 16.9 |
| 2023 | 4 | 3,927,557 | 13,677,865 | 23.7 |
| 2023 | 5 | 4,840,130 | 18,517,994 | 32.1 |
| 2023 | 6 | 5,785,712 | 24,303,706 | 42.2 |
| 2023 | 7 | 5,390,409 | 29,694,115 | 51.5 |
| 2023 | 8 | 4,129,386 | 33,823,500 | 58.7 |
| 2023 | 9 | 5,714,731 | 39,538,231 | 68.6 |
| 2023 | 10 | 5,134,331 | 44,672,561 | 77.5 |
| 2023 | 11 | 5,815,308 | 50,487,869 | 87.6 |
| 2023 | 12 | 7,152,348 | 57,640,217 | 100.0 |
| 2024 | 1 | 6,022,738 | 6,022,738 | 9.2 |
| 2024 | 2 | 2,463,147 | 8,485,885 | 13.0 |

36 rows × 5 columns (first 14 shown)

`PARTITION BY yr` restarts the running total every January (year-to-date). The percentage uses a second window (the whole year) as the denominator.

# Moving averages

A moving average smooths noisy data and shows the trend. Here is the 7-day average of daily units sold:

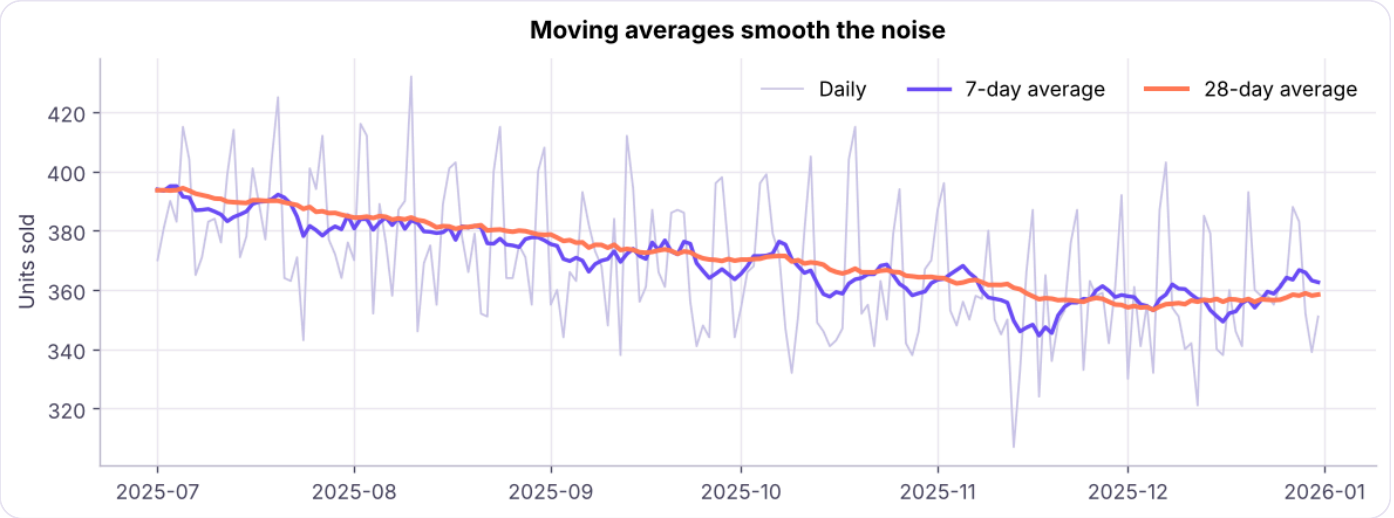
SQL

```sql
SELECT date, units_sold,
       ROUND(AVG(units_sold * 1.0) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT
ROW), 1) AS avg_7_day
FROM daily_sales
WHERE date ⩾ '2025-11-01'
ORDER BY date;
```

RESULT

| date | units_sold | avg_7_day |
|---|---|---|
| 2025-11-01 | 387 | 387.0 |
| 2025-11-02 | 396 | 391.5 |
| 2025-11-03 | 353 | 378.7 |
| 2025-11-04 | 348 | 371.0 |
| 2025-11-05 | 356 | 368.0 |
| 2025-11-06 | 350 | 365.0 |
| 2025-11-07 | 358 | 364.0 |
| 2025-11-08 | 357 | 359.7 |

61 rows × 3 columns (first 8 shown)




Daily units sold and the 7-day and 28-day moving averages (last 6 months of 2025). The weekly pattern (weekend peaks) disappears in the averages; the 28-day line shows the trend.

The first six rows of a 7-row window have fewer than seven rows available: SQL averages whatever exists (partial windows). Check `COUNT(*) OVER (...)` if you need to exclude them.

## LAG and LEAD: compare with previous and next rows

| Function | Returns |
|---|---|
| `LAG(col, n, default)` | The value n rows **before** the current row (default n = 1) |
| `LEAD(col, n, default)` | The value n rows **after** |

SQL

```sql
WITH monthly AS (
    SELECT YEAR(order_date) * 100 + MONTH(order_date) AS ym, SUM(sales) AS sales
    FROM superstore
    GROUP BY YEAR(order_date) * 100 + MONTH(order_date)
)
SELECT ym,
       ROUND(sales, 0)                                                    AS sales,
       ROUND(LAG(sales)  OVER (ORDER BY ym), 0)                           AS prev_month,
       ROUND(100.0 * (sales - LAG(sales) OVER (ORDER BY ym)) / LAG(sales) OVER (ORDER BY ym), 1)
AS mom_pct,
       ROUND(LAG(sales, 12) OVER (ORDER BY ym), 0)                        AS same_month_last_year,
       ROUND(100.0 * (sales - LAG(sales, 12) OVER (ORDER BY ym)) / LAG(sales, 12) OVER (ORDER BY
ym), 1) AS yoy_pct,
       ROUND(LEAD(sales) OVER (ORDER BY ym), 0)                           AS next_month
FROM monthly
ORDER BY ym;
```

RESULT

| ym | sales | prev_month | mom_pct | same_month_last_year | yoy_pct | next_month |
|---|---|---|---|---|---|---|
| 202301 | 2,447,151 | NULL | NULL | NULL | NULL | 3,861,852 |
| 202302 | 3,861,852 | 2,447,151 | 57.8 | NULL | NULL | 3,441,306 |
| 202303 | 3,441,306 | 3,861,852 | -10.9 | NULL | NULL | 3,927,557 |
| 202304 | 3,927,557 | 3,441,306 | 14.1 | NULL | NULL | 4,840,130 |
| 202305 | 4,840,130 | 3,927,557 | 23.2 | NULL | NULL | 5,785,712 |
| 202306 | 5,785,712 | 4,840,130 | 19.5 | NULL | NULL | 5,390,409 |
| 202307 | 5,390,409 | 5,785,712 | -6.8 | NULL | NULL | 4,129,386 |
| 202308 | 4,129,386 | 5,390,409 | -23.4 | NULL | NULL | 5,714,731 |
| 202309 | 5,714,731 | 4,129,386 | 38.4 | NULL | NULL | 5,134,331 |
| 202310 | 5,134,331 | 5,714,731 | -10.2 | NULL | NULL | 5,815,308 |
| 202311 | 5,815,308 | 5,134,331 | 13.3 | NULL | NULL | 7,152,348 |
| 202312 | 7,152,348 | 5,815,308 | 23.0 | NULL | NULL | 6,022,738 |

36 rows × 7 columns (first 12 shown)

- **MoM** compares with `LAG(sales, 1)`; **YoY** compares with `LAG(sales, 12)` when there is exactly one row per month. This is the SQL equivalent of the pivot table's "Difference From" settings (Excel Lesson 8).
- The first row has no previous value, so `LAG` returns NULL (or the default you supply: `LAG(sales, 1, 0)`).

**WATCH OUT** **Gaps in the data**

`LAG(sales, 12)` assumes every month exists. If a month has no sales it also has no row, so LAG(12) points at the wrong month. Build a calendar table (a list of every month) and LEFT JOIN the sales onto it first.

## Time between events

"How many days passed between each customer's consecutive orders?"

SQL

```
WITH o AS (
    SELECT customer_id, order_date,
           LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_order
    FROM orders
)
SELECT COUNT(*)                                          AS repeat_orders,
       ROUND(AVG(DATEDIFF(day, prev_order, order_date) * 1.0), 1) AS avg_days_between_orders,
       MIN(DATEDIFF(day, prev_order, order_date))        AS shortest_gap,
       MAX(DATEDIFF(day, prev_order, order_date))        AS longest_gap
FROM o
WHERE prev_order IS NOT NULL;
```

RESULT

| repeat_orders | avg_days_between_orders | shortest_gap | longest_gap |
|---|---|---|---|
| 2415 | 186.6 | 0 | 1001 |

1 rows × 4 columns

## FIRST_VALUE, LAST_VALUE, NTH_VALUE

SQL

```
SELECT department, emp_id, monthly_income,
       FIRST_VALUE(emp_id) OVER (PARTITION BY department ORDER BY monthly_income DESC)
AS top_earner,
       LAST_VALUE(emp_id)  OVER (PARTITION BY department ORDER BY monthly_income DESC
                                 ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
AS lowest_earner,
       monthly_income - FIRST_VALUE(monthly_income) OVER (PARTITION BY department ORDER BY
monthly_income DESC) AS gap_to_top
FROM employees
ORDER BY department, monthly_income DESC;
```

RESULT

| department | emp_id | monthly_income | top_earner | lowest_earner | gap_to_top |
|---|---|---|---|---|---|
| Finance | E00519 | 209833 | E00519 | E00322 | 0 |
| Finance | E00145 | 185500 | E00519 | E00322 | -24333 |
| Finance | E00336 | 174417 | E00519 | E00322 | -35416 |
| Finance | E00201 | 169500 | E00519 | E00322 | -40333 |
| Finance | E01172 | 168167 | E00519 | E00322 | -41666 |
| Finance | E01151 | 155167 | E00519 | E00322 | -54666 |
| Finance | E00759 | 152083 | E00519 | E00322 | -57750 |
| Finance | E01004 | 144667 | E00519 | E00322 | -65166 |

1,500 rows × 6 columns (first 8 shown)

`LAST_VALUE` needs the explicit full frame (`UNBOUNDED FOLLOWING`); with the default frame it just returns the current row.

## Distribution functions

| Function | Meaning |
|---|---|
| `PERCENT_RANK()` | (rank − 1) ÷ (rows − 1): 0 for the lowest, 1 for the highest |
| `CUME_DIST()` | Share of rows with a value ≤ the current one |
| `PERCENTILE_CONT(p) WITHIN GROUP (ORDER BY x) OVER (…)` | Interpolated percentile (e.g. median at p = 0.5) |
| `PERCENTILE_DISC(p) …` | An actual value from the data |

SQL

```sql
SELECT DISTINCT department,
       ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY monthly_income) OVER (PARTITION BY
department), 0) AS median_income,
       ROUND(AVG(monthly_income) OVER (PARTITION BY department), 0)
AS mean_income
FROM employees
ORDER BY department;
```

RESULT

| department | median_income | mean_income |
|---|---|---|
| Finance | 77,709 | 83,681 |
| HR | 86,833 | 87,805 |
| IT | 68,750 | 79,051 |
| Operations | 92,667 | 92,461 |
| R&D | 70,250 | 80,255 |
| Sales | 74,709 | 82,448 |

6 rows × 3 columns

In SQL Server, `PERCENTILE_CONT` is a window function (needs `OVER`); PostgreSQL uses it as an aggregate (`GROUP BY`). Mean above median is the signature of a right-skewed distribution (Statistics, ML track).

## Cumulative share: the Pareto in SQL

Which products produce the first 80% of sales?

SQL

```sql
WITH p AS (
    SELECT product_id, SUM(sales) AS sales FROM order_items GROUP BY product_id
), c AS (
    SELECT product_id, sales,
           SUM(sales) OVER (ORDER BY sales DESC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) * 1.0 / SUM(sales) OVER () AS cum_share,
           ROW_NUMBER() OVER (ORDER BY sales DESC) AS rnk
    FROM p
)
SELECT COUNT(*) AS products_for_80pct_of_sales,
       (SELECT COUNT(*) FROM p)  AS total_products
FROM c
WHERE cum_share ⩽ 0.8;
```

RESULT

| products_for_80pct_of_sales | total_products |
|---|---|
| 20 | 78 |

1 rows × 2 columns

## Gaps and islands: streaks

A classic: find runs of consecutive days. The trick is that `date - ROW_NUMBER()` is **constant within a streak**.

SQL

```sql
WITH days AS (
    SELECT DISTINCT order_date AS d FROM orders WHERE order_date ⩾ '2025-12-01'
), g AS (
    SELECT d, DATEADD(day, -ROW_NUMBER() OVER (ORDER BY d), d) AS grp FROM days
)
SELECT MIN(d) AS streak_start, MAX(d) AS streak_end, COUNT(*) AS days_in_streak
FROM g
GROUP BY grp
ORDER BY streak_start;
```

RESULT

| streak_start | streak_end | days_in_streak |
|---|---|---|
| 2025-12-01 | 2025-12-31 | 31 |

1 rows × 3 columns

Each group of consecutive order days becomes one row: all of December 2025 had orders every day, so there is a single streak. The same pattern finds consecutive login days, uninterrupted employment spells and missing-sequence ranges.

## Summary

**KEY TAKEAWAYS**

- **Frames** (`ROWS BETWEEN … AND …`) choose the rows around the current row; always write `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` for running totals (the default RANGE frame includes ties).
- Running totals, YTD, cumulative % and moving averages use aggregate functions with frames.
- `LAG`/`LEAD` read the previous/next row (MoM, YoY, time between events); fill calendar gaps first.
- `FIRST_VALUE`/`LAST_VALUE` need an explicit full frame for LAST_VALUE.
- `PERCENT_RANK`, `CUME_DIST`, `PERCENTILE_CONT` describe position and median within a distribution.
- Gaps and islands: `value - ROW_NUMBER()` is constant within a consecutive run.

### Quick quiz

6 questions · tap an answer to check it

**1.** Which frame gives a 3-month moving average (current and two previous rows)?

A `ROWS BETWEEN 3 PRECEDING AND CURRENT ROW`

B `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`

C `ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING`

D `RANGE 3`

**Answer: B.** Two preceding rows plus the current row make three rows.

**2.** What does `LAG(sales, 12) OVER (ORDER BY ym)` return for monthly data?

A Sales 12 months in the future

B Sales for the same month last year (if no month is missing)

C The 12th highest sales

D The average of 12 months

**Answer: B.** LAG reads n rows back; with one row per month that is 12 months earlier.

**3.** Why does LAST_VALUE often return the current row's value?

A The default frame ends at the current row, so you must extend it to UNBOUNDED FOLLOWING

B LAST_VALUE is broken in SQL Server

C It only works with PARTITION BY

D ORDER BY is missing

**Answer: A.** Specify ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

**4.** How do you restart a running total every year?

A ORDER BY year

B PARTITION BY year in the OVER clause

C GROUP BY year only

D Use LAG

**Answer: B.** Each partition has its own running total.

**5.** In the gaps-and-islands trick, what stays constant inside a streak of consecutive dates?

A The date

B The row number

C The date minus its ROW_NUMBER

D The count

**Answer: C.** Both increase by 1 per row, so their difference is constant.

**6.** Mean income is much higher than median income. What does that suggest?

A Symmetric data

B A right-skewed distribution with a few very high values

C Missing data

D A data error

**Answer: B.** A long right tail pulls the mean above the median.

EXERCISE **Time-series practice**

(1) Monthly sales per region with MoM %; (2) for each customer, the days since their previous order and a flag for "returned within 30 days"; (3) a 3-month moving average of sales by category; (4) the month when cumulative 2025 sales first exceeded ₹5 crore.

**Show solution (worked answer)**

SQL

```sql
-- (1)
WITH m AS (SELECT region, YEAR(order_date)*100+MONTH(order_date) AS ym, SUM(sales) AS sales
FROM superstore GROUP BY region, YEAR(order_date)*100+MONTH(order_date))
SELECT region, ym, ROUND(sales,0) AS sales,
       ROUND(100.0*(sales - LAG(sales) OVER (PARTITION BY region ORDER BY ym)) / LAG(sales)
OVER (PARTITION BY region ORDER BY ym), 1) AS mom_pct FROM m ORDER BY region, ym;
-- (2)
WITH o AS (SELECT customer_id, order_date, LAG(order_date) OVER (PARTITION BY customer_id
ORDER BY order_date) AS prev FROM orders)
SELECT customer_id, order_date, DATEDIFF(day, prev, order_date) AS days_since_prev,
       CASE WHEN DATEDIFF(day, prev, order_date) ⩽ 30 THEN 1 ELSE 0 END AS within_30_days
FROM o WHERE prev IS NOT NULL;
-- (3)
WITH m AS (SELECT category, YEAR(order_date)*100+MONTH(order_date) AS ym, SUM(sales) AS
sales FROM superstore GROUP BY category, YEAR(order_date)*100+MONTH(order_date))
SELECT category, ym, AVG(sales) OVER (PARTITION BY category ORDER BY ym ROWS BETWEEN 2
PRECEDING AND CURRENT ROW) AS ma3 FROM m;
-- (4)
WITH m AS (SELECT MONTH(order_date) AS mth, SUM(sales) AS sales FROM superstore WHERE
YEAR(order_date) = 2025 GROUP BY MONTH(order_date)),
     c AS (SELECT mth, SUM(sales) OVER (ORDER BY mth ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) AS cum FROM m)
SELECT TOP 1 mth FROM c WHERE cum > 50000000 ORDER BY mth;
```

INTERVIEW CORNER **Questions you may be asked**

**Q: How do you calculate a running total in SQL?** `SUM(x) OVER (ORDER BY d ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`, optionally partitioned.

**Q: How do you calculate month-on-month growth?** Aggregate to months, then `(sales - LAG(sales) OVER (ORDER BY month)) / LAG(sales) OVER (ORDER BY month)`; use `LAG(…, 12)` for YoY.

**Q: ROWS vs RANGE in a window frame?** ROWS counts physical rows; RANGE includes all rows with the same ORDER BY value as the current row (peers). ROWS is predictable and faster.

**Q: How would you find consecutive login days?** Gaps and islands: `date - ROW_NUMBER() OVER (PARTITION BY user ORDER BY date)` as a group key, then GROUP BY it.

SQL FOR DATA ANALYSIS · LESSON 13

# CTEs, Views, Temporary Tables, Stored Procedures & Functions

Write queries other people can read and reuse. Use CTEs (including recursive CTEs for hierarchies and calendars), create views for reusable logic and security, use temp tables and table variables, and build stored procedures, user-defined functions, parameters and basic error handling in T-SQL.

SQL class 6 · SQL Server 2022, 2026 · Advanced · 130 min

**By the end of this lesson you can**

- Structure complex queries with one or more CTEs and explain CTE vs subquery vs temp table
- Write recursive CTEs for hierarchies (org charts) and number or date series
- Create, alter and use views, and explain what views are good for
- Choose between temp tables, table variables and CTEs
- Write stored procedures with parameters, scalar and table-valued functions, and TRY...CATCH

## CTE: a named, readable subquery

A **Common Table Expression** is a named result set defined with `WITH` that exists for **one statement**. It makes complex logic read top to bottom, like paragraphs, instead of inside-out.

SQL

```
WITH name1 AS (
    SELECT …
),
name2 AS (
    SELECT … FROM name1 …          -- later CTEs can use earlier ones
)
SELECT … FROM name2 …;
```

### From nested subqueries to CTEs

Question: *"Which regions had above-average 2025 sales per customer?"*

SQL

```sql
WITH customer_sales AS (                        -- step 1: sales per customer in 2025
    SELECT c.region, c.customer_id, SUM(oi.sales) AS sales
    FROM order_items AS oi
    JOIN orders    AS o ON o.order_id    = oi.order_id
    JOIN customers AS c ON c.customer_id = o.customer_id
    WHERE o.order_date ⩾ '2025-01-01' AND o.order_date < '2026-01-01'
    GROUP BY c.region, c.customer_id
),
region_avg AS (                                 -- step 2: average per region
    SELECT region, COUNT(*) AS customers, AVG(sales) AS avg_sales_per_customer
    FROM customer_sales
    GROUP BY region
)
SELECT region, customers,
       ROUND(avg_sales_per_customer, 0)                      AS avg_sales_per_customer,
       ROUND((SELECT AVG(sales) FROM customer_sales), 0)      AS overall_avg
FROM region_avg
WHERE avg_sales_per_customer > (SELECT AVG(sales) FROM customer_sales)   -- step 3: compare
ORDER BY avg_sales_per_customer DESC;
```

RESULT

| region | customers | avg_sales_per_customer | overall_avg |
|---|---|---|---|
| West | 156 | 142,513 | 125,805 |
| South | 160 | 130,010 | 125,805 |

2 rows × 4 columns

`customer_sales` is used **twice**, which with plain subqueries would mean copy-pasting the same block. CTEs also make window-function filters (Lesson 11) clean.

> **TIP** **CTE style**
>
> Name CTEs for what they contain (`customer_sales`, not `cte1`); keep each CTE doing one job; comment each step. If a CTE needs a long explanation, it probably should be two CTEs.

## CTE, subquery, view and temp table

| | Subquery / derived table | CTE | View | Temp table |
|---|---|---|---|---|
| Lives for | The statement | The statement | Permanently (a saved query) | The session (`#t`) |
| Stores data? | No | No (re-evaluated) | No (unless indexed/materialised) | **Yes** |
| Reusable in other queries | No | No | **Yes** | In the same session |
| Can be referenced several times | Only by repeating | **Yes**, in the same statement | Yes | Yes |
| Recursive | No | **Yes** | No | No |
| Can be indexed | No | No | Indexed views only | **Yes** |
| Best for | Simple one-off logic | Readability, multi-step logic, recursion | Sharing logic, hiding complexity, security | Large intermediate results used many times |

> **NOTE** **A CTE is not a cache**
>
> In SQL Server the CTE's query may run each time it is referenced. If an expensive CTE is used many times, load it into a **temp table** once.

## Recursive CTEs

A recursive CTE has an **anchor** (starting rows) and a **recursive member** that refers to the CTE itself, joined with `UNION ALL`. It repeats until no new rows are produced.

### Number and date series

SQL

```sql
WITH numbers AS (
    SELECT 1 AS n                    -- anchor
    UNION ALL
    SELECT n + 1 FROM numbers WHERE n < 7      -- recursive member with a stop condition
)
SELECT n FROM numbers;
```

RESULT

| n |
|---|
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |
| 6 |
| 7 |

7 rows × 1 columns

A **calendar table** of every day in 2026, handy for filling gaps before LAG/LEAD (Lesson 12):

SQL

```sql
WITH calendar AS (
    SELECT CAST('2026-01-01' AS DATE) AS d
    UNION ALL
    SELECT CAST(DATEADD(day, 1, d) AS DATE) FROM calendar WHERE d < CAST('2026-12-31' AS DATE)
)
SELECT COUNT(*) AS days, MIN(d) AS first_day, MAX(d) AS last_day,
       SUM(CASE WHEN DATENAME(weekday, d) IN ('Saturday', 'Sunday') THEN 1 ELSE 0 END) AS
weekend_days
FROM calendar;
```

RESULT

| days | first_day | last_day | weekend_days |
|---|---|---|---|
| 365 | 2026-01-01 | 2026-12-31 | 104 |

1 rows × 4 columns

In SQL Server, the default limit is 100 recursions, so a year-long calendar needs `OPTION (MAXRECURSION 400)` after the final SELECT (use `0` for no limit, with care).

## Hierarchies: org chart

SQL

```sql
CREATE TABLE org (emp_id INT, name VARCHAR(20), manager_id INT);
INSERT INTO org VALUES (1,'Varun',NULL), (2,'Asha',1), (3,'Ravi',1), (4,'Meera',2),
(5,'Kabir',2), (6,'Zoya',4), (7,'Farhan',3);

WITH tree AS (
    SELECT emp_id, name, manager_id, 0 AS level, CAST(name AS VARCHAR(200)) AS path
    FROM org WHERE manager_id IS NULL                          -- anchor: the top
    UNION ALL
    SELECT o.emp_id, o.name, o.manager_id, t.level + 1, CAST(CONCAT(t.path, ' > ', o.name) AS
VARCHAR(200))
    FROM org AS o
    JOIN tree AS t ON o.manager_id = t.emp_id                  -- walk down one level
)
SELECT level, CONCAT(REPLICATE('    ', level), name) AS org_chart, path
FROM tree
ORDER BY path;
```

RESULT

```
Commands completed successfully.
(7 rows affected)
```

| level | org_chart | path |
|---|---|---|
| 0 | Varun | Varun |
| 1 | Asha | Varun > Asha |
| 2 | Kabir | Varun > Asha > Kabir |
| 2 | Meera | Varun > Asha > Meera |
| 3 | Zoya | Varun > Asha > Meera > Zoya |
| 1 | Ravi | Varun > Ravi |
| 2 | Farhan | Varun > Ravi > Farhan |

7 rows × 3 columns

Recursive CTEs also solve bill-of-materials, category trees and graph-path problems.

## Views

A **view** is a saved SELECT with a name. Querying it runs the stored query; it holds no data of its own.

SQL

```sql
CREATE VIEW vw_order_detail AS
SELECT o.order_id, o.order_date, o.ship_mode,
       c.customer_id, c.customer_name, c.segment, c.city, c.region,
       p.product_id, p.product_name, p.category, p.sub_category,
       oi.quantity, oi.discount, oi.sales, oi.profit
FROM order_items AS oi
JOIN orders    AS o ON o.order_id    = oi.order_id
JOIN customers AS c ON c.customer_id = o.customer_id
JOIN products  AS p ON p.product_id  = oi.product_id;

SELECT category, region, ROUND(SUM(sales), 0) AS sales
FROM vw_order_detail
WHERE order_date ⩾ '2025-01-01'
GROUP BY category, region
ORDER BY category, region;
```

RESULT

| category | region | sales |
|---|---|---|
| Furniture | East | 2,875,816 |
| Furniture | North | 2,688,426 |
| Furniture | South | 3,759,688 |
| Furniture | West | 3,436,316 |
| Office Supplies | East | 1,075,076 |
| Office Supplies | North | 833,204 |
| Office Supplies | South | 1,018,343 |
| Office Supplies | West | 1,044,699 |
| Technology | East | 14,959,309 |
| Technology | North | 12,911,446 |
| Technology | South | 16,023,523 |
| Technology | West | 17,750,953 |

12 rows × 3 columns

Now analysts query one friendly object instead of repeating a four-table join. Power BI and Excel can connect to the view too.

| Use views to | Notes |
|---|---|
| Hide complexity | One name for a multi-join query |
| Provide one definition of a metric | "Net sales" defined once, used everywhere |
| Secure data | `GRANT SELECT ON vw_sales_summary TO analysts` without granting access to base tables; expose only some columns/rows |
| Keep apps working when tables change | Change the view, not every report |

SQL Server syntax: `CREATE VIEW dbo.vw_x AS …`, `ALTER VIEW`, `DROP VIEW IF EXISTS`, `CREATE OR ALTER VIEW`. Rules: no `ORDER BY` inside a view (unless with TOP); views can usually be `INSERT`/`UPDATE`d only if they map simply to one table. **Indexed (materialised) views** store the results physically for speed (`WITH SCHEMABINDING` + a unique clustered index).

## Temporary tables and table variables

SQL

```sql
-- Temp table: lives for the session, stored in tempdb, can be indexed, has statistics
SELECT customer_id, SUM(sales) AS sales
INTO   #cust_sales                      -- the # prefix makes it local temporary
FROM   superstore
GROUP  BY customer_id;

CREATE INDEX ix_cust ON #cust_sales (customer_id);
SELECT TOP 10 * FROM #cust_sales ORDER BY sales DESC;
DROP TABLE IF EXISTS #cust_sales;

-- Table variable: scoped to the batch; for small row counts
DECLARE @top TABLE (customer_id VARCHAR(10), sales DECIMAL(14,2));
INSERT INTO @top SELECT TOP 5 customer_id, SUM(sales) FROM superstore GROUP BY customer_id ORDER
BY SUM(sales) DESC;
SELECT * FROM @top;
```

##name is a **global** temp table visible to other sessions. Temp tables vanish when the session ends.

## Stored procedures

A **stored procedure** is a named, saved block of T-SQL that can take parameters, run many statements, and return results. It is compiled and its plan cached, and permissions can be granted on the procedure instead of the tables.

SQL

```sql
CREATE OR ALTER PROCEDURE dbo.usp_sales_by_region
    @from_date DATE,
    @to_date   DATE,
    @category  VARCHAR(30) = NULL          -- optional parameter with a default
AS
BEGIN
    SET NOCOUNT ON;                        -- suppress "(n rows affected)" messages

    SELECT c.region,
           COUNT(DISTINCT o.order_id) AS orders,
           SUM(oi.sales)              AS sales,
           SUM(oi.profit)             AS profit
    FROM dbo.order_items AS oi
    JOIN dbo.orders      AS o ON o.order_id    = oi.order_id
    JOIN dbo.customers   AS c ON c.customer_id = o.customer_id
    JOIN dbo.products    AS p ON p.product_id  = oi.product_id
    WHERE o.order_date >= @from_date
      AND o.order_date <  DATEADD(day, 1, @to_date)
      AND (@category IS NULL OR p.category = @category)     -- optional filter
    GROUP BY c.region
    ORDER BY sales DESC;
END;
GO

EXEC dbo.usp_sales_by_region @from_date = '2025-01-01', @to_date = '2025-12-31';
EXEC dbo.usp_sales_by_region '2025-01-01', '2025-12-31', 'Technology';
```

## Output parameters, variables, IF and loops

SQL

```sql
CREATE OR ALTER PROCEDURE dbo.usp_add_order_note
    @order_id VARCHAR(20),
    @note     NVARCHAR(200),
    @rows     INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT EXISTS (SELECT 1 FROM dbo.orders WHERE order_id = @order_id)
    BEGIN
        RAISERROR('Order %s does not exist.', 16, 1, @order_id);   -- or THROW 50001, 'Order
does not exist.', 1;
        RETURN;
    END;

    UPDATE dbo.orders SET note = @note WHERE order_id = @order_id;
    SET @rows = @@ROWCOUNT;                                        -- rows changed by the last
statement
END;
GO

DECLARE @n INT;
EXEC dbo.usp_add_order_note 'IN-2025-00042', N'Gift wrap', @rows = @n OUTPUT;
SELECT @n AS rows_updated;
```

| Control flow | Syntax |
|---|---|
| Condition | `IF … BEGIN … END ELSE BEGIN … END` |
| Loop | `WHILE @i ⩽ 10 BEGIN … SET @i += 1; END` |
| Exit / return | `RETURN` |
| Row count | `@@ROWCOUNT` |
| Dynamic SQL | `EXEC sp_executesql @sql, N'@x INT', @x = 5` (use parameters; never concatenate user input: SQL injection) |

## Error handling: TRY…CATCH

SQL

```sql
BEGIN TRY
    BEGIN TRAN;
        UPDATE dbo.accounts SET balance = balance - 5000 WHERE acct_id = 1;
        UPDATE dbo.accounts SET balance = balance + 5000 WHERE acct_id = 2;
    COMMIT;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK;
    SELECT ERROR_NUMBER() AS err_no, ERROR_MESSAGE() AS err_msg, ERROR_LINE() AS err_line;
    THROW;                                      -- re-raise to the caller
END CATCH;
```

## User-defined functions (UDFs)

| Type | Returns | Example use |
|---|---|---|
| **Scalar** | One value | `dbo.fn_margin(profit, sales)` |
| **Inline table-valued** | A table, from one SELECT | `dbo.fn_orders_by_city('Pune')`: like a parameterised view |
| **Multi-statement table-valued** | A table built in a body | Complex logic (usually slower) |

SQL

```sql
CREATE OR ALTER FUNCTION dbo.fn_margin (@profit DECIMAL(12,2), @sales DECIMAL(12,2))
RETURNS DECIMAL(6,2)
AS
BEGIN
    RETURN CASE WHEN @sales = 0 THEN NULL ELSE 100.0 * @profit / @sales END;
END;
GO

CREATE OR ALTER FUNCTION dbo.fn_sales_by_city (@city VARCHAR(30))
RETURNS TABLE
AS RETURN (
    SELECT order_id, order_date, sales, profit FROM dbo.vw_order_detail WHERE city = @city
);
GO

SELECT order_id, dbo.fn_margin(profit, sales) AS margin_pct FROM dbo.fn_sales_by_city('Pune');
```

**WATCH OUT** **Scalar UDFs can be slow**

Older SQL Server versions call a scalar UDF once per row, which can make a query many times slower than writing the expression inline (SQL Server 2019+ can inline some). Prefer inline table-valued functions or views for set-based logic.

### Procedure vs function vs view

| | View | Stored procedure | Function |
|---|---|---|---|
| Parameters | No | Yes | Yes |
| Can change data (INSERT/UPDATE) | Limited | **Yes** | No (side-effect free) |
| Used in SELECT / FROM | Yes | No (EXEC) | **Yes** |
| Returns | A table | Result sets, output parameters, a status code | A value or a table |

## Triggers (awareness)

A **trigger** runs automatically on INSERT, UPDATE or DELETE: audit logs, maintaining summary tables. Use sparingly: they hide logic and can slow writes. Modern designs prefer explicit procedures, constraints, temporal tables or change data capture.

DO IT WITH AI **Do it with AI**

Prompt: *"Convert this nested-subquery SQL into readable CTEs with clear names and comments, keep the result identical, and explain each step."* AI is good at refactoring; verify equality by running both versions and comparing with EXCEPT in both directions (Lesson 9).

## Summary

KEY TAKEAWAYS

- **CTE** (`WITH name AS (…)`): a named, readable subquery for one statement; can be referenced many times; **recursive CTEs** generate series and walk hierarchies (anchor + `UNION ALL` + recursive member).
- **View**: a saved query for reuse, one definition of a metric, and security. **Temp tables** (`#t`) store intermediate data and can be indexed; table variables suit small sets.
- **Stored procedures** take parameters, run multiple statements, change data and handle errors with `TRY…CATCH`; **functions** return values or tables and can be used inside queries.
- Prefer set-based SQL; avoid row-by-row loops and scalar UDFs on big tables; never concatenate user input into dynamic SQL.

### Quick quiz

6 questions · tap an answer to check it

**1.** A CTE exists for:

A The whole session

B A single statement

C Until the server restarts

D As long as the database

**Answer: B.** A CTE is defined and used within one statement.

**2.** What are the two parts of a recursive CTE?

A SELECT and WHERE

B An anchor member and a recursive member joined with UNION ALL

C INSERT and UPDATE

D A view and a function

**Answer: B.** The anchor gives starting rows; the recursive member repeats until nothing new appears.

**3.** Which object stores data physically for the session and can be indexed?

A CTE

B View

C Temporary table

D Subquery

**Answer: C.**

**temp tables live in tempdb for the session.**

**4.** Which can modify data and return an OUTPUT parameter?

A View

B Stored procedure

C Scalar function

D CTE

**Answer: B.** Procedures can run INSERT/UPDATE/DELETE and return output parameters.

**5.** **Why grant access to a view instead of the base table?**

A Views are faster

B You can expose only selected columns and rows and hide the underlying tables

C Views cannot be dropped

D Tables cannot be queried

**Answer: B.** Views are a security and simplification layer.

**6.** **A recursive CTE stops with an error after 100 levels. Fix?**

A Add DISTINCT

B Add OPTION (MAXRECURSION n) (and a proper stop condition)

C Use a view

D Use TOP

**Answer: B.** SQL Server's default recursion limit is 100.

EXERCISE **Build reusable objects**

(1) A view `vw_monthly_sales` with year, month, category and total sales. (2) A CTE query that lists, for each region, the best-selling sub-category and its share of the region's sales. (3) A stored procedure `usp_top_customers (@year INT, @n INT)` that returns the top n customers by sales for a year. (4) A recursive CTE that lists all employees under manager 2 in the `org` table with their level.

**Show solution (worked answer)**

SQL

```
-- (1)
CREATE VIEW vw_monthly_sales AS
SELECT YEAR(order_date) AS yr, MONTH(order_date) AS mth, category, SUM(sales) AS sales FROM
vw_order_detail GROUP BY YEAR(order_date), MONTH(order_date), category;
-- (2)
WITH rs AS (SELECT region, sub_category, SUM(sales) AS sales FROM superstore GROUP BY
region, sub_category),
     r  AS (SELECT *, RANK() OVER (PARTITION BY region ORDER BY sales DESC) AS rnk,
100.0*sales/SUM(sales) OVER (PARTITION BY region) AS pct FROM rs)
SELECT region, sub_category, ROUND(pct,1) AS pct_of_region FROM r WHERE rnk = 1;
-- (3) T-SQL
CREATE OR ALTER PROCEDURE usp_top_customers @year INT, @n INT AS
BEGIN
  SELECT TOP (@n) customer_id, customer_name, SUM(sales) AS sales FROM vw_order_detail
WHERE YEAR(order_date) = @year
  GROUP BY customer_id, customer_name ORDER BY sales DESC;
END;
-- (4)
WITH t AS (SELECT emp_id, name, 1 AS level FROM org WHERE manager_id = 2
           UNION ALL SELECT o.emp_id, o.name, t.level + 1 FROM org o JOIN t ON o.manager_id
= t.emp_id)
SELECT * FROM t;
```

INTERVIEW CORNER

### Questions you may be asked

**Q: CTE vs temp table vs subquery?** Subquery: inline, single use. CTE: named, readable, statement-scoped, supports recursion, may be re-evaluated. Temp table: materialised data for the session, indexable: use for large intermediate results reused many times.

**Q: View vs stored procedure?** A view is a saved SELECT you query like a table; a procedure is a program with parameters and multiple statements that can change data.

**Q: What is a recursive CTE and where do you use it?** A CTE that references itself, for hierarchies (org charts, categories), series generation and graph traversal.

**Q: How do you prevent SQL injection in dynamic SQL?** Use parameterised `sp_executesql` and QUOTENAME for identifiers; never concatenate raw user input.

SQL FOR DATA ANALYSIS · LESSON 14

# Transactions, Indexes, Performance & Security

Make SQL safe and fast. Understand transactions and ACID (BEGIN, COMMIT, ROLLBACK, SAVE), isolation and locking, indexes (clustered, non-clustered, covering), how to read an execution plan, a checklist for writing fast queries, and permissions with GRANT, REVOKE and roles (DCL).

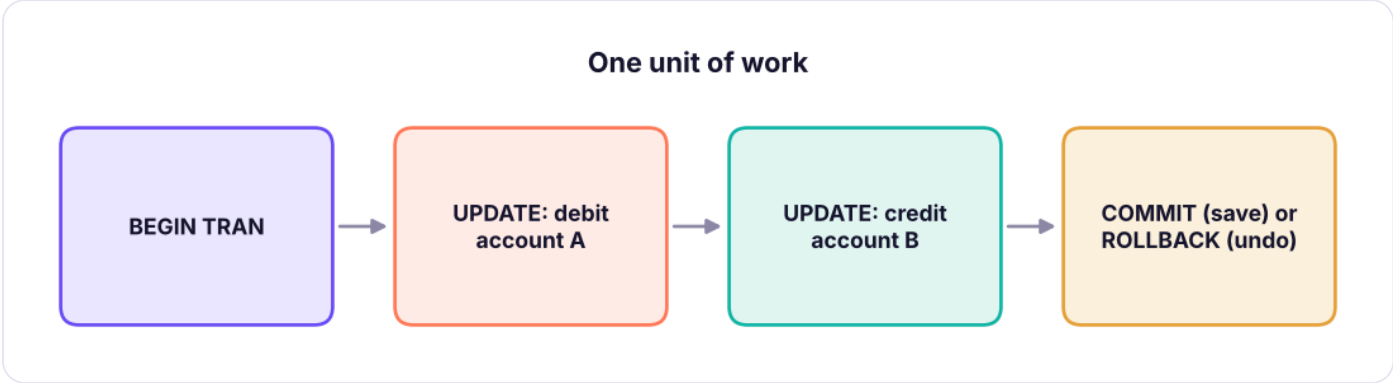
SQL class 6 (extended) · ✦ SQL Server 2022, 2026 · Advanced · ⏱ 110 min

**By the end of this lesson you can**

- Explain ACID and group statements into transactions that commit or roll back together
- Describe isolation levels, locks, blocking and deadlocks at an analyst level
- Choose between clustered and non-clustered indexes and recognise queries that can or cannot use them
- Read the basics of an execution plan and apply a performance checklist
- Grant and revoke permissions with roles, and apply least privilege

## Transactions and ACID

A **transaction** is a group of statements treated as one unit of work: **all succeed (commit) or none do (rollback)**. Moving ₹5,000 between two accounts is two updates; if the server crashes between them, money would vanish without a transaction.




A bank transfer as a transaction. Either both updates are saved, or both are undone.

| Property | Meaning | Example |
|---|---|---|
| **A**tomicity | All or nothing | Both account updates happen, or neither |
| **C**onsistency | Data stays valid (constraints hold) | Total money before = total after; no negative balance if a CHECK forbids it |
| **I**solation | Concurrent transactions do not see each other's half-finished work | Another user does not see the debit without the credit |
| **D**urability | Once committed, it survives a crash | Transaction log is written to disk first |

## Transaction commands

| Command | Purpose |
|---|---|
| `BEGIN TRANSACTION` (`BEGIN TRAN`) | Start a transaction |
| `COMMIT` | Make all changes permanent |
| `ROLLBACK` | Undo everything since BEGIN (or since a savepoint) |
| `SAVE TRANSACTION name` | Create a savepoint inside the transaction |
| `ROLLBACK TRANSACTION name` | Undo back to the savepoint only |
| `@@TRANCOUNT` | How many transactions are open (should be 0 when you leave) |

By default SQL Server runs in **autocommit** mode: every single statement is its own transaction. Wrap several statements in `BEGIN TRAN … COMMIT` to group them.

## Seeing a rollback

SQL

```sql
CREATE TABLE accounts (acct_id INT PRIMARY KEY, holder VARCHAR(20), balance DECIMAL(12,2));
INSERT INTO accounts VALUES (1, 'Asha', 20000), (2, 'Ravi', 5000);

BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 5000 WHERE acct_id = 1;
UPDATE accounts SET balance = balance + 5000 WHERE acct_id = 2;
SELECT * FROM accounts ORDER BY acct_id;      -- inside the transaction: changes visible to us
ROLLBACK;                                     -- something went wrong: undo both updates

SELECT * FROM accounts ORDER BY acct_id;      -- balances are back to the originals
```

RESULT

```
Commands completed successfully.
(2 rows affected)
Commands completed successfully.
(1 row affected)
```

```
(1 row affected)
Commands completed successfully.
```

RESULT SET 1

| acct_id | holder | balance |
|---|---|---|
| 1 | Asha | 15,000 |
| 2 | Ravi | 10,000 |

2 rows × 3 columns

RESULT SET 2

| acct_id | holder | balance |
|---|---|---|
| 1 | Asha | 20,000 |
| 2 | Ravi | 5,000 |

2 rows × 3 columns

### The safe-update pattern

The pattern that protects you from the "UPDATE without WHERE" disaster (Lesson 4):

SQL

```sql
BEGIN TRAN;

UPDATE dbo.staff SET salary = salary * 1.05 WHERE dept_id = 10;

SELECT @@ROWCOUNT AS rows_changed;                    -- expected: 2?
SELECT emp_id, salary FROM dbo.staff WHERE dept_id = 10;     -- sanity-check

-- If everything looks right:
COMMIT;
-- Otherwise:
-- ROLLBACK;
```

**WATCH OUT** **Never leave a transaction open**

An open transaction holds locks that block other users and keeps the log from truncating. Always end with COMMIT or ROLLBACK; check `SELECT @@TRANCOUNT;` if a query window seems "stuck".

## Concurrency: isolation, locks and deadlocks

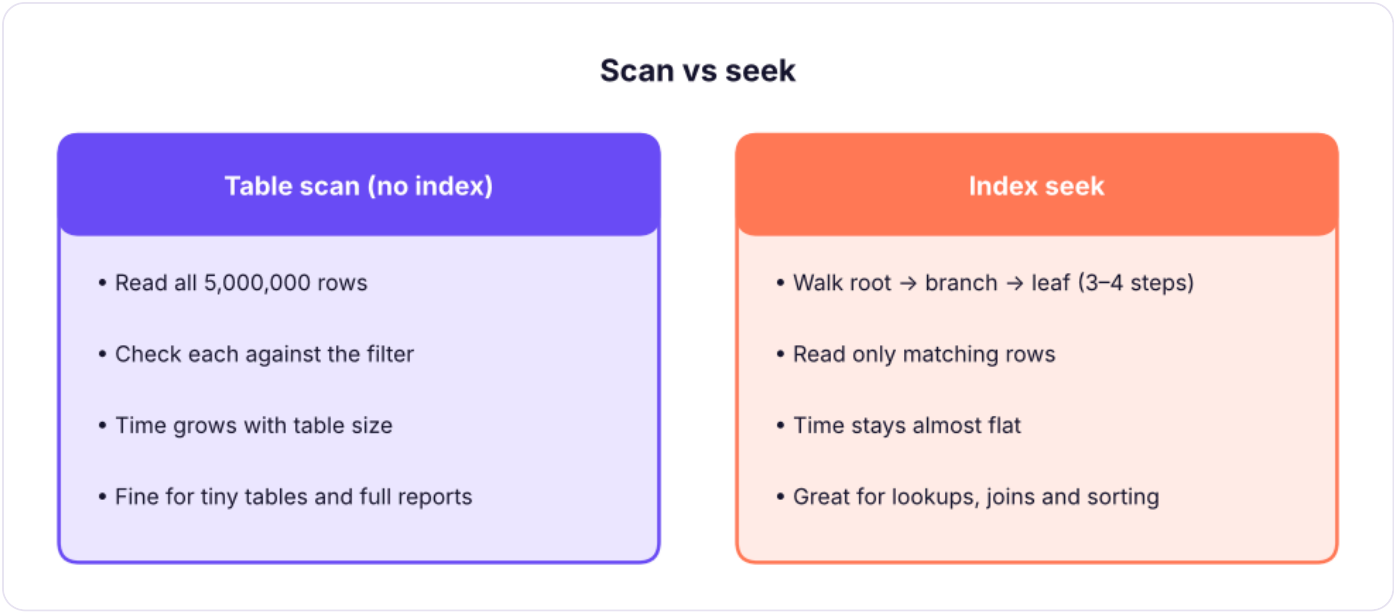
When many users work at once, the database uses **locks** to keep transactions isolated. **Isolation levels** trade consistency for concurrency:

| Level | Prevents | Notes |
|---|---|---|
| `READ UNCOMMITTED` | Nothing (allows dirty reads; `NOLOCK` hint) | Can read data that is later rolled back; avoid for reports needing accuracy |
| `READ COMMITTED` (default) | Dirty reads | A row you read can change before you read it again |
| `REPEATABLE READ` | Dirty and non-repeatable reads | Holds read locks longer |
| `SERIALIZABLE` | Also phantom rows | Safest, least concurrent |
| `SNAPSHOT` / RCSI | Readers do not block writers (row versions) | Common in modern systems, uses tempdb |

- **Blocking**: one session waits for a lock held by another. Fix: shorter transactions, indexes, row versioning.
- **Deadlock**: two sessions each hold a lock the other needs; SQL Server kills one (the "victim", error 1205). Fix: access tables in the same order, keep transactions short, retry on error 1205.

## Indexes: how databases find rows fast

Without an index, a query like `WHERE customer_id = 'C0042'` scans **every row** (table scan). An **index** is a sorted structure (a B-tree) that lets the database jump straight to matching rows, like the index of a book.




A table scan reads every row. An index seek follows a short path through a B-tree.

| Index type | What it is | How many per table |
|---|---|---|
| **Clustered** | Defines the physical order of the table's rows; the table *is* the index. Created by default with a PRIMARY KEY | **One** |
| **Non-clustered** | A separate sorted copy of selected columns with pointers to the rows | Many (up to 999) |
| **Unique** | Enforces uniqueness (created for UNIQUE constraints) | Many |
| **Covering** (`INCLUDE`) | A non-clustered index that contains every column a query needs, so the table is never touched | |
| **Filtered** | Indexes only rows matching a condition | |
| **Columnstore** | Column-wise compressed storage for analytics on huge tables | |

SQL

```sql
CREATE CLUSTERED INDEX cx_orders_date ON dbo.orders (order_date);           -- if not already clustered by the PK

CREATE NONCLUSTERED INDEX ix_orders_customer
    ON dbo.orders (customer_id)
    INCLUDE (order_date, ship_mode);                                        -- covering index for "orders per customer" queries

CREATE NONCLUSTERED INDEX ix_items_product ON dbo.order_items (product_id);   -- foreign keys deserve indexes

DROP INDEX ix_orders_customer ON dbo.orders;
```

SQL

```sql
CREATE INDEX ix_super_city ON superstore (city);
SELECT index_name, table_name FROM duckdb_indexes() WHERE table_name = 'superstore';
```

RESULT

| index_name | table_name |
|---|---|
| ix_super_city | superstore |

1 rows × 2 columns

## Which queries can use an index?

| Pattern | Uses an index? | Why |
|---|---|---|
| `WHERE customer_id = 'C0042'` | **Yes** (seek) | Equality on the leading column |
| `WHERE order_date ⩾ '2025-01-01' AND order_date < '2026-01-01'` | **Yes** | Range on an indexed column |
| `WHERE YEAR(order_date) = 2025` | **No** (scan) | The function must run on every row: **non-SARGable** |
| `WHERE customer_name LIKE 'Ra%'` | Yes | Prefix match |
| `WHERE customer_name LIKE '%Rao'` | No | Leading wildcard |
| `WHERE UPPER(city) = 'PUNE'` | No | Function on the column (the default collation is already case-insensitive) |
| `WHERE city = 'Pune' OR product_id = 'P0012'` | Often no | OR across different columns |
| `WHERE CAST(order_id AS VARCHAR) = '1001'` | No | Type conversion on the column |

> **TIP** **SARGable predicates**
>
> **SARG** = Search ARGument able. Keep the **column bare** on one side of the comparison and do the maths on the other side: use `order_date ⩾ '2025-01-01' AND order_date < '2026-01-01'` instead of `YEAR(order_date) = 2025`, and `sales > 100 / 1.18` instead of `sales * 1.18 > 100`.

### Index trade-offs

Indexes speed up **reads** but slow down **writes** (every INSERT/UPDATE/DELETE must also update each index) and use disk space. Index columns used in `WHERE`, `JOIN`, `ORDER BY` and `GROUP BY`, and foreign keys; do not index everything. Analytic warehouses (Snowflake, BigQuery) use different techniques (partitioning, clustering, columnar storage) instead of manual indexes.

## Execution plans

In SSMS press `Ctrl + L` (estimated plan) or `Ctrl + M` then run (actual plan). The plan is a diagram of operators read **right to left**. Things to look for:

| Operator | Meaning | Concern |
|---|---|---|
| Index Seek | Direct lookup | Good |
| Index / Clustered Index Scan | Reads many rows | Fine for big reports, suspicious for single-row lookups |
| Table Scan | Reads the heap | Usually add an index |
| Key Lookup | Fetches extra columns from the table per row | Consider a covering index (`INCLUDE`) |
| Hash / Merge / Nested Loops Join | Join algorithms | Hash for big unsorted inputs; nested loops for small; merge for sorted |
| Sort | Sorting rows | Expensive; an index in the right order avoids it |
| Thick arrows, big estimated vs actual row differences | Bad estimates | Update statistics |

`SET STATISTICS IO, TIME ON;` shows logical reads (the best measure of work) and CPU time for each query.

## Performance checklist for analysts

1. **Select only the columns and rows you need**: avoid `SELECT *`; filter early.
2. **Make filters SARGable** (no functions on columns; use ranges for dates).
3. **Join on indexed keys** with matching data types (an `INT` joined to a `VARCHAR` forces conversions).
4. **Aggregate before joining** when joining big tables (reduce rows first, avoids row explosion).
5. **UNION ALL** instead of UNION; **EXISTS** instead of `IN (huge list)`; avoid `DISTINCT` as a bandage for join duplicates.
6. **Avoid row-by-row logic** (cursors, scalar UDF in WHERE); think in sets.
7. **Test on realistic data**; look at the actual plan; keep statistics updated.
8. **Pull less data into BI tools**: summarise in SQL, or create views and aggregate tables.
9. **Limit while developing**: `SELECT TOP 100` or `WHERE` on a date range, then remove it.

## Security: DCL

Data Control Language gives and removes permissions. Follow the **principle of least privilege**: grant only what each person or application needs.

SQL

```sql
-- Logins (server level) and users (database level)
CREATE LOGIN priya_analyst WITH PASSWORD = 'Use-A-Strong-Passphrase-1!';
CREATE USER  priya_analyst FOR LOGIN priya_analyst;

-- Roles group permissions
CREATE ROLE analyst_role;
GRANT SELECT ON SCHEMA::dbo        TO analyst_role;          -- read all tables in dbo
DENY  SELECT ON dbo.customers (phone, email) TO analyst_role;     -- but hide personal columns
GRANT EXECUTE ON dbo.usp_sales_by_region TO analyst_role;

ALTER ROLE analyst_role ADD MEMBER priya_analyst;            -- SQL Server 2012+: ALTER ROLE …
ADD MEMBER

REVOKE SELECT ON dbo.orders FROM analyst_role;                -- take back a permission
```

| Statement | Effect |
|---|---|
| `GRANT` | Gives a permission |
| `REVOKE` | Removes a previously granted (or denied) permission |
| `DENY` | Explicitly blocks a permission (overrides grants) |
| Fixed roles | `db_datareader` (read all), `db_datawriter`, `db_owner` (full control: avoid) |

Other protections: **Row-Level Security** (a region manager sees only their rows), **Dynamic Data Masking** (show `XXXX-1234`), **Always Encrypted** and **TDE** (encryption), **auditing**, and never embedding passwords in scripts (use Windows/Entra ID authentication or a secret store). In India, personal data handling also falls under the **DPDP Act 2023**: collect only what you need and restrict who can see it.

DO IT WITH AI **Do it with AI**

Paste a slow query and its actual execution plan (as XML or text) into an assistant: *"Explain what this plan is doing and suggest indexes or rewrites. Do not invent columns."* AI can spot scans, key lookups and non-SARGable filters quickly, but test every suggestion; an index that helps one query can hurt a hundred writes.

## Summary

KEY TAKEAWAYS

- A **transaction** groups statements to commit or roll back together; **ACID** = atomicity, consistency, isolation, durability. Use `BEGIN TRAN … COMMIT/ROLLBACK`, check `@@ROWCOUNT`, never leave transactions open.
- Isolation levels and locks keep concurrent users consistent; blocking and deadlocks are managed with short transactions and consistent access order.
- **Indexes** (clustered = row order, one per table; non-clustered = separate structure; covering = `INCLUDE`) speed up reads and slow writes. **SARGable** filters keep the column bare.
- Read execution plans for scans, key lookups and bad estimates; use `SET STATISTICS IO, TIME ON`.
- **DCL**: `GRANT`, `REVOKE`, `DENY`, roles, least privilege; protect personal data.

## Quick quiz

6 questions · tap an answer to check it

**1. Which ACID property says that a committed transaction survives a power failure?**

A Atomicity

B Consistency

C Isolation

D Durability

**Answer: D.** Durability: committed changes are written to the log/disk.

**2. Which filter can use an index on order_date?**

A `WHERE YEAR(order_date) = 2025`

B `WHERE order_date ⩾ '2025-01-01' AND order_date < '2026-01-01'`

C `WHERE CONVERT(VARCHAR, order_date, 23) LIKE '2025%'`

D `WHERE DATEPART(year, order_date) = 2025`

**Answer: B.** The column must be bare (SARGable); functions on the column force a scan.

**3.** How many clustered indexes can a table have?

A None

B One

C One per column

D Unlimited

**Answer: B.** The clustered index defines the physical order, so there can be only one.

**4.** You ran `BEGIN TRAN; UPDATE …;` and closed the window. What is the risk?

A The transaction stays open, holding locks and blocking other users

B The update is committed automatically

C The table is dropped

D Nothing

**Answer: A.** Open transactions keep locks until committed or rolled back.

**5.** What does DENY do?

A Removes a permission like REVOKE

B Explicitly blocks a permission, overriding grants

C Deletes a login

D Grants read-only access

**Answer: B.** DENY takes precedence over GRANT.

**6.** In an execution plan, a "Key Lookup" repeated for many rows suggests:

- **A** The table is empty
- **B** Adding the needed columns to the index with INCLUDE (a covering index)
- **C** Using UNION instead of UNION ALL
- **D** Dropping the clustered index

**Answer: B.** Key lookups fetch missing columns from the table row by row; covering the query avoids them.

EXERCISE

### Make it faster

This query is slow on 50 million order lines: `SELECT * FROM order_items WHERE YEAR(order_date) = 2025 AND UPPER(region) = 'WEST' ORDER BY sales DESC;` Rewrite it and suggest an index.

**Show solution (worked answer)**

```sql
SELECT item_id, order_id, product_id, quantity, sales, profit      -- only the needed columns
FROM dbo.order_items
WHERE order_date ⩾ '2025-01-01' AND order_date < '2026-01-01'     -- SARGable range instead of YEAR()
  AND region = 'West'                                              -- default collation is case-insensitive; no UPPER needed
ORDER BY sales DESC;

CREATE NONCLUSTERED INDEX ix_items_date_region
ON dbo.order_items (order_date, region) INCLUDE (order_id, product_id, quantity, sales, profit);
```

Check the actual plan for a seek, compare logical reads with `SET STATISTICS IO ON`, and consider whether the extra index slows nightly loads.

INTERVIEW CORNER

## Questions you may be asked

**Q: What is ACID?** Atomicity (all or nothing), Consistency (valid state), Isolation (concurrent transactions do not interfere), Durability (committed data persists).

**Q: Clustered vs non-clustered index?** A clustered index sorts and stores the table rows in index order (one per table). A non-clustered index is a separate structure with pointers to rows (many per table).

**Q: Why is `WHERE YEAR(date) = 2025` slow?** It applies a function to every row, so the index on date cannot be used (non-SARGable). Use a date range.

**Q: How do you find and fix a slow query?** Look at the actual execution plan and I/O statistics, find scans/lookups/bad estimates, make predicates SARGable, add or adjust indexes, reduce columns/rows early, and re-test.

SQL FOR DATA ANALYSIS · LESSON 15

# End-to-End SQL Case Study & Interview Practice

Put everything together. Work through a complete analysis of the retail business (data quality, KPIs, trends, customer segmentation, retention, product and discount analysis) using joins, CTEs and window functions, then practise 25 classic SQL interview questions with model answers.

SQL class 5 (end-to-end case study) · ✦ 2026 · Advanced · ⏱ 180 min

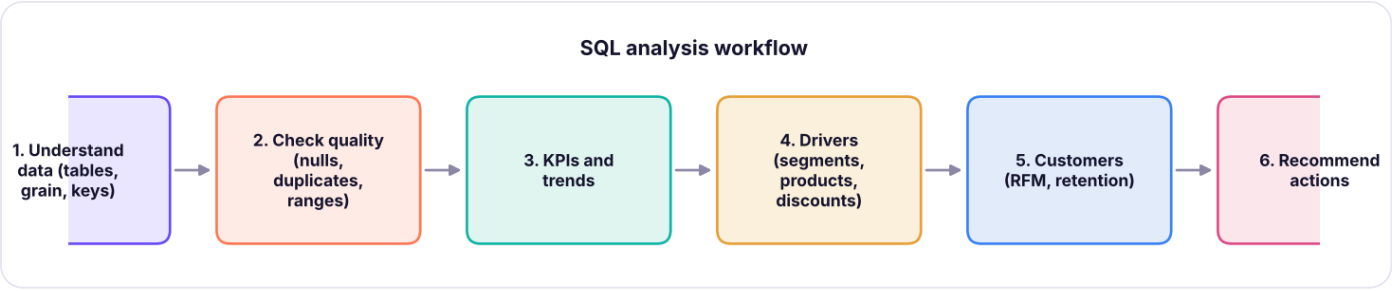
**By the end of this lesson you can**

- Run a structured SQL analysis: understand the data, check quality, calculate KPIs, find drivers, and recommend actions
- Combine joins, CTEs, aggregation and window functions in multi-step queries
- Build customer segments (RFM), cohort and retention views in SQL
- Answer common SQL interview questions with clear, correct queries
- Explain results in business language

## The business problem

CASE STUDY **Retail growth review**

You are the data analyst at an Indian retailer selling Furniture, Office Supplies and Technology through four regions. Sales grew from 2023 to 2025, but the CFO suspects profit is not keeping pace. Management wants answers to five questions: **(1)** Is the data trustworthy? **(2)** How are sales and profit really trending? **(3)** Where are we losing money and why? **(4)** Who are our best customers and are they staying? **(5)** What should we do next quarter?




A repeatable SQL analysis workflow.

## Step 1: understand the data

SQL

```
SELECT 'orders'       AS tbl, COUNT(*) AS n, MIN(order_date) AS first_date, MAX(order_date) AS
last_date FROM orders
UNION ALL SELECT 'order_items', COUNT(*), NULL, NULL FROM order_items
UNION ALL SELECT 'customers',   COUNT(*), NULL, NULL FROM customers
UNION ALL SELECT 'products',    COUNT(*), NULL, NULL FROM products;
```

RESULT

| tbl | n | first_date | last_date |
|---|---|---|---|
| orders | 3200 | 2023-01-01 | 2025-12-31 |
| order_items | 6323 | NULL | NULL |
| customers | 800 | NULL | NULL |
| products | 78 | NULL | NULL |

4 rows × 4 columns

**Grain**: `order_items` has one row per product line in an order; `orders` one row per order; `customers` one row per customer. KPIs like "orders" must use `COUNT(DISTINCT order_id)`, and "average order value" needs order-level totals (Lesson 10).

## Step 2: data-quality checks

Always check before analysing. A compact data-quality report:

SQL

```
SELECT
    (SELECT COUNT(*) FROM order_items WHERE sales IS NULL OR profit IS NULL OR quantity IS NULL)
AS null_measures,
    (SELECT COUNT(*) FROM order_items WHERE sales ⩽ 0 OR quantity ⩽ 0)
AS non_positive,
    (SELECT COUNT(*) FROM order_items WHERE discount < 0 OR discount > 1)
AS bad_discount,
    (SELECT COUNT(*) FROM orders WHERE ship_date < order_date)
AS ship_before_order,
    (SELECT COUNT(*) FROM order_items oi LEFT JOIN orders o ON o.order_id = oi.order_id WHERE
o.order_id IS NULL) AS orphan_items,
    (SELECT COUNT(*) FROM orders o LEFT JOIN customers c ON c.customer_id = o.customer_id WHERE
c.customer_id IS NULL) AS orders_without_customer,
    (SELECT COUNT(*) - COUNT(DISTINCT item_id) FROM order_items)
AS duplicate_item_ids;
```

RESULT

| null_measures | non_positive | bad_discount | ship_before_order | orphan_items | orders_without |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |

1 rows × 7 columns

All zeros: the data is clean, so the numbers can be trusted. In real projects every non-zero here becomes a ticket to fix or a documented exclusion. Also check **business-rule** outliers:

SQL

```
SELECT ROUND(MIN(sales), 0) AS min_sales, ROUND(MAX(sales), 0) AS max_sales,
       ROUND(AVG(sales), 0) AS avg_sales,
       MIN(discount) AS min_discount, MAX(discount) AS max_discount,
       MIN(quantity) AS min_qty, MAX(quantity) AS max_qty
FROM order_items;
```

RESULT

| min_sales | max_sales | avg_sales | min_discount | max_discount | min_qty | max_qty |
|---|---|---|---|---|---|---|
| 42 | 619,290 | 31,854 | 0 | 0.4 | 1 | 7 |

1 rows × 7 columns

## Step 3: KPIs and trends

SQL

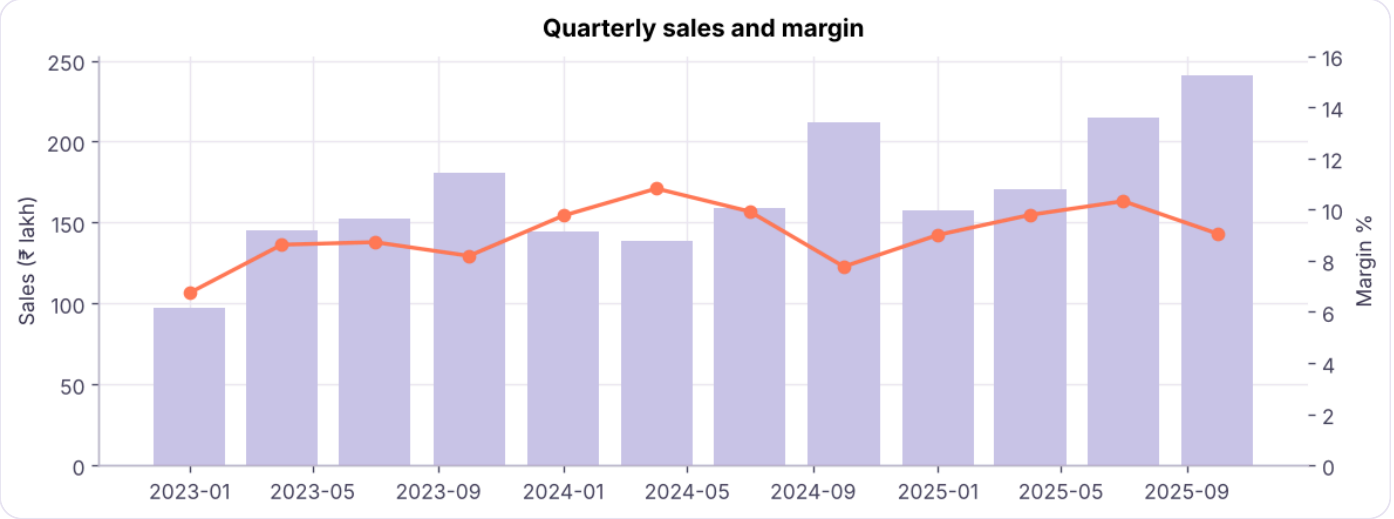
```sql
WITH yearly AS (
    SELECT YEAR(o.order_date)             AS yr,
           COUNT(DISTINCT o.order_id)     AS orders,
           COUNT(DISTINCT o.customer_id)  AS customers,
           SUM(oi.sales)                  AS sales,
           SUM(oi.profit)                 AS profit
    FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
    GROUP BY YEAR(o.order_date)
)
SELECT yr, orders, customers,
       ROUND(sales, 0)                                          AS sales,
       ROUND(100.0 * (sales - LAG(sales) OVER (ORDER BY yr)) / LAG(sales) OVER (ORDER BY yr), 1)
AS sales_growth_pct,
       ROUND(profit, 0)                                         AS profit,
       ROUND(100.0 * (profit - LAG(profit) OVER (ORDER BY yr)) / LAG(profit) OVER (ORDER BY yr),
1) AS profit_growth_pct,
       ROUND(100.0 * profit / sales, 1)                         AS margin_pct,
       ROUND(sales / orders, 0)                                 AS avg_order_value
FROM yearly
ORDER BY yr;
```

RESULT

| yr | orders | customers | sales | sales_growth_pct | profit | profit_growth_pct | m |
|---|---|---|---|---|---|---|---|
| 2023 | 883 | 540 | 57,640,217 | NULL | 4,725,378 | NULL | 8 |
| 2024 | 1085 | 590 | 65,396,556 | 13.5 | 6,135,992 | 29.9 | 9 |
| 2025 | 1232 | 623 | 78,376,797 | 19.8 | 7,490,793 | 22.1 | 9 |

3 rows × 9 columns

**Finding 1.** Sales grew 13.5% in 2024 and 19.8% in 2025, and profit grew even faster (29.9% and 22.1%), so the overall margin edged up from 8.2% to 9.6%. The growth is real, but a 9.6% margin is low compared with the 17–18% earned on lines sold **without** a discount (you will see this in Step 4). The CFO's concern is about the *quality* of profit: let's look at where the money is made and lost.




Quarterly sales and profit margin from the same data. Sales climb steadily and peak each Q4 (festive season); the margin moves in a narrow band below 12%.

## Step 4: where do we lose money?

SQL

```sql
SELECT p.category, p.sub_category,
       COUNT(*)                                          AS lines,
       ROUND(SUM(oi.sales), 0)                           AS sales,
       ROUND(SUM(oi.profit), 0)                          AS profit,
       ROUND(100.0 * SUM(oi.profit) / SUM(oi.sales), 1)  AS margin_pct,
       ROUND(AVG(oi.discount) * 100, 1)                  AS avg_discount_pct
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category, p.sub_category
ORDER BY profit;
```

RESULT

| category | sub_category | lines | sales | profit | margin_pct | avg_discount_pct |
|---|---|---|---|---|---|---|
| Furniture | Tables | 505 | 14,046,224 | -816,042 | -5.8 | 17.5 |
| Furniture | Bookcases | 477 | 11,274,849 | -422,786 | -3.7 | 18.3 |
| Office Supplies | Art | 462 | 245,985 | 36,526 | 14.8 | 7.6 |
| Office Supplies | Binders | 494 | 401,219 | 61,074 | 15.2 | 7.4 |
| Office Supplies | Paper | 559 | 493,232 | 75,040 | 15.2 | 7.2 |
| Furniture | Furnishings | 442 | 1,429,307 | 78,632 | 5.5 | 7.4 |
| Furniture | Chairs | 493 | 8,091,474 | 342,043 | 4.2 | 8.3 |
| Technology | Accessories | 445 | 3,178,930 | 369,554 | 11.6 | 7.5 |
| Office Supplies | Storage | 507 | 2,659,340 | 397,228 | 14.9 | 7.6 |
| Office Supplies | Appliances | 492 | 6,333,529 | 944,927 | 14.9 | 7.5 |
| Technology | Phones | 488 | 31,919,223 | 3,663,117 | 11.5 | 7.1 |
| Technology | Machines | 491 | 42,590,335 | 5,186,711 | 12.2 | 6.7 |

13 rows × 7 columns (first 12 shown)

**Finding 2.** Tables and Bookcases lose money, and they also carry the heaviest average discounts. Is discounting the cause? Compare margin by discount level and by whether the line is Furniture:

SQL

```
SELECT CASE WHEN p.category = 'Furniture' THEN 'Furniture' ELSE 'Other' END AS group_name,
       CASE WHEN oi.discount = 0 THEN '0%' WHEN oi.discount ⩽ 0.2 THEN '1-20%' ELSE '>20%' END
AS discount_band,
       COUNT(*)                                          AS lines,
       ROUND(SUM(oi.profit), 0)                          AS profit,
       ROUND(100.0 * SUM(oi.profit) / SUM(oi.sales), 1)  AS margin_pct
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY CASE WHEN p.category = 'Furniture' THEN 'Furniture' ELSE 'Other' END,
         CASE WHEN oi.discount = 0 THEN '0%' WHEN oi.discount ⩽ 0.2 THEN '1-20%' ELSE '>20%'
END
ORDER BY group_name, discount_band;
```

RESULT

| group_name | discount_band | lines | profit | margin_pct |
|---|---|---|---|---|
| Furniture | 0% | 850 | 1,861,761 | 11.7 |
| Furniture | 1–20% | 509 | -317,393 | -3.9 |
| Furniture | >20% | 558 | -2,362,521 | -21.9 |
| Other | 0% | 2630 | 19,857,203 | 18.4 |
| Other | 1–20% | 1434 | 1,014,534 | 2.0 |
| Other | >20% | 342 | -1,701,419 | -18.9 |

6 rows × 5 columns

**Finding 3.** In every group, discounts above 20% destroy margin. Furniture is hit hardest because it is discounted more often. What would profit be if lines discounted above 20% had been sold at a 20% discount instead? A rough "what-if":

SQL

```sql
WITH x AS (
    SELECT oi.sales, oi.profit, oi.discount,
           oi.sales / (1 - oi.discount)           AS list_value,
           (oi.sales / (1 - oi.discount)) * 0.8   AS sales_at_20,
           oi.sales - oi.profit                   AS cost
    FROM order_items oi WHERE oi.discount > 0.2
)
SELECT COUNT(*)                                   AS lines_over_20pct,
       ROUND(SUM(profit), 0)                      AS actual_profit,
       ROUND(SUM(sales_at_20 - cost), 0)          AS profit_if_capped_at_20pct,
       ROUND(SUM(sales_at_20 - cost) - SUM(profit), 0) AS potential_gain
FROM x;
```

RESULT

| lines_over_20pct | actual_profit | profit_if_capped_at_20pct | potential_gain |
|---|---|---|---|
| 900 | -4,063,940 | -422,474 | 3,641,466 |

1 rows × 4 columns

(This ignores that a lower discount might lose some of those sales. It is an upper bound to size the opportunity, and it must be labelled as such for management.)

## Step 5: customers

### Top customers and concentration

SQL

```sql
WITH c AS (
    SELECT o.customer_id, SUM(oi.sales) AS sales, SUM(oi.profit) AS profit, COUNT(DISTINCT
o.order_id) AS orders
    FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
    GROUP BY o.customer_id
), r AS (
    SELECT *, NTILE(10) OVER (ORDER BY sales DESC) AS decile FROM c
)
SELECT decile,
       COUNT(*)                                        AS customers,
       ROUND(100.0 * SUM(sales) / SUM(SUM(sales)) OVER (), 1) AS pct_of_sales,
       ROUND(100.0 * SUM(profit) / SUM(SUM(profit)) OVER (), 1) AS pct_of_profit
FROM r
GROUP BY decile
ORDER BY decile;
```

RESULT

| decile | customers | pct_of_sales | pct_of_profit |
|---|---|---|---|
| 1 | 79 | 28.8 | 34.1 |
| 2 | 79 | 18.7 | 21.0 |
| 3 | 79 | 14.6 | 14.3 |
| 4 | 79 | 11.3 | 10.4 |
| 5 | 79 | 9.0 | 8.8 |
| 6 | 78 | 6.7 | 5.3 |
| 7 | 78 | 5.2 | 2.5 |
| 8 | 78 | 3.4 | 2.2 |
| 9 | 78 | 1.9 | 1.1 |
| 10 | 78 | 0.5 | 0.3 |

10 rows × 4 columns

## RFM segmentation

**RFM** scores each customer by **R**ecency (days since last order), **F**requency (number of orders) and **M**onetary value (total sales). The reference date is the day after the last order in the data.

SQL

```sql
WITH base AS (
    SELECT o.customer_id,
           DATEDIFF(day, MAX(o.order_date), '2026-01-01')  AS recency_days,
           COUNT(DISTINCT o.order_id)                      AS frequency,
           SUM(oi.sales)                                   AS monetary
    FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
    GROUP BY o.customer_id
), scored AS (
    SELECT *,
           NTILE(4) OVER (ORDER BY recency_days DESC) AS r,        -- 4 = most recent
           NTILE(4) OVER (ORDER BY frequency)         AS f,
           NTILE(4) OVER (ORDER BY monetary)          AS m
    FROM base
)
SELECT customer_id, recency_days, frequency, ROUND(monetary, 0) AS monetary, r, f, m,
       CASE WHEN r ⩾ 3 AND f ⩾ 3 AND m ⩾ 3 THEN 'Champions'
            WHEN r ⩾ 3 AND f ⩾ 2             THEN 'Loyal'
            WHEN r ⩽ 2 AND f ⩾ 3 AND m ⩾ 3   THEN 'At risk (high value)'
            WHEN r = 1                       THEN 'Lost / hibernating'
            ELSE 'Potential' END AS segment
FROM scored
ORDER BY monetary DESC;
```

RESULT

| customer_id | recency_days | frequency | monetary | r | f | m | segment |
|---|---|---|---|---|---|---|---|
| C0741 | 441 | 6 | 1,204,439 | 1 | 4 | 4 | At risk (high value) |
| C0092 | 11 | 8 | 1,140,540 | 4 | 4 | 4 | Champions |
| C0306 | 278 | 7 | 1,126,867 | 2 | 4 | 4 | At risk (high value) |
| C0781 | 130 | 5 | 1,123,527 | 2 | 3 | 4 | At risk (high value) |
| C0443 | 92 | 7 | 1,113,058 | 3 | 4 | 4 | Champions |
| C0720 | 12 | 6 | 1,100,213 | 4 | 4 | 4 | Champions |
| C0493 | 514 | 6 | 1,004,382 | 1 | 4 | 4 | At risk (high value) |
| C0028 | 266 | 5 | 1,000,189 | 2 | 3 | 4 | At risk (high value) |
| C0630 | 331 | 5 | 1,000,175 | 1 | 3 | 4 | At risk (high value) |
| C0262 | 70 | 6 | 952,738 | 3 | 4 | 4 | Champions |

785 rows × 8 columns (first 10 shown)

SQL

```sql
WITH base AS (
    SELECT o.customer_id, DATEDIFF(day, MAX(o.order_date), '2026-01-01') AS recency_days,
           COUNT(DISTINCT o.order_id) AS frequency, SUM(oi.sales) AS monetary
    FROM orders o JOIN order_items oi ON oi.order_id = o.order_id GROUP BY o.customer_id
), scored AS (
    SELECT *, NTILE(4) OVER (ORDER BY recency_days DESC) AS r, NTILE(4) OVER (ORDER BY
frequency) AS f, NTILE(4) OVER (ORDER BY monetary) AS m FROM base
), seg AS (
    SELECT *, CASE WHEN r ⩾ 3 AND f ⩾ 3 AND m ⩾ 3 THEN 'Champions' WHEN r ⩾ 3 AND f ⩾ 2
THEN 'Loyal'
                   WHEN r ⩽ 2 AND f ⩾ 3 AND m ⩾ 3 THEN 'At risk (high value)' WHEN r = 1 THEN
'Lost / hibernating' ELSE 'Potential' END AS segment FROM scored
)
SELECT segment, COUNT(*) AS customers, ROUND(SUM(monetary), 0) AS sales,
       ROUND(100.0 * SUM(monetary) / SUM(SUM(monetary)) OVER (), 1) AS pct_of_sales
FROM seg GROUP BY segment ORDER BY sales DESC;
```

RESULT

| segment | customers | sales | pct_of_sales |
|---|---|---|---|
| Champions | 178 | 80,496,072 | 40.0 |
| At risk (high value) | 92 | 42,616,303 | 21.2 |
| Potential | 198 | 29,644,898 | 14.7 |
| Lost / hibernating | 169 | 24,817,833 | 12.3 |
| Loyal | 148 | 23,838,464 | 11.8 |

5 rows × 4 columns

The "At risk (high value)" group is the first win: high-spending customers who have gone quiet deserve a personal outreach before they churn. The same RFM idea is implemented with pandas in the ML track.

## Retention and cohorts

A **cohort** groups customers by the year of their first order. **Retention** shows how many come back in later years.

SQL

```sql
WITH first_order AS (
    SELECT customer_id, YEAR(MIN(order_date)) AS cohort_year FROM orders GROUP BY customer_id
), activity AS (
    SELECT DISTINCT customer_id, YEAR(order_date) AS active_year FROM orders
)
SELECT f.cohort_year,
       COUNT(DISTINCT f.customer_id)                                              AS cohort_size,
       COUNT(DISTINCT CASE WHEN a.active_year = f.cohort_year     THEN a.customer_id END) AS y0,
       COUNT(DISTINCT CASE WHEN a.active_year = f.cohort_year + 1 THEN a.customer_id END) AS y1,
       COUNT(DISTINCT CASE WHEN a.active_year = f.cohort_year + 2 THEN a.customer_id END) AS y2
FROM first_order f
JOIN activity a ON a.customer_id = f.customer_id
GROUP BY f.cohort_year
ORDER BY f.cohort_year;
```

RESULT

| cohort_year | cohort_size | y0 | y1 | y2 |
|---|---|---|---|---|
| 2023 | 540 | 540 | 397 | 415 |
| 2024 | 193 | 193 | 156 | 0 |
| 2025 | 52 | 52 | 0 | 0 |

3 rows × 5 columns

Divide y1 by `cohort_size` to get year-1 retention. About 74% of the 2023 cohort (397 of 540) ordered again in 2024, and some who skipped a year returned in 2025: this is a business with sticky customers, so reducing discounts is likely safer than the CFO fears.

## Step 6: recommendations

KEY TAKEAWAYS **Findings and actions**

1. **Sales and profit are growing, but the margin is only 9.6%**, far below the 17–18% earned on undiscounted lines.
2. **Technology produces nearly all the profit**; **Furniture is net loss-making** because Tables and Bookcases are sold with heavy discounts.
3. **Discounts above 20% lose money in every category.** Cap discounts at 20% (the what-if sizes the upper-bound upside at about ₹36 lakh over three years; the real gain will be lower if some customers buy less), test in two regions with an A/B design.
4. **Customer base is sticky** (strong year-on-year retention), so price discipline is lower-risk than assumed.
5. **Re-engage "At risk (high value)" customers** with a targeted offer instead of blanket discounts.
6. Build a **monthly SQL view/dashboard** with these KPIs, with data-quality checks as a first step.

## SQL interview practice: 25 questions with answers

Run these on the retail database. Each answer is a runnable query.

INTERVIEW CORNER **Q1. Second-highest sales line.**

SQL

```sql
SELECT MAX(sales) AS second_highest FROM order_items WHERE sales < (SELECT MAX(sales) FROM order_items);
```

RESULT

| second_highest |
| --- |
| 551,530 |

1 rows × 1 columns

INTERVIEW CORNER

## Q2. Customers with at least 8 orders.

SQL

```
SELECT customer_id, COUNT(*) AS orders FROM orders GROUP BY customer_id HAVING COUNT(*) ≥ 8 ORDER BY orders DESC;
```

RESULT

| customer_id | orders |
|---|---|
| C0219 | 10 |
| C0396 | 10 |
| C0291 | 9 |
| C0664 | 9 |
| C0110 | 9 |
| C0384 | 9 |
| C0596 | 9 |
| C0419 | 9 |
| C0271 | 9 |
| C0527 | 9 |
| C0719 | 9 |
| C0069 | 9 |

41 rows × 2 columns (first 12 shown)

INTERVIEW CORNER

## Q3. Duplicate customer names (possible duplicate records).

SQL

```sql
SELECT customer_name, COUNT(*) AS n FROM customers GROUP BY customer_name HAVING COUNT(*)
> 1 ORDER BY n DESC;
```

RESULT

| customer_name | n |
|---|---|
| Sneha Joshi | 6 |
| Ananya Gupta | 5 |
| Rahul Shah | 5 |
| Asha Rao | 5 |
| Ananya Iyer | 5 |
| Saanvi Khan | 5 |
| Rohan Gupta | 5 |
| Reyansh Shah | 4 |
| Simran Khan | 4 |
| Vikram Kulkarni | 4 |
| Arjun Sharma | 4 |
| Nikhil Das | 4 |

238 rows × 2 columns (first 12 shown)

INTERVIEW CORNER

## Q4. Highest-selling product in each category.

SQL

```sql
WITH t AS (SELECT p.category, p.product_name, SUM(oi.sales) AS sales FROM order_items oi
JOIN products p ON p.product_id = oi.product_id GROUP BY p.category, p.product_name),
     r AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY category ORDER BY sales DESC) AS rn
FROM t)
SELECT category, product_name, ROUND(sales, 0) AS sales FROM r WHERE rn = 1;
```

RESULT

| category | product_name | sales |
|---|---|---|
| Furniture | Crest Table 572 | 3,122,680 |
| Technology | Orbit Copier 746 | 16,026,341 |
| Office Supplies | Crest Appliance 402 | 1,909,628 |

3 rows × 3 columns

INTERVIEW CORNER

## Q5. Month-over-month growth for 2025.

SQL

```sql
WITH m AS (SELECT MONTH(order_date) AS mth, SUM(sales) AS sales FROM superstore WHERE
YEAR(order_date) = 2025 GROUP BY MONTH(order_date))
SELECT mth, ROUND(sales, 0) AS sales, ROUND(100.0 * (sales - LAG(sales) OVER (ORDER BY
mth)) / LAG(sales) OVER (ORDER BY mth), 1) AS mom_pct FROM m ORDER BY mth;
```

RESULT

| mth | sales | mom_pct |
|---|---|---|
| 1 | 5,357,761 | NULL |
| 2 | 3,879,338 | -27.6 |
| 3 | 6,512,645 | 67.9 |
| 4 | 6,699,805 | 2.9 |
| 5 | 6,421,274 | -4.2 |
| 6 | 3,968,572 | -38.2 |
| 7 | 7,190,087 | 81.2 |
| 8 | 5,802,184 | -19.3 |
| 9 | 8,468,060 | 45.9 |
| 10 | 6,140,290 | -27.5 |
| 11 | 10,284,516 | 67.5 |
| 12 | 7,652,267 | -25.6 |

12 rows × 3 columns

INTERVIEW CORNER

## Q6. Running total of sales by month in 2025.

SQL

```sql
WITH m AS (SELECT MONTH(order_date) AS mth, SUM(sales) AS sales FROM superstore WHERE
YEAR(order_date) = 2025 GROUP BY MONTH(order_date))
SELECT mth, ROUND(sales, 0) AS sales, ROUND(SUM(sales) OVER (ORDER BY mth ROWS BETWEEN
UNBOUNDED PRECEDING AND CURRENT ROW), 0) AS running_total FROM m ORDER BY mth;
```

RESULT

| mth | sales | running_total |
|---|---|---|
| 1 | 5,357,761 | 5,357,761 |
| 2 | 3,879,338 | 9,237,099 |
| 3 | 6,512,645 | 15,749,743 |
| 4 | 6,699,805 | 22,449,548 |
| 5 | 6,421,274 | 28,870,822 |
| 6 | 3,968,572 | 32,839,394 |
| 7 | 7,190,087 | 40,029,481 |
| 8 | 5,802,184 | 45,831,665 |
| 9 | 8,468,060 | 54,299,725 |
| 10 | 6,140,290 | 60,440,015 |
| 11 | 10,284,516 | 70,724,530 |
| 12 | 7,652,267 | 78,376,797 |

12 rows × 3 columns

INTERVIEW CORNER

## Q7. Customers who ordered in every year (2023, 2024 and 2025).

SQL

```sql
SELECT COUNT(*) AS customers_all_three_years
FROM (SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(DISTINCT
YEAR(order_date)) = 3) AS t;
```

RESULT

| customers_all_three_years |
| --- |
| 311 |

1 rows × 1 columns

INTERVIEW CORNER

## Q8. Employees earning more than their department's average (HR data).

SQL

```sql
SELECT COUNT(*) AS above_dept_avg
FROM (SELECT emp_id, monthly_income, AVG(monthly_income) OVER (PARTITION BY department) AS
avg_income FROM employees) AS t
WHERE monthly_income > avg_income;
```

RESULT

| above_dept_avg |
| --- |
| 638 |

1 rows × 1 columns

INTERVIEW CORNER

## Q9. Delete duplicates keeping the lowest id (pattern).

SQL

```sql
WITH d AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_name, city ORDER BY
customer_id) AS rn FROM dbo.customers)
DELETE FROM d WHERE rn > 1;
```

INTERVIEW CORNER

### Q10. Customers with no orders in the last 6 months of 2025 but orders before that (churn candidates).

SQL

```sql
SELECT COUNT(*) AS churn_candidates
FROM (SELECT customer_id FROM orders WHERE order_date < '2025-07-01' GROUP BY customer_id
      EXCEPT
      SELECT customer_id FROM orders WHERE order_date ⩾ '2025-07-01') AS t;
```

RESULT

| churn_candidates |
| --- |
| 310 |

1 rows × 1 columns

INTERVIEW CORNER

### Q11. Percentage of orders shipped late (more than 5 days).

SQL

```sql
SELECT ROUND(100.0 * SUM(CASE WHEN DATEDIFF(day, order_date, ship_date) > 5 THEN 1 ELSE 0
END) / COUNT(*), 1) AS pct_late FROM orders;
```

RESULT

| pct_late |
| --- |
| 31.3 |

1 rows × 1 columns

INTERVIEW CORNER

## Q12. Median order-line sales.

SQL

```sql
SELECT DISTINCT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY sales) OVER () AS median_sales
FROM order_items;
```

RESULT

| median_sales |
| --- |
| 8,430 |

1 rows × 1 columns

INTERVIEW CORNER

## Q13. Categories where more than 30% of lines are loss-making.

SQL

```sql
SELECT p.category, ROUND(100.0 * AVG(CASE WHEN oi.profit < 0 THEN 1.0 ELSE 0 END), 1) AS
pct_loss_lines
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category HAVING AVG(CASE WHEN oi.profit < 0 THEN 1.0 ELSE 0 END) > 0.3;
```

RESULT

| category | pct_loss_lines |
| --- | --- |
| Furniture | 48.9 |

1 rows × 2 columns

INTERVIEW CORNER

### Q14. Cumulative percentage of sales by customer (Pareto): how many customers make 50% of sales?

SQL

```
WITH c AS (SELECT customer_id, SUM(sales) AS s FROM order_items oi JOIN orders o ON
o.order_id = oi.order_id GROUP BY customer_id),
     p AS (SELECT s, SUM(s) OVER (ORDER BY s DESC ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) * 1.0 / SUM(s) OVER () AS cum FROM c)
SELECT COUNT(*) AS customers_for_half_of_sales, (SELECT COUNT(*) FROM c) AS
total_customers FROM p WHERE cum ⩽ 0.5;
```

RESULT

| customers_for_half_of_sales | total_customers |
|---|---|
| 170 | 785 |

1 rows × 2 columns

INTERVIEW CORNER

### Q15. Swap two values in a column (pattern).

SQL

```
UPDATE dbo.staff SET gender = CASE gender WHEN 'M' THEN 'F' WHEN 'F' THEN 'M' ELSE gender
END;
```

INTERVIEW CORNER

### Q16. Find consecutive order days (streaks) for a customer: see Lesson 12 (gaps and islands).

INTERVIEW CORNER

## Q17. Average order value per segment.

SQL

```sql
SELECT segment, ROUND(AVG(t.total), 0) AS aov
FROM (SELECT c.segment, o.order_id, SUM(oi.sales) AS total FROM orders o JOIN customers c
ON c.customer_id = o.customer_id JOIN order_items oi ON oi.order_id = o.order_id GROUP BY
c.segment, o.order_id) AS t
GROUP BY segment;
```

RESULT

| segment | aov |
|---|---|
| Consumer | 63,451 |
| Corporate | 61,899 |
| Home Office | 63,237 |

3 rows × 2 columns

INTERVIEW CORNER

## Q18. Products priced above the average of their sub-category.

SQL

```sql
SELECT COUNT(*) AS products_above_subcat_avg
FROM (SELECT product_id, list_price, AVG(list_price) OVER (PARTITION BY sub_category) AS
avg_price FROM products) AS t WHERE list_price > avg_price;
```

RESULT

| products_above_subcat_avg |
|---|
| 36 |

1 rows × 1 columns

INTERVIEW CORNER

## Q19. Rows in A not in B without NOT IN trap: use NOT EXISTS (Lesson 10).

INTERVIEW CORNER

## Q20. Pivot category sales into columns by year.

SQL

```sql
SELECT category,
       ROUND(SUM(CASE WHEN YEAR(order_date) = 2023 THEN sales ELSE 0 END), 0) AS y2023,
       ROUND(SUM(CASE WHEN YEAR(order_date) = 2024 THEN sales ELSE 0 END), 0) AS y2024,
       ROUND(SUM(CASE WHEN YEAR(order_date) = 2025 THEN sales ELSE 0 END), 0) AS y2025
FROM superstore GROUP BY category ORDER BY category;
```

RESULT

| category | y2023 | y2024 | y2025 |
|---|---|---|---|
| Furniture | 9,952,975 | 12,128,633 | 12,760,246 |
| Office Supplies | 2,611,874 | 3,550,109 | 3,971,321 |
| Technology | 45,075,369 | 49,717,814 | 61,645,230 |

3 rows × 4 columns

INTERVIEW CORNER

## Q21. First and last order date and the number of days between for each customer (top 3 longest tenures).

SQL

```sql
SELECT TOP 3 customer_id, MIN(order_date) AS first_order, MAX(order_date) AS last_order,
DATEDIFF(day, MIN(order_date), MAX(order_date)) AS days_active
FROM orders GROUP BY customer_id ORDER BY days_active DESC;
```

RESULT

| customer_id | first_order | last_order | days_active |
|---|---|---|---|
| C0079 | 2023-01-08 | 2025-12-27 | 1084 |
| C0192 | 2023-01-24 | 2025-12-31 | 1072 |
| C0070 | 2023-01-09 | 2025-12-13 | 1069 |

3 rows × 4 columns

INTERVIEW CORNER

### Q22. Which month has the highest sales in each year?

SQL

```sql
WITH m AS (SELECT YEAR(order_date) AS yr, MONTH(order_date) AS mth, SUM(sales) AS sales
FROM superstore GROUP BY YEAR(order_date), MONTH(order_date)),
     r AS (SELECT *, RANK() OVER (PARTITION BY yr ORDER BY sales DESC) AS rnk FROM m)
SELECT yr, mth, ROUND(sales, 0) AS sales FROM r WHERE rnk = 1 ORDER BY yr;
```

RESULT

| yr | mth | sales |
|---|---|---|
| 2023 | 12 | 7,152,348 |
| 2024 | 11 | 7,284,664 |
| 2025 | 11 | 10,284,516 |

3 rows × 3 columns

INTERVIEW CORNER

### Q23. How do you find the number of rows in each table of a database?

SQL

```sql
SELECT t.name AS table_name, SUM(p.rows) AS row_count
FROM sys.tables t JOIN sys.partitions p ON p.object_id = t.object_id AND p.index_id IN (0,
1)
GROUP BY t.name ORDER BY row_count DESC;
```

INTERVIEW CORNER

### Q24. Explain the difference between RANK() and ROW_NUMBER() and when each is right: Lesson 11.

INTERVIEW CORNER

### Q25. How would you explain the result of a query to a non-technical manager?

State the answer first, in one sentence with numbers ("Furniture loses ₹8.2 lakh a year, almost entirely on heavily discounted tables and bookcases"), then give the 1–2 facts that support it, the assumption or limitation, and the action you recommend.

## Summary

KEY TAKEAWAYS

- A good analysis follows a workflow: understand the grain, check data quality, calculate KPIs, find drivers, study customers, recommend actions.
- Always check data quality first; label what-if estimates as estimates.
- Combine CTEs, joins and window functions to build RFM segments, concentration (Pareto), cohorts and retention.
- In interviews, state your assumptions, clarify the grain, write clean, formatted SQL, and explain the result in business language.

## Quick quiz

6 questions · tap an answer to check it

**1. Why must "number of orders" use COUNT(DISTINCT order_id) when joining orders to order_items?**

A DISTINCT is faster

B Each order appears once per item, so COUNT(*) would count lines

C order_id is a text column

D It avoids NULLs

**Answer: B.** The join repeats the order for each of its items.

**2. In RFM, a customer with recency 400 days, frequency 1 and low monetary value is probably:**

A A champion

B Lost or hibernating

C A loyal customer

D A new high-value customer

**Answer: B.** Long time since the last order, few orders and low spend.

**3.** A cohort table shows 2023 cohort size 500, year-1 active 350. Year-1 retention is:

A 35%

B 70%

C 50%

D 150%

**Answer: B.** 350 ÷ 500 = 70%.

**4.** Which should you do before reporting KPIs?

A Build a dashboard

B Check data quality: nulls, duplicates, orphans, ranges

C Email the CFO

D Add indexes

**Answer: B.** Untrusted data produces untrusted numbers.

**5.** The what-if "profit if discounts were capped at 20%" is best described as:

A A forecast

B A guarantee

C An upper-bound estimate that ignores possible lost sales

D A data error

**Answer: C.** It assumes demand would not change, so it overstates the benefit.

**6.** NTILE(10) over customers ordered by sales gives:

A Ten customers

B Ten equal-sized groups (deciles) of customers

C The top 10% only

D The median

**Answer: B.** NTILE splits rows into n buckets.

EXERCISE **Capstone: your own analysis**

Using only SQL on the retail database, prepare a one-page analysis answering: (1) Which three cities have the highest profit and which three the lowest? (2) What is the average days to ship by ship mode, and does a slower mode save money (profit per line)? (3) Which product's sales fell the most from 2024 to 2025? (4) Propose two actions with the numbers that support them.

**Show solution (worked answer)**

Hints: (1) `GROUP BY city` with `ORDER BY profit` ascending and descending using TOP 3 in two queries or `RANK()` in a CTE. (2) Join orders to order_items, `AVG(DATEDIFF(day, order_date, ship_date))` and `SUM(profit)/COUNT(*)` by ship_mode. (3) Pivot product sales by year with conditional aggregation and compute the difference. (4) Example actions: cap Furniture discounts at 20% in two pilot regions; re-engage at-risk high-value customers. Present each action with a baseline number, an expected impact (labelled as an estimate) and a way to measure success.

TRACK 3

# Power BI

ETL with Power Query, star-schema modelling, DAX measures and time intelligence, interactive reports, case studies, publishing, security and Copilot.

1. Getting Started with Power BI & Your First HR Report
2. Power Query, Star Schema & Data Modelling (Adventure Cycles)
3. DAX Fundamentals: Measures, Filter Context & CALCULATE
4. DAX Time Intelligence, Ranking, Parameters & Dynamic Measures
5. Visuals, Filters, Drill-through, Tooltips, Buttons & Report Design
6. Power BI Service, Connectivity Modes, Security & Capstone

POWER BI · LESSON 1

# Getting Started with Power BI & Your First HR Report

Understand what Power BI is and how its parts fit together, install Power BI Desktop, tour the three views, follow the ETL and analytics stages, and build a complete first report on HR data: cards, bar and column charts, a donut, slicers, a year column with DAX, groups and small multiples.

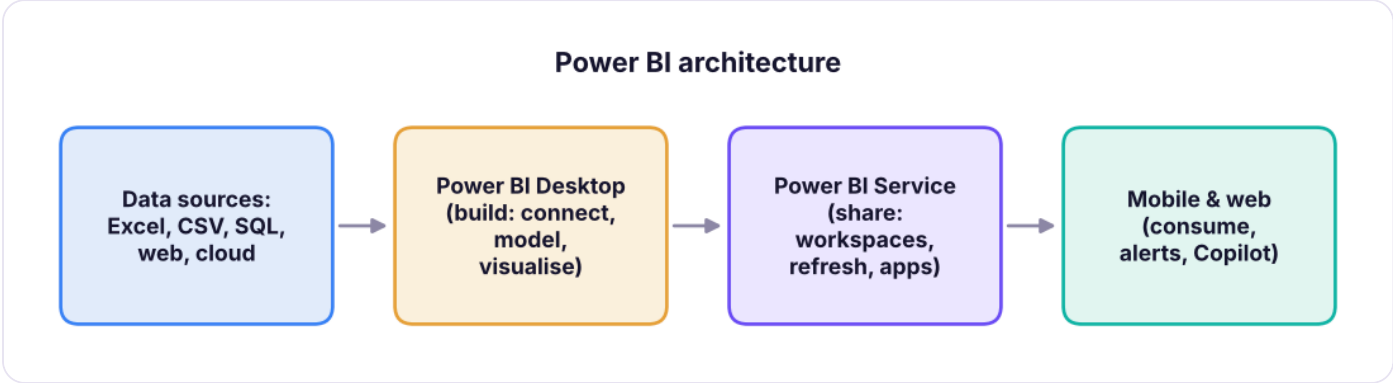
Power BI class 1 | ✦ Power BI Desktop 2026 (Copilot, Fabric, PBIP) | Beginner | ⏱ 120 min

**By the end of this lesson you can**

- Describe Power BI Desktop, the Service and Mobile, and how data flows between them
- Explain ETL and the stages of the data-analytics process, and map them to Power BI tools
- Navigate Report, Table and Model views, and the Fields, Visualizations and Filters panes
- Load a CSV, check data types and build cards, bar/column charts, a matrix, a donut and slicers
- Add a calculated column with DAX (YEAR), create data groups, and use small multiples

## What is Power BI?

**Power BI** is Microsoft's business-intelligence platform: it connects to data, cleans and models it, and turns it into interactive reports and dashboards that people can share. In 2026 it is part of **Microsoft Fabric**, Microsoft's all-in-one analytics platform, and it is one of the most requested tools in analyst job posts, alongside Excel, SQL and Tableau.




The Power BI components. You build in Desktop, publish to the Service, and consume on the web or mobile.

| Component | What it is | Who uses it |
|---|---|---|
| **Power BI Desktop** | Free Windows application to build data models and reports ( `.pbix` ) | Analysts, developers |
| **Power BI Service** (app.powerbi.com) | Cloud site to publish, share, schedule refresh, create dashboards and apps | Analysts and report consumers |
| **Power BI Mobile** | iOS/Android apps to view reports and get alerts | Managers on the go |
| **Power BI Report Builder** | Paginated, print-ready reports (invoices, statements) | Report developers |
| **On-premises data gateway** | Secure bridge so the cloud can refresh data from your own servers | Admins |
| **Power BI Report Server** | Self-hosted option for organisations that cannot use the cloud | IT |
| **Copilot** | AI assistant that writes DAX, builds report pages and summarises insights (needs a Fabric/Premium capacity) | Everyone |

> **TIP** **Report vs dashboard**
>
> In Power BI a **report** is one or more pages of interactive visuals built on a data model. A **dashboard** (Service only) is a single page of **pinned tiles** from different reports. Many teams use reports only, or share them as an **app**.

## Licences in brief

| Licence | Use |
|---|---|
| Power BI Desktop | Free: build and save locally |
| Free (Service) | Personal use in "My workspace"; cannot share with others |
| **Pro** | Publish and share with other Pro users (about US$14 per user per month) |
| **Premium Per User / Fabric capacity** | Larger models, AI features, share with free users |

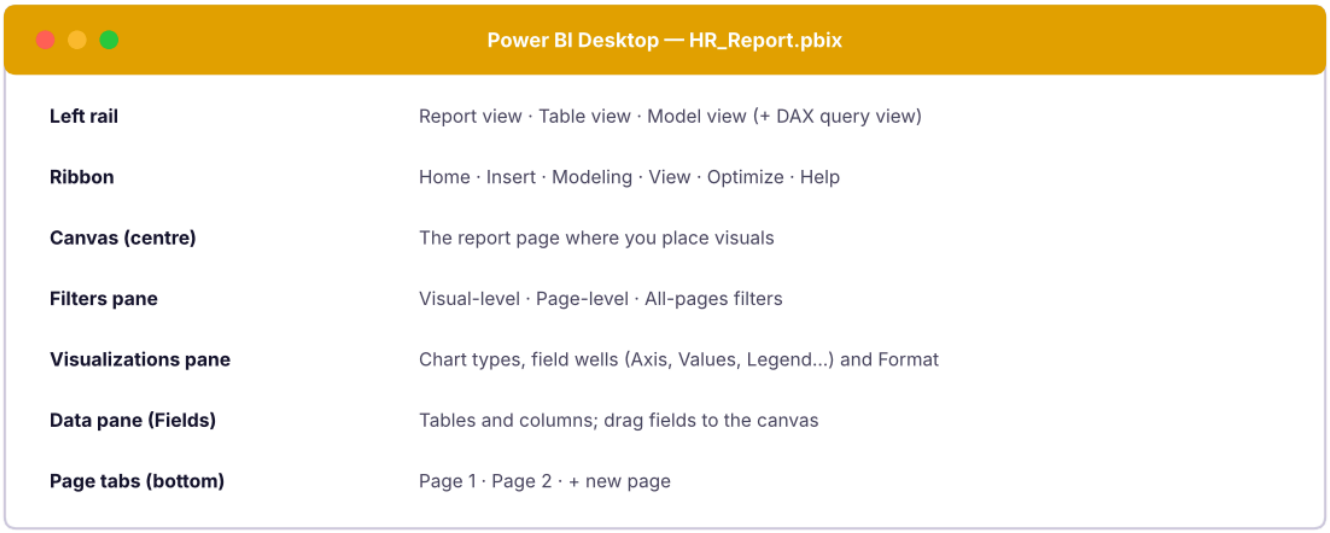
(Prices change; check Microsoft's current pricing page.)

# Installing Power BI Desktop

STEP BY STEP **Install**

1. **Easiest:** open the **Microsoft Store**, search "Power BI Desktop" and click Install (it updates itself every month).
2. Or download it from the Microsoft Power BI website → Download → Power BI Desktop.
3. Requirements: Windows 10/11 (64-bit), 4 GB+ RAM (8 GB+ recommended). **There is no Mac version**: Mac users can use the Service in a browser, a Windows virtual machine, or Windows 365.
4. Start the app and sign in with a work or school account (optional for building locally).

## The three views

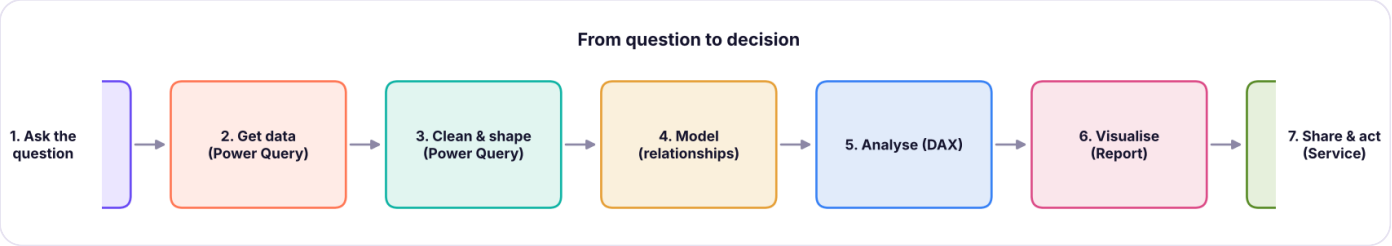



The Power BI Desktop window: left rail with three views, canvas in the middle, panes on the right.

| View | Icon | Use it to |
|---|---|---|
| **Report** | bar chart | Design the report pages |
| **Table** (Data) | grid | Inspect the data and create calculated columns and measures |
| **Model** | diagram | See and edit **relationships** between tables |
| **DAX query view** | `{ }` | Run DAX queries and test measures |

## ETL and the analytics process

**ETL** = **E**xtract (get data from sources), **T**ransform (clean and reshape it), **L**oad (store it in the model). In Power BI, Extract and Transform are done in **Power Query**, and Load puts the result into the data model. (You met Power Query in Excel Lesson 12: it is the same engine.)




The data analytics stages, mapped to Power BI.

Always start with the **question**: "Why are people leaving?" decides which data and visuals you need. The worst Power BI reports are tours of every chart type with no question.

## Your first report: HR attrition

CASE STUDY **The brief**

HR wants a one-page report on the company's 1,500 employees: how many there are, how many left (attrition), where attrition is highest, and whether overtime, job satisfaction or income play a role. Use `hr_employees.csv`.

The data (first rows):

Python

```python
import pandas as pd
hr = pd.read_csv("hr_employees.csv", parse_dates=["hire_date"])
print(hr.shape)
hr.head(5)
```

OUTPUT

```
(1500, 15)
```

| | emp_id | gender | age | department | job_title | education | hire_date | years_at_compa |
|---|---|---|---|---|---|---|---|---|
| **0** | E00001 | Male | 39 | Operations | Operations Manager | Bachelor's | 2025-05-26 | 0 |
| **1** | E00002 | Female | 45 | IT | Tester | Bachelor's | 2020-10-17 | 5 |
| **2** | E00003 | Male | 41 | IT | Tester | Bachelor's | 2019-05-06 | 6 |
| **3** | E00004 | Male | 21 | R&D | Data Scientist | Bachelor's | 2025-08-23 | 0 |
| **4** | E00005 | Male | 41 | Sales | Sales Executive | Master's | 2015-08-24 | 10 |

5 rows × 15 columns

## Step 1: get the data

**STEP BY STEP** **Load the CSV**

1. Home → **Get data → Text/CSV** → choose `hr_employees.csv`.
2. The preview shows the columns and detected data types. Click **Transform Data** (not Load) to check types in Power Query.
3. Confirm: `age`, `years_at_company`, `monthly_income`, `job_satisfaction`, `performance_rating` are whole numbers; `hire_date` is **Date**; the rest are text.
4. Home → **Close & Apply**.

In the Data pane you now see the table `hr_employees` with a **Σ** next to numeric fields (they can be summed) and no symbol next to text fields.

## Step 2: KPI cards

Drag fields onto the canvas, then choose the **Card** visual:

- **Employees** = Count of `emp_id`
- **Attrition (leavers)**: use a measure (Lesson 3 explains measures): `Leavers = CALCULATE ( COUNTROWS ( hr_employees ), hr_employees[attrition] = "Yes" )`
- **Attrition %** = `DIVIDE ( [Leavers], COUNTROWS ( hr_employees ) )`, formatted as a percentage
- **Average monthly income** = Average of `monthly_income`

Python

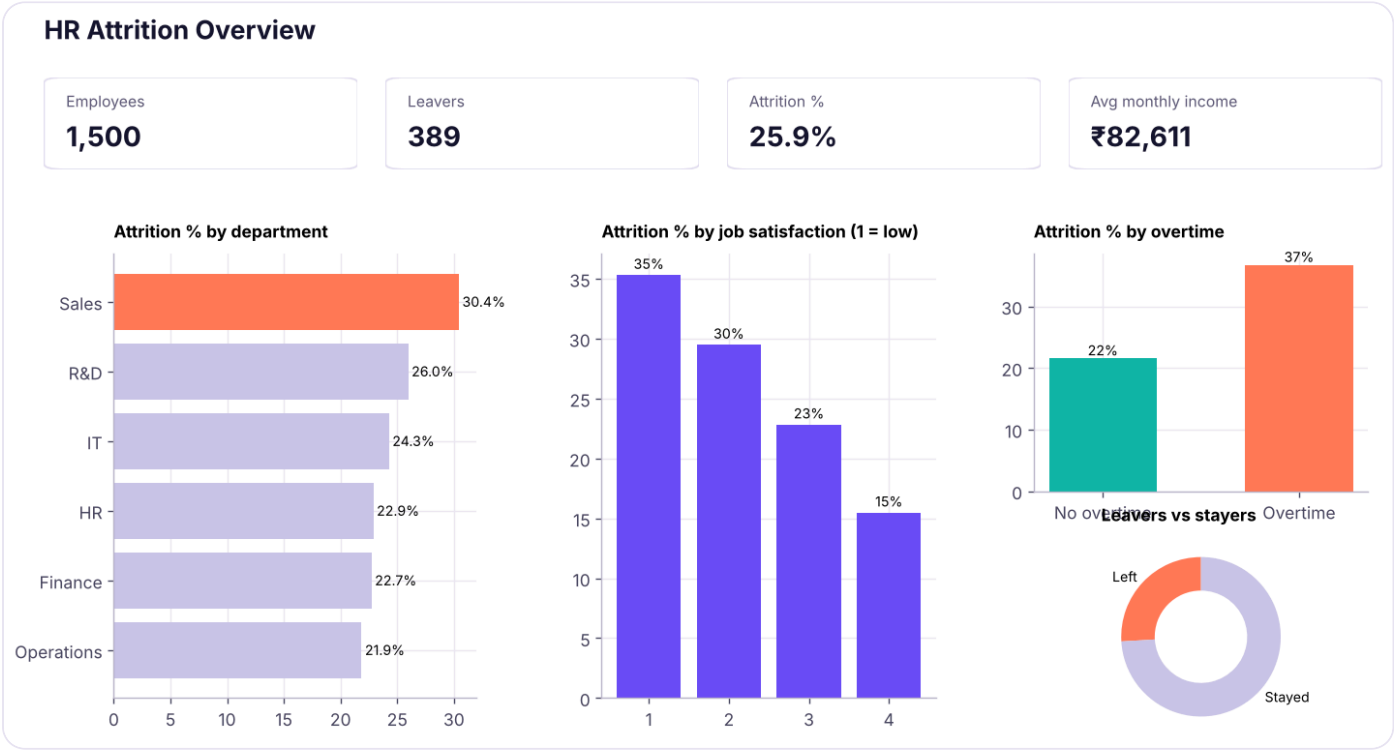
```python
n = len(hr); leavers = (hr.attrition == "Yes").sum()
print(f"Employees      : {n:,}")
print(f"Leavers        : {leavers:,}")
print(f"Attrition %    : {leavers / n:.1%}")
print(f"Avg income (₹) : {hr.monthly_income.mean():,.0f}")
```

OUTPUT

```
Employees      : 1,500
Leavers        : 389
Attrition %    : 25.9%
Avg income (₹) : 82,611
```

## Step 3: bar, column and donut charts

- **Clustered bar chart**: Y-axis `department`, X-axis = `Attrition %` → ranks departments.
- **Clustered column chart**: X-axis `job_satisfaction`, Y-axis `Attrition %`.
- **Donut chart**: Legend `attrition`, Values = count of `emp_id`.
- **Matrix**: Rows `department`, Columns `overtime`, Values `Attrition %` (turns into a heat map with conditional formatting, Lesson 5).
- **Slicers**: `gender`, `education`, `city`.




The finished HR page, drawn from the real data: KPI cards, attrition by department, by satisfaction and overtime, and the leaver/stayer donut.

## Step 4: a calculated column with DAX

We want to see joining trends by year, but the table has only `hire_date`. Add a **calculated column**: Table view (or Modeling) → **New column**:

DAX

```
Hire Year = YEAR ( hr_employees[hire_date] )
```

A calculated column is evaluated **row by row when the data loads** and stored in the model, just like a new column in Excel. Now a line/column chart of **headcount by hire year** works:

Python

```
hr["hire_year"] = hr.hire_date.dt.year
by_year = hr.groupby("hire_year").agg(hired=("emp_id", "count"), left=("left", "sum"))
by_year["attrition_%"] = (by_year.left / by_year.hired * 100).round(1)
by_year.tail(8)
```

OUTPUT

| | hired | left | attrition_% |
|---|---|---|---|
| **hire_year** | | | |
| **2019** | 82 | 16 | 19.5 |
| **2020** | 85 | 20 | 23.5 |
| **2021** | 120 | 22 | 18.3 |
| **2022** | 136 | 35 | 25.7 |
| **2023** | 196 | 60 | 30.6 |
| **2024** | 234 | 71 | 30.3 |
| **2025** | 284 | 93 | 32.7 |
| **2026** | 64 | 17 | 26.6 |

8 rows × 3 columns

## Step 5: data groups and bins

Groups combine values into custom categories, such as the "age band" below, without changing the source data:

- **Bins** (numbers): right-click the field → **New group** → Group type *Bin* → bin size 5 → creates `age (bins)`.

- **List groups** (text): select several values → right-click → **Group** → rename, for example group *Mumbai* and *Pune* as "Maharashtra".

For flexible logic, a calculated column is more transparent:

DAX

```
Age Band =
SWITCH (
    TRUE (),
    hr_employees[age] < 25, "Under 25",
    hr_employees[age] < 35, "25-34",
    hr_employees[age] < 45, "35-44",
    "45 and over"
)
```

Python

```
bands = pd.cut(hr.age, [0, 24, 34, 44, 120], labels=["Under 25", "25-34", "35-44", "45 and over"])
hr.groupby(bands, observed=False).agg(employees=("emp_id", "count"), attrition_pct=("left", lambda s: round(s.mean() * 100, 1)))
```

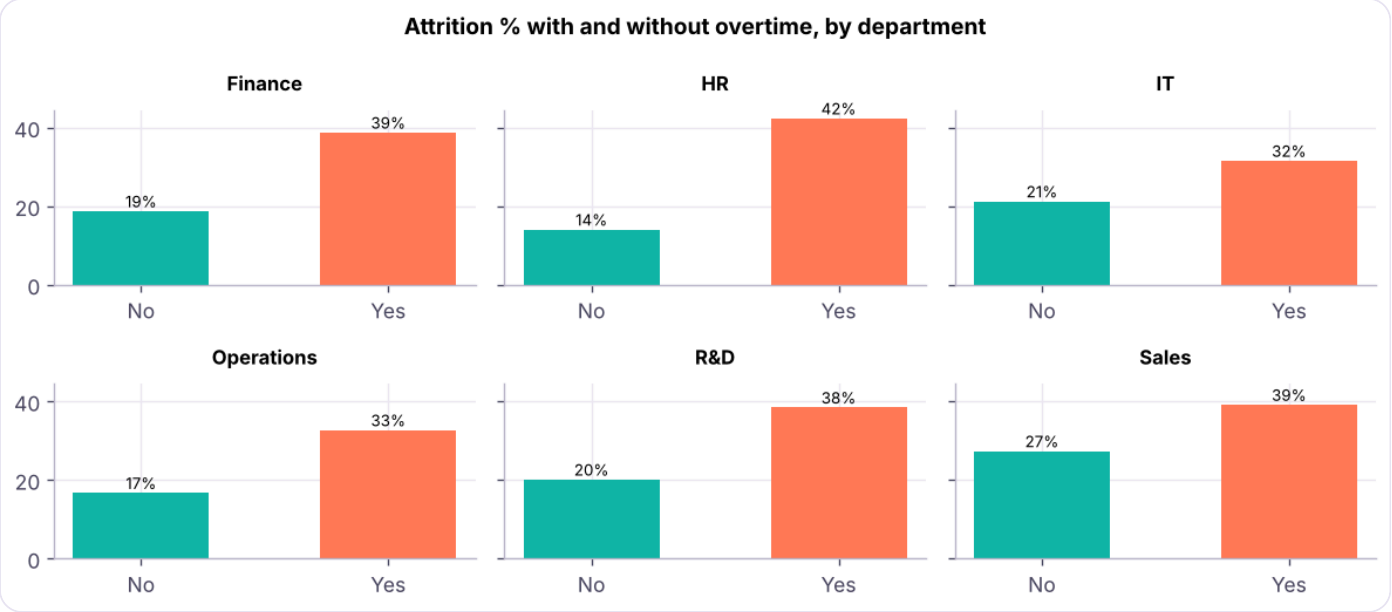
OUTPUT

| | employees | attrition_pct |
|---|---|---|
| **age** | | |
| **Under 25** | 148 | 35.1 |
| **25–34** | 624 | 26.9 |
| **35–44** | 574 | 23.5 |
| **45 and over** | 154 | 22.1 |

4 rows × 2 columns

## Step 6: small multiples

**Small multiples** split one visual into a grid of identical mini-charts by a field. Drag `department` into the visual's **Small multiples** well: you get one small column chart of attrition by satisfaction for every department. This lets you compare patterns across groups at a glance without a cluttered legend.




Small multiples: attrition % by overtime for each department. The same pattern repeats everywhere, which says overtime is a company-wide issue, not a one-department issue.

### What does the report tell us?

Python

```
t = hr.groupby("overtime").left.agg(["count", "mean"]); t["mean"] = (t["mean"] * 100).round(1)
t.columns = ["employees", "attrition_%"]
print(t.to_string()); print()
low = hr[hr.job_satisfaction ⩽ 2].left.mean() * 100; high = hr[hr.job_satisfaction ⩾
3].left.mean() * 100
print(f"Attrition for satisfaction 1-2: {low:.1f}%   vs   3-4: {high:.1f}%")
```

OUTPUT

```
           employees  attrition_%
overtime
No              1067         21.6
Yes              433         36.7

Attrition for satisfaction 1-2: 32.5%   vs   3-4: 19.4%
```

Insight sentences for a manager: *overtime workers leave far more often than others; low satisfaction scores go with higher attrition; Sales has the highest attrition among departments*. Notice the careful wording: these are **associations** in the data, not proof of cause. (The ML track shows how to test them properly.)

## Good habits from day one

1. **Name things clearly**: `Attrition %`, not `Measure 3`; rename pages ("HR Overview").
2. **Put measures in a dedicated table** later (Lesson 3) and format them (percentage, thousands separator).

3. **Check data types** in Power Query before loading.
4. **One idea per page**; the title should say the insight.
5. **Save often** (`.pbix`), and use clear version names.
6. **Do not use pie charts** with many slices; prefer bars (ranked) and use donuts only for 2–3 parts.

**2026 UPDATE** **Power BI in 2026**

- **Copilot** can create a report page from a prompt ("Build an attrition overview for HR"), write DAX measures and summarise a page. It needs a paid Fabric/Premium capacity and your data to be well-modelled and named.
- **Power BI Project (PBIP)** format stores the report as text files, so teams can use **Git** and code review.
- **TMDL** (Tabular Model Definition Language) and **DAX query view** make models easier to script and test.
- **Direct Lake** (Fabric) reads Delta tables directly with import-like speed. (See Lesson 6.)

## Summary

**KEY TAKEAWAYS**

- Power BI = Desktop (build) + Service (share) + Mobile (consume), now inside Microsoft Fabric.
- ETL: **Extract and Transform in Power Query, Load into the model**; analytics stages run from question to decision.
- Three views: **Report**, **Table**, **Model**; build with the Fields, Visualizations and Filters panes.
- A **card** shows a KPI; bar/column charts compare categories; a donut shows 2–3 parts; a matrix is a pivot table; slicers filter.
- A DAX **calculated column** (`YEAR`, `SWITCH`) is computed row by row at load time; **groups/bins** categorise without changing the data; **small multiples** repeat a chart per category.

## Quick quiz

6 questions · tap an answer to check it

**1.** In Power BI, where do you perform the Extract and Transform steps of ETL?

- **A** Report view
- **B** Power Query
- **C** The Service
- **D** The Model view

**Answer: B.** Power Query connects to sources and transforms data; Close & Apply loads it into the model.

**2.** Which view shows relationships between tables?

- **A** Report view
- **B** Table view
- **C** Model view
- **D** Filters pane

**Answer: C.** Relationships are created and edited in Model view.

**3.** Power BI Desktop is available for:

- **A** Windows and Mac
- **B** Windows only
- **C** Mac only
- **D** Browsers only

**Answer: B.** Desktop is Windows-only; Mac users use the Service or a Windows virtual machine.

**4.** What does a calculated column do?

A Runs only when a slicer changes

B Computes a value for every row when the data is loaded and stores it in the model

C Replaces a measure

D Changes the source file

**Answer: B.** Calculated columns are row-by-row and stored; measures calculate on the fly.

**5.** Which visual repeats the same chart for every value of a field?

A Matrix

B Slicer

C Small multiples

D Card

**Answer: C.** Small multiples create a grid of mini-charts split by a category.

**6.** Which licence do you need to share a report with colleagues who also have Pro licences?

A None, Desktop is enough

B Free Service licence

C Power BI Pro (or Premium Per User)

D Gateway licence

**Answer: C.** Sharing in the Service requires Pro for creators and viewers (unless the workspace is on Premium/Fabric capacity).

EXERCISE

### Build the HR page

Build the page shown above in Power BI Desktop with `hr_employees.csv`. Then add: (1) a slicer for `city` and `gender`; (2) a matrix of attrition % by `job_title` and `education`; (3) an `Age Band` column and a column chart of attrition % by band; (4) a title text box that states your main finding.

**Show solution (worked answer)**

(1) Slicer visual, fields `city`, `gender` (Format → Slicer settings → Style: Tile or Dropdown). (2) Matrix: Rows `job_title`, Columns `education`, Values `Attrition %` (turn on Cell elements → Background colour for a heat map). (3) Use the `SWITCH` formula in this lesson; chart X-axis `Age Band`, Y-axis `Attrition %`; sort the axis in the right order using a sort-by-column. (4) Insert → Text box, for example "Overtime is linked to about 15 more attrition points company-wide." Verify the claim against your visuals before you publish.

INTERVIEW CORNER

### Questions you may be asked

**Q: What is the difference between Power BI Desktop and Service?** Desktop is the free authoring tool for models and reports on Windows; the Service is the cloud platform to publish, share, refresh on a schedule, build dashboards and apps, and apply security.

**Q: What is ETL and where is it done in Power BI?** Extract, Transform, Load: Power Query connects to sources and applies transformation steps; the result is loaded into the VertiPaq in-memory model.

**Q: Calculated column vs measure?** A calculated column is evaluated per row at refresh time and stored (uses memory); a measure is evaluated at query time in the current filter context (uses CPU). Use measures for aggregations such as sums and ratios.

**Q: Report vs dashboard?** A report is multi-page and interactive, built on one model. A dashboard is a single page of pinned tiles from several reports, available only in the Service.

POWER BI · LESSON 2

# Power Query, Star Schema & Data Modelling (Adventure Cycles)

Prepare and model data the professional way. Clean data in Power Query, load a star schema (fact and dimension tables), create one-to-many relationships, understand filter direction, build and mark a date table, create hierarchies, and avoid the most common modelling mistakes, using the Adventure Cycles case study.

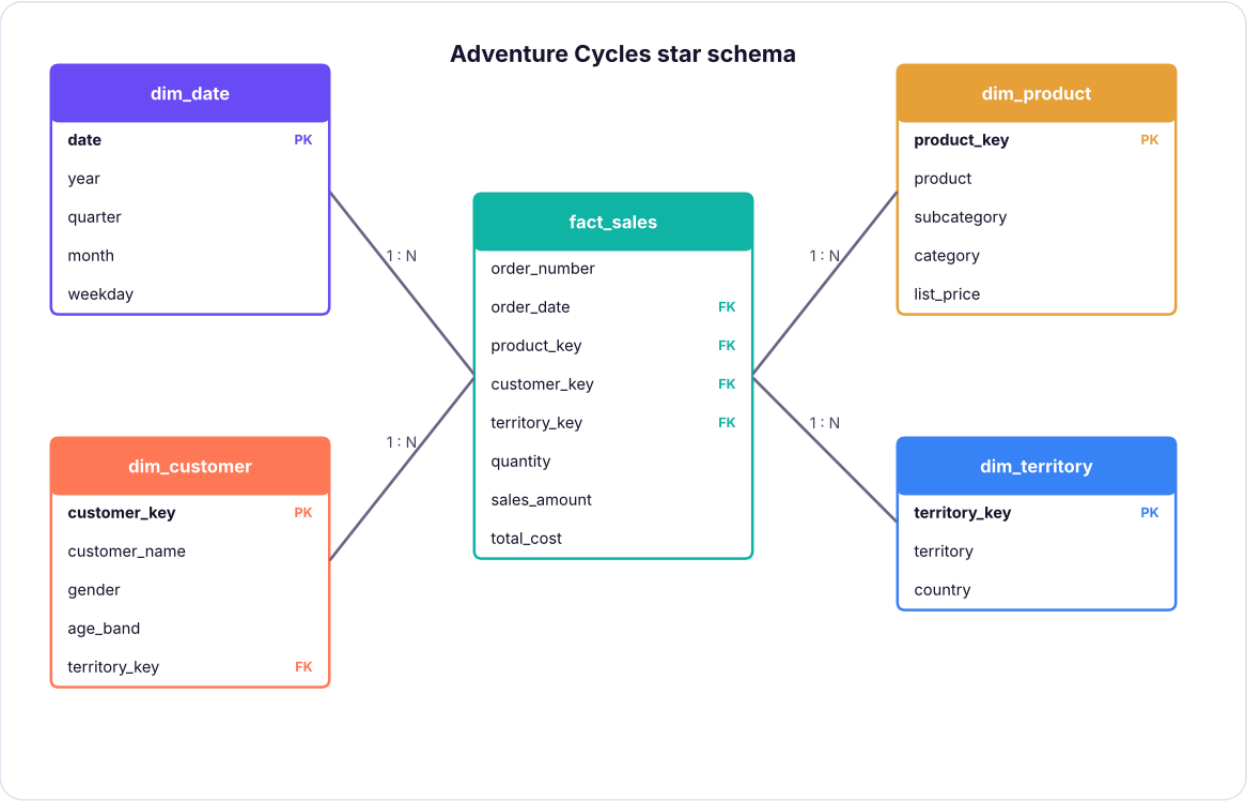
Power BI class 3 | Power BI Desktop 2026 | Intermediate | 140 min

**By the end of this lesson you can**

- Apply common Power Query steps and read the Applied Steps and M code
- Explain fact tables, dimension tables and the star schema, and why they beat one wide table
- Create relationships with the right cardinality and filter direction, and explain how filters flow
- Build a date table, mark it as a date table and add hierarchies
- Diagnose common modelling problems (many-to-many, ambiguous paths, missing dates)

## Why modelling matters more than visuals

Beautiful visuals on a bad model give wrong or slow answers. In Power BI the **data model** decides what numbers mean: how tables relate, which table filters which, and how fast DAX runs. The golden rule: **model as a star schema**.




A star schema: one central fact table of events/numbers surrounded by descriptive dimension tables. Filters flow from dimensions (one side) to the fact (many side).

| | Fact table | Dimension table |
|---|---|---|
| Describes | Events or transactions (a sale, a click, a shipment) | The "who, what, where, when" of those events |
| Rows | Many (millions) | Fewer (thousands) |
| Columns | Keys + numeric measures (quantity, amount, cost) | Keys + descriptive attributes (name, category, city) |
| Changes | Grows constantly | Changes rarely |
| Example | `fact_sales` | `dim_product`, `dim_customer`, `dim_date` |

**Why a star beats one big flat table:** smaller file (text stored once), faster visuals, one place to fix a name, correct filtering across several facts, and simpler DAX. A **snowflake** (dimensions normalised into sub-dimensions) works but adds joins; prefer a star in Power BI.

## The Adventure Cycles dataset

A fictional bicycle company sells bikes, components, clothing and accessories in India, the UAE and Singapore. The model has one fact table and four dimensions (plus the date table):

Python

```
import pandas as pd
D = "adventure_cycles/"
fact = pd.read_csv(D + "fact_sales.csv", parse_dates=["order_date"]); prod = pd.read_csv(D +
"dim_product.csv")
cust = pd.read_csv(D + "dim_customer.csv"); terr = pd.read_csv(D + "dim_territory.csv"); dates =
pd.read_csv(D + "dim_date.csv", parse_dates=["date"])
for name, df in [("fact_sales", fact), ("dim_product", prod), ("dim_customer", cust),
("dim_territory", terr), ("dim_date", dates)]:
    print(f"{name:14s} {df.shape[0]:>6,} rows × {df.shape[1]} columns")
fact.head(4)
```

OUTPUT

```
fact_sales      9,000 rows × 9 columns
dim_product        45 rows × 6 columns
dim_customer    1,200 rows × 5 columns
dim_territory       6 rows × 3 columns
dim_date        1,096 rows × 6 columns
```

| | order_number | order_date | product_key | customer_key | territory_key | quantity | unit_pri |
|---|---|---|---|---|---|---|---|
| 0 | SO50387 | 2023-01-01 | 11 | 103 | 1 | 3 | 114,40 |
| 1 | SO55797 | 2023-01-01 | 32 | 798 | 1 | 3 | 200 |
| 2 | SO51236 | 2023-01-01 | 9 | 750 | 5 | 1 | 126,70 |
| 3 | SO55082 | 2023-01-01 | 13 | 759 | 1 | 1 | 79,600 |

4 rows × 9 columns

## Power Query: prepare each table

Open **Transform Data**. Typical steps for each table (all recorded in **Applied Steps**):

| Table | Steps |
|---|---|
| `fact_sales` | Set types (`order_date` Date; `quantity` Whole number; amounts **Fixed decimal number**), remove blank rows, add `Profit = [sales_amount] - [total_cost]` as a custom column, keep only needed columns |
| `dim_product` | Trim/clean text, set `list_price` and `standard_cost` as decimals, remove duplicates on `product_key` |
| `dim_customer` | Replace `M`/`F` with Male/Female, remove duplicates on `customer_key` |
| `dim_territory` | Check there is one row per `territory_key` |
| `dim_date` | Set `date` to Date; sort `month` by `month_no` later in the model |

**Query settings** (right pane) shows each step; click any step to see the data at that point, and delete a step to undo it. Rename queries (`Sales`, `Product`…) because those names become table names in the model.

## Common transformations (from class and beyond)

| Goal | Where | Notes |
|---|---|---|
| Promote headers, set types | Home | Always check types: wrong types break relationships |
| Remove duplicates / blank rows | Home → Remove rows | Needed for dimension keys |
| Split column by delimiter | Transform → Split Column | e.g. `IN-2025-00042` |
| Replace values, trim, clean, format | Transform | |
| **Unpivot columns** | Transform → Unpivot | Turn Jan, Feb, Mar columns into rows (Excel Lesson 12) |
| **Merge queries** | Home → Merge | A join; use to bring a category into the fact (but prefer relationships) |
| **Append queries** | Home → Append | Stack monthly files |
| Group by | Transform → Group By | Pre-aggregate large tables |
| Conditional / Custom column | Add Column | M formulas |
| **Reference vs Duplicate** | Right-click query | *Reference* creates a new query that starts from the output of another (changes flow through); *Duplicate* copies the steps |
| **Parameters** | Home → Manage Parameters | A variable such as a file path or a start date, used in queries (Lesson 4) |
| Disable load | Right-click query | Keep helper queries out of the model |

## What is "query folding"?

When the source is a database (SQL Server, Snowflake), Power Query **translates your steps into SQL** and lets the database do the work, instead of downloading everything first. A step that cannot be translated "breaks folding" and everything after it runs locally. Check by right-clicking a step → *View Native Query*. Keep foldable steps (filter, select columns, merge, group) **before** non-foldable ones.

Power Query M

```
let
    Source   = Csv.Document(File.Contents("C:\Data\fact_sales.csv"), [Delimiter=",",
Encoding=65001]),
    Promoted = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
    Typed    = Table.TransformColumnTypes(Promoted, {{"order_date", type date}, {"quantity",
Int64.Type},
                  {"sales_amount", Currency.Type}, {"total_cost", Currency.Type}}),
    Profit   = Table.AddColumn(Typed, "profit", each [sales_amount] - [total_cost],
Currency.Type)
in
    Profit
```

## Relationships

A **relationship** links a column in one table to a column in another so that filters can flow between them. In **Model view**, drag `dim_product[product_key]` onto `fact_sales[product_key]`. Power BI usually detects relationships on same-named columns (Home → Manage relationships → Autodetect).

### Cardinality

| Type | Meaning | Use |
|---|---|---|
| **One-to-many (1:*)** | A key is unique in one table, repeated in the other | The standard: dimension → fact |
| One-to-one (1:1) | Unique on both sides | Splitting a wide table |
| Many-to-many (*:*) | Duplicates on both sides | Avoid; use a bridge table |

Check uniqueness before creating a one-to-many: if the "one" side has duplicates Power BI will complain.

Python

```
checks = {"dim_product[product_key]": prod.product_key, "dim_customer[customer_key]":
cust.customer_key, "dim_territory[territory_key]": terr.territory_key, "dim_date[date]":
dates.date}
for name, col in checks.items(): print(f"{name:30s} unique: {col.is_unique}")
orphans = {"product_key": ~fact.product_key.isin(prod.product_key), "customer_key":
~fact.customer_key.isin(cust.customer_key), "territory_key":
~fact.territory_key.isin(terr.territory_key)}
for k, m in orphans.items(): print(f"fact rows with no matching {k}: {m.sum()}")
```

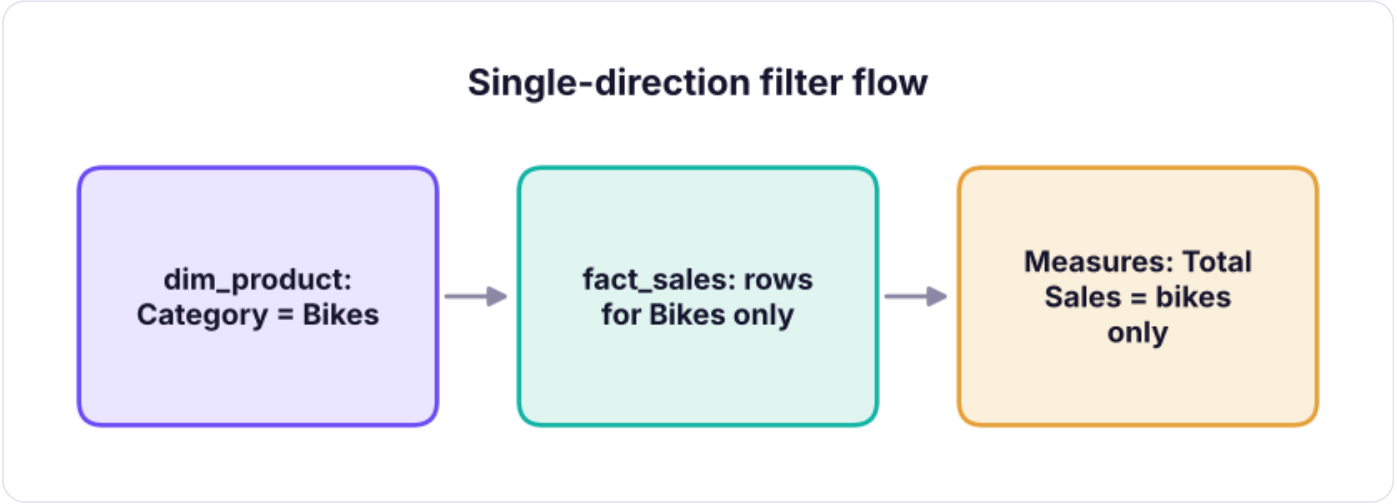
OUTPUT

```
dim_product[product_key]       unique: True
dim_customer[customer_key]     unique: True
dim_territory[territory_key]   unique: True
dim_date[date]                 unique: True
fact rows with no matching product_key: 0
```

```
fact rows with no matching customer_key: 0
fact rows with no matching territory_key: 0
```

Unique keys on the "one" side and no orphans on the "many" side: the model is safe.

## Filter direction




Filters flow along relationships from the one side to the many side (single direction). Selecting Bikes in dim_product filters fact_sales, which then filters the other dimensions only if bi-directional filtering is on.

| Setting | Behaviour | Advice |
|---|---|---|
| **Single** (dimension → fact) | A selection in a dimension filters the fact | Default and best |
| **Both** (bi-directional) | Filters also flow back from fact to dimension | Use only when needed (many-to-many, "show only customers who bought"); can cause ambiguity and slowness |
| **Active** vs inactive relationship | Only one active path between two tables; an inactive one can be switched on in DAX with `USERELATIONSHIP` | e.g. Order Date and Ship Date both link to the date table |

## Seeing the model work

Total sales by product category and by country, using the relationships (a join in pandas):

Python

```python
m = fact.merge(prod, on="product_key").merge(terr, on="territory_key")
m["profit"] = m.sales_amount - m.total_cost
out = m.pivot_table(index="category", columns="country", values="sales_amount", aggfunc="sum",
margins=True, margins_name="Total") / 1e6
out.round(1).rename_axis("Sales (₹ million)")
```

OUTPUT

| country | India | Singapore | UAE | Total |
|---|---|---|---|---|
| **Sales (₹ million)** | | | | |
| **Accessories** | 6.7 | 1.7 | 1.3 | 9.8 |
| **Bikes** | 320.6 | 74.8 | 71.3 | 466.8 |
| **Clothing** | 6.0 | 1.3 | 1.1 | 8.5 |
| **Components** | 73.5 | 18.8 | 17.2 | 109.6 |
| **Total** | 406.9 | 96.7 | 91.0 | 594.6 |

5 rows × 4 columns

In Power BI you get this with a matrix: Rows `category`, Columns `country`, Values `SUM(sales_amount)`. No join code needed: the relationships do it.

## The date table

Time intelligence (year-to-date, same period last year, Lesson 4) requires a **proper date table**: one row per day with **no gaps**, covering all dates in the data, and **marked as a date table**.

DAX

```
Date =
VAR MinDate = DATE ( YEAR ( MIN ( fact_sales[order_date] ) ), 1, 1 )
VAR MaxDate = DATE ( YEAR ( MAX ( fact_sales[order_date] ) ), 12, 31 )
RETURN
ADDCOLUMNS (
    CALENDAR ( MinDate, MaxDate ),
    "Year",       YEAR ( [Date] ),
    "Quarter",    "Q" & QUARTER ( [Date] ),
    "Month No",   MONTH ( [Date] ),
    "Month",      FORMAT ( [Date], "mmm" ),
    "Year-Month", FORMAT ( [Date], "yyyy-mm" ),
    "Weekday",    FORMAT ( [Date], "ddd" )
)
```

STEP BY STEP **Set it up**

1. Model → **New table** → paste the DAX above (or load `dim_date.csv`).
2. Select the table → **Table tools → Mark as date table** → choose the `Date` column. Power BI validates that dates are unique and continuous.
3. Create the relationship `Date[Date]` → `fact_sales[order_date]` (one-to-many).
4. **Sort `Month` by `Month No`** (select `Month` → Column tools → Sort by column) so April does not come before January.
5. Hide the raw `order_date` in the fact table so report authors use the date table.

Python

```
print("Fact order dates :", fact.order_date.min().date(), "to", fact.order_date.max().date())
print("Date table       :", dates.date.min().date(), "to", dates.date.max().date(), "| rows:",
len(dates))
full = pd.date_range(dates.date.min(), dates.date.max())
print("Continuous (no gaps):", len(full) == len(dates) and (dates.date.sort_values().values ==
full.values).all())
dates.head(4)
```

OUTPUT

```
Fact order dates : 2023-01-01 to 2025-12-30
Date table       : 2023-01-01 to 2025-12-31 | rows: 1096
Continuous (no gaps): True
```

| | date | year | quarter | month_no | month | weekday |
|---|---|---|---|---|---|---|
| 0 | 2023-01-01 | 2023 | Q1 | 1 | Jan | Sun |
| 1 | 2023-01-02 | 2023 | Q1 | 1 | Jan | Mon |
| 2 | 2023-01-03 | 2023 | Q1 | 1 | Jan | Tue |
| 3 | 2023-01-04 | 2023 | Q1 | 1 | Jan | Wed |

4 rows × 6 columns

WATCH OUT **Do not rely on auto date/time**

Power BI can create hidden date tables automatically ("Auto date/time"). They bloat the model and cannot be customised or shared. Turn it off (File → Options → Data Load → Time intelligence) and always use your own marked date table.

## Hierarchies

A **hierarchy** lets users drill down through levels with one click. In the Data pane, right-click `category` in `dim_product` → **Create hierarchy**, then drag `subcategory` and `product` onto it. Visuals with a hierarchy get drill-down buttons (arrows at the top-right of the visual): from category → sub-category → product. The date table's `Year → Quarter → Month → Date` is the classic one.

## Common modelling problems

| Symptom | Likely cause | Fix |
|---|---|---|
| Totals repeat the same number for every category | No relationship between the table you filter and the fact | Create the relationship |
| "A circular dependency was detected" / ambiguous paths | Two paths between two tables | Make one inactive, or remove the bi-directional filter |
| Many-to-many warning | Duplicate keys on both sides | Build a clean dimension with unique keys, or a bridge table |
| Blank row ("(Blank)") appears in visuals | Fact rows whose key is missing in the dimension (orphans) | Add the missing rows to the dimension, or fix keys |
| YTD / last-year measures give wrong or blank results | Date table not marked, has gaps, or not related | Fix the date table |
| Slow model, big file | Wide tables, high-cardinality text columns, auto date/time | Star schema, remove unused columns, reduce precision, disable auto date/time |
| Months sort alphabetically | Missing sort-by-column | Sort `Month` by `Month No` |

**TIP** **Best-practice checklist**

1. Star schema with surrogate integer keys. 2. One-to-many, single-direction relationships. 3. One marked date table. 4. Hide keys and technical columns from report view. 5. Disable auto date/time. 6. Remove columns you do not use. 7. Name tables and columns for business users. 8. Put measures in a separate `_Measures` table (Lesson 3).

**DO IT WITH AI** **Do it with AI**

Ask Copilot or any assistant: *"Review this list of tables and columns and propose a star schema with a fact table and dimensions, the keys, and the relationships"* and paste your column lists. Compare its answer with the checklist above, and verify key uniqueness yourself in Power Query (Column profile → *Distinct* equals *Count*).

## Summary

KEY TAKEAWAYS

- Model as a **star schema**: facts (events and numbers) surrounded by dimensions (descriptions), joined by **one-to-many, single-direction** relationships.
- Power Query prepares each table: types, cleaning, unpivot, merge/append, group, parameters; **query folding** pushes work to the source.
- Filters flow from the **one side** (dimension) to the **many side** (fact); use bi-directional filters sparingly; one active relationship per table pair, others inactive (`USERELATIONSHIP`).
- A continuous, **marked date table** is required for time intelligence; sort month names by month number; turn off auto date/time.
- Hierarchies enable drill-down; most "wrong totals" come from missing relationships, duplicate keys or orphans.

## Quick quiz

6 questions · tap an answer to check it

**1. In a star schema, which table typically contains the numeric measures such as quantity and sales amount?**

A Dimension table

B Fact table

C Date table

D Parameter table

**Answer: B.** Facts hold events and measures; dimensions hold descriptions.

**2. A one-to-many relationship between dim_product and fact_sales means:**

A A product appears once in sales

B Each product_key appears once in dim_product and many times in fact_sales

C Both tables have unique keys

D The relationship is bidirectional

**Answer: B.** The "one" side must have unique keys.

**3.** Why is a bi-directional relationship risky?

A It is not allowed

B It can create ambiguous filter paths and slow performance

C It deletes data

D It only works with dates

**Answer: B.** Use it only when you must (for example many-to-many scenarios).

**4.** What does "Mark as date table" require?

A A column of text

B A unique, continuous Date column with no gaps

C A relationship to every table

D A measure

**Answer: B.** Time-intelligence functions need a valid contiguous date table.

**5.** Months show as Apr, Aug, Dec, Feb... in alphabetical order. Fix?

A Rename the months

B Use a pie chart

C Sort the Month column by the Month No column

D Delete the date table

**Answer: C.** Sort by column controls the display order.

**6.** What does Power Query's "query folding" do?

A Compresses the PBIX file

B Translates steps into the source system's query (e.g. SQL) so the database does the work

C Hides the query

D Merges two queries

**Answer: B.** Folding improves refresh performance for database sources.

EXERCISE **Model it yourself**

Load the five Adventure Cycles CSV files, set types, create the four relationships, mark `dim_date` as the date table, hide key columns, create a Category → Subcategory → Product hierarchy, and build a matrix of sales by `category` (rows) and `year` (columns).

**Show solution (worked answer)**

In Power Query set `order_date` and `date` to Date. In Model view drag `product_key`, `customer_key`, `territory_key` from each dimension to the fact, and `date` to `order_date`. Table tools → Mark as date table → `date`. Right-click the key columns → Hide in report view. Create the hierarchy by right-clicking `category`. In Report view add a Matrix with Rows `dim_product[category]`, Columns `dim_date[year]`, Values `Sum of sales_amount`. Compare with the pandas pivot: Bikes should be about ₹46.7 crore of the total ₹59.5 crore.

INTERVIEW CORNER

### Questions you may be asked

**Q: What is a star schema and why use it in Power BI?** One fact table joined to several dimension tables with one-to-many relationships. It compresses well, filters predictably, and makes DAX simple and fast.

**Q: How do filters propagate in a Power BI model?** From the one side to the many side along active relationships, in the filter direction. Bi-directional relationships also filter from many back to one.

**Q: Why do you need a date table?** Built-in time-intelligence DAX (YTD, SAMEPERIODLASTYEAR) needs a continuous, marked date table; it also gives you year/quarter/month columns, fiscal calendars and holidays.

**Q: How do you handle two dates in the fact table (order date and ship date)?** Role-playing dimension: one date table with one active relationship (order date) and one inactive (ship date) activated with `USERELATIONSHIP` in specific measures, or two copies of the date table.

POWER BI · LESSON 3

# DAX Fundamentals: Measures, Filter Context & CALCULATE

Learn DAX, the formula language of Power BI. Understand calculated columns vs measures, row context vs filter context, write core aggregations and iterators (SUMX), master CALCULATE and FILTER, ALL/REMOVEFILTERS and ALLSELECTED, handle division safely, use variables, and build a library of business measures.

Power BI classes 2–3 | Power BI Desktop 2026 | Intermediate | 150 min

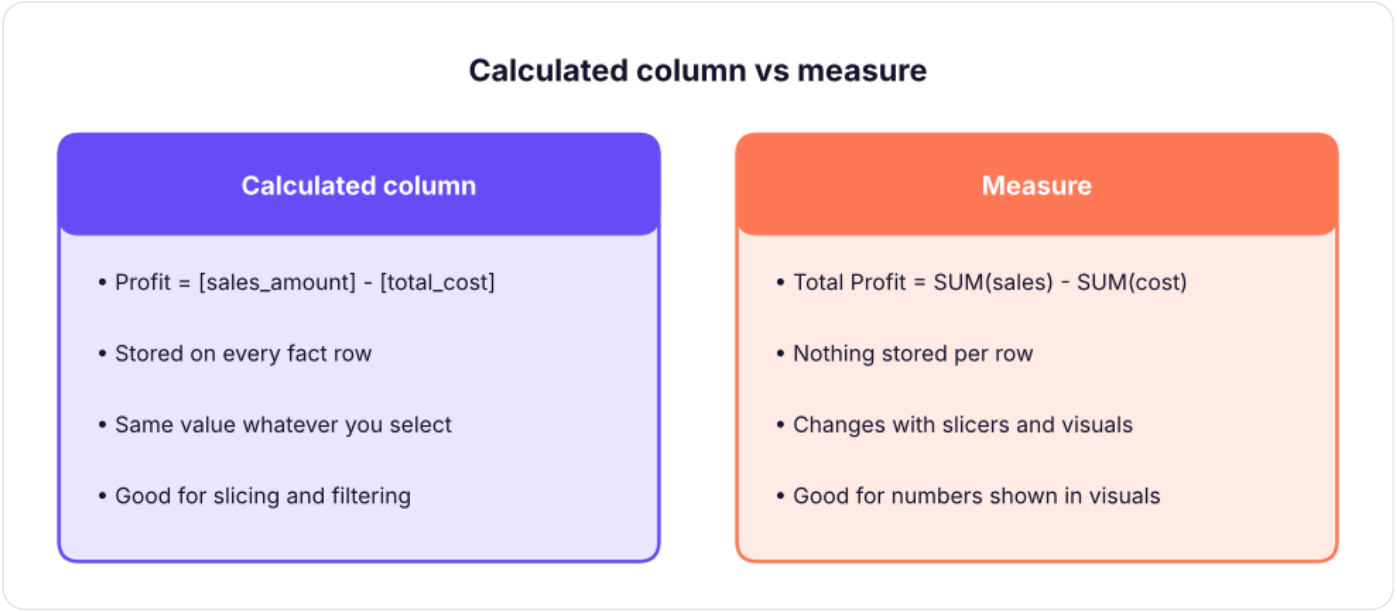
**By the end of this lesson you can**

- Explain the difference between a calculated column and a measure, and when to use each
- Describe row context and filter context and predict what a measure returns in a visual
- Write aggregation measures and iterators (SUM, COUNTROWS, DISTINCTCOUNT, SUMX, AVERAGEX)
- Use CALCULATE with filter arguments, ALL/REMOVEFILTERS and ALLSELECTED to build ratios and percent-of-total
- Use variables, DIVIDE, IF/SWITCH and display folders to write clean, reusable measures

## What is DAX?

**DAX** (Data Analysis Expressions) is the formula language used in Power BI, Excel's Data Model and Analysis Services. It looks like Excel formulas, but it works on **columns and tables** and recalculates for whatever the user selects in a visual. DAX is the skill that separates a report builder from a Power BI developer.

| | Calculated column | Measure |
|---|---|---|
| Evaluated | **Row by row** when data is refreshed | **On the fly**, for the current filter selection |
| Stored in the model | Yes (uses memory) | No (only the formula) |
| Result depends on slicers? | No | **Yes** |
| Use for | New attributes to slice by (Age Band, Hire Year, Profit per row) | Totals, ratios, KPIs, growth |
| Appears in | Rows, columns, slicers, filters | Values area, cards, tooltips |




The same table, two ways. A calculated column adds a value to every row. A measure returns one number computed from whichever rows are currently filtered.

**TIP** **Rule of thumb**

If you want to **put it on an axis or slicer**, make a column. If you want to **put it in the Values well**, make a measure. In a star schema, almost every number is a measure.

## DAX syntax in five minutes

```dax
Total Sales = SUM ( fact_sales[sales_amount] )
```

- `Total Sales` is the **measure name** (spaces allowed, no brackets when defining it).
- `=` starts the formula.

- `SUM` is a **function**; `fact_sales[sales_amount]` is a **fully qualified column** (table name, then column in square brackets). Always include the table name for columns; **never** include it for measures: `[Total Sales]`.
- Strings use double quotes; comments use `--` or `//` or `/* … */`.
- Operators: `+ - * /`, `&` (join text), `= <> > < >= <=`, `&&` (AND), `||` (OR), `IN { "A", "B" }`.
- Spaces and line breaks are ignored; format with **daxformatter.com** or the formatting button.

### Creating measures

Home → **New measure** (or right-click a table). Put all measures in an empty table named `_Measures` (Home → Enter data → name `_Measures` → Load, then hide its default column) so they are easy to find. Use **display folders** (Model view → Properties) to group them.

## Basic aggregations

DAX

```
Total Sales      = SUM ( fact_sales[sales_amount] )
Total Cost       = SUM ( fact_sales[total_cost] )
Total Profit     = [Total Sales] - [Total Cost]
Profit Margin %  = DIVIDE ( [Total Profit], [Total Sales] )
Units Sold       = SUM ( fact_sales[quantity] )
Orders           = DISTINCTCOUNT ( fact_sales[order_number] )
Order Lines      = COUNTROWS ( fact_sales )
Avg Order Value  = DIVIDE ( [Total Sales], [Orders] )     -- in this data every order has one
line, so Orders = Order Lines
Customers        = DISTINCTCOUNT ( fact_sales[customer_key] )
```

Check the values in Python; they are what a card visual with no filters would display:

Python

```
import pandas as pd
D = "adventure_cycles/"
f = pd.read_csv(D + "fact_sales.csv", parse_dates=["order_date"]); p = pd.read_csv(D +
"dim_product.csv"); t = pd.read_csv(D + "dim_territory.csv")
sales, cost = f.sales_amount.sum(), f.total_cost.sum()
print(f"Total Sales     : {sales:,.0f}")
print(f"Total Cost      : {cost:,.0f}")
print(f"Total Profit    : {sales - cost:,.0f}")
print(f"Profit Margin % : {(sales - cost) / sales:.1%}")
print(f"Units Sold      : {f.quantity.sum():,}")
print(f"Orders          : {f.order_number.nunique():,}")
print(f"Order Lines     : {len(f):,}")
print(f"Avg Order Value : {sales / f.order_number.nunique():,.0f}")
print(f"Customers       : {f.customer_key.nunique():,}")
```

OUTPUT

```
Total Sales      : 594,559,665
Total Cost       : 384,392,790
Total Profit     : 210,166,875
Profit Margin %  : 35.3%
Units Sold       : 22,703
Orders           : 9,000
Order Lines      : 9,000
Avg Order Value  : 66,062
Customers        : 1,200
```

**WATCH OUT** **DIVIDE, not /**

`[A] / [B]` returns an error or infinity when B is 0 or blank. `DIVIDE ( [A], [B] )` returns BLANK (or an alternate result: `DIVIDE ( [A], [B], 0 )`). Always use DIVIDE for ratios.

## Filter context: the heart of DAX

When you put a measure in a visual, DAX evaluates it **once per cell**, with a **filter context** made of everything that filters the data for that cell: row and column headers, slicers, page/report filters, and cross-filtering from other visuals.




Filter context for one cell of a matrix (Category = Bikes, Country = India, Year = 2025 slicer). DAX applies all of them to the fact table before it sums.

Same measure, different context in each cell. The matrix below is exactly what Power BI shows for `Total Sales` with Category on rows and Country on columns:

Python

```
m = f.merge(p, on="product_key").merge(t, on="territory_key")
m["year"] = m.order_date.dt.year
print((m.pivot_table(index="category", columns="country", values="sales_amount", aggfunc="sum")
/ 1e6).round(1).to_string())
print("\nSame measure, with a Year = 2025 slicer:")
print((m[m.year == 2025].pivot_table(index="category", columns="country", values="sales_amount",
aggfunc="sum") / 1e6).round(1).to_string())
```

OUTPUT

```
country      India  Singapore   UAE
category
Accessories    6.7        1.7   1.3
Bikes        320.6       74.8  71.3
Clothing       6.0        1.3   1.1
Components    73.5       18.8  17.2

Same measure, with a Year = 2025 slicer:
country      India  Singapore   UAE
category
Accessories    2.2        0.7   0.4
Bikes        102.3       24.6  22.0
Clothing       2.0        0.3   0.4
Components    22.9        7.1   5.1
```

### Row context

A **row context** exists when DAX goes through a table row by row: in a **calculated column**, and inside **iterator** functions ( `SUMX` , `AVERAGEX` , `FILTER` ...). In row context, column references mean "this row's value".

DAX

```
-- Calculated column on fact_sales: row context gives each row's own values
Line Profit = fact_sales[sales_amount] - fact_sales[total_cost]
```

Row context does **not** automatically filter other tables; **CALCULATE** converts a row context into a filter context ("context transition").

## Iterators: SUMX, AVERAGEX, MAXX

`SUM ( column )` adds one column. When you need to **calculate per row first** and then aggregate, use an iterator: `SUMX ( table, expression )` .

DAX

```
-- Revenue at list price (unit_price x quantity), calculated row by row
List Value = SUMX ( fact_sales, fact_sales[quantity] * fact_sales[unit_price] )

-- Average margin per order line (not the same as total profit / total sales!)
Avg Line Margin % = AVERAGEX ( fact_sales, DIVIDE ( fact_sales[sales_amount] -
fact_sales[total_cost], fact_sales[sales_amount] ) )

-- Most profitable single line
Max Line Profit = MAXX ( fact_sales, fact_sales[sales_amount] - fact_sales[total_cost] )
```

Python

```
f["line_profit"] = f.sales_amount - f.total_cost
list_value = (f.quantity * f.unit_price).sum()
avg_line_margin = ((f.sales_amount - f.total_cost) / f.sales_amount).mean()
print(f"List Value        : {list_value:,.0f}")
print(f"Realised/List     : {f.sales_amount.sum() / list_value:.1%}  (average price
realisation)")
print(f"Avg Line Margin % : {avg_line_margin:.1%}   vs overall margin {(f.sales_amount.sum() -
f.total_cost.sum()) / f.sales_amount.sum():.1%}")
print(f"Max Line Profit   : {f.line_profit.max():,.0f}")
```

OUTPUT

```
List Value        : 617,201,300
Realised/List     : 96.3%  (average price realisation)
Avg Line Margin % : 34.5%   vs overall margin 35.3%
Max Line Profit   : 174,060
```

The average of line margins (34.5%) differs slightly from the overall margin (35.3%) because large lines count more in the total than in a simple average. Choose the definition the business wants, and name the measure clearly.

## CALCULATE: change the filter context

`CALCULATE ( expression, filter1, filter2, … )` evaluates the expression in a **modified** filter context. It is the most important DAX function.

DAX

```
Bike Sales   = CALCULATE ( [Total Sales], dim_product[category] = "Bikes" )
India Sales  = CALCULATE ( [Total Sales], dim_territory[country] = "India" )
Bike Sales India 2025 =
    CALCULATE ( [Total Sales], dim_product[category] = "Bikes", dim_territory[country] =
"India", dim_date[year] = 2025 )
High Value Lines = CALCULATE ( COUNTROWS ( fact_sales ), fact_sales[sales_amount] > 200000 )
```

Multiple filter arguments are combined with **AND**. A simple column filter like `dim_product[category] = "Bikes"` **replaces** any existing filter on that column.

Python

```
bike = f.merge(p, on="product_key").query("category == 'Bikes'").sales_amount.sum()
print(f"Bike Sales          : {bike:,.0f}")
india_2025_bikes = m[(m.category == "Bikes") & (m.country == "India") & (m.year ==
2025)].sales_amount.sum()
print(f"Bike Sales India 2025: {india_2025_bikes:,.0f}")
print(f"High Value Lines    : {(f.sales_amount > 200000).sum():,}")
```

OUTPUT

```
Bike Sales          : 466,754,465
Bike Sales India 2025: 102,271,520
High Value Lines    : 797
```

## Filter functions: ALL, REMOVEFILTERS, ALLSELECTED, ALLEXCEPT, KEEPFILTERS

| Function | Effect | Typical use |
|---|---|---|
| `ALL ( table or column )` / `REMOVEFILTERS ( … )` | Ignores filters on the table/column | Grand totals, denominators |
| `ALLSELECTED ( … )` | Ignores filters from the visual's own rows/columns but keeps slicers and page filters | "% of visible total" |
| `ALLEXCEPT ( table, col1, … )` | Removes all filters on the table except those columns | Subtotals per group |
| `VALUES ( column )` | The distinct values visible in the current context | Counting / looping over what is filtered |
| `SELECTEDVALUE ( column, alt )` | The single selected value (or alternate) | Titles, dynamic measures |
| `KEEPFILTERS ( filter )` | Intersects with, rather than replaces, existing filters | Advanced |

### Percent of total

DAX

```
Sales % of Total =
DIVIDE ( [Total Sales], CALCULATE ( [Total Sales], REMOVEFILTERS ( dim_product ) ) )

Sales % of Visible Total =
DIVIDE ( [Total Sales], CALCULATE ( [Total Sales], ALLSELECTED ( dim_product ) ) )

Share Within Country =
DIVIDE ( [Total Sales], CALCULATE ( [Total Sales], ALLEXCEPT ( dim_territory,
dim_territory[country] ) ) )
```

Python

```
cat = m.groupby("category").sales_amount.sum()
print((100 * cat / cat.sum()).round(1).rename("Sales % of Total").to_string())
```

OUTPUT

```
category
Accessories     1.6
Bikes          78.5
Clothing        1.4
Components     18.4
```

### FILTER: row-by-row conditions

`FILTER ( table, condition )` is an iterator that returns the rows where the condition is TRUE. Use it when a filter needs a **measure or several columns**; for a simple single-column comparison, the plain `column = value` form inside CALCULATE is faster.

DAX

```
-- Sales from products with a list price above 1,00,000
Premium Sales = CALCULATE ( [Total Sales], FILTER ( dim_product, dim_product[list_price] >
100000 ) )

-- Number of products whose total sales beat 1 crore
Strong Products = COUNTROWS ( FILTER ( VALUES ( dim_product[product] ), [Total Sales] > 10000000
) )
```

Python

```
prem = f.merge(p, on="product_key").query("list_price > 100000").sales_amount.sum()
ps = f.groupby("product_key").sales_amount.sum()
print(f"Premium Sales  : {prem:,.0f}")
print(f"Strong Products: {(ps > 10_000_000).sum()}  (products with sales above ₹1 crore)")
```

OUTPUT

```
Premium Sales  : 303,639,995
Strong Products: 20  (products with sales above ₹1 crore)
```

## Variables and readability

`VAR … RETURN` stores intermediate results: easier to read, easier to debug, and faster (each variable is evaluated once).

DAX

```
Profit Margin vs Overall =
VAR CurrentMargin = DIVIDE ( [Total Profit], [Total Sales] )
VAR OverallMargin = CALCULATE ( DIVIDE ( [Total Profit], [Total Sales] ), REMOVEFILTERS () )
RETURN
    CurrentMargin - OverallMargin
```

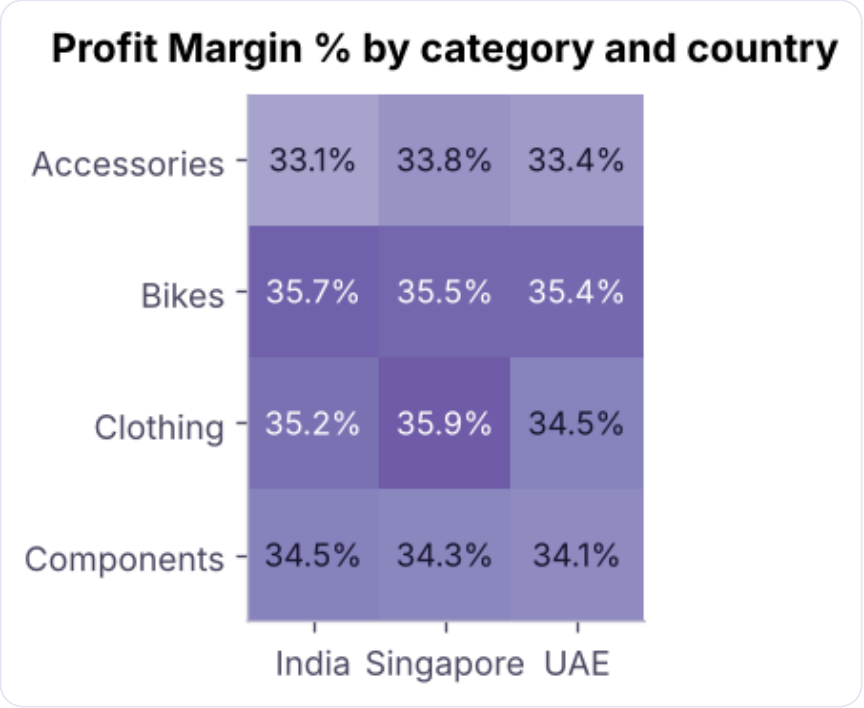
# Logical and text measures

```DAX
Margin Status =
SWITCH (
    TRUE (),
    ISBLANK ( [Profit Margin %] ), "No data",
    [Profit Margin %] ≥ 0.4,       "Excellent",
    [Profit Margin %] ≥ 0.3,       "Good",
    "Review"
)

Selected Country Title =
"Sales for " & SELECTEDVALUE ( dim_territory[country], "all countries" )
```

| Function | Purpose |
| --- | --- |
| `IF ( test, a, b )`, `SWITCH ( expr, v1, r1, … )`, `SWITCH ( TRUE (), cond1, r1, … )` | Branching |
| `ISBLANK`, `COALESCE ( x, 0 )`, `BLANK ()` | Handle empty values |
| `HASONE ( col )`, `ISFILTERED ( col )` | Test the current context |
| `FORMAT ( value, "0.0%" )`, `CONCATENATEX ( table, col, ", " )` | Text |

## Measures in action: a profit-margin matrix




Profit margin % by category and country: the matrix you would build with the Profit Margin % measure and conditional formatting (darker = higher).

## Common DAX mistakes

| Mistake | Fix |
|---|---|
| Using a calculated column where a measure is needed (results do not respond to slicers) | Create a measure |
| Dividing with `/` | `DIVIDE` |
| Writing `SUM ( [Measure] )` | `SUM` takes columns; use `SUMX ( table, [Measure] )` for measures over rows |
| Averaging ratios ( `AVERAGE` of a margin column) | Compute `DIVIDE ( SUM numerator, SUM denominator )` unless you really want the average of ratios |
| Using `FILTER ( ALL ( table ), … )` when a column filter works | Prefer simple column filters inside CALCULATE |
| Forgetting that a measure's result depends on context; testing in a card | Test in a table or matrix with the fields you care about |
| Hard-coding values ( `"Bikes"` ) in many measures | Use a parameter table or SELECTEDVALUE |

DO IT WITH AI **Do it with AI**

Copilot and chat assistants write decent DAX, but they invent column names and often misuse context. A reliable prompt gives the **exact table and column names**, the **relationships**, and the **expected result**: *"Tables: fact_sales(order_date, product_key, sales_amount, total_cost), dim_product(product_key, category), dim_date(date). Write a DAX measure for each category's share of total sales that ignores the category filter but respects year slicers. Explain the filter context."* Then test the measure in a table next to your own pandas check.

## Summary

KEY TAKEAWAYS

- **Calculated columns** run row by row at refresh and are stored; **measures** run at query time in the current **filter context**.
- Filter context = row/column headers + slicers + filters + cross-filtering; measures are re-evaluated for every cell.
- Row context exists in calculated columns and iterators ( `SUMX` , `AVERAGEX` , `FILTER` ); `CALCULATE` converts it to filter context.
- `CALCULATE ( expression, filters )` changes the filters; `REMOVEFILTERS` / `ALL` , `ALLSELECTED` , `ALLEXCEPT` remove them for percent-of-total calculations.
- Use `DIVIDE` , variables ( `VAR … RETURN` ), `SWITCH ( TRUE (), … )` , clear names, a `_Measures` table and display folders.

### Quick quiz

6 questions · tap an answer to check it

**1. Which is evaluated for every cell of a matrix based on slicers and row/column headers?**

A A calculated column

B A measure

C A relationship

D A parameter

**Answer: B.** Measures respond to filter context; calculated columns are fixed at refresh.

**2.** Which function lets you compute a total that ignores the Category filter (for a denominator)?

A `SUM`

B `CALCULATE ( [Total Sales], REMOVEFILTERS ( dim_product[category] ) )`

C `FILTER ( [Total Sales] )`

D `COUNTROWS`

**Answer: B.** REMOVEFILTERS (or ALL) clears the filter inside CALCULATE.

**3.** Which is the safest way to compute a margin?

A `[Profit] / [Sales]`

B `DIVIDE ( [Profit], [Sales] )`

C `IF ( [Sales] = 0, 0, [Profit] ) / [Sales]`

D `SUM ( [Profit] ) / SUM ( [Sales] )`

**Answer: B.** DIVIDE handles zero and blank denominators gracefully.

**4.** `SUMX ( fact_sales, fact_sales[quantity] * fact_sales[unit_price] )` is needed because:

A SUM cannot add numbers

B The product must be calculated per row before adding

C SUMX is faster than SUM

D It creates a calculated column

**Answer: B.** SUM works on one existing column; SUMX evaluates an expression per row.

**5.** What is row context?

- **A** The current row being evaluated in a calculated column or iterator
- **B** The filters from slicers
- **C** The relationship direction
- **D** A table filter

**Answer: A.** Row context refers to "this row" during row-by-row evaluation.

**6.** What does `CALCULATE ( [Total Sales], dim_product[category] = "Bikes" )` return in a visual that shows each country?

- **A** Total sales for all products
- **B** Bike sales for each country in the visual
- **C** Bike sales for all countries on every row
- **D** An error

**Answer: B.** The Category filter is replaced by Bikes, while the Country filter from the visual still applies.

EXERCISE **Write the measures**

Using the Adventure Cycles model, write DAX for: (1) Profit and margin % for **Clothing** only; (2) each country's share of total sales (works in a visual with country on rows); (3) the number of customers who bought more than one product line; (4) a title measure that says "Top category: ..." for the category with the highest sales in the current context.

**Show solution (worked answer)**

DAX

```
-- (1)
Clothing Profit  = CALCULATE ( [Total Profit], dim_product[category] = "Clothing" )
Clothing Margin  = DIVIDE ( [Clothing Profit], CALCULATE ( [Total Sales],
dim_product[category] = "Clothing" ) )
-- (2)
Country Share = DIVIDE ( [Total Sales], CALCULATE ( [Total Sales], REMOVEFILTERS (
dim_territory ) ) )
-- (3)
Multi-line Customers =
COUNTROWS ( FILTER ( VALUES ( fact_sales[customer_key] ), CALCULATE ( COUNTROWS (
fact_sales ) ) > 1 ) )
-- (4)
Top Category =
VAR t = ADDCOLUMNS ( VALUES ( dim_product[category] ), "@s", [Total Sales] )
VAR top1 = TOPN ( 1, t, [@s], DESC )
RETURN "Top category: " & MAXX ( top1, dim_product[category] )
```

Verify each in a table visual next to a pandas check.

INTERVIEW CORNER

### Questions you may be asked

**Q: What is the difference between a calculated column and a measure?** A column is computed per row at refresh and stored; a measure is computed at query time for the current filter context and returns a single value.

**Q: Explain filter context and row context.** Filter context is the set of filters (axes, slicers, filters) applied to the model when a measure is evaluated. Row context is the "current row" while iterating a table (calculated columns, iterators). CALCULATE turns row context into filter context.

**Q: What does CALCULATE do?** Evaluates an expression in a modified filter context; its filter arguments add, replace or remove filters.

**Q: ALL vs ALLSELECTED?** ALL removes all filters from the specified table/column (grand total); ALLSELECTED removes only the filters coming from the visual's own rows and columns but keeps outer filters such as slicers, giving a total of what is visible.

**Q: SUM vs SUMX?** SUM adds a column; SUMX evaluates an expression for each row of a table and sums the results (needed for products of columns, conditional logic per row).

POWER BI · LESSON 4

# DAX Time Intelligence, Ranking, Parameters & Dynamic Measures

Make dashboards answer "compared with when?". Build year-to-date, month-to-date, same-period-last-year, month-on-month and year-on-year measures, rolling averages, rankings with RANKX and TOPN, what-if parameters, field parameters and dynamic measure switching, and know visual calculations and calculation groups.

● Power BI classes 3–4

✦ Power BI Desktop 2026 (field parameters, calculation groups, visual calculations)

Advanced

⏱ 150 min

**By the end of this lesson you can**

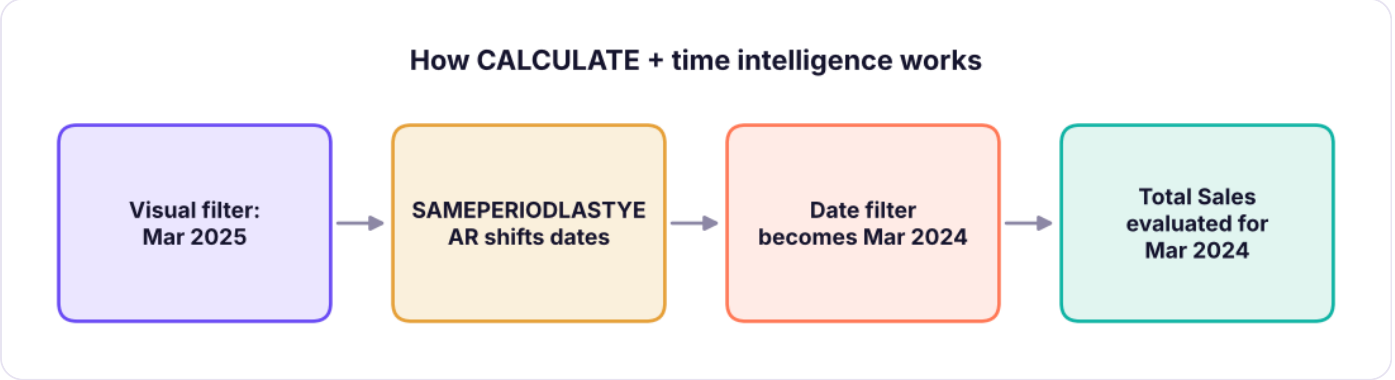
- Explain what time intelligence needs (a marked date table) and use TOTALYTD, DATESYTD, SAMEPERIODLASTYEAR, DATEADD and PARALLELPERIOD
- Build YoY %, MoM %, YTD, rolling 3-month and running-total measures
- Rank items with RANKX and show the Top N with TOPN, including an "Others" bucket
- Create what-if parameters and dynamic measure switching with field parameters
- Describe calculation groups, visual calculations and DAX query view, and when to use them

## What time intelligence needs

Time-intelligence functions move or expand the **date filter**: "this year to date", "the same period last year". They only work when:

1. You have a **date table** with every date (Lesson 2),
2. it is **marked as a date table**, and
3. it is **related** to the fact table's date column.

Always use the **date table's** `Date` column inside time functions, never the fact table's date.




Time intelligence changes the date filter, then CALCULATE re-evaluates the measure. SAMEPERIODLASTYEAR shifts the selected dates back exactly one year.

## Core measures

DAX

```
Total Sales = SUM ( fact_sales[sales_amount] )

-- Cumulative totals
Sales YTD = TOTALYTD ( [Total Sales], dim_date[date] )
Sales QTD = TOTALQTD ( [Total Sales], dim_date[date] )
Sales MTD = TOTALMTD ( [Total Sales], dim_date[date] )
-- Indian financial year ending 31 March:
Sales FYTD = TOTALYTD ( [Total Sales], dim_date[date], "31-3" )

-- Previous periods
Sales LY    = CALCULATE ( [Total Sales], SAMEPERIODLASTYEAR ( dim_date[date] ) )
Sales PM    = CALCULATE ( [Total Sales], DATEADD ( dim_date[date], -1, MONTH ) )
Sales PQ    = CALCULATE ( [Total Sales], DATEADD ( dim_date[date], -1, QUARTER ) )

-- Growth
YoY %  = DIVIDE ( [Total Sales] - [Sales LY], [Sales LY] )
MoM %  = DIVIDE ( [Total Sales] - [Sales PM], [Sales PM] )
YoY Δ  = [Total Sales] - [Sales LY]
```

| Function | Returns | Notes |
|---|---|---|
| `TOTALYTD ( expr, dates, [year_end] )` | Year-to-date total | Shortcut for `CALCULATE ( expr, DATESYTD ( dates ) )` |
| `DATESYTD / DATESQTD / DATESMTD` | The date set from the period start to the last visible date | Use inside CALCULATE |
| `SAMEPERIODLASTYEAR ( dates )` | The selected dates shifted back one year | Needs continuous dates |
| `DATEADD ( dates, n, interval )` | Shift by n days, months, quarters or years | Month/quarter shifts handle month-end well |
| `PARALLELPERIOD ( dates, n, interval )` | The **full** period shifted (full month, quarter, year) | |
| `DATESBETWEEN ( dates, start, end )` | Custom ranges | |
| `DATESINPERIOD ( dates, end, -n, interval )` | Rolling window ending at `end` | Rolling 3/12 months |
| `PREVIOUSMONTH / PREVIOUSQUARTER / PREVIOUSYEAR` | Whole previous period | |
| `EOMONTH`, `EDATE`, `DATEDIFF` | Date arithmetic | |

Let's compute what these measures return for our data (monthly sales, year-to-date, last year and YoY):

Python

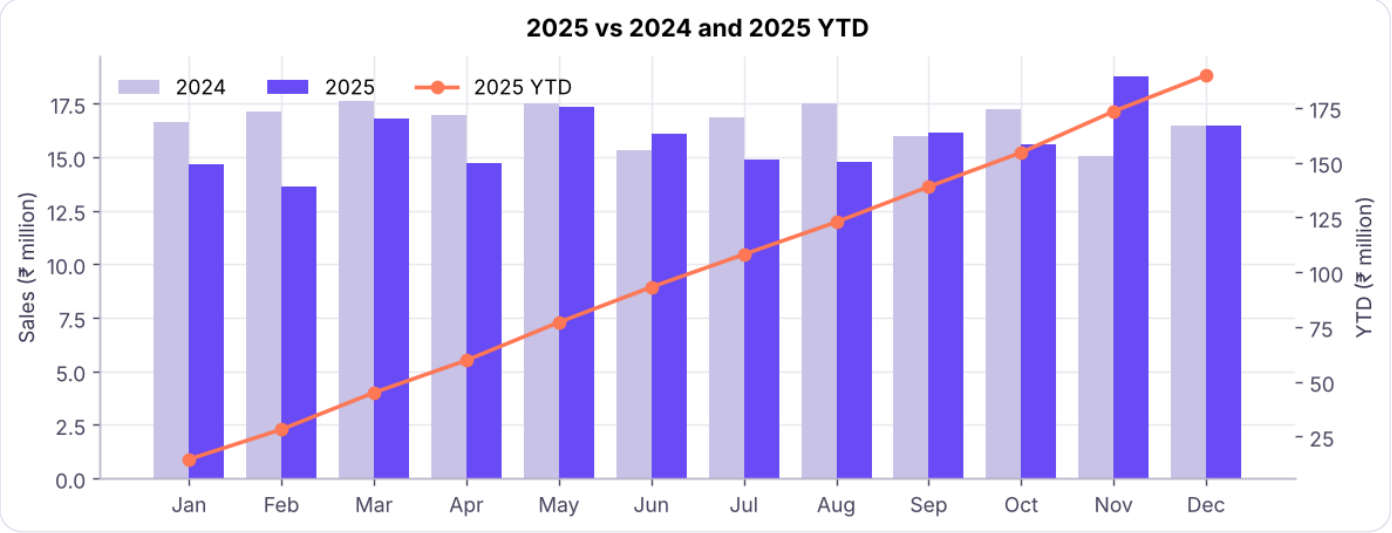
```
import pandas as pd
D = "adventure_cycles/"
f = pd.read_csv(D + "fact_sales.csv", parse_dates=["order_date"]); p = pd.read_csv(D +
"dim_product.csv")
f["ym"] = f.order_date.dt.to_period("M")
mth = f.groupby("ym").sales_amount.sum().to_frame("Total Sales")
mth["Sales PM"] = mth["Total Sales"].shift(1)
mth["MoM %"] = (mth["Total Sales"] / mth["Sales PM"] - 1) * 100
mth["Sales LY"] = mth["Total Sales"].shift(12)
mth["YoY %"] = (mth["Total Sales"] / mth["Sales LY"] - 1) * 100
mth["Sales YTD"] = mth.groupby(mth.index.year)["Total Sales"].cumsum()
out = mth[mth.index.year == 2025].copy()
out["Total Sales"] = (out["Total Sales"] / 1e6).round(1); out["Sales LY"] = (out["Sales LY"] /
1e6).round(1); out["Sales YTD"] = (out["Sales YTD"] / 1e6).round(1)
out[["Total Sales", "Sales LY", "YoY %", "MoM %", "Sales YTD"]].round(1).rename_axis("2025
month")
```

OUTPUT

| | Total Sales | Sales LY | YoY % | MoM % | Sales YTD |
|---|---|---|---|---|---|
| **2025 month** | | | | | |
| **2025-01** | 14.6 | 16.7 | -12.1 | -11.2 | 14.6 |
| **2025-02** | 13.6 | 17.1 | -20.5 | -6.9 | 28.3 |
| **2025-03** | 16.8 | 17.6 | -4.8 | 23.1 | 45.1 |
| **2025-04** | 14.7 | 17.0 | -13.2 | -12.1 | 59.8 |
| **2025-05** | 17.4 | 17.5 | -1.0 | 17.7 | 77.2 |
| **2025-06** | 16.1 | 15.3 | 5.0 | -7.4 | 93.2 |
| **2025-07** | 14.9 | 16.9 | -11.5 | -7.2 | 108.1 |
| **2025-08** | 14.8 | 17.5 | -15.6 | -1.0 | 122.9 |
| **2025-09** | 16.2 | 16.0 | 1.0 | 9.4 | 139.1 |
| **2025-10** | 15.6 | 17.3 | -9.7 | -3.5 | 154.7 |
| **2025-11** | 18.8 | 15.1 | 24.6 | 20.5 | 173.4 |
| **2025-12** | 16.5 | 16.5 | 0.0 | -12.2 | 189.9 |

12 rows × 5 columns

(Values in ₹ million.) In a Power BI matrix with `dim_date[month]` on rows and these measures in the Values area you would see exactly this.




Monthly sales for 2025 against the same month in 2024, with year-to-date sales on the right axis. This is the classic "this year vs last year" chart.

COMMON MISTAKES **Common time-intelligence bugs**

- **Date table not marked / has gaps**: `SAMEPERIODLASTYEAR` returns blank or wrong values.
- **Using the fact table's date column** in the function: use `dim_date[date]`.
- **Comparing partial periods**: the current month has data for 10 days, last year's month has 31, so YoY looks terrible. Compare "to date" with `DATESYTD` and `SAMEPERIODLASTYEAR`, or restrict to complete months.
- **Fiscal year**: pass `"31-3"` as the year-end argument to `TOTALYTD`, and create `Fiscal Year` and `Fiscal Quarter` columns in the date table.
- **Showing a "last year" value in months with no data** (future months): wrap with `IF ( [Total Sales] <> BLANK (), … )`.

## Rolling windows and running totals

DAX

```
-- Rolling 3-month sales ending at the last visible date
Sales Rolling 3M =
CALCULATE ( [Total Sales], DATESINPERIOD ( dim_date[date], MAX ( dim_date[date] ), -3, MONTH ) )

-- Rolling 3-month average of monthly sales (average over the months in the window)
Avg Monthly Sales R3M =
DIVIDE ( [Sales Rolling 3M], 3 )      -- valid once three full months exist in the window

-- Running total across all dates up to the current one
Sales Running Total =
CALCULATE ( [Total Sales], FILTER ( ALL ( dim_date[date] ), dim_date[date] ⩽ MAX (
dim_date[date] ) ) )
```

In SQL this was `SUM() OVER (ORDER BY … ROWS BETWEEN …)` (SQL Lesson 12). DAX does the same thing by redefining the date filter.

Python

```
mth["Rolling 3M"] = mth["Total Sales"].rolling(3).sum()
mth["Rolling 3M avg"] = mth["Total Sales"].rolling(3).mean()
(mth[["Total Sales", "Rolling 3M", "Rolling 3M avg"]].tail(6) / 1e6).round(1).rename_axis("month
(₹ million)")
```

OUTPUT

| | Total Sales | Rolling 3M | Rolling 3M avg |
|---|---|---|---|
| month (₹ million) | | | |
| 2025-07 | 14.9 | 48.3 | 16.1 |
| 2025-08 | 14.8 | 45.8 | 15.3 |
| 2025-09 | 16.2 | 45.8 | 15.3 |
| 2025-10 | 15.6 | 46.5 | 15.5 |
| 2025-11 | 18.8 | 50.5 | 16.8 |
| 2025-12 | 16.5 | 50.9 | 17.0 |

6 rows × 3 columns

## Ranking: RANKX and TOPN

DAX

```
-- Rank of each product category by sales within the current context
Category Rank = RANKX ( ALL ( dim_product[category] ), [Total Sales], , DESC, DENSE )

-- Show only the top 5 products: sales of a product if it is in the top 5, else blank
Top 5 Product Sales =
VAR top5 = TOPN ( 5, ALL ( dim_product[product] ), [Total Sales], DESC )
RETURN CALCULATE ( [Total Sales], KEEPFILTERS ( top5 ) )

-- Everything outside the top 5, as one "Others" value
Others Sales = [Total Sales] - [Top 5 Product Sales]
```

For simple Top N visuals, you do not need DAX: in the **Filters pane**, choose the visual's field → Filter type **Top N** → Show items Top 5 → By value `Total Sales` → Apply filter. Use DAX when the Top N must respond to slicers in a custom way or you need an "Others" bar.

Python

```
prod_sales = f.merge(p, on="product_key").groupby(["category",
"product"]).sales_amount.sum().sort_values(ascending=False)
top5 = prod_sales.head(5)
print((top5 / 1e6).round(1).to_string())
print(f"\nTop 5 total : {top5.sum() / 1e6:,.1f} M   |   Others: {(prod_sales.sum() - top5.sum())
/ 1e6:,.1f} M   |   Top 5 share: {top5.sum() / prod_sales.sum():.1%}")
```

OUTPUT

```
category  product
Bikes     Road Bike E-331        51.3
```

```
          Mountain Bike C-245    42.4
          Touring Bike A-507     39.7
          Mountain Bike D-662    36.2
          Mountain Bike B-168    35.9

Top 5 total : 205.4 M   |   Others: 389.2 M   |   Top 5 share: 34.5%
```

# Parameters

## What-if parameter

**Modeling → New parameter → Numeric range**: name `Price Change %`, from −20 to +20, increment 1, default 0, and tick **Add slicer**. Power BI creates a table and a measure `Price Change % Value`. Use it in your own measure to simulate:

DAX

```
Simulated Sales = [Total Sales] * ( 1 + [Price Change % Value] / 100 )
Simulated Profit = [Simulated Sales] - [Total Cost]
```

Dragging the slicer instantly shows what profit would be with a 5% price increase (ignoring demand changes: always say so on the page).

Python

```
base_sales, base_cost = f.sales_amount.sum(), f.total_cost.sum()
rows = [(chg, base_sales * (1 + chg / 100) / 1e6, (base_sales * (1 + chg / 100) - base_cost) /
1e6) for chg in (-10, -5, 0, 5, 10)]
pd.DataFrame(rows, columns=["Price change %", "Simulated sales (₹ M)", "Simulated profit (₹
M)"]).round(1)
```

OUTPUT

| | Price change % | Simulated sales (₹ M) | Simulated profit (₹ M) |
|---|---|---|---|
| **0** | -10 | 535.1 | 150.7 |
| **1** | -5 | 564.8 | 180.4 |
| **2** | 0 | 594.6 | 210.2 |
| **3** | 5 | 624.3 | 239.9 |
| **4** | 10 | 654.0 | 269.6 |

5 rows × 3 columns

## Power Query parameters

**Home → Manage parameters** creates parameters used inside queries, for example `StartDate` or a `ServerName`. Use them to switch between a test and a production database, or to filter large tables at load time. In the Service, parameters can be changed in dataset settings.

## Dynamic measure switching

A very popular pattern: a slicer that lets users choose which measure the chart shows (Sales, Profit, Orders).

**The modern way: Field parameters.** Modeling → **New parameter → Fields**: add `Total Sales`, `Total Profit`, `Orders`; tick **Add slicer**. Power BI creates a table with the three measures; use that table's field on a chart's axis or values and the slicer swaps the measure. A field parameter can also switch **axis** fields (Category, Country, Year).

**The classic way: a disconnected table + SWITCH.**

```dax
-- Table "Measure Selector" (Enter data): columns Measure Name, Sort
Selected Measure =
SWITCH (
    SELECTEDVALUE ( 'Measure Selector'[Measure Name], "Sales" ),
    "Sales",  [Total Sales],
    "Profit", [Total Profit],
    "Orders", [Orders]
)
```

Format strings can be dynamic too (Power BI 2024+ "dynamic format string"): thousands for sales, percentage for margin.

## KPI visuals and forecasting

- **KPI visual**: needs a value (indicator), a trend axis (date) and a target (for example last year's sales). Colours show good/bad.
- **Gauge**: current vs target; min/max/target set in the format pane.
- **Forecasting**: in a **line chart** with a date axis, Analytics pane → **Forecast** → set forecast length (for example 6 months), confidence interval (95%) and seasonality (12 points). Power BI uses exponential smoothing (ETS). It is a quick what-comes-next view, not a replacement for proper forecasting models (ML track).
- **Analytics pane** also offers constant lines, average, min/max, percentile lines, trend lines, and **anomaly detection**.

## Newer DAX features worth knowing

| Feature | What it is |
| --- | --- |
| **Calculation groups** | A reusable "time-intelligence switcher": define `Current`, `YTD`, `LY`, `YoY %` once and apply them to every measure (created in Tabular Editor or in the model view in recent versions) |
| **Visual calculations** | DAX written **inside a visual** that can refer to other fields in the visual (running sum, moving average, % of parent) with simple functions such as `RUNNINGSUM`, `MOVINGAVERAGE`, `PREVIOUS`, `FIRST` |
| **DAX query view** | `EVALUATE` queries to test and debug measures; Copilot can write DAX here |
| **User-defined functions (DAX UDFs)** | Reusable functions defined with the `FUNCTION` keyword (preview) |
| **INFO.* functions** | Query model metadata (`INFO.TABLES()`, `INFO.MEASURES()`) |

DAX

```
-- Visual calculation (typed in the visual's formula bar)
Running Sales = RUNNINGSUM ( [Total Sales] )
Moving Avg 3  = MOVINGAVERAGE ( [Total Sales], 3 )
```

DO IT WITH AI

### Do it with AI

Copilot can write time-intelligence measures from a sentence ("Show sales year to date versus last year"). It works best if the date table is marked, columns and tables have clear names, and you add **descriptions** to measures (Model view → Properties). Check the generated DAX uses your `dim_date[date]` (not the fact table's date), and confirm one value by hand.

## Summary

KEY TAKEAWAYS

- Time intelligence needs a **continuous, marked date table**, related to the fact; always reference the date table's column.
- `TOTALYTD`, `SAMEPERIODLASTYEAR`, `DATEADD`, `PARALLELPERIOD`, `DATESINPERIOD` build YTD, last-year, MoM, YoY and rolling windows; `DIVIDE ( current - previous, previous )` gives growth.
- `RANKX` ranks, `TOPN` selects the top items (add an "Others" measure); the Filters-pane **Top N** is the no-code route.
- **What-if parameters** simulate scenarios; **field parameters** let users switch measures or axes; the disconnected table + `SWITCH` pattern does the same in older models.
- Know calculation groups, visual calculations and the DAX query view for advanced work.

## Quick quiz

6 questions · tap an answer to check it

**1.** Which measure shows year-to-date sales?

A `SUM ( fact_sales[sales_amount] )`

B `TOTALYTD ( [Total Sales], dim_date[date] )`

C `SAMEPERIODLASTYEAR ( [Total Sales] )`

D `RANKX ( dim_date, [Total Sales] )`

**Answer: B.** TOTALYTD accumulates from the start of the year to the current date.

**2.** SAMEPERIODLASTYEAR returns blank for every month. What should you check first?

A The theme

B That the date table is continuous, marked as a date table and related to the fact

C The colour of the visual

D The number of slicers

**Answer: B.** Time intelligence relies on a valid date table.

**3.** Which formula gives month-on-month growth?

A `[Total Sales] - [Sales PM]`

B `DIVIDE ( [Total Sales] - [Sales PM], [Sales PM] )`

C `SUM ( [Sales PM] ) / [Total Sales]`

D `DATEADD ( [Total Sales], -1, MONTH )`

**Answer: B.** Growth rate = (current − previous) ÷ previous, using DIVIDE.

**4.** What does a field parameter let users do?

A Change the report theme

B Switch which measures or fields a visual shows with a slicer

C Refresh the data

D Create relationships

**Answer: B.** Field parameters create a slicer that swaps fields in visuals.

**5.** In the Indian financial year (April–March), how do you build YTD sales?

A `TOTALYTD ( [Total Sales], dim_date[date] )` always works

B `TOTALYTD ( [Total Sales], dim_date[date], "31-3" )`

C `DATESMTD`

D `PARALLELPERIOD ( … )`

**Answer: B.** The third argument sets the year-end date.

**6.** What is the correct way to compute "Others" after a Top 5?

A Add a second visual

B Total sales minus the Top 5 measure

C Use FILTER ALL

D Use PREVIOUSMONTH

**Answer: B.** Others = overall total − Top N total.

EXERCISE

### Time-intelligence measures

For Adventure Cycles write measures for: (1) sales for the last 12 months ending at the selected date; (2) YoY % by category; (3) the best month of each year; (4) a what-if parameter "Discount %" from 0 to 30 that reduces `Total Sales` and shows simulated profit margin.

**Show solution (worked answer)**

DAX

```
-- (1)
Sales L12M = CALCULATE ( [Total Sales], DATESINPERIOD ( dim_date[date], MAX (
dim_date[date] ), -12, MONTH ) )
-- (2) put dim_product[category] on rows and dim_date[year] on columns
YoY % = DIVIDE ( [Total Sales] - [Sales LY], [Sales LY] )
-- (3)
Best Month =
VAR t = ADDCOLUMNS ( VALUES ( dim_date[month] ), "@s", [Total Sales] )
RETURN MAXX ( TOPN ( 1, t, [@s], DESC ), dim_date[month] )
-- (4)  parameter table 'Discount %' with measure [Discount % Value]
Simulated Margin = DIVIDE ( [Total Sales] * ( 1 - [Discount % Value] / 100 ) - [Total
Cost], [Total Sales] * ( 1 - [Discount % Value] / 100 ) )
```

Validate (2) and (3) against pandas group-bys.

INTERVIEW CORNER

### Questions you may be asked

**Q: What do you need for time-intelligence functions to work?** A date table with continuous dates covering the data, marked as a date table, related to the fact table by date.

**Q: How would you calculate YoY growth?** `Sales LY = CALCULATE ( [Total Sales], SAMEPERIODLASTYEAR ( dim_date[date] ) )` then `YoY % = DIVIDE ( [Total Sales] - [Sales LY], [Sales LY] )`.

**Q: How can users choose which metric a chart shows?** Field parameters (modern) or a disconnected table with a `SWITCH ( SELECTEDVALUE (…), … )` measure.

**Q: How do you show Top N plus an "Others" bar?** A Top N measure using TOPN/KEEPFILTERS and an Others measure = total − Top N.

POWER BI · LESSON 5

# Visuals, Filters, Drill-through, Tooltips, Buttons & Report Design

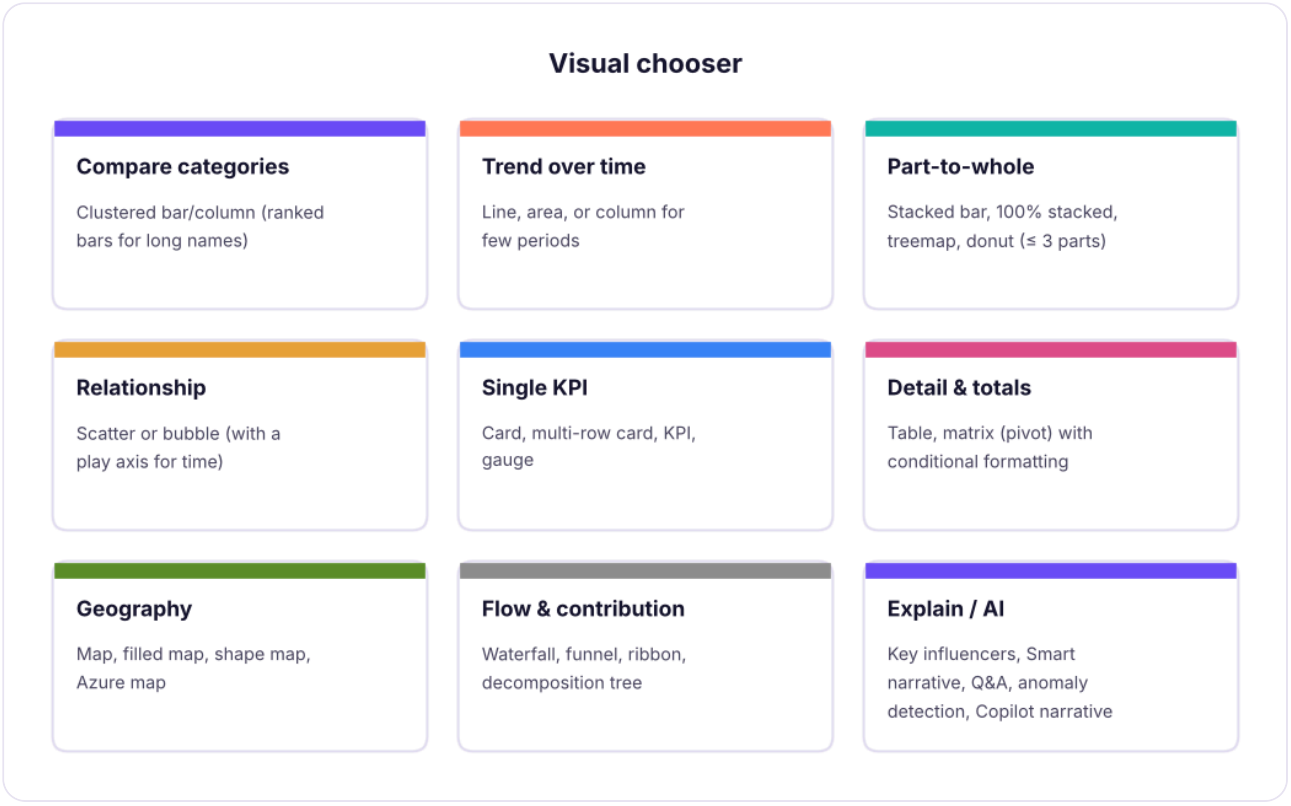
Turn a model into an interactive, well-designed report. Choose visuals, format them (cards, matrix, charts, gauge, KPI, maps, images), use visual, page and report filters, edit interactions, drill-down and drill-through, custom tooltips, bookmarks and buttons, conditional formatting, themes and design principles.

Power BI classes 2–4 | Power BI Desktop 2026 | Intermediate | 150 min

**By the end of this lesson you can**

- Pick the right visual for each question and configure its field wells and formatting
- Apply visual, page and report-level filters, Top N, relative date filters and slicer sync
- Control cross-filtering with Edit interactions; add drill-down, drill-through and report-page tooltips
- Build navigation with buttons, bookmarks, page navigation and a selection pane
- Apply themes and conditional formatting, and follow layout, colour and accessibility rules

# Choosing the visual




Match the visual to the question.

| Visual | Fields | Good for | Watch out |
|---|---|---|---|
| **Card** / multi-row card | Values | One or few headline numbers | Format display units (K, M, Cr) and add a clear label |
| **Clustered bar / column** | Axis, Values, Legend | Comparisons | Sort descending, start at zero, ≤ 15 bars |
| **Stacked / 100% stacked** | Axis, Legend, Values | Composition | Only the bottom segment is easy to compare |
| **Line / area** | X-axis (date), Y-axis, Legend | Trends | Limit to ~5 lines; use a continuous date axis |
| **Line + clustered column** | Shared axis, column values, line values | Two measures with different scales (secondary axis) | Label both axes |
| **Waterfall** | Category, Y-axis | Bridge from start to end (profit walk) | Needs an ordered category |
| **Scatter / bubble** | X, Y, Size, Details, Play axis | Relationships and outliers | Add Details so each point is an entity |
| **Treemap** | Group, Values | Hierarchical share of total | Many tiny rectangles are unreadable |
| **Donut / pie** | Legend, Values | 2–3 parts | More than 4 slices = use a bar |
| **Matrix** | Rows, Columns, Values | Pivot tables with drill and subtotals | Use conditional formatting for heat maps |
| **Table** | Columns | Exact values, detail list | Keep it short; a table is not a dashboard |
| **KPI** / **Gauge** | Value, Target, Trend axis | Progress to a target | Gauges waste space; prefer KPI cards |
| **Funnel** | Stage, Values | Drop-off between stages | |
| **Map / Filled map** | Location, Size, Legend | Geography | Geocoding needs clean place names ( `City, State, Country` ) |
| **Slicer** | Field | Filter the page | Styles: list, dropdown, tiles, between (range), relative date |
| **Decomposition tree** | Analyze, Explain by | Drill into drivers | Interactive exploration |
| **Key influencers** | Analyze, Explain by | AI finds what drives a metric | Needs a binary/numeric target |

| Visual | Fields | Good for | Watch out |
|---|---|---|---|
| **Smart narrative** | Auto-text | Written summary that updates | Check the wording |

## Formatting essentials

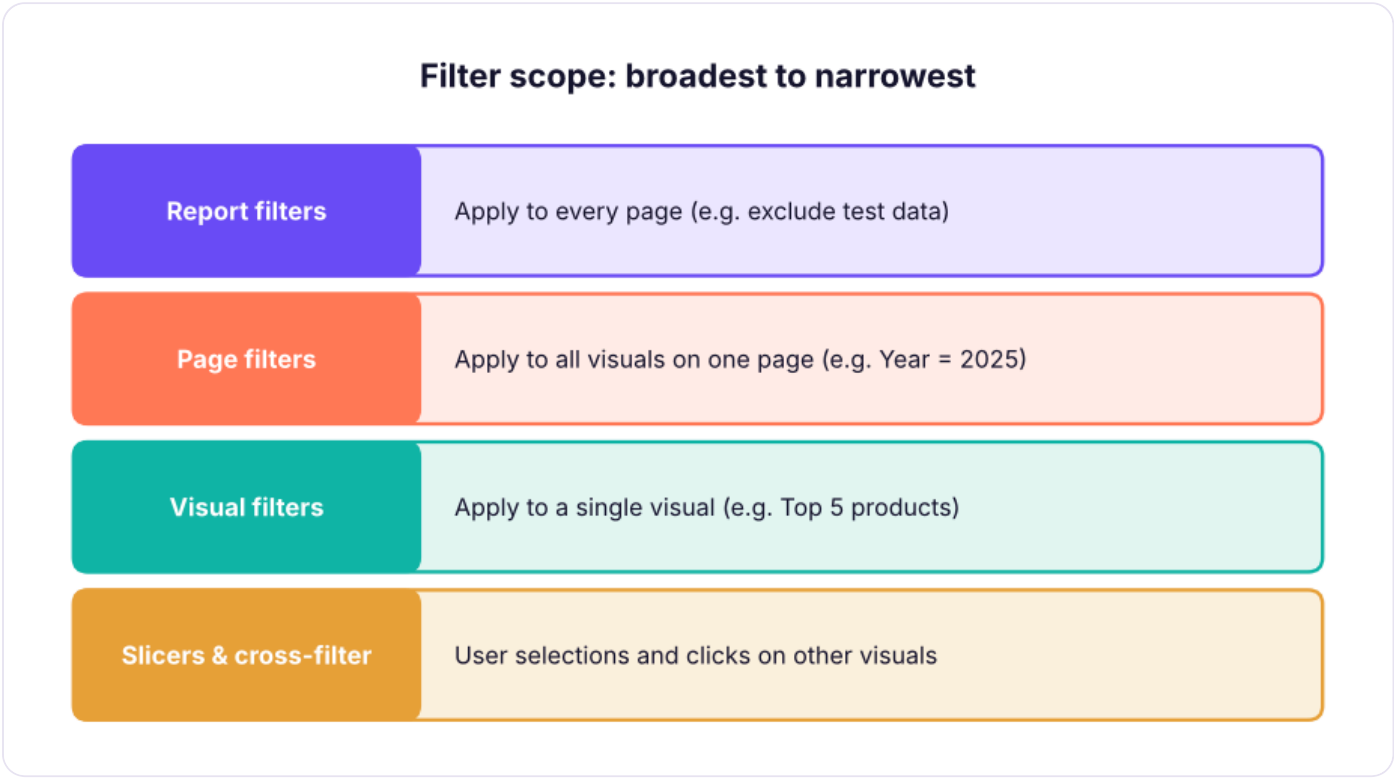
Select a visual → **Format visual** (paint-roller icon): **Visual** tab (axis, labels, colours, legend, gridlines, title) and **General** (title, size, position, background, border, alt text).

| Task | Where |
|---|---|
| Display units and decimals | Format → Callout value / Data labels → Display units: Thousands, Lakhs (not built in; use Millions) or custom format strings |
| Sort | `…` menu → Sort axis (by value or by another field) |
| Colour a highlighted series | Format → Colors → per-category colours or conditional |
| Titles that state the insight | General → Title → fx (dynamic title from a measure) |
| Accessibility | General → Alt text; keep contrast high; do not rely on colour alone |
| Copy formatting | Home → **Format painter** |
| Align / distribute | Format → Arrange → Align and distribute; hold `Shift` to multi-select |
| Group visuals | Select several → Format → Group |

**TIP** **Dynamic titles**

Create a measure `Title Sales = "Sales by " & SELECTEDVALUE ( dim_territory[country], "all countries" ) & " in " & SELECTEDVALUE ( dim_date[year], "all years" )` and bind it in Title → fx → *Field value*. The title now reflects the user's selections.

# Filters: three levels




The three filter levels. Visual filters affect one visual, page filters one page, report filters every page.

Use the **Filters pane** (right side). Filter types:

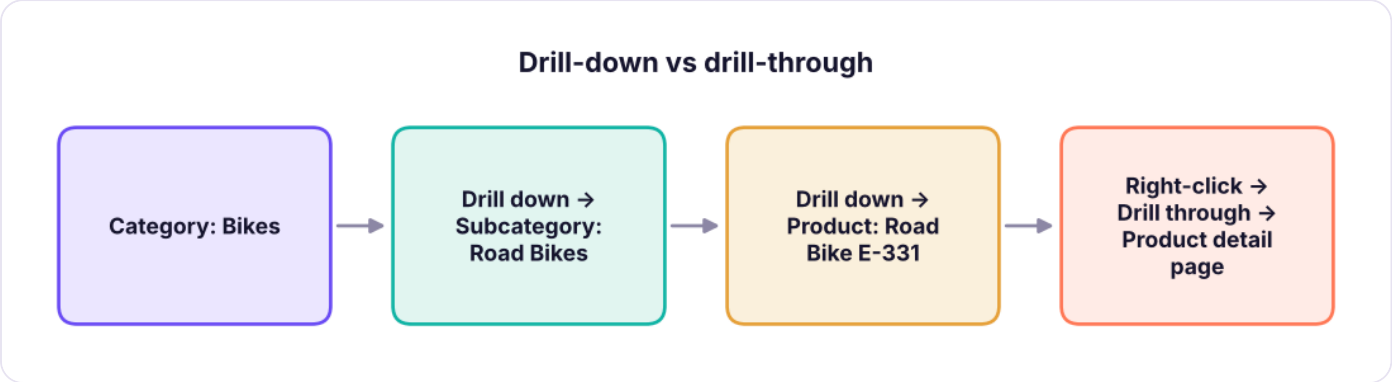
| Filter type | Behaviour |
|---|---|
| **Basic** | Tick values from a list |
| **Advanced** | "contains", "starts with", "is greater than", with AND/OR |
| **Top N** | Top/Bottom N items by a measure (even one not in the visual) |
| **Relative date** | "Last 6 months", "this year", "next 7 days" |
| **Relative time** | Last 4 hours etc. |

Tips: **Require single selection** in a slicer; **Sync slicers** across pages (View → Sync slicers); the **"Include" / "Exclude"** options on a selection filter in or out; lock the filters pane or hide it for consumers.

## Edit interactions

By default clicking a bar **cross-filters** other visuals. Select the source visual → Format → **Edit interactions** → then on each target visual click the **Filter**, **Highlight** or **None** icon. Example: a KPI card for "overall sales" should ignore the clicks on the country chart (set to None), while the product chart should filter.

# Drill-down and drill-through




Drill-down moves through a hierarchy inside one visual. Drill-through jumps to another page that is filtered by the selected item.

- **Drill-down**: needs a hierarchy (Lesson 2). Use the arrow buttons at the top of the visual: ↑ drill up, ↓ drill mode, ⇊ go to the next level in the hierarchy, ⇈ expand all.
- **Drill-through**: create a **detail page** → in the Visualizations pane, drag `product` into **Drill through → Add drill-through fields here** → Power BI adds a back button automatically. Users right-click a product in any visual → **Drill through → Product detail**. Use it for "customer 360", "order detail" and "product detail" pages. **Keep all filters** keeps slicer context.
- **Cross-report drill-through** links to a separate report in the Service.

## Report-page tooltips

A **tooltip page** shows a mini-report when hovering over a data point.

STEP BY STEP **Build one**

1. Add a new page; Format page → Page information → **Allow use as tooltip** on; Page size → **Tooltip**.
2. Add visuals (a KPI, a small trend line, top 3 products) using fields that respond to the hovered item.
3. In the main visual: Format → General → **Tooltips** → Type **Report page** → choose the page.
4. Hide the tooltip page from navigation (right-click the tab → Hide page).

## Buttons, bookmarks and navigation

| Feature | What it does | Use |
|---|---|---|
| **Buttons** (Insert → Buttons) | Back, Bookmark, Page navigation, Q&A, Web URL, Apply all slicers, Clear all slicers | Menus, navigation bars, reset filter |
| **Bookmarks** (View → Bookmarks) | Save the current state (filters, slicer selections, visibility, spotlight) of a page | Story-telling views, toggle between visuals |
| **Selection pane** (View → Selection) | List, name, show or hide every object on a page | Required for bookmark toggles |
| **Page navigator** (Insert → Navigator) | Auto-generated tab buttons for pages or bookmarks | Menu |
| **Conditional visibility** | Bookmarks that show/hide visuals | "Chart ⇄ Table" toggle |

STEP BY STEP **Chart / table toggle with bookmarks**

1. Place a column chart and a table on top of each other.
2. Selection pane: hide the table → View → Bookmarks → **Add** → name it "Chart view"; make the table visible and hide the chart → Add → "Table view". Untick **Data** in each bookmark's options so slicers are not reset.
3. Insert two buttons and set Action → **Bookmark** → choose the matching bookmark.

## Conditional formatting

| Where | How |
|---|---|
| **Table / matrix cells** | Format → Cell elements → Background colour / Font colour / **Data bars** / **Icons** / Web URL → fx → *Gradient*, *Rules* or *Field value* |
| **Column / bar colour** | Format → Colors → fx → rules (green above target, red below) |
| **Card text colour or title** | fx on the callout/title |
| **Based on a measure** | Create a measure returning a hex colour: `Margin Colour = IF ( [Profit Margin %] ≥ 0.35, "#1E8E5A", "#C0392B" )` then Field value |

| Subcategory | Sales (₹ M) | Profit (₹ M) | Margin % |
|---|---|---|---|
| Mountain Bikes | 161.9 | 54.9 | 33.9% |
| Road Bikes | 155.7 | 56.1 | 36.0% |
| Touring Bikes | 149.2 | 55.2 | 37.0% |
| Frames | 76.1 | 27.3 | 35.8% |
| Wheels | 33.4 | 10.5 | 31.3% |
| Helmets | 8.2 | 2.7 | 33.1% |
| Jerseys | 5.8 | 2.2 | 37.2% |
| Gloves | 2.7 | 0.8 | 30.8% |
| Bottles | 1.6 | 0.5 | 34.0% |

A matrix with conditional formatting: a heat-map background on sales and traffic-light colours on margin (rules: ≥ 36% green, otherwise amber/red).

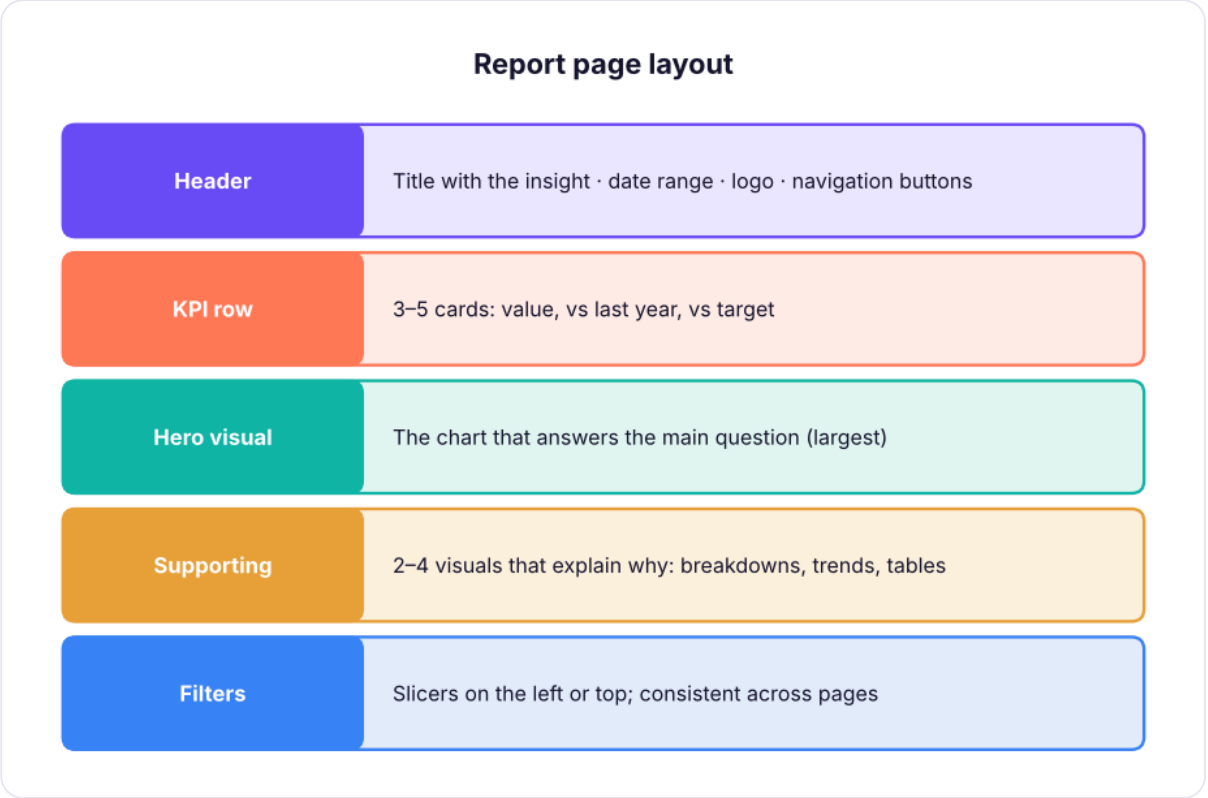
## Themes

**View → Themes** offers built-in themes; you can download community themes or **Customize current theme** (colours, fonts, defaults). A **theme JSON file** (View → Themes → Browse for themes) applies one consistent look to every visual:

```JSON
{
  "name": "SkillNavora",
  "dataColors": ["#6C4CF5", "#FF7A59", "#12B5A6", "#E9A23B", "#3B82F6", "#E04F8A"],
  "background": "#FFFFFF", "foreground": "#17142E", "tableAccent": "#6C4CF5",
  "textClasses": { "title": { "fontFace": "Segoe UI Semibold", "fontSize": 14, "color":
"#17142E" } }
}
```

Apply the company theme to every report so they look consistent, and edit colours in one place.

# Report design principles




A layout that works: headline KPIs on top, the main insight chart large, supporting visuals below, filters on one side, consistent spacing.

1. **Start from the audience and decision**, not from the data: what will they do differently?
2. **Hierarchy**: most important top-left; big numbers first; details lower.
3. **Less is more**: 5–8 visuals per page; remove gridlines, borders and chart junk.
4. **Consistent colour logic**: one brand colour plus one highlight; red and green only for bad/good; the same colour always means the same thing.
5. **Align and space** visuals on a grid; same sizes for similar charts.
6. **Label for meaning**: insight titles, units (₹, %), clear axis titles, "as of" dates.
7. **Accessible**: contrast, alt text, tab order (Format → Tab order), do not use colour alone, test on mobile layout (View → Mobile layout).
8. **Performance**: fewer visuals, summarised data, no huge tables; check Optimize → Performance analyzer.

## Q&A, images and mobile

- **Q&A visual**: users type "total sales by country in 2025" and get a chart. Improve it with **synonyms** (Model view → Linguistic schema / Q&A setup).
- **Images and logos**: Insert → Image (or text box). For images **in data** (product photos), set the column's **Data category → Image URL**.
- **Mobile layout**: View → Mobile layout; drag key visuals into a phone-shaped canvas.

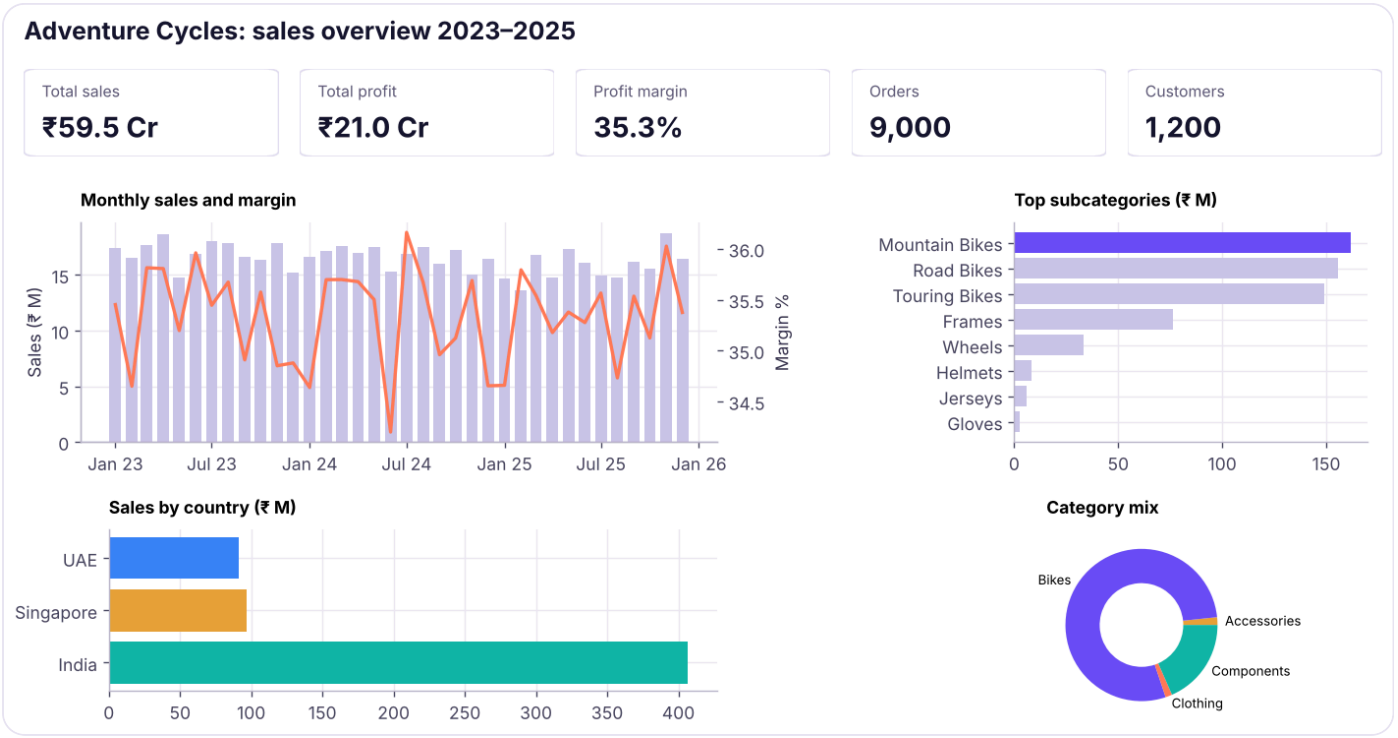
- **Accessibility and exporting**: export to PDF or PowerPoint (File → Export), analyse in Excel, subscribe in the Service.

> **2026 UPDATE** **Copilot and AI visuals in 2026**
>
> Copilot can generate a report page from a prompt, suggest visuals for your fields, write summaries ("Summarise this page") and answer questions about the data. **Key influencers**, **Decomposition tree** and **Smart narrative** are built-in AI visuals; **anomaly detection** (Analytics pane) flags unusual spikes. Always verify AI-generated visuals and text against real numbers.

## A mini case: the Adventure Cycles overview page

Plan the page before you build it:

| Element | Fields / measure | Visual |
|---|---|---|
| Title | Dynamic title measure | Text box |
| KPI row | `Total Sales`, `Total Profit`, `Profit Margin %`, `Orders`, each with vs-last-year | 4–5 cards |
| Hero | Monthly sales and profit margin | Line + column |
| Breakdown 1 | Sales by category (Top N subcategories) | Bar |
| Breakdown 2 | Sales by country | Map or bar |
| Drill-through | Product detail page | Table + KPIs |
| Filters | Year, Country, Category | Slicers |




The Adventure Cycles overview page from the real data: KPI cards, monthly trend, subcategory ranking and country share.

# Summary

**KEY TAKEAWAYS**

- Choose visuals from the question; format with insight titles, units, sorting and accessible colours.
- Filters work at **visual, page and report** level (Basic, Advanced, Top N, Relative date); use slicers, sync them across pages, and **Edit interactions** to control cross-filtering.
- **Drill-down** needs a hierarchy; **drill-through** jumps to a detail page; **tooltip pages** show mini-reports on hover.
- **Buttons + bookmarks + the Selection pane** build navigation and chart/table toggles.
- Use **conditional formatting** (rules, gradients, field value from a measure) and **themes** for consistency.
- Design for the decision: hierarchy, few visuals, consistent colour logic, alignment, labels, accessibility and mobile layout.

## Quick quiz

6 questions · tap an answer to check it

**1.** Which filter level affects every page of a report?

A Visual-level

B Page-level

C Report-level (All pages)

D Slicer-level

**Answer: C.** Report filters apply to all pages.

**2.** How do you jump from a product in a chart to a detail page for that product?

A Drill-down

B Drill-through (create a drill-through page with the product field)

C Add a slicer

D Use a tooltip

**Answer: B.** Drill-through passes the selected value as a filter to another page.

**3.** What do bookmarks capture?

A Only the page name

B The state of a page: filters, slicers, visibility and more

C The data model

D The theme

**Answer: B.** Bookmarks save a view that buttons can switch to.

**4.** Which feature stops a card from reacting when you click a bar chart?

A Sync slicers

B Edit interactions → set the card to None

C Drill-through

D Alt text

**Answer: B.** Edit interactions controls how visuals filter each other.

**5.** When is a donut chart appropriate?

A With 12 categories

B With 2–3 parts of a whole

C To show a time trend

D To compare 20 products

**Answer: B.** More than a few slices are hard to compare; use bars.

**6.** What is the benefit of a theme JSON file?

A One consistent set of colours and fonts applied to all visuals and reusable across reports

B It refreshes data

C It creates relationships

D It adds measures

**Answer: A.** Themes standardise the look and make branding quick.

EXERCISE

### Build the overview page

Using the Adventure Cycles model: (1) build the overview page above; (2) add a **Year** slicer and a **Country** slicer; (3) create a drill-through page for **Product** with sales, profit, margin and a monthly trend; (4) add a bookmarked toggle between "Sales" and "Profit" using a field parameter or bookmarks; (5) apply conditional formatting to the margin in a matrix and write a dynamic title.

**Show solution (worked answer)**

(1) Cards for the five measures; line + clustered column for monthly sales and margin; bar for top subcategories; bar for country; donut for category mix. (2) Slicer visuals with `dim_date[year]` (tile style) and `dim_territory[country]` (dropdown). (3) New page → drag `dim_product[product]` into Drill through; add a back button. (4) Modeling → New parameter → Fields → add `Total Sales`, `Total Profit`; use the parameter on the chart values; place its slicer as a toggle. (5) Matrix of `subcategory` × `country` with `Profit Margin %`; Cell elements → Background colour → Rules; title measure with `SELECTEDVALUE`.

INTERVIEW CORNER

### Questions you may be asked

**Q: How do you design an effective Power BI dashboard?** Start from the audience and decision; put headline KPIs top-left; use few visuals with insight titles; consistent colour and spacing; allow drill-down/through; test with users; check performance and accessibility.

**Q: Drill-down vs drill-through?** Drill-down navigates levels of a hierarchy within a visual; drill-through navigates to another page filtered by the selected item.

**Q: What are the filter levels and how do slicers differ?** Visual, page and report filters are set by the author in the Filters pane; slicers are on-canvas controls that users change, and they can be synced across pages.

**Q: How would you let users toggle between a chart and a table?** Two bookmarks that show/hide the visuals through the Selection pane, and two buttons linked to those bookmarks.

POWER BI · LESSON 6

# Power BI Service, Connectivity Modes, Security & Capstone

Take a report from Desktop to the people who need it. Understand Import, DirectQuery, Live connection, composite models and Direct Lake, publish to the Service, schedule refresh with a gateway, share through workspaces and apps, secure data with row-level security, tune performance, work with Copilot and Fabric, and complete the Adventure Cycles capstone.

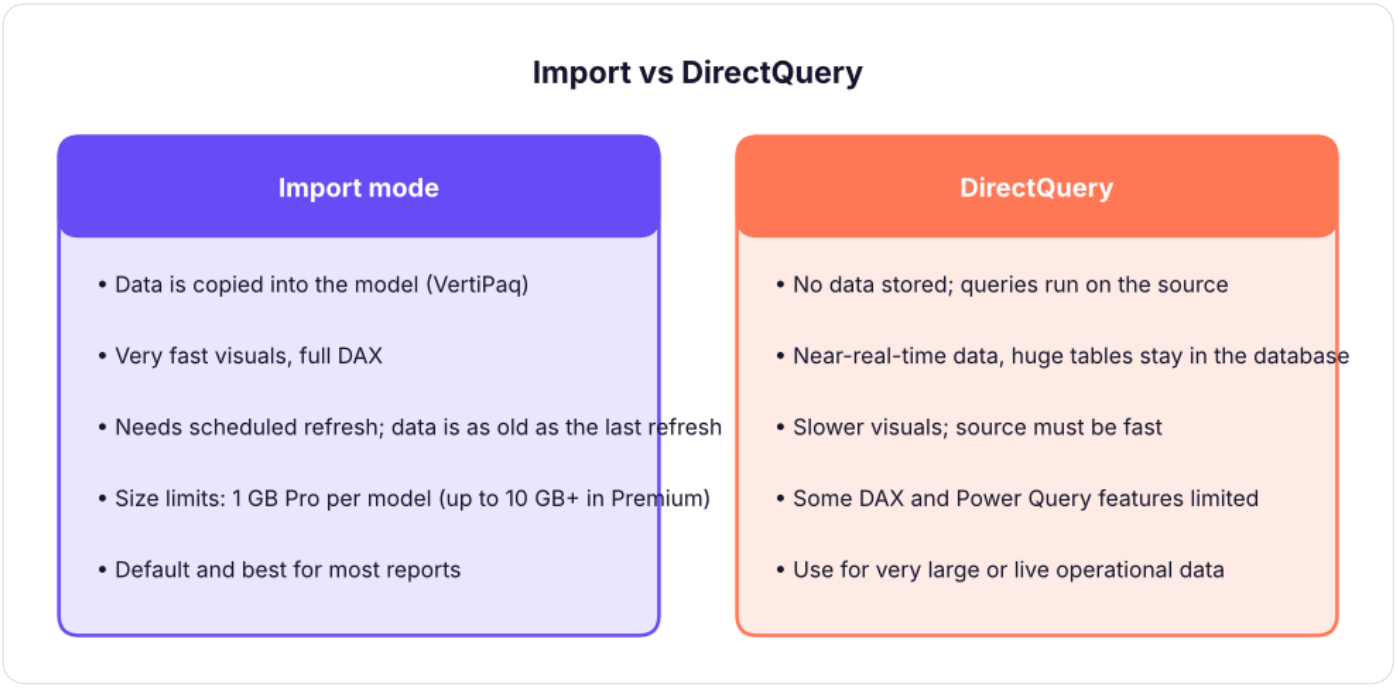
Power BI class 4 | Power BI Service & Microsoft Fabric 2026 | Advanced | 150 min

**By the end of this lesson you can**

- Compare Import, DirectQuery, Live connection, Composite and Direct Lake modes and choose one
- Publish a report, configure scheduled refresh and a gateway, and share via workspaces, apps and links
- Implement and test row-level security (static and dynamic) with DAX filters
- Use Performance Analyzer and best practices to speed up a model
- Describe Microsoft Fabric, Copilot and governance basics, and deliver an end-to-end capstone

## Connectivity modes

How does Power BI get its data: **copy it** into the model (Import) or **ask the source each time** (DirectQuery)?




Import copies data into Power BI's compressed in-memory model; DirectQuery sends a query to the source for every interaction.

| Mode | Data stored in Power BI? | Refresh | Typical use |
|---|---|---|---|
| **Import** | Yes (compressed copy) | Scheduled (up to 8/day Pro, 48/day Premium) | Most reports; Excel, CSV, SQL |
| **DirectQuery** | No | Live per interaction | Large SQL databases, near-real-time dashboards |
| **Live connection** | No: connects to an existing **semantic model** (Analysis Services or a published Power BI dataset) | Live | One governed model shared by many reports |
| **Composite model** | Mix: some tables Import, some DirectQuery, plus other semantic models | Both | Large fact in DirectQuery, small dimensions imported (dual storage mode) |
| **Direct Lake** (Fabric) | Reads Delta tables in OneLake directly | Automatic on data change | Import-like speed with no copy; Fabric only |

TIP **A practical rule**

Start with **Import**. Move to DirectQuery or Direct Lake only if data volume or freshness requirements force you to. Reduce the data first: only needed columns, aggregated tables, filtered date range.

# Publishing and the Service

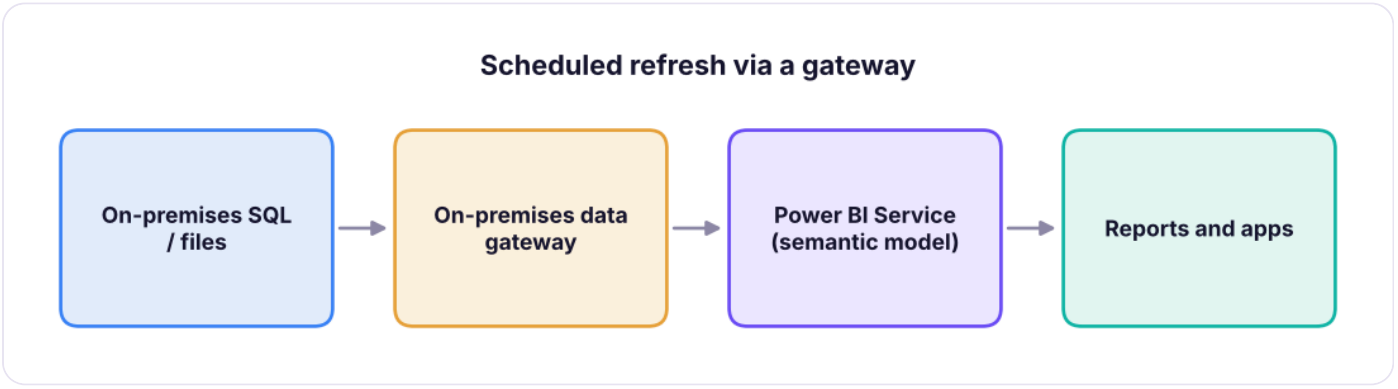
STEP BY STEP **Publish a report**

1. In Desktop: **Home → Publish** → sign in with a work/school account → choose a **workspace** (not "My workspace" if you want to share).
2. The Service now has a **semantic model** (the data and model) and a **report** (the visuals): the same objects in two items.
3. Open the report in the browser, check it, and **Share** with people or add it to an **app**.
4. Configure **scheduled refresh** (Settings → Semantic model → Scheduled refresh) with credentials; for on-premises sources install a **gateway**.

## Workspaces, apps and sharing

| Concept | Meaning |
|---|---|
| **Workspace** | A shared container for semantic models, reports, dashboards. Roles: **Admin, Member, Contributor, Viewer** |
| **App** | A packaged, read-only, curated set of reports/dashboards published to a wider audience (best way to distribute) |
| **Share link** | Give a person or group access to one report (they need appropriate licence) |
| **Dashboard** | One page of **pinned tiles** from several reports; supports alerts and mobile |
| **Subscriptions & alerts** | Email snapshots on a schedule; data alerts on card/gauge thresholds |
| **Export / Analyze in Excel / Embed** | PDF/PowerPoint export, live pivot tables on the model, secure embedding in portals |
| **Deployment pipelines** | Move content Dev → Test → Prod with controlled changes |
| **Endorsement, sensitivity labels, lineage view** | Governance: certified datasets, data classification, impact analysis |

## Refresh and gateways




A scheduled refresh for on-premises data goes through a gateway installed inside the company network.

- **Cloud sources** (Azure SQL, SharePoint Online, OneDrive files) refresh directly without a gateway.
- A **standard gateway** serves many users; a **personal gateway** is for one person's testing.
- **Incremental refresh** (right-click a table → Incremental refresh) loads only new or changed partitions (for example the last 7 days) instead of the whole fact table: essential for big tables. It needs `RangeStart` and `RangeEnd` parameters (Power Query).
- Refresh failures (expired credentials, changed schema, gateway offline) appear in the refresh history; enable failure notifications.

# Row-level security (RLS)

**RLS** restricts the rows a user can see: a country manager sees only their country.

## Static RLS

STEP BY STEP **Create roles in Desktop**

1. **Modeling → Manage roles → Create**. Name the role `India`.
2. Select the table `dim_territory` and add a DAX filter: `[country] = "India"`.
3. Create roles `UAE` and `Singapore` the same way. **Save**.
4. **View as** (Modeling → View as) → choose a role to test what that person sees.
5. Publish; in the Service: **Semantic model → Security** → add users or groups to each role.

Because filters flow from `dim_territory` to `fact_sales` through the relationship, the India role sees only India's sales everywhere:

Python

```
import pandas as pd
D = "adventure_cycles/"
f = pd.read_csv(D + "fact_sales.csv"); t = pd.read_csv(D + "dim_territory.csv"); m = f.merge(t,
on="territory_key")
role_view = m.groupby("country").agg(orders=("order_number", "count"), sales=("sales_amount",
"sum"))
role_view["sales"] = (role_view.sales / 1e6).round(1)
role_view.rename(columns={"sales": "sales (₹ M) visible to that role"}).assign(**{"all-company
total (₹ M)": round(f.sales_amount.sum() / 1e6, 1)})
```

OUTPUT

| | orders | sales (₹ M) visible to that role | all-company total (₹ M) |
|---|---|---|---|
| **country** | | | |
| **India** | 6135 | 406.9 | 594.6 |
| **Singapore** | 1487 | 96.7 | 594.6 |
| **UAE** | 1378 | 91.0 | 594.6 |

3 rows × 3 columns

## Dynamic RLS

One role for everyone, driven by a **security table** mapping user emails to territories:

DAX

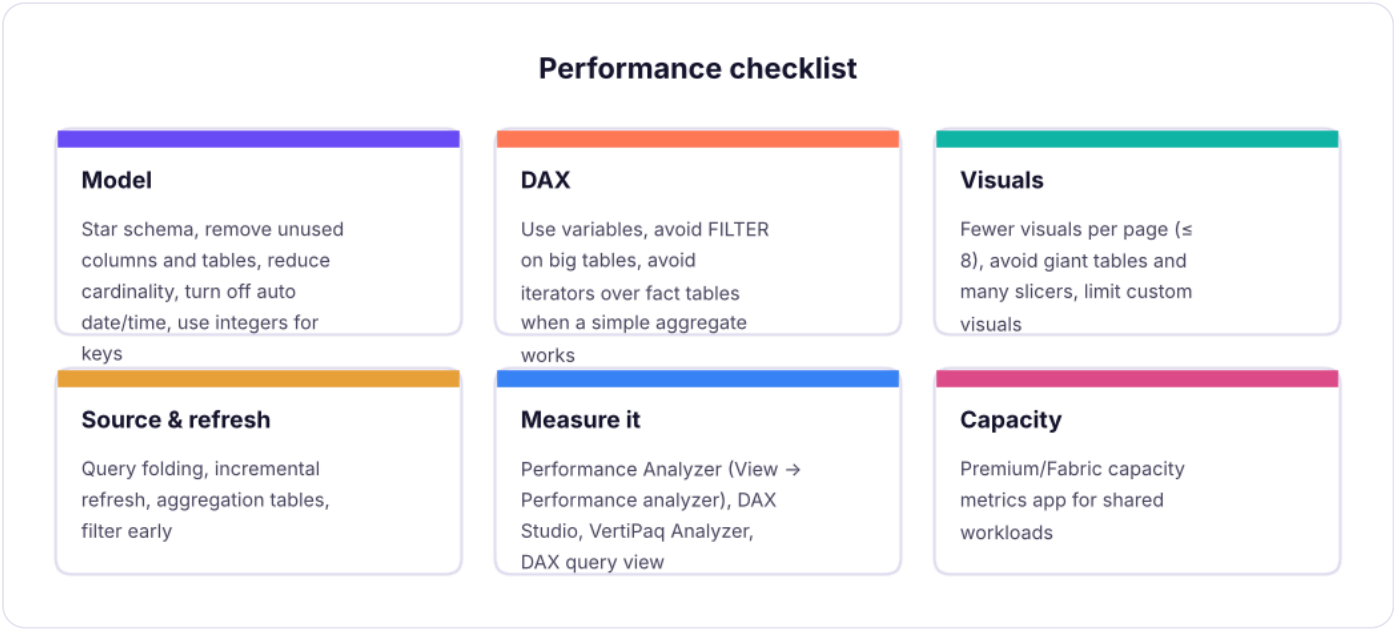
```
-- Table 'UserSecurity'(user_email, territory_key); role filter on dim_territory:
[territory_key]
    IN CALCULATETABLE ( VALUES ( UserSecurity[territory_key] ),
                        UserSecurity[user_email] = USERPRINCIPALNAME () )
```

`USERPRINCIPALNAME()` returns the signed-in user's email. Add a person to the table and they get access automatically: no role edits.

WATCH OUT **RLS pitfalls**

- RLS applies to **Viewers** only. Workspace Admins, Members and Contributors can see all data.
- RLS does not hide column names or the existence of tables; use **object-level security** for that.
- Filters must flow along relationships: apply RLS to a **dimension**, not the fact, where possible.
- RLS with DirectQuery depends on the source's behaviour; test with *View as*.
- Do not store secrets or personal data you do not need in the model. In India, handling personal data is governed by the DPDP Act 2023.

## Performance tuning




Where Power BI report time goes, and where to look.

**Performance Analyzer** (View → Performance analyzer → Start recording → refresh visuals) shows, for each visual, the time spent in **DAX query**, **Visual display** and **Other**. If DAX dominates, optimise the measure or model; if display dominates, simplify the visual.

Practical wins:

| Problem | Fix |
|---|---|
| File is huge | Remove columns you do not use; store IDs as integers; split date and time; use `Whole number`/`Fixed decimal` rather than text |
| Slow card or matrix | Rewrite measures with variables; avoid nested `FILTER ( ALL ( bigTable ) )` |
| Slow refresh | Query folding, incremental refresh, filter at source |
| Slow DirectQuery | Aggregations table (Import) in front of the big table; index the source |
| Many identical measures | Calculation groups or one base measure referenced by others |

## Microsoft Fabric, Copilot and AI

**Microsoft Fabric** unifies data engineering, data factory, data warehouse, data science, real-time analytics and Power BI over one data lake called **OneLake** (Delta/Parquet files). Power BI's semantic models can use **Direct Lake** over lakehouse tables. For data professionals: learn SQL (this track), Python (next track), and then Fabric notebooks, pipelines and lakehouses.

| Fabric item | Role |
|---|---|
| **Lakehouse / Warehouse** | Stores data (files + SQL endpoint) |
| **Dataflow Gen2** | Power Query in the cloud |
| **Notebook** | Python/Spark for data science and engineering |
| **Pipeline** | Orchestrates data movement |
| **Semantic model** | The Power BI model (formerly "dataset") |
| **Eventstream / Real-time** | Streaming data |

**Copilot in Power BI** (needs a Fabric or Premium capacity): *create a report page* from a prompt, *suggest measures* and *write DAX*, *summarise* a page or report, and *answer questions* about the data. **Prep data for AI** (add descriptions, synonyms, verified answers) improves the answers. Treat it as a fast assistant, not an authority.

## Governance and good practice

1. **One certified semantic model per subject area**, many thin reports on top (live connection).
2. Name items consistently (`Sales – Overview`, `Sales_Model`); keep an **owner** and a description.
3. Version control: **PBIP** + Git, deployment pipelines, a test workspace.
4. Document measures (description property) and keep a measure dictionary.
5. Limit **who can export** data; use sensitivity labels.
6. Monitor usage (usage metrics report) and remove unused reports.
7. Train users: the best dashboard is useless if nobody trusts or understands it.

# Capstone: Adventure Cycles performance review

CASE STUDY **The brief**

The sales director wants a 3-page Power BI report for the Adventure Cycles leadership team and country managers: **(1) Executive summary**, **(2) Products and customers**, **(3) Country detail (drill-through)**. It must show YTD and vs-last-year numbers, let managers filter by year, country and category, and each country manager must see **only their own country**.

### Deliverables checklist

| Step | What to produce |
|---|---|
| 1. Requirements | Audience, questions, KPIs, refresh frequency, security |
| 2. Data | Load the five CSV tables; clean in Power Query |
| 3. Model | Star schema, relationships, marked date table, hierarchies (Lessons 1–2) |
| 4. Measures | Sales, Profit, Margin %, Orders, AOV, YTD, LY, YoY %, Rank, Top 5 + Others (Lessons 3–4) |
| 5. Report | Three pages, theme, tooltips, drill-through, bookmarks, mobile layout (Lesson 5) |
| 6. Security | RLS roles by country, tested with *View as* |
| 7. Publish | Workspace, scheduled refresh, app, subscriptions (this lesson) |
| 8. Documentation | One-page "how to read this report" and a measure dictionary |

The analysis the report must support: verify your page numbers against this independent calculation.

Python

```
m = f.merge(t, on="territory_key").merge(pd.read_csv(D + "dim_product.csv"), on="product_key");
m["profit"] = m.sales_amount - m.total_cost
m["order_date"] = pd.to_datetime(m.order_date); m["year"] = m.order_date.dt.year
kpi = m.groupby("year").agg(sales=("sales_amount", "sum"), profit=("profit", "sum"), orders=
("order_number", "nunique"))
kpi["margin_%"] = (kpi.profit / kpi.sales * 100).round(1); kpi["YoY_sales_%"] =
(kpi.sales.pct_change() * 100).round(1)
kpi[["sales", "profit"]] = (kpi[["sales", "profit"]] / 1e6).round(1)
kpi
```

OUTPUT

| | sales | profit | orders | margin_% | YoY_sales_% |
|---|---|---|---|---|---|
| year | | | | | |
| 2023 | 204.2 | 72.2 | 3004 | 35.4 | NaN |
| 2024 | 200.4 | 70.8 | 3022 | 35.3 | -1.9 |
| 2025 | 189.9 | 67.2 | 2974 | 35.4 | -5.2 |

3 rows × 5 columns

Python

```
c = m.groupby("country").agg(sales=("sales_amount", "sum"), profit=("profit", "sum"))
c["margin_%"] = (c.profit / c.sales * 100).round(1); c["share_%"] = (c.sales / c.sales.sum() *
100).round(1); c[["sales", "profit"]] = (c[["sales", "profit"]] / 1e6).round(1)
bike_share = (m[m.category == "Bikes"].sales_amount.sum() / m.sales_amount.sum() * 100).round(1)
print(c.to_string()); print(f"\nBikes are {bike_share}% of total sales; Components {m[m.category
== 'Components'].sales_amount.sum() / m.sales_amount.sum() * 100:.1f}%.")
```

OUTPUT

```
           sales  profit  margin_%  share_%
country
India      406.9   144.1      35.4     68.4
Singapore   96.7    34.1      35.2     16.3
UAE         91.0    32.0      35.1     15.3

Bikes are 78.5% of total sales; Components 18.4%.
```

**KEY TAKEAWAYS** **What a good leadership summary says**

1. **Sales are shrinking, not growing**: ₹204.2 M (2023) → ₹200.4 M (2024, −1.9%) → ₹189.9 M (2025, −5.2%), while the margin holds steady at about 35%. The problem is **volume or demand**, not pricing or cost.
2. **Bikes** generate 78.5% of sales and **India** 68.4%, so any decline in Indian bike sales moves the whole company: a concentration risk.
3. Singapore (16.3% of sales) and the UAE (15.3%) earn the same ~35% margin as India, so growth there is as profitable as at home.
4. Recommended next steps: find where the 2025 decline comes from (by category, country and month, using the YoY measures from Lesson 4), set recovery targets per country, and track them monthly in the report.

(Verify each statement against the tables above before you present it; the numbers, not the template, are the story.)

## Summary

**KEY TAKEAWAYS**

- **Import** is fast and the default; **DirectQuery** keeps data in the source (slower, near-live); **Live connection** reuses a shared semantic model; **Composite** mixes modes; **Direct Lake** reads Fabric Delta tables directly.
- Publish to a **workspace**, schedule refresh (gateway for on-premises, incremental refresh for big facts), distribute through **apps**, and use subscriptions and alerts.
- **RLS** filters rows by role (static) or by the signed-in user (dynamic, `USERPRINCIPALNAME()`); it applies to Viewers and must be tested with *View as*.
- Tune performance with a star schema, lean columns, efficient DAX and **Performance Analyzer**.
- Fabric unifies the data stack; Copilot speeds up building; governance (certified models, pipelines, labels) keeps it trustworthy.

### Quick quiz

6 questions · tap an answer to check it

**1. Which mode stores a compressed copy of the data inside the Power BI model?**

**A** Import

**B** DirectQuery

**C** Live connection

**D** Streaming

**Answer: A.** Import loads the data into the in-memory VertiPaq model.

**2.** When is an on-premises data gateway needed?

A For any Excel file

B To schedule refresh from data sources inside a company network

C To publish reports

D To use slicers

**Answer: B.** Gateways let the cloud service reach on-premises sources.

**3.** A user with the Admin role in the workspace is also in an RLS role. What do they see?

A Only the role's rows

B All data, because RLS does not apply to Admin, Member and Contributor roles

C Nothing

D Only their own data

**Answer: B.** RLS applies to Viewers; test with View as.

**4.** What does incremental refresh do?

A Refreshes slicers only

B Refreshes only new or changed partitions of a large table

C Deletes old data

D Converts the model to DirectQuery

**Answer: B.** It speeds up refresh by loading only recent data.

**5.** Which tool shows time spent in DAX vs visual display?

A Model view

B Performance Analyzer

C Selection pane

D Bookmarks

**Answer: B.** Performance Analyzer breaks down each visual's load time.

**6.** Which function enables dynamic row-level security by user?

A `TODAY ()`

B `SELECTEDVALUE ()`

C `USERPRINCIPALNAME ()`

D `RELATED ()`

**Answer: C.** It returns the logged-in user's email for comparison with a security table.

EXERCISE

### Deploy and secure

(1) Create RLS roles for the three countries and test each with *View as*. (2) Create a dynamic RLS design using a `UserSecurity` table (write the table and the DAX filter). (3) List five changes you would make to a 2 GB model that refreshes in 45 minutes. (4) Write the checklist you would give a colleague before publishing a report to leadership.

**Show solution (worked answer)**

(1) Manage roles → filter `dim_territory[country]`; View as each role and confirm the totals equal the pandas numbers above. (2) Table: `user_email, territory_key`; filter on `dim_territory`: `[territory_key] IN CALCULATETABLE ( VALUES ( UserSecurity[territory_key] ), UserSecurity[user_email] = USERPRINCIPALNAME () )`. (3) Remove unused columns; star schema and integer keys; incremental refresh on the fact; push transformations to the source (query folding); aggregate to the needed grain; disable auto date/time. (4) Numbers reconciled with source; filters and slicers tested; titles state insights; RLS tested; refresh scheduled and tested; owner and description added; performance checked; accessibility and mobile layout checked; app published with notes.

INTERVIEW CORNER

### Questions you may be asked

**Q: Import vs DirectQuery?** Import copies and compresses data into the model (fast, needs refresh, size limits); DirectQuery queries the source live (fresh, handles huge data, slower, DAX limitations).

**Q: What is row-level security and how do you implement it?** Restricting the rows a user can see: define roles with DAX filters on tables (static), or use USERPRINCIPALNAME and a security table (dynamic); assign users in the Service; test with View as.

**Q: How do you improve a slow Power BI report?** Measure with Performance Analyzer; fix the model (star schema, fewer columns), optimise DAX, simplify visuals, use aggregations/incremental refresh, and reduce data volume.

**Q: What is a semantic model and why use live connections?** The data model (tables, relationships, measures); multiple reports can connect live to one governed model so everyone uses the same definitions.

TRACK 4

# AI for Data Analysis

Generative AI for analysts: prompting, AI vs GenAI vs agentic AI, AI over SQL (Metabase, MinusX), Copilot and Gemini in Excel, and office agents.

1. Generative AI for Analysts: How It Works, Prompting & AI vs GenAI vs Agentic AI
2. AI with SQL: Text-to-SQL, Docker, Metabase & MinusX
3. AI in Excel & Office: Copilot, Gemini, Claude, Python in Excel & Agents
4. The AI-Assisted Analyst Workflow, Agents & Governance (Capstone)

AI FOR DATA ANALYSIS · LESSON 1

# Generative AI for Analysts: How It Works, Prompting & AI vs GenAI vs Agentic AI

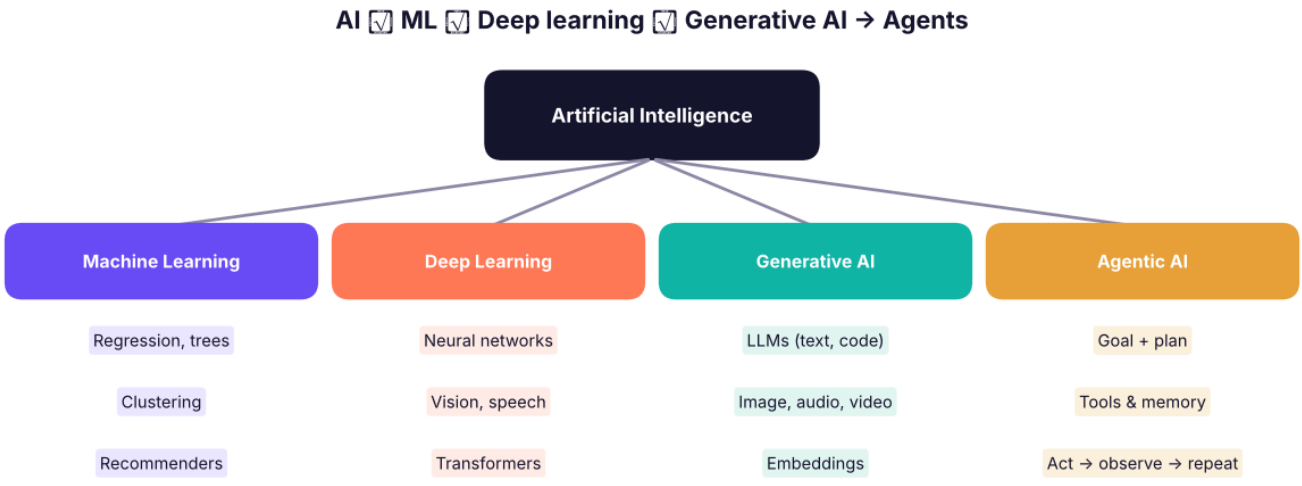
Understand what generative AI is and how large language models behave (tokens, context, temperature, hallucination), how AI, machine learning, deep learning, GenAI and agentic AI differ, and how to write prompts that give analysts accurate, checkable results, with reusable templates and privacy rules.

DVA with AI · class 1 | ✦ Models and tools as of 2026 | Beginner | ⏱ 110 min

**By the end of this lesson you can**

- Explain the relationship between AI, ML, deep learning, generative AI and agentic AI
- Describe in plain language how an LLM generates text and why it can be confidently wrong (hallucination)
- Write prompts using role, context, task, format and constraints, with examples and step-by-step reasoning
- Use prompt templates for common analyst tasks and know what never to paste into an AI tool
- Choose a suitable tool and check an AI answer before using it

## The AI family tree




How the terms relate. Each ring is a subset of the one before it. Agentic AI uses generative models as its brain but adds goals, tools and loops.

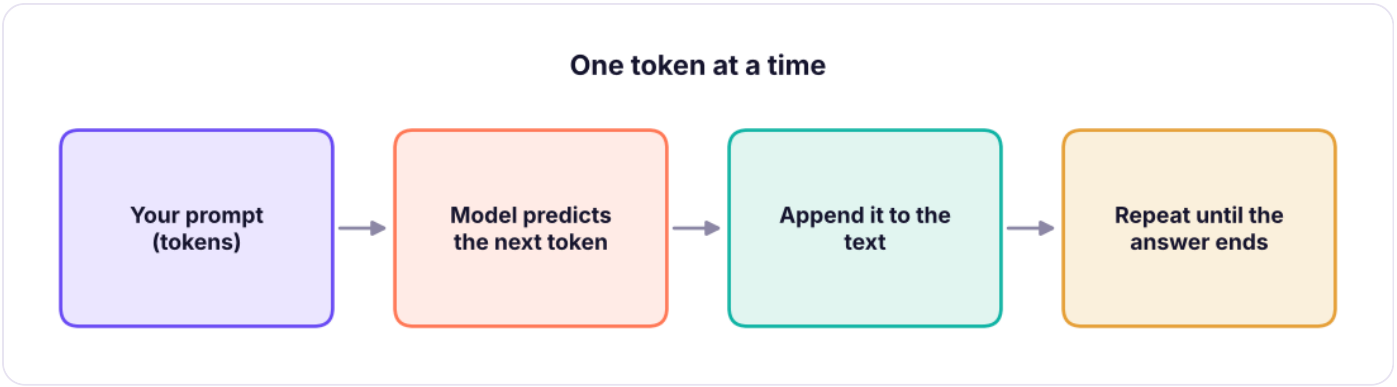
| Term | What it means | Example |
|---|---|---|
| **AI** | Machines doing tasks that normally need human intelligence | Rule-based chess engine, spam filter |
| **Machine learning (ML)** | Systems that **learn patterns from data** instead of following hand-written rules | Predict loan default from past loans |
| **Deep learning** | ML with many-layered neural networks | Image recognition, speech-to-text |
| **Generative AI (GenAI)** | Models that **create new content** (text, images, code, audio) | ChatGPT, Claude, Gemini, Copilot, Midjourney |
| **LLM** | Large language model: the GenAI engine for text and code | GPT, Claude, Gemini, Llama |
| **Agentic AI** | An LLM **plus goals, tools and a loop**: it plans, calls tools (run SQL, search, edit files), checks results and continues until the goal is met | An analyst agent that explores a database and writes the report |
| **AGI** | Hypothetical AI at human level across tasks | Not here yet; ignore the hype |

### AI vs GenAI vs agentic AI

| | Traditional ML | Generative AI | Agentic AI |
|---|---|---|---|
| Output | A prediction (number, label) | New content (text, code, image) | A completed multi-step task |
| Input | Structured data | A prompt (text, files, images) | A goal in plain language |
| Control | You train and deploy a model | You ask once, get one answer | The system decides the next step |
| Example | "Will this customer churn?" → 0.82 | "Summarise these reviews" → paragraph | "Find why sales fell in Q3 and draft a memo" → runs queries, charts, writes |
| Main risk | Bias, drift | Hallucination | Hallucination **plus actions**: wrong tool calls, runaway cost, security |

## How an LLM works (without maths)

An LLM is trained on vast amounts of text to predict **the next token** (a word or word piece). Repeating that prediction one token at a time produces sentences, code and tables. It has learnt statistical patterns of language, facts and reasoning styles, but it does **not look things up** unless it is given a tool or documents.




Generating an answer: the prompt is split into tokens; the model repeatedly predicts the next token from everything so far.

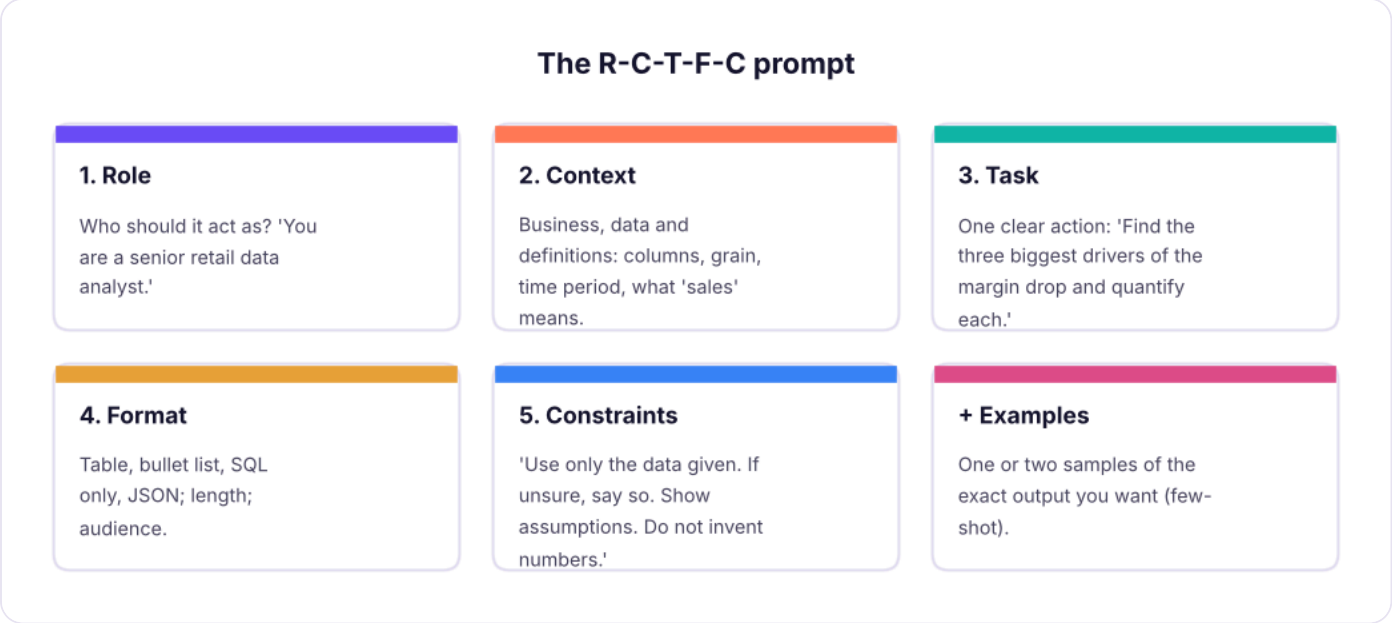
| Concept | Meaning | Why analysts care |
|---|---|---|
| **Token** | Roughly ¾ of an English word (about 4 characters) | Limits and costs are measured in tokens |
| **Context window** | How much text the model can consider at once (prompt + answer) | A long file may not fit; the model forgets what is outside |
| **Temperature** | Randomness of word choice | Low (0–0.3) for SQL, maths and extraction; higher for brainstorming |
| **System prompt** | Hidden instructions that set behaviour | Custom GPTs/Projects store your standing instructions |
| **Training cut-off** | The model's knowledge stops at some date | It may not know recent tools or prices; give it documents |
| **Grounding / RAG** | Supplying real documents or data in the prompt (Retrieval-Augmented Generation) | Cuts hallucination: "answer only from this file" |
| **Tool use / function calling** | The model asks the app to run a calculator, database query or code | Makes answers computed instead of guessed |
| **Multimodal** | Accepts images, PDFs, audio as well as text | Paste a chart screenshot and ask what it shows |

WATCH OUT **Hallucination**

Because an LLM generates *plausible* text, it can state false things with complete confidence: invented statistics, non-existent functions, wrong column names, fake citations. **Never use an AI answer as a fact until you have checked it**: run the code, recompute the number, open the source.

## Prompt engineering for analysts

A prompt is an instruction plus the material the model needs. The most reliable structure has five parts:




A prompt recipe: Role, Context, Task, Format, Constraints.

## Weak vs strong prompt

**Weak:** *"Analyse my sales data."*

**Strong:**

```text
Role: You are a senior retail data analyst for an Indian omnichannel retailer.
Context: Table superstore(order_id, order_date, region, category, sub_category, sales, discount,
profit),
         one row per order line, Jan 2023-Dec 2025, amounts in rupees. Margin = profit / sales.
Task: Explain why overall margin is only about 9%. Look at category, sub-category and discount
band.
Format: (1) a 3-bullet summary for the CFO, (2) a table of the 5 worst sub-categories (sales,
profit, margin),
        (3) the SQL Server queries you used.
Constraints: Use only the columns listed. State any assumption. If a number cannot be computed
from the table, say so.
```

The strong prompt names the grain, the metric definition, the output format and the rules for uncertainty. The result is far more likely to be right *and* checkable.

## Core techniques

| Technique | How | Use for |
|---|---|---|
| **Be specific** | Name columns, periods, units, audience | Everything |
| **Few-shot examples** | Show 2–3 input→output pairs | Classification, extraction, tone matching |
| **Step by step** | "First list the assumptions, then compute, then summarise" | Multi-step analysis; makes errors visible |
| **Ask for checks** | "Then list three ways this answer could be wrong" | Self-critique |
| **Structured output** | "Return JSON with keys..." | Feeding results into code |
| **Delimiters** | Put data in `"""triple quotes"""` or XML tags | Stops the model confusing instructions with data |
| **Iterate** | Treat the first answer as a draft; refine with follow-ups | Always |
| **Give it the schema, not the data** | Column names and 5 sample rows, not the whole file | Privacy, token limits |

## Prompt templates for common analyst jobs

| Job | Template (replace brackets) |
|---|---|
| **Explain a formula** | "Explain this Excel formula step by step for a beginner and give a simpler alternative: [formula]" |
| **Write SQL** | "Dialect: SQL Server. Tables: [schema with keys and grain]. Write a query that [question]. Return only SQL, then list assumptions." |
| **Debug code/SQL** | "This [language] query gives [wrong result / this error]. Expected: [x]. Query: [code]. Find the cause and show the fix." |
| **Clean data** | "Here are 10 sample values of column [x]: [list]. Propose normalisation rules and a mapping table; flag ambiguous cases." |
| **Interpret results** | "Here is a result table: [table]. Write 3 insights for a non-technical manager, each with a number, and 2 caveats." |
| **Choose a chart** | "I want to show [message] to [audience] with [data]. Recommend a chart and what to label." |
| **Draft a report** | "Turn these findings into a one-page memo with: situation, key findings, recommendation, risks. Keep it under 250 words." |
| **Interview practice** | "Act as an interviewer for a data analyst role; ask one SQL question at a time and give feedback on my answer." |

**EXAMPLE** **From vague to checkable**

Prompt: *"Explain this formula:* `=IFERROR(INDEX($C:$C,MATCH(1,($A:$A=F2)*($B:$B=G2),0)),"Not found")` *"* A good answer says it looks up C where A equals F2 **and** B equals G2 (a two-condition lookup), returns "Not found" if none, and notes it needs array evaluation in older Excel. You can verify by testing it on 3 rows.

## Choosing and using AI tools

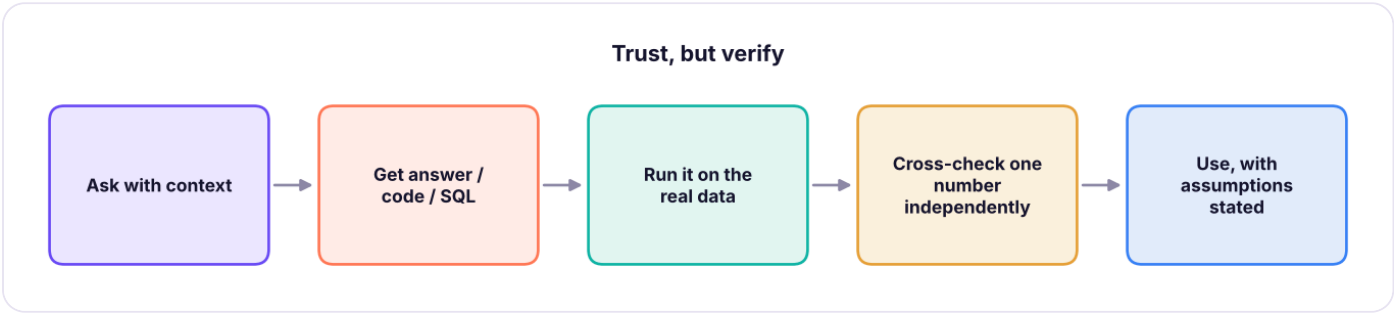
| Tool type | Examples | Good for | Watch out |
|---|---|---|---|
| **General chat assistants** | Claude, ChatGPT, Gemini, Copilot, Grok, Perplexity | Explaining, drafting, code, brainstorming | Data privacy; hallucination |
| **Search-grounded** | Perplexity, Gemini with search, Copilot with web | Fresh facts with citations | Check the cited pages; sources vary in quality |
| **Document/notebook tools** | NotebookLM, Projects, Custom GPTs | Answering **only from your files** | Quality depends on the files |
| **Embedded in apps** | Copilot in Excel/Power BI, Gemini in Sheets, Metabase/ MinusX, Claude in Excel | Work where the data is | Permissions; see next lessons |
| **Coding assistants and agents** | Claude Code, Cursor, GitHub Copilot, Codex | Writing and running code, analysis scripts | Review diffs; run tests; secrets |

## Never paste

1. **Personal data** (names with phone numbers, emails, Aadhaar/PAN, health, salary of identifiable people) unless your company's approved tool and policy allow it.
2. **Confidential business data** (unreleased financials, customer lists, contracts, source code, credentials, API keys) into public tools.
3. **Anything you are not allowed to share**: client data under NDA.

Use **enterprise versions** with data-protection terms, **mask or sample** the data (replace names with IDs, share the schema and 5 fake rows), and follow your organisation's AI policy. In India, personal data is also covered by the Digital Personal Data Protection Act, 2023.

# A verification habit for every AI answer




The analyst's rule: generate with AI, verify with data.

Here is why verification matters. Suppose an AI writes the following query for "average order value per region". It looks right, runs without error and returns plausible numbers, but it averages order **lines**, not **orders**:

Python

```
import duckdb, pandas as pd
con = duckdb.connect()
con.execute("CREATE TABLE superstore AS SELECT * FROM read_csv_auto('superstore_sales.csv')")
wrong = con.execute("SELECT region, ROUND(AVG(sales), 0) AS avg_order_value FROM superstore
GROUP BY region ORDER BY region").df()
right = con.execute("""SELECT region, ROUND(AVG(order_total), 0) AS avg_order_value
                       FROM (SELECT region, order_id, SUM(sales) AS order_total FROM superstore
GROUP BY region, order_id) GROUP BY region ORDER BY region""").df()
pd.DataFrame({"region": wrong.region, "AI query (per line)": wrong.avg_order_value, "Correct
(per order)": right.avg_order_value})
```

OUTPUT

| | region | AI query (per line) | Correct (per order) |
|---|---|---|---|
| 0 | East | 30,521 | 60,684 |
| 1 | North | 32,038 | 61,892 |
| 2 | South | 32,580 | 64,925 |
| 3 | West | 32,192 | 64,067 |

4 rows × 3 columns

The AI's version understates order value by roughly half, because an average order has about two lines. A one-line sanity check (total sales ÷ number of orders) would have caught it:

Python

```python
tot = con.execute("SELECT SUM(sales) / COUNT(DISTINCT order_id) FROM superstore").fetchone()[0]
print(f"Total sales / distinct orders = {tot:,.0f}")
```

OUTPUT

```
Total sales / distinct orders = 62,942
```

COMMON MISTAKES **Common AI-assistant failure modes**

1. **Wrong grain**: counting lines as orders, double-counting after a join (Lesson 8 in SQL).
2. **Invented columns or functions**: the code references a field that does not exist.
3. **Silent assumptions**: it picks a date range, currency or definition without saying.
4. **Plausible but unverified numbers**: especially when asked to "estimate" or "recall" data it cannot see.
5. **Over-agreeing**: if you assert something wrong, it may follow you. Ask it to challenge your assumption.
6. **Stale knowledge**: new tool versions, prices, laws.

## Responsible use

- **Accountability stays with you**: you sign off on the analysis, not the model.
- **Bias**: models mirror training data; check outputs for unfair assumptions about people or regions.
- **Transparency**: tell stakeholders when AI helped, and keep the prompt and verification notes.
- **Copyright and attribution**: do not present generated text or images as someone else's work; check licences.
- **Security**: treat prompts and uploaded files as data leaving your machine; beware prompt injection when an AI reads untrusted documents or web pages.

DO IT WITH AI **Do it with AI**

Pick a task you did earlier in this course (for example the SQL discount analysis). Ask an AI assistant to produce the answer from the schema, run its SQL, and compare with the result in this handbook. Where it differs, work out *why* (grain? date range? definition?). This single exercise teaches more about AI limits than any lecture.

## Summary

KEY TAKEAWAYS

- AI ⊃ ML ⊃ deep learning ⊃ generative AI; **agentic AI** = an LLM with goals, tools and a loop.
- LLMs predict the next token; they do not look things up unless given documents or tools. They can **hallucinate**, so verify every answer.
- Good prompts have **role, context, task, format and constraints**, use examples and step-by-step reasoning, and give the schema (not the sensitive data).
- Use templates for SQL, formulas, cleaning, insights, memos and interview practice; iterate.
- Never paste personal or confidential data into public tools; mask or sample, and follow your company policy.
- The analyst's loop: ask with context → run on real data → cross-check one number → state assumptions.

## Quick quiz

6 questions · tap an answer to check it

**1. Which statement best describes agentic AI?**

A A model that only classifies images

B An LLM-based system that plans, uses tools and repeats steps to achieve a goal

C Any chatbot

D A rule-based macro

**Answer: B.** Agents add goals, tools and a decision loop on top of a language model.

**2. Why can an LLM state wrong facts confidently?**

A It lacks memory

B It generates plausible text from patterns and does not verify facts unless given sources or tools

C It is connected to the internet

D It always copies its training data

**Answer: B.** Fluency is not accuracy.

**3. Which setting is best for generating SQL?**

A High temperature

B Low temperature

C No context

D Maximum length

**Answer: B.** Low temperature reduces random variation for precise tasks.

**4. What makes a prompt more reliable for analysis?**

A Shorter wording

B Role, context (schema and definitions), a clear task, an output format and constraints

C Asking for "everything"

D Hiding the table names

**Answer: B.** Specifics reduce guesswork and make the answer checkable.

**5. A teammate wants to paste a customer list with phone numbers into a public chatbot. Best advice?**

A Go ahead

B Do not; mask or sample the data, or use an approved enterprise tool

C Delete the email column only

D Convert it to PDF first

**Answer: B.** Personal data should not go into tools not approved for it.

**6.** **An AI query returns an "average order value" half the size of total sales ÷ orders. The likely error is:**

- **A** Wrong chart
- **B** Averaging order lines instead of order totals (wrong grain)
- **C** Too many decimals
- **D** A missing index

**Answer: B.** Always sanity-check results against a simple independent calculation.

EXERCISE **Prompt rewrite**

Rewrite this weak prompt into a strong one and list three checks you will run on the answer: *"Tell me about churn in my telecom data."* The data is `telecom_churn.csv` with columns customer_id, senior_citizen, tenure_months, contract, internet_service, payment_method, monthly_charges, total_charges, support_calls, churn (Yes/No).

**Show solution (worked answer)**

Strong prompt: "Role: senior telecom data analyst. Context: table telecom_churn(...columns...), one row per subscriber, churn = Yes/No. Task: find which contract types, tenure bands and support-call counts have the highest churn rate and quantify each. Format: a table (group, customers, churners, churn %), three bullets for a retention manager, and the SQL/pandas code used. Constraints: use only these columns, state assumptions, do not infer causes you cannot test, flag small groups (< 50 customers)." Checks: (1) recompute the overall churn rate by hand (count of Yes ÷ rows) and match it; (2) confirm each group's customer counts add up to the total; (3) spot-check one group with an independent query.

INTERVIEW CORNER

### Questions you may be asked

**Q: Difference between AI, ML and generative AI?** AI is the broad field; ML learns patterns from data to predict or classify; generative AI (a deep-learning subset) creates new content such as text, code and images.

**Q: What is a hallucination and how do you reduce it?** Confident but false output. Reduce it by grounding answers in provided documents or data (RAG), using tools that compute results, lowering temperature, asking for sources and assumptions, and verifying independently.

**Q: How do you write a good prompt for a data task?** State the role, context (schema, grain, definitions), a single clear task, the required output format, and constraints (use only given data, state assumptions); give examples; iterate.

**Q: What is agentic AI?** Systems where an LLM plans and executes multi-step tasks using tools, observes results and continues until a goal is met, with human oversight for risky actions.

AI FOR DATA ANALYSIS · LESSON 2

# AI with SQL: Text-to-SQL, Docker, Metabase & MinusX

Use AI to write, explain and debug SQL safely. Learn how text-to-SQL works and where it fails, give the model the right schema, protect your database with read-only access, install Docker, run Metabase on your own data, connect an AI assistant such as MinusX, and validate every generated query.

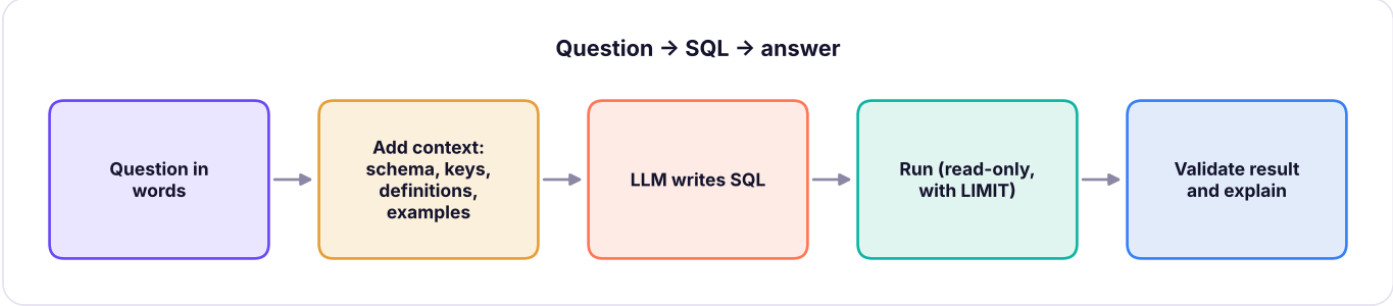
DVA with AI · class 2 | ✦ Docker, Metabase and AI assistants, 2026 | Intermediate | ⏱ 130 min

**By the end of this lesson you can**

- Explain how text-to-SQL works and why context (schema, definitions, examples) decides quality
- Write a schema-aware prompt and review generated SQL for the usual errors
- Install Docker and run Metabase locally; connect a database; build questions, models and dashboards
- Describe how an in-browser AI assistant such as MinusX works with Metabase, and its privacy trade-offs
- Apply guardrails: read-only roles, row limits, query review and result validation

## What is text-to-SQL?

**Text-to-SQL** turns a business question in plain language into a SQL query: "Which region had the highest profit in 2025?" → `SELECT region, SUM(profit) …`. LLMs are good at SQL syntax, but **correct SQL for *your* database** needs your context.




Text-to-SQL pipeline. Quality depends mostly on the context you supply, not on the model alone.

## What goes wrong without context

| Problem | Example | Prevention |
|---|---|---|
| **Wrong tables/columns** | Uses `customer_name` in `orders`, where it does not exist | Provide the schema (tables, columns, types, keys) |
| **Wrong grain / fan-out** | Joins orders to items and then `COUNT(*)` for "orders" | State the grain: "one row per order line"; add "count distinct order_id" rule |
| **Wrong definitions** | "Active customer", "net sales", "this year" | Provide a metric dictionary |
| **Dialect mismatch** | `LIMIT` instead of `TOP` in SQL Server | Name the database and version |
| **Dates** | Uses `YEAR(date) = 2025` (slow) or the wrong fiscal year | State calendar vs fiscal year and preferred patterns |
| **Hallucinated values** | `WHERE region = 'Northern'` when data says `North` | Provide sample values for categorical columns |
| **Unsafe actions** | `DELETE` / `DROP` suggestions | Read-only connection; review before running |

## A schema prompt that works

Text

```
You are a senior analytics engineer. Write SQL for Microsoft SQL Server 2022.

SCHEMA (all in dbo):
customers(customer_id PK, customer_name, segment, city, state, region)
  - segment in ('Consumer','Corporate','Home Office'); region in ('North','South','East','West')
orders(order_id PK, order_date DATE, ship_date DATE, ship_mode, customer_id FK → customers)
order_items(item_id PK, order_id FK → orders, product_id FK → products, quantity, discount,
sales, profit)
  - one row per product line; sales and profit in rupees
products(product_id PK, product_name, category, sub_category, list_price)

DEFINITIONS:
- "Orders" = COUNT(DISTINCT orders.order_id). "Margin %" = SUM(profit) / SUM(sales).
- Financial year runs April-March. Use half-open date ranges (≥ start AND < next start).

QUESTION: For FY2025 (Apr 2025-Mar 2026), show margin % by category and by discount band (0, 1-
20%, >20%).

RULES: Return only SQL; add comments; no SELECT *; list any assumptions after the query.
```

Notice the pieces: dialect, schema with keys, valid values, grain, definitions, date convention and output rules. This is the difference between a lucky answer and a reliable one.

TIP **Keep a reusable "context file"**

Save your schema, definitions and sample queries in one text file (or in an AI **Project** / custom instructions). Paste it into every session. Many BI tools (Metabase, Power BI Copilot, Snowflake Cortex, Looker) call this a **semantic layer** or "AI context": descriptions, synonyms and verified queries.

## Reviewing AI-generated SQL: a checklist

1. **Does every table/column exist?** (Run it; fix any "invalid column name".)
2. **Is the grain right?** After each join, did rows multiply? Are distinct counts used for orders and customers?
3. **Are filters correct?** Date ranges half-open? Status filters? NULL handling?
4. **Do aggregates match the definition?** `SUM(profit)/SUM(sales)` not `AVG(margin)`.
5. **Does the total reconcile?** Compare with a simple query: `SELECT SUM(sales) FROM order_items`.
6. **Is it safe and cheap?** No writes; a `TOP` / `LIMIT` while testing; no accidental cross join.
7. **Is it readable?** CTEs, aliases and comments you could explain in an interview.

Let's see this in practice. A typical AI answer to "profit margin by discount band" that forgets `SUM(profit)/SUM(sales)` and averages row margins instead:

SQL

```sql
SELECT CASE WHEN discount = 0 THEN '0%' WHEN discount ⩽ 0.2 THEN '1–20%' ELSE '>20%' END AS
discount_band,
       ROUND(AVG(profit / sales) * 100, 1) AS ai_margin_pct          -- average of row margins
(unweighted)
FROM order_items
GROUP BY CASE WHEN discount = 0 THEN '0%' WHEN discount ⩽ 0.2 THEN '1–20%' ELSE '>20%' END
ORDER BY discount_band;
```

RESULT

| discount_band | ai_margin_pct |
|---|---|
| 0% | 18.2 |
| 1–20% | 2.0 |
| >20% | -20.5 |

3 rows × 2 columns

```sql
SELECT CASE WHEN discount = 0 THEN '0%' WHEN discount ⩽ 0.2 THEN '1–20%' ELSE '>20%' END AS
discount_band,
       ROUND(100.0 * SUM(profit) / SUM(sales), 1) AS correct_margin_pct     -- weighted: total
profit / total sales
FROM order_items
GROUP BY CASE WHEN discount = 0 THEN '0%' WHEN discount ⩽ 0.2 THEN '1–20%' ELSE '>20%' END
ORDER BY discount_band;
```

RESULT

| discount_band | correct_margin_pct |
|---|---|
| 0% | 17.5 |
| 1–20% | 1.2 |
| >20% | -20.6 |

3 rows × 2 columns

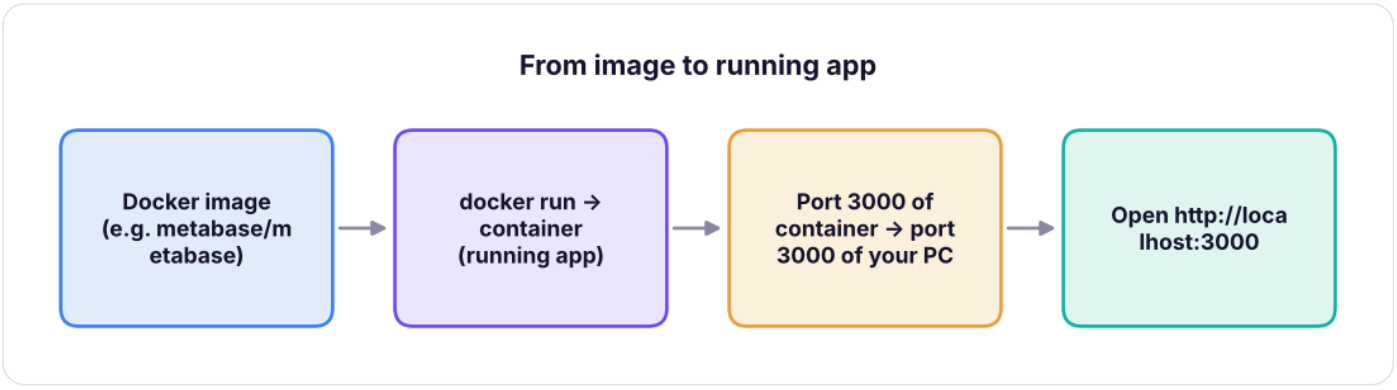
The two queries differ in every band (18.2% vs 17.5% and 2.0% vs 1.2% for the first two) because large order lines matter more in the weighted version than small ones. Which one the business wants must be written in the **definitions** part of your prompt. The checklist question "does the aggregate match the definition?" catches this.

## Guardrails for letting AI near a database

| Guardrail | How |
|---|---|
| **Read-only login** | Create a database user with `SELECT` only on approved views (`GRANT SELECT ON …`); never give AI tools `db_owner` |
| **Expose views, not raw tables** | A `vw_sales_for_ai` view with clean names, no personal data columns |
| **Row and time limits** | Timeouts, `TOP 1000`, query cost limits |
| **Masking** | Hide phone, email, ID columns; use dynamic data masking |
| **Human review** | Show the SQL before running it; log queries |
| **Verified questions** | Keep a library of approved queries for key metrics |
| **No secrets in prompts** | Connection strings and passwords stay in the tool's secret store |

## Docker: running tools in containers

**Docker** packages an application and everything it needs into a **container**, so it runs the same on any machine. Instead of installing Java, a database and an app by hand, you run one command.




An image is a read-only template; a container is a running instance of it. Many containers can run from one image.

STEP BY STEP **Install Docker Desktop**

1. Download **Docker Desktop** for Windows, Mac or Linux from the Docker website and install it. On Windows it uses **WSL 2** (Windows Subsystem for Linux); the installer enables it, and you may need to restart.
2. Start Docker Desktop and wait until it says "Engine running".
3. Open a terminal (PowerShell, Terminal) and check: `docker --version` and `docker run hello-world`.
4. Note your machine needs virtualisation enabled in BIOS and about 4 GB of free RAM for small tools.

| Command | Meaning |
|---|---|
| `docker pull metabase/metabase:latest` | Download an image |
| `docker run -d -p 3000:3000 --name metabase metabase/metabase` | Run it in the background (`-d`), map port 3000, name it |
| `docker ps` / `docker ps -a` | List running / all containers |
| `docker logs -f metabase` | Follow its logs (see start-up progress) |
| `docker stop metabase` / `docker start metabase` | Stop / start again |
| `docker rm metabase` | Delete the container (not the image) |
| `docker images` | List downloaded images |
| `docker compose up -d` | Start everything defined in a `compose.yaml` file |

## Metabase: open-source BI for everyone

**Metabase** is an open-source business-intelligence tool: connect a database, ask questions with a point-and-click **question builder** (no SQL needed), or write SQL, then save charts into **dashboards**. It is popular in start-ups because non-technical colleagues can self-serve.

STEP BY STEP **Run Metabase and connect your data**

1. Start it: `docker run -d -p 3000:3000 --name metabase metabase/metabase`. First start takes a minute or two (check `docker logs -f metabase`).
2. Open **http://localhost:3000**, create the admin account, and choose **Add your data**.
3. Pick your database type (PostgreSQL, MySQL, SQL Server, BigQuery, Snowflake, SQLite files via drivers, CSV upload in newer versions), enter host, port, database name, username and password. **Use the read-only user.** If the database also runs on your PC, use the host `host.docker.internal` from inside the container.
4. Metabase scans the tables. Browse them under **Browse data**.
5. **New → Question**: choose a table, pick columns, add **Filter**, **Summarize** (`Sum of sales` by `region`), choose a visualisation, **Save**.
6. **New → Dashboard**: add saved questions, add filters (date, region) and connect them to the question fields.

For anything you want to keep, run Metabase with a proper application database instead of the default embedded one, and mount a volume, for example:

YAML

```yaml
# compose.yaml
services:
  metabase:
    image: metabase/metabase:latest
    ports: ["3000:3000"]
    environment:
      MB_DB_TYPE: postgres
      MB_DB_DBNAME: metabase
      MB_DB_PORT: 5432
      MB_DB_USER: metabase
      MB_DB_PASS: ${METABASE_DB_PASSWORD}      # from a .env file, never hard-coded
      MB_DB_HOST: postgres
    depends_on: [postgres]
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: metabase
      POSTGRES_DB: metabase
      POSTGRES_PASSWORD: ${METABASE_DB_PASSWORD}
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:
```

## Metabase concepts

| Concept | Meaning |
|---|---|
| **Question** | A saved query (builder or SQL) with a visualisation |
| **Model** | A curated, cleaned table (a saved query with documented columns) for people to build on |
| **Metric** | A named, reusable calculation such as "Net revenue" |
| **Dashboard** | A page of questions with shared filters |
| **Collection** | Folders with permissions |
| **Segment / filters** | Reusable row filters |
| **Permissions** | Which groups see which databases and tables; row-level sandboxing (paid plans) |
| **Metabot / AI features** | Built-in AI assistance for questions and SQL in recent versions (availability depends on the edition and plan; check Metabase's current documentation) |

Try it on the retail data: connect to your SQL Server `RetailDB`, then build three questions: *Sales by region*, *Monthly sales* (summarise by `order_date: Month`), and *Top 10 customers*. Then verify each one against this handbook's tables:

```sql
SELECT region, ROUND(SUM(sales), 0) AS sum_of_sales FROM superstore GROUP BY region ORDER BY sum_of_sales DESC;
```

RESULT

| region | sum_of_sales |
|---|---|
| South | 53,952,769 |
| West | 51,958,521 |
| North | 49,018,161 |
| East | 46,484,119 |

4 rows × 2 columns

## MinusX: an AI assistant inside your BI tool

**MinusX** is an open-source AI assistant delivered as a **Chrome extension** that adds a chat sidebar to tools such as **Metabase** and **Jupyter notebooks**. Instead of copying your data into a chatbot, you work in Metabase and ask the sidebar things like *"Why did sales drop in March?"* or *"Write a query for monthly profit by category"*. According to its public description, it reads a simplified view of the page (metadata such as table and column names), then operates the tool by clicking, typing and writing queries, and lets you accept or reject its edits.

| Aspect | How it behaves | What you should still do |
|---|---|---|
| **Context** | Sees the page, selected tables/columns and the query editor | Make sure the right database and tables are selected |
| **Actions** | Writes and runs SQL, edits questions, explains results | Read the SQL before accepting |
| **Models** | Supports several LLM providers (you may supply your own API key) | Check the provider's data policy |
| **Privacy** | Vendor says raw data is not sent; metadata and your prompts are | Verify the current policy; never expose sensitive tables |
| **Cost** | Free and paid plans; model API usage may cost | Check current pricing |

STEP BY STEP

### Using an AI assistant with Metabase

1. Install the Chrome extension from the Chrome Web Store (search "MinusX") and pin it; open your Metabase tab and open the sidebar.
2. Add your model/API key in settings if required.
3. Select the database/table in Metabase so the assistant knows the context.
4. Ask a precise question ("Monthly sales and profit by category for 2025, as a line chart; sales = SUM(sales), profit = SUM(profit)").
5. Review the generated SQL, run it, check totals against a simple query, then save it as a question.
6. Ask follow-ups: "Explain the biggest drop", "Break it down by region", "Why is Furniture negative?"

WATCH OUT

### AI assistants and sensitive data

Anything the assistant can see, the model provider may receive. Use a **read-only role limited to non-sensitive views**, keep personal columns out of the connected schema, use an enterprise model agreement for company data, and keep a record of which prompts were run. If your company bans browser extensions that read pages, ask IT before installing any.

## Other ways to use AI with SQL

| Use | How |
|---|---|
| **Explain a query** | Paste the query (not the data) and ask for a line-by-line explanation and the business meaning |
| **Optimise** | Paste query + execution plan + table sizes; ask for rewrites and indexes; test each |
| **Convert dialects** | "Convert this SQL Server query to PostgreSQL; list functions that differ" |
| **Generate test data** | "Create INSERT statements for 20 fictional customers across four regions" (no real data) |
| **Write documentation** | Column descriptions, data dictionaries, comments |
| **Data quality checks** | "Write queries to find nulls, duplicates, orphans and out-of-range values for these tables" |
| **Platform AI** | Snowflake Cortex Analyst, BigQuery Gemini, Databricks Genie, SSMS Copilot, Azure Data Studio/VS Code Copilot, Fabric Copilot: all follow the same principle: context in, SQL out, you verify |

2026 UPDATE

### Trend: the semantic layer

Teams that succeed with text-to-SQL do not rely on the raw schema: they define metrics (net sales, active customer) once in a **semantic layer** (dbt Semantic Layer, Cube, LookML, Power BI semantic models, Metabase models/metrics) and let AI query *that*. The AI then picks from governed metrics instead of reinventing definitions, which removes most wrong-number errors.

## Summary

KEY TAKEAWAYS

- Text-to-SQL works when you provide **dialect, schema with keys, valid values, grain, metric definitions and date rules**.
- Review every query: columns exist, **grain** is right, filters correct, aggregates match the definition, totals reconcile, nothing writes.
- Protect the database with **read-only logins on curated views**, row limits and human review; never expose secrets or personal columns.
- **Docker** runs tools in containers (`docker run -d -p 3000:3000 metabase/metabase`); **Metabase** gives self-service questions, models and dashboards.
- An assistant such as **MinusX** adds an AI sidebar to Metabase; treat it as a fast junior analyst whose SQL you review, and check privacy terms.
- A governed **semantic layer** makes AI answers consistent.

## Quick quiz

6 questions · tap an answer to check it

**1. Why does giving the AI your schema and metric definitions improve results?**

- **A** It makes the model larger
- **B** The model can use real table and column names and your definitions instead of guessing
- **C** It speeds up the database
- **D** It removes the need to review the SQL

**Answer: B.** Context is the main driver of text-to-SQL accuracy; review is still required.

**2. Which command starts Metabase in a container on port 3000?**

- **A** `docker build metabase`
- **B** `docker run -d -p 3000:3000 --name metabase metabase/metabase`
- **C** `docker stop metabase`
- **D** `metabase install`

**Answer: B.** `-d` runs detached and `-p 3000:3000` maps the port.

**3. Which login is safest for an AI tool?**

- **A** The database owner
- **B** A read-only user restricted to curated views
- **C** The admin account
- **D** An anonymous login

**Answer: B.** Least privilege limits damage if the AI makes a mistake.

**4.** A generated query averages `profit/sales` row by row to get a margin. Why is that risky?

- **A** It is invalid SQL
- **B** It is an unweighted average of ratios and may not match the business definition SUM(profit)/SUM(sales)
- **C** It is slower
- **D** It drops NULLs

**Answer: B.** Weighted vs unweighted aggregates differ; define the metric in the prompt.

**5.** What is a semantic layer?

- **A** A type of chart
- **B** A governed set of definitions for tables, metrics and relationships that BI and AI tools use
- **C** A SQL function
- **D** A backup system

**Answer: B.** It standardises metrics so every query uses the same logic.

**6.** What should you do before saving an AI-generated Metabase question?

- **A** Nothing
- **B** Read the SQL and reconcile at least one total against a simple independent query
- **C** Share it with everyone
- **D** Delete the database

**Answer: B.** Validation protects you from plausible but wrong numbers.

EXERCISE

### Schema prompt and review

(1) Write a schema prompt for the `employees` table (HR data: emp_id, gender, age, department, job_title, education, hire_date, years_at_company, monthly_income, job_satisfaction, overtime, performance_rating, city, attrition) including at least three definitions. (2) Ask any AI assistant for a query that returns attrition % by department and overtime. (3) Review the result with the checklist and reconcile the overall attrition % (25.9%) with a simple query.

**Show solution (worked answer)**

Definitions to include: "Attrition %" = employees with attrition = 'Yes' ÷ all employees; `monthly_income` is in rupees; "tenure band" = 0–2, 3–5, 6+ years. Reconciliation query: `SELECT ROUND(100.0 * SUM(CASE WHEN attrition = 'Yes' THEN 1 ELSE 0 END) / COUNT(*), 1) FROM employees;` should return 25.9. The weighted total from the grouped result (sum of leavers ÷ sum of employees across all groups) must also equal 25.9.

INTERVIEW CORNER

### Questions you may be asked

**Q: How would you use AI to speed up SQL work safely?** Provide schema, definitions and dialect; generate and explain queries; review for grain, filters and definitions; run read-only with limits; reconcile totals; keep verified queries in a library.

**Q: What is Docker and why is it useful for data tools?** A container platform that packages apps with their dependencies so the same setup runs anywhere; useful to spin up Metabase, databases or Jupyter in one command and share reproducible environments.

**Q: What are the risks of connecting an AI assistant to a database?** Data exposure to a model provider, destructive queries, wrong answers presented as facts, and prompt injection; mitigate with read-only access to curated views, masking, review and logging.

**Q: What is Metabase?** An open-source BI tool where users can ask questions with a visual builder or SQL, build dashboards and share them, with permission control by group.

AI FOR DATA ANALYSIS · LESSON 3

# AI in Excel & Office: Copilot, Gemini, Claude, Python in Excel & Agents

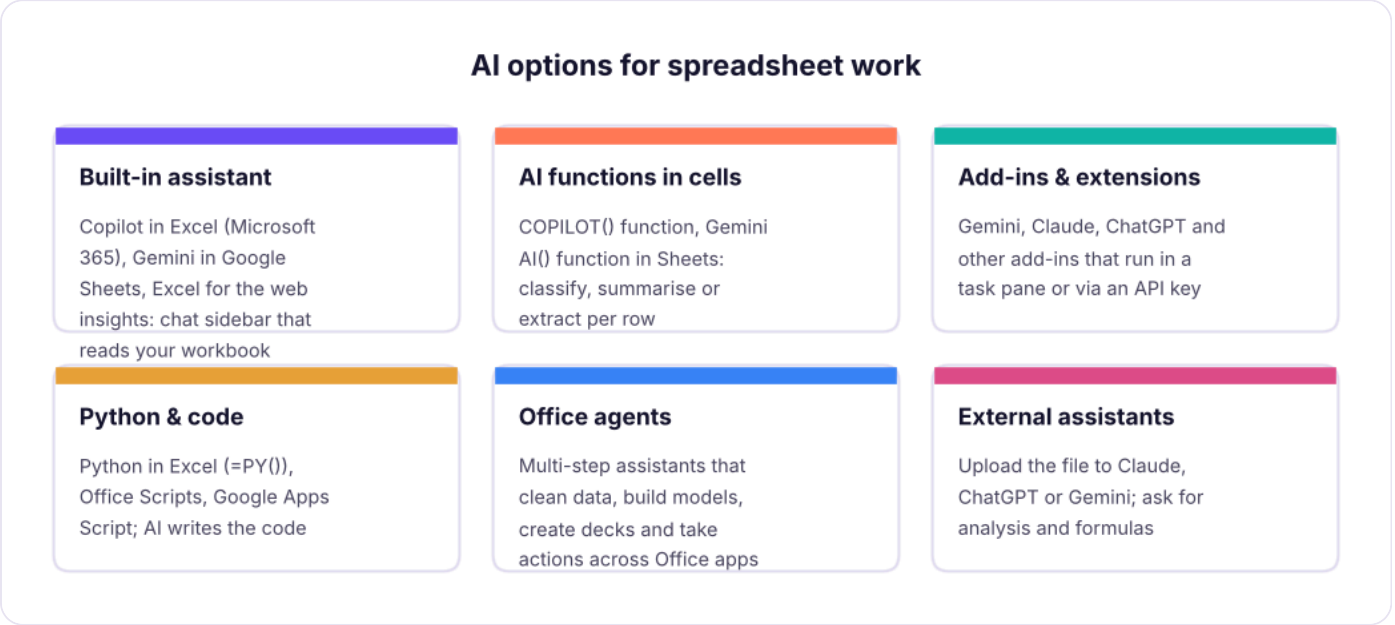
Use AI where analysts work every day. Work with Copilot in Excel (formulas, insights, charts, the COPILOT function), Gemini in Google Sheets and Excel add-ins, Claude and other assistants for formulas and files, Python in Excel, and office agents that perform multi-step tasks, and learn how to prompt, verify and stay safe.

DVA with AI · class 3 | ✦ Microsoft 365 Copilot, Gemini and agent features as of 2026 | Intermediate | ⏱ 120 min

**By the end of this lesson you can**

- Prepare a workbook so AI tools understand it (Tables, clear headers, no merged cells)
- Use Copilot and Gemini-style assistants to explain, write and debug formulas, create pivots and charts, and summarise data
- Use the Excel COPILOT function and Python in Excel for text, classification and analysis inside cells
- Describe what office agents do and what controls they need
- Verify AI-generated formulas and numbers with independent checks

# The AI-in-Excel landscape




Four ways AI shows up in spreadsheets.

## Step 0: make your workbook AI-ready

AI assistants can only reason about data they can *see and parse*. The same rules that make pivot tables work (Excel Lesson 1) make AI work:

1. **Convert data to an Excel Table** ( `Ctrl + T` ) with a clear name ( `tblSales` ) and one header row.
2. **Descriptive column names** ( `order_date` , `sales_inr` ) instead of `Col1` ; add units.
3. No merged cells, blank rows or notes inside the data; keep totals outside.
4. One sheet per dataset; a small **Data dictionary** sheet that defines columns and metrics (AI reads it too).
5. Remove or mask sensitive columns before connecting an AI service.
6. Save to **OneDrive/SharePoint** (Copilot needs cloud-saved files with AutoSave on).

## Copilot in Excel

*Needs a Microsoft 365 Copilot licence and your data formatted as a Table or range with headers.*

| Task | Example prompt |
|---|---|
| **Explain** | "Explain what column H does and how it is calculated" |
| **Formulas** | "Add a column that shows profit margin %" / "Write a formula that looks up the manager's name from the Managers table" |
| **Insights** | "Show me insights about sales by region" → Copilot adds pivot tables and charts |
| **Highlight** | "Highlight rows where discount > 20% and profit is negative" (conditional formatting) |
| **Sort / filter** | "Sort by sales descending and filter to Technology" |
| **Pivot & charts** | "Create a pivot table of sales by category and year with YoY %" |
| **Clean up** | "Fix the inconsistent city names" / "Split full name into first and last name" |
| **Analyse with Python** | "Run a correlation between discount and profit and plot it" (uses Python in Excel) |

STEP BY STEP **A safe Copilot workflow**

1. Open your workbook (saved in OneDrive) → **Home → Copilot**.
2. Select the Table (or let Copilot detect it).
3. Ask one thing at a time, with column names: *"Add a column margin_pct = profit / sales, formatted as a percentage."*
4. **Read the proposed formula** and its explanation before clicking **Insert column**.
5. Check three rows by hand, and one total against a plain `SUM`.
6. Undo (`Ctrl + Z`) if wrong, and rephrase with more detail.

## The COPILOT() function

Microsoft 365 also provides a `COPILOT` **function** that sends text and cell ranges to the model and returns results into cells that **spill and recalculate**:

Excel formula

```
=COPILOT("Classify this customer review as Positive, Neutral or Negative", D2:D100)
=COPILOT("Summarise the main complaint in 6 words or fewer", D2)
=COPILOT("Extract the city name from this address", A2)
```

Use it for text tasks (classify, summarise, extract, translate, tag) over **small to moderate** ranges, not for arithmetic: use normal formulas for numbers. Results can change between recalculations and each call consumes model capacity; availability, limits and licensing change, so check Microsoft's current documentation. Convert important results to values (copy → paste values) once checked.

Here is the same classify-sentiment idea applied to the product reviews dataset, with the (non-AI) ground-truth labels already in the file, so you can see how to *measure* an AI's accuracy:

Python

```
import pandas as pd
rev = pd.read_csv("product_reviews.csv")
print(rev.shape)
rev[["review_id", "product", "rating", "review_text", "sentiment"]].head(6)
```

OUTPUT

```
(1200, 5)
```

| | review_id | product | rating | review_text | sentiment |
|---|---|---|---|---|---|
| **0** | 1 | Speaker | 4 | Very comfortable to use all day | positive |
| **1** | 2 | Smartwatch | 5 | Very comfortable to use all day | positive |
| **2** | 3 | Smartphone | 4 | Delivery was quick and packaging was neat. great value for money, totally worth it | positive |
| **3** | 4 | Earbuds | 1 | Battery drains too fast | negative |
| **4** | 5 | Laptop | 2 | Delivery was late and the box was damaged | negative |
| **5** | 6 | Laptop | 2 | Refund process is very slow. heats up a lot while charging | negative |

6 rows × 5 columns

Python

```
# How would you evaluate an AI classifier? Compare its labels with a trusted label set.
# Here a deliberately weak keyword rule plays the role of "the AI" so you can see the evaluation method.
# (These reviews are synthetic and templated, so real reviews will be much harder to classify.)
neg_words = ("stopped", "late", "poor", "crashing")          # a deliberately small word list: an imperfect classifier
pos_words = ("excellent", "great", "amazing", "premium")
def guess(t):
    t = t.lower()
    n = sum(w in t for w in neg_words); p = sum(w in t for w in pos_words)
    return "negative" if n > p else ("positive" if p > n else "neutral")
rev["pred"] = rev.review_text.apply(guess)
acc = (rev.pred == rev.sentiment).mean()
print(f"Agreement with trusted labels: {acc:.1%}\n")
print(pd.crosstab(rev.sentiment, rev.pred, rownames=["actual"], colnames=["predicted"]))
```

OUTPUT

```
Agreement with trusted labels: 56.0%

predicted  negative  neutral  positive
actual
negative        204      206         0
neutral           0      175         0
positive          0      322       293
```

The cross-tab (a **confusion matrix**, ML track Lesson 15) shows *where* a classifier goes wrong. Do the same with an AI column: take 100 reviews you labelled yourself, compare, and only then trust it on 10,000.

## Gemini in Google Sheets and with Excel

- **Gemini in Google Sheets** (Workspace plans): a side panel that can *create tables, write formulas, summarise, generate charts and analyse trends*, plus an **`AI()` function** to run a prompt per row (classify, summarise, extract) similar to `COPILOT()`.
- **Gemini and other LLM add-ins for Excel**: some marketplaces offer add-ins or custom functions (for example `=GEMINI("prompt", A2)`) that call a model through **your own API key**. They are useful for batch text tasks, but you must protect the key, watch the **cost per call**, and avoid sending sensitive data.
- **Apps Script / Office Scripts**: AI can write the script; you review and run it.

COMMON MISTAKES **API-key hygiene for add-ins**

Never paste an API key into a shared workbook or a cell others can read; use the add-in's secure settings, set a **spending limit** on the key, and rotate it if exposed. A key in a shared file is a billing and security incident waiting to happen.

## Claude, ChatGPT and other assistants with your files

Upload a CSV or XLSX to a general assistant that can **run code** (analysis/code-execution tools). It reads the file, writes Python, runs it, and shows charts and tables: closer to an analyst than a chatbot that only talks.

A good first message with a file:

```text
I've attached superstore_sales.csv (one row per order line, Jan 2023–Dec 2025, amounts in rupees).
1) Profile the data: row count, columns and types, missing values, duplicates, date range.
2) Compute total sales, profit and margin by year and category.
3) Find the three biggest drivers of low margin; quantify each.
4) Show your Python code, and tell me which assumptions you made.
Do not guess: if something cannot be computed from the file, say so.
```

Typical good behaviour: it prints the profile, shows code, makes charts. What you still do: spot-check numbers, make sure it used the right columns, and ask for the code so you can re-run it yourself. Assistants that run code produce **reproducible** analysis; those that only "think" may fabricate numbers.

| Assistant strengths | Check |
|---|---|
| Writing and explaining formulas (even LET, LAMBDA, XLOOKUP, array formulas) | Test on sample rows |
| Cleaning plans and Power Query M code | Preview the result; keep a backup |
| Building DAX measures | Validate against a pivot |
| Summarising and drafting commentary | Verify each number quoted |
| Designing dashboards and chart choices | Compare with your audience's needs |

## Python in Excel

=PY( turns a cell into a Python editor (Formulas → Insert Python). It uses **pandas, matplotlib, seaborn, statsmodels and scikit-learn**, runs in Microsoft's secure cloud (no local Python needed), and reads Excel ranges with `xl()`.

```python
df = xl("tblSales[#All]", headers=True)          # Excel Table → pandas DataFrame
df.groupby("region")["profit"].sum().sort_values(ascending=False)
```

Ask Copilot: *"Use Python to show the correlation between discount and profit and draw a scatter plot."* Behind the scenes it writes the =PY() code. Check what it computed against this handbook's result:

```python
sup = pd.read_csv("superstore_sales.csv")
print(f"Correlation discount vs profit margin (lines): {sup.discount.corr(sup.profit / sup.sales):.2f}")
print(sup.groupby(sup.discount.map(lambda d: f"{d:.0%}")).apply(lambda g: round(g.profit.sum() / g.sales.sum() * 100, 1)).rename("margin_%").to_string())
```

OUTPUT

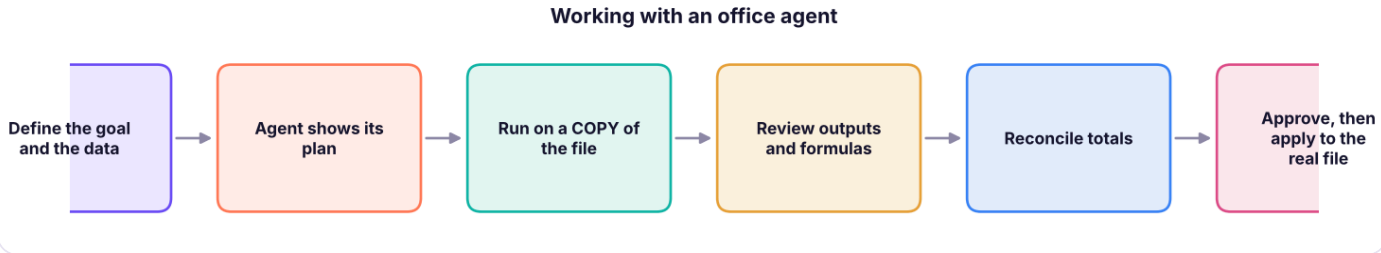
```
Correlation discount vs profit margin (lines): -0.91
discount
0%     17.5
10%     5.9
15%     0.9
20%    -4.6
30%   -19.0
40%   -26.3
```

A strong negative correlation, and margin turning negative from a 20% discount upwards: the same story as the SQL and Power BI lessons, arrived at a third way. Cross-checking tools against each other is how you build trust.

## Office agents: the next step

**Agents** go beyond answering: given a goal ("Clean this export, build a pivot by region, add a chart, and draft a three-slide summary"), they **plan, use the spreadsheet or Office apps as tools, check results and iterate**, then present the outcome for your approval. Microsoft, Google, Anthropic, OpenAI and others have all released agent modes for office work; names and capabilities change quickly, so evaluate any specific product (including whichever "Frontier"-style agent your organisation provides) against the questions below.

| Question to ask about any agent | Why it matters |
|---|---|
| What can it **read** and **write**? (files, mail, calendar, web) | Scope of risk |
| Does it **show its plan and steps** and ask before acting? | Oversight |
| Can I **undo** or review changes (version history)? | Recoverability |
| Where does data **go** (tenant, region, retention, training use)? | Compliance |
| How does it handle **untrusted content** (prompt injection in emails/web pages)? | Security |
| Does it produce **formulas and sources** or just pasted values? | Auditability |
| What does it **cost per task**? | Budget |




A safe agent workflow: small steps, visible plan, human approval at risky points, independent checks.

## What AI does well and badly in spreadsheets

| Strong | Weak / risky |
|---|---|
| Explaining unfamiliar formulas and files | Large-scale precise calculation (use formulas/Python) |
| Drafting formulas, DAX, SQL, M code, VBA | Knowing your **business rules** unless told |
| Text tasks: classify, summarise, extract, translate | Consistent results every time (non-deterministic) |
| Suggesting charts and structure | Hidden assumptions (date ranges, definitions) |
| Debugging errors (`#N/A`, `#SPILL!`) | Anything involving **sensitive data** in the wrong tool |
| Documentation and commentary | Replacing review and sign-off |

## A checklist before you trust an AI-built spreadsheet

1. Totals reconcile with the source (`SUM` of raw vs summary).
2. Row counts match (no lost or duplicated rows).
3. Formulas are **visible** (not pasted values) and use absolute references correctly.
4. Spot-check five random rows by hand.
5. Edge cases: blanks, zero, negative, text in number columns, duplicates.
6. Units and currency are labelled; dates are real dates.
7. Assumptions are written on the sheet.
8. A second tool or person confirms the headline number.

## Summary

**KEY TAKEAWAYS**

- Prepare data as **Tables** with clear headers, no merged cells, and a data dictionary; save in the cloud for Copilot.
- **Copilot** explains and writes formulas, creates pivots, charts, highlights and cleans data; `COPILOT()` and Sheets' `AI()` apply prompts per row for text tasks.
- General assistants that can **run code** make reproducible analyses from uploaded files; always request the code and assumptions.
- **Python in Excel** (`=PY()`) brings pandas into cells; AI can write it, you verify it.
- **Agents** plan and act across apps: work on a copy, require visible plans, approvals, undo, audit trail and clear data-handling terms.
- Verify with reconciliation, spot checks, edge cases and a second tool; protect keys and sensitive data.

### Quick quiz

6 questions · tap an answer to check it

**1.** What should you do first to help an AI understand your Excel data?

A Merge headers

B Convert it to a clearly named Excel Table with descriptive headers and no merged cells

C Colour the cells

D Hide the columns

**Answer: B.** Structured, well-named data is easier for AI (and pivot tables) to use.

**2.** The COPILOT() function is best used for:

A Precise financial arithmetic

B Text tasks such as classifying, summarising or extracting over a range

C Replacing SUM

D Sorting

**Answer: B.** Use normal formulas for numbers; use AI functions for language tasks.

**3.** Why ask an assistant to show its Python code?

A To make the answer longer

B So the analysis is reproducible and can be checked

C Because code is always correct

D To avoid cleaning data

**Answer: B.** Visible code lets you verify and re-run the work.

**4.** Which is a sensible control for an office agent?

A Let it edit the live file without review

B Run it on a copy, review its plan and outputs, and keep version history

C Give it all passwords

D Disable logging

**Answer: B.** Human oversight and recoverability are essential.

**5.** An AI classifier labels reviews; how do you measure its quality?

A Read three results

B Compare its labels with a trusted labelled sample and look at accuracy and the confusion matrix

C Use temperature 1.0

D Ask it if it is right

**Answer: B.** Evaluate against ground truth, not self-reports.

**6.** Where should an API key for an Excel add-in be stored?

A In a visible cell

B In the add-in's secure settings with a spending limit

C In the sheet name

D In an email

**Answer: B.** Keys in shared files can be abused and billed to you.

EXERCISE

### AI-assisted analysis, verified

Using `superstore_sales.csv`: (1) ask an AI assistant with code execution to profile the data and compute margin by category and discount band; (2) write the pivot table version yourself; (3) compare the two and list any differences; (4) create three `COPILOT()`-style prompts (or Gemini `AI()` prompts) for a column of customer comments, and say how you would evaluate them on 50 labelled examples.

**Show solution (worked answer)**

Expected margins by discount band: 0% → 17.5%, 1–20% → 1.2%, >20% → −20.6% (overall 9.1%). Differences usually come from averaging margins instead of total profit ÷ total sales, or from using order lines versus orders. Prompts: "Classify as Positive, Neutral or Negative (one word)"; "Extract the product mentioned (or 'none')"; "Summarise the complaint in under 8 words". Evaluation: label 50 comments yourself, run the prompt, compute accuracy and a confusion matrix, inspect errors, refine the prompt, and re-test on a fresh sample.

INTERVIEW CORNER

### Questions you may be asked

**Q: How would you use AI tools to speed up Excel analysis responsibly?** Prepare structured data, ask specific questions, review generated formulas and pivots, reconcile totals, document assumptions, keep sensitive data out of unapproved tools, and keep a human sign-off.

**Q: What is Python in Excel?** A feature that runs Python (pandas, matplotlib, scikit-learn) in cells via `=PY()`, in a secure cloud, reading Excel ranges with `xl()`, useful for analysis and charts that are hard with formulas.

**Q: What is an AI agent in an office context and what are its risks?** A system that plans and executes multi-step tasks using apps and files; risks include wrong actions, data leakage, prompt injection and lack of audit; mitigate with scoped permissions, approvals, copies and logs.

**Q: How do you check an AI-generated formula?** Test on a few rows including edge cases, compare totals with a simple independent calculation, read the formula's explanation, and make sure references and units are right.

AI FOR DATA ANALYSIS · LESSON 4

# The AI-Assisted Analyst Workflow, Agents & Governance (Capstone)

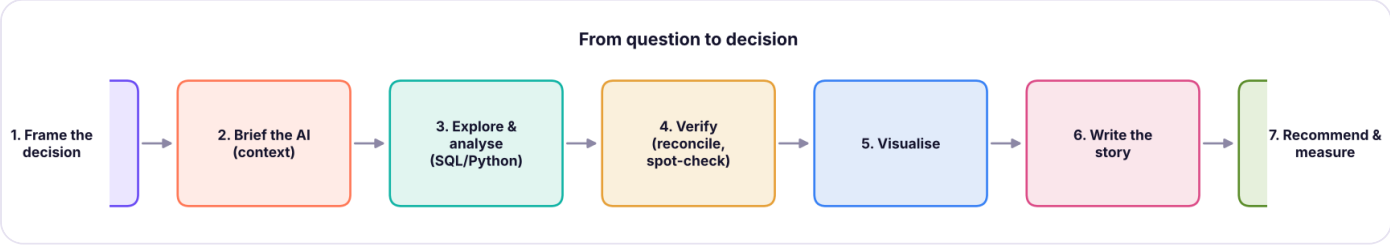
Combine everything into one repeatable workflow: frame a question, brief the AI, analyse with code, verify, visualise and write the story. See how an analyst agent works (plan, tools, observe, repeat) with a runnable toy example, learn MCP and tool-calling basics, and finish with governance, ethics and a telecom-churn capstone.

DVA with AI · wrap-up | ✦ 2026 | Intermediate | ⏱ 140 min

**By the end of this lesson you can**

- Apply a seven-step AI-assisted analysis workflow from question to recommendation
- Explain the agent loop, tool calling and the Model Context Protocol (MCP) at a conceptual level
- Build a tiny agent-style loop that plans, runs SQL tools and reports, and describe where an LLM would sit
- Design human-in-the-loop controls, logging and evaluation for AI analytics
- Complete an end-to-end churn analysis and explain it to a business audience

## The AI-assisted analyst workflow




Seven steps. AI accelerates several of them, but the analyst owns the first and the last two.

| Step | What you do | Where AI helps | Your responsibility |
|---|---|---|---|
| 1. Frame | Decision, audience, metric definitions, constraints | Brainstorm questions, critique your framing | Own the decision question |
| 2. Brief | Schema, definitions, sample rows, rules | Turn notes into a clean brief | No sensitive data; correct definitions |
| 3. Analyse | Profile, clean, aggregate, model | Draft SQL/Python, explain code, suggest checks | Run it; read it |
| 4. Verify | Reconcile totals, test edge cases, second method | Write test queries, find anomalies | Sign off the numbers |
| 5. Visualise | Choose and label charts | Suggest chart types, write plotting code | Honest, readable charts |
| 6. Write | Insight, caveat, action | Draft commentary, simplify language | Every claim traceable to a number |
| 7. Recommend | Action, owner, metric, date | Pre-mortem: "how could this fail?" | Business judgement |

## Case: why do telecom customers leave?

CASE STUDY **Brief**

A telecom company has 3,000 subscribers in `telecom_churn.csv` (tenure, contract type, internet service, payment method, charges, support calls, churn Yes/No). The retention head asks: **Who is most likely to churn, and what should we do about it?** We will follow the workflow and use AI-style assistance where it helps, checking everything.

### Steps 1–2: frame and brief

Text

```
Decision: choose which customers get a retention offer next quarter (budget: 300 offers).
Metric: churn rate = churners / customers in the group. Report groups with at least 50 customers
only.
Schema: telecom_churn(customer_id, senior_citizen 0/1, tenure_months, contract {Month-to-month,
One year, Two year},
        internet_service {Fiber, DSL, No internet}, payment_method, monthly_charges,
total_charges, support_calls, churn {Yes, No})
Rules: use only this table; state assumptions; do not claim causes; show code.
```

## Step 3: profile and analyse

Python

```
import pandas as pd, numpy as np
tc = pd.read_csv("telecom_churn.csv")
tc["churned"] = (tc.churn == "Yes").astype(int)
print(f"{len(tc):,} customers | overall churn {tc.churned.mean():.1%} | missing values:
{int(tc.isna().sum().sum())} | duplicate IDs: {int(tc.customer_id.duplicated().sum())}")
tc.describe(include="all").T[["count", "unique", "top", "mean", "min", "max"]].round(1).head(10)
```

OUTPUT

```
3,000 customers | overall churn 25.7% | missing values: 0 | duplicate IDs: 0
```

| | count | unique | top | mean | min | max |
|---|---|---|---|---|---|---|
| **customer_id** | 3000 | 3000 | T00001 | NaN | NaN | NaN |
| **senior_citizen** | 3,000 | NaN | NaN | 0.17 | 0 | 1 |
| **tenure_months** | 3,000 | NaN | NaN | 36.06 | 0 | 72 |
| **contract** | 3000 | 3 | Month-to-month | NaN | NaN | NaN |
| **internet_service** | 3000 | 3 | Fiber | NaN | NaN | NaN |
| **payment_method** | 3000 | 4 | Card | NaN | NaN | NaN |
| **monthly_charges** | 3,000 | NaN | NaN | 67.07 | 20 | 131.74 |
| **total_charges** | 3,000 | NaN | NaN | 2,418.32 | 19.58 | 8,724.35 |
| **support_calls** | 3,000 | NaN | NaN | 1.28 | 0 | 6 |
| **churn** | 3000 | 2 | No | NaN | NaN | NaN |

10 rows × 6 columns

Churn by one factor at a time (the same GROUP BY thinking as SQL Lesson 7):

Python

```
def churn_by(col, bins=None, min_n=50):
    x = tc[col] if bins is None else pd.cut(tc[col], bins)
    g = tc.groupby(x, observed=True).agg(customers=("churned", "size"), churners=("churned",
"sum"))
    g["churn_%"] = (g.churners / g.customers * 100).round(1)
    return g[g.customers ⩾ min_n]
print(churn_by("contract").to_string()); print();
print(churn_by("internet_service").to_string()); print()
print(churn_by("support_calls").to_string()); print(); print(churn_by("tenure_months", bins=[-1,
6, 12, 24, 48, 72]).to_string())
```

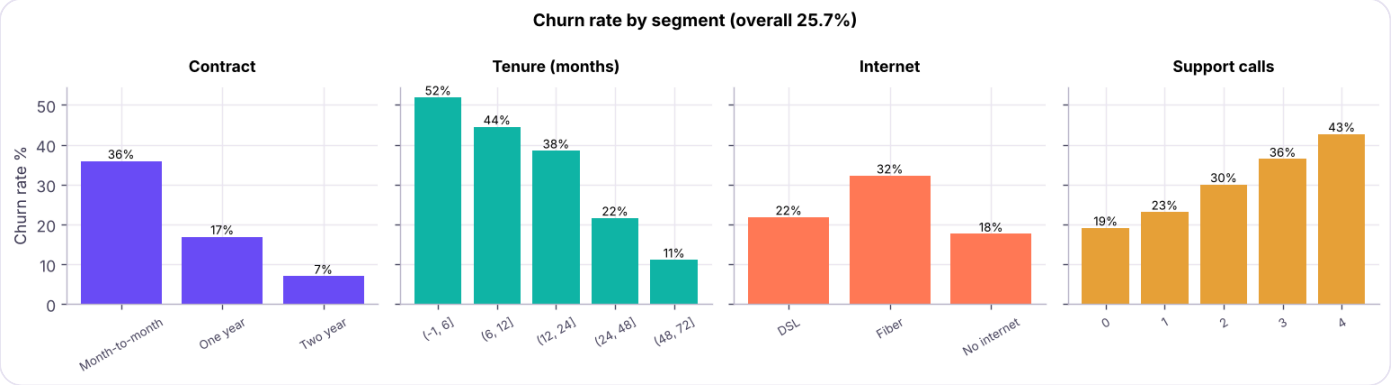
OUTPUT

```
               customers  churners  churn_%
contract
Month-to-month       1690       607     35.9
One year              739       125     16.9
Two year              571        40      7.0

                  customers  churners  churn_%
internet_service
DSL                    1047       228     21.8
Fiber                  1368       441     32.2
No internet             585       103     17.6

               customers  churners  churn_%
support_calls
0                    822       157     19.1
1                   1101       254     23.1
2                    649       195     30.0
3                    294       107     36.4
4                    110        47     42.7

               customers  churners  churn_%
tenure_months
(-1, 6]              318       165     51.9
(6, 12]              238       106     44.5
(12, 24]             452       174     38.5
(24, 48]            1011       218     21.6
(48, 72]             981       109     11.1
```




Churn rate by contract type, tenure band, internet service and support calls. Month-to-month customers, new customers, fibre users and frequent callers churn most.

## Step 4: verify before you believe

Python

```
# 1. Reconcile: group counts add up; 2. a second method gives the same answer; 3. check small groups
g = churn_by("contract", min_n=0)
print("Group customers sum to total :", g.customers.sum() == len(tc))
print("Group churners sum to total  :", g.churners.sum() == tc.churned.sum())
import duckdb
sql = duckdb.sql("SELECT contract, COUNT(*) n, ROUND(100.0*AVG(CASE WHEN churn='Yes' THEN 1 ELSE 0 END),1) churn_pct FROM tc GROUP BY contract ORDER BY contract").df()
print("SQL cross-check:\n", sql.to_string(index=False))
print("\nCombinations with fewer than 50 customers are excluded from rankings; check:", (tc.groupby(['contract','internet_service']).size() < 50).sum(), "such groups")
```

OUTPUT

```
Group customers sum to total : True
Group churners sum to total  : True
SQL cross-check:
       contract    n  churn_pct
Month-to-month 1690       35.9
      One year  739       16.9
      Two year  571        7.0

Combinations with fewer than 50 customers are excluded from rankings; check: 0 such groups
```

Associations are **not causes**: month-to-month customers may also be newer or on fibre. To look at several factors together, combine them:

Python

```
tc["tenure_band"] = pd.cut(tc.tenure_months, [-1, 12, 72], labels=["≤12 months", ">12 months"])
combo = tc.groupby(["contract", "tenure_band"], observed=True).agg(customers=("churned",
"size"), churn_pct=("churned", lambda s: round(s.mean() * 100, 1)))
combo[combo.customers ⩾ 50]
```

OUTPUT

| | | customers | churn_pct |
|---|---|---|---|
| **contract** | **tenure_band** | | |
| **Month-to-month** | **≤12 months** | 306 | 67.0 |
| | **>12 months** | 1384 | 29.0 |
| **One year** | **≤12 months** | 151 | 33.8 |
| | **>12 months** | 588 | 12.6 |
| **Two year** | **≤12 months** | 99 | 15.2 |
| | **>12 months** | 472 | 5.3 |

6 rows × 2 columns

## Steps 5–7: prioritise and recommend

With a budget of 300 offers, focus on the **highest-risk group that is also large enough to matter**. A simple **risk × size** table:

Python

```
seg = tc.groupby(["contract", "internet_service"], observed=True).agg(customers=("churned",
"size"), churn_pct=("churned", "mean"), avg_charges=("monthly_charges", "mean"))
seg["expected_churners"] = (seg.customers * seg.churn_pct).round(0)
seg["churn_pct"] = (seg.churn_pct * 100).round(1); seg["avg_charges"] = seg.avg_charges.round(0)
seg = seg[seg.customers ⩾ 50].sort_values("expected_churners", ascending=False)
seg.head(6)
```

OUTPUT

| | | customers | churn_pct | avg_charges | expected_churners |
|---|---|---|---|---|---|
| **contract** | **internet_service** | | | | |
| **Month-to-month** | **Fiber** | 743 | 43.5 | 87 | 323 |
| | **DSL** | 617 | 30.8 | 61 | 190 |
| | **No internet** | 330 | 28.5 | 31 | 94 |
| **One year** | **Fiber** | 341 | 26.7 | 86 | 91 |
| | **DSL** | 252 | 11.1 | 61 | 28 |
| **Two year** | **Fiber** | 284 | 9.5 | 87 | 27 |

6 rows × 4 columns

**KEY TAKEAWAYS** **A recommendation a manager can act on**

1. **Target** the top segments by *expected churners*, not only by churn rate: they hold most of the preventable loss.
2. **Offer** a discounted switch to a 1-year contract for month-to-month customers in their first year, and a service check for fibre customers with 2+ support calls.
3. **Measure** with an A/B test: random half of the target list gets the offer; compare churn after 90 days (the test design is in the ML track, hypothesis-testing lessons).
4. **Limit**: this analysis is descriptive; it does not prove that contracts *cause* churn or estimate the offer's effect.

## How an AI agent does analysis

An agent turns the workflow above into a loop that an LLM drives:




The agent loop. The model decides the next action, a tool executes it, and the result goes back into the model's context.

| Piece | Meaning | Analytics example |
|---|---|---|
| **Goal** | What to achieve | "Find the top churn drivers and draft a memo" |
| **Tools** | Functions the model can call | `run_sql(query)`, `run_python(code)`, `plot(spec)`, `search_docs(text)` |
| **Memory / context** | What it has seen so far | Previous queries and results |
| **Planner (LLM)** | Chooses the next action | "Check churn by contract next" |
| **Guardrails** | Limits | Read-only DB, max queries, no network, human approval for writes |
| **Evaluation** | How you know it worked | Unit tests on metrics, golden questions, human review |

## A toy agent you can run (no LLM)

To make the loop concrete, here is a miniature "analyst agent" in which a hard-coded planner stands in for the LLM. It has **one tool** (read-only SQL) and a rule: stop when it has found the top three churn drivers.

Python

```python
import duckdb
db = duckdb.connect(); db.execute("CREATE TABLE tc AS SELECT * FROM
read_csv_auto('telecom_churn.csv')")

def run_sql(q):                                  # TOOL: read-only SQL with a row limit
    assert q.strip().lower().startswith("select"), "only SELECT statements are allowed"
    return db.execute(q).df().head(20)

def planner(memory):                             # stands in for the LLM: decides the next tool
call
    todo = [c for c in ["contract", "internet_service", "payment_method", "support_calls"] if c
not in memory]
    if todo and len(memory) < 4:
        c = todo[0]
        return f"SELECT {c} AS grp, COUNT(*) AS n, ROUND(100.0*AVG(CASE WHEN churn='Yes' THEN 1
ELSE 0 END),1) AS churn_pct FROM tc GROUP BY {c} HAVING COUNT(*) ⩾ 50", c
    return None, None

memory, log = {}, []
while True:
    q, key = planner(memory)
    if q is None: break
    res = run_sql(q); memory[key] = res; log.append(f"ran query on '{key}' → best group:
{res.sort_values('churn_pct').iloc[-1].grp} ({res.churn_pct.max()}%)")
print("\n".join(log))
spread = {k: v.churn_pct.max() - v.churn_pct.min() for k, v in memory.items()}
print("\nDrivers ranked by spread in churn rate (percentage points):")
for k, v in sorted(spread.items(), key=lambda x: -x[1]): print(f"  {k:18s} {v:5.1f}")
```

OUTPUT

```
ran query on 'contract' → best group: Month-to-month (35.9%)
ran query on 'internet_service' → best group: Fiber (32.2%)
ran query on 'payment_method' → best group: Net banking (28.5%)
ran query on 'support_calls' → best group: 4.0 (42.7%)

Drivers ranked by spread in churn rate (percentage points):
  contract            28.9
  support_calls       23.6
  internet_service    14.6
  payment_method       5.0
```

A real agent replaces `planner` with an LLM call that reads the memory and writes the next SQL, adds many more tools, and decides when to stop. Everything else (read-only guard, row limit, log) is the same safety scaffolding you should demand from any agent.

## Tool calling and MCP

- **Tool (function) calling**: the model returns a structured request such as `{"tool": "run_sql", "arguments": {"query": "SELECT …"}}`; your code runs it and sends the result back.

- **Model Context Protocol (MCP)** is an open standard for connecting AI applications to **tools and data sources** through a common interface, so one connector (a "server") for your database, files or SaaS app works with many AI clients. For analysts it means an assistant can query a **governed** database or semantic layer through an approved server instead of you pasting data around.
- **RAG (retrieval-augmented generation)** gives the model *documents* (metric definitions, past reports) to ground its answers (GenAI track).
- **Multi-agent systems** split work among specialised agents (a SQL agent, a chart agent, a reviewer); more moving parts mean more to test.

## Evaluating AI analytics

| What to evaluate | How |
|---|---|
| **Correctness of numbers** | A set of "golden questions" with known answers (total sales by year, churn by contract) run on every change |
| **SQL quality** | Execution accuracy (does the result match the reference result?), not just text similarity |
| **Faithfulness of narrative** | Every number in the text must appear in the tables; automatic check by extracting numbers |
| **Safety** | Attempted writes blocked, personal data not exposed, prompt-injection tests |
| **Cost and latency** | Tokens, tool calls and time per task |
| **Usefulness** | Do users make better decisions? Track adoption and feedback |

## Governance, ethics and your career

- **Policy**: know your organisation's approved tools, data-classification rules and retention.
- **Human in the loop** for decisions that affect people (credit, hiring, pricing, health): AI assists; accountable humans decide.
- **Fairness**: test whether results differ unfairly across groups (gender, age, region) before acting.
- **Explainability**: record the data, prompt, code and checks behind each insight; be able to re-run it.
- **Skills that stay valuable**: asking the right question, domain knowledge, data modelling, SQL/Python fluency to *read and verify* AI output, statistics and experimentation, storytelling and stakeholder management. The analysts who thrive are those who use AI for speed **and** can prove their answers.

2026 UPDATE

### What is changing in 2026

Analytics agents are moving into the BI tools themselves (Copilot in Power BI/Fabric, Gemini in Looker and BigQuery, Snowflake Cortex, Databricks Genie, Metabase AI features). Vendors compete on accuracy with governed semantic layers, so expect "ask a question of certified metrics" to become a standard feature. Learn the concepts here and re-check each product's current capabilities, pricing and data-handling terms before adopting it.

# Summary

**KEY TAKEAWAYS**

- Follow a **seven-step workflow**: frame, brief, analyse, verify, visualise, write, recommend; AI accelerates but you own framing, verification and judgement.
- Always **verify**: reconcile totals, cross-check with a second method, exclude tiny groups, and remember associations are not causes.
- An **agent** = goal + LLM planner + tools + memory + guardrails in a loop; tool calling and **MCP** connect models to tools and governed data.
- Evaluate AI analytics with golden questions, execution accuracy, faithfulness checks, safety tests and cost tracking.
- Keep humans accountable, log the work, check fairness and follow data policy.

## Quick quiz

6 questions · tap an answer to check it

**1. In the agent loop, which part executes a SQL query?**

A The planner

B A tool

C The memory

D The prompt

**Answer: B.** The LLM plans; tools perform actions such as running SQL.

**2. What is MCP?**

A A chart type

B An open protocol for connecting AI applications to tools and data sources

C A database engine

D A prompt format

**Answer: B.** The Model Context Protocol standardises how AI clients use external tools and data.

**3. Why report churn only for groups with at least 50 customers?**

A To shorten the report

B Small groups give unstable percentages that can mislead

C Because SQL requires it

D To hide data

**Answer: B.** Small samples produce noisy rates.

**4. Which statement about our churn findings is correct?**

A Contracts cause churn

B Month-to-month customers churn more, but this is an association that needs testing before claiming cause

C Fibre causes churn

D Support calls prove dissatisfaction

**Answer: B.** Descriptive analysis shows patterns, not causes; use an experiment to test an intervention.

**5. What is a "golden question" set?**

A Questions that cost the most

B Questions with known correct answers used to test an AI analytics system after each change

C A list of dashboards

D Training data

**Answer: B.** They enable regression testing of answers.

**6.** Which control best limits damage if an agent writes a bad query?

A A longer prompt

B A read-only database role with row limits and logging

C A bigger model

D A louder warning

**Answer: B.** Technical guardrails beat instructions alone.

EXERCISE **Run the workflow on a new question**

Using `superstore_sales.csv`, follow the seven steps for: *"Which customer segment × region combinations should sales focus on next quarter?"* Write the brief, run the analysis (use groupby or SQL), verify with two methods, produce one chart and write a five-sentence recommendation that includes one limitation.

**Show solution (worked answer)**

Brief: define "focus" (for example profit, not just sales; minimum 100 order lines). Analysis: group by segment and region, compute sales, profit, margin and lines; filter groups with ≥ 100 lines. Verify: group totals equal the overall totals; repeat with SQL; check one group manually. Chart: bubble or bar of profit by segment × region. Recommendation example: "Focus on [top profit segment-region] (profit ₹X, margin Y%); protect [healthy group]; reduce discounts in [low-margin group]. Limitation: three years of data, no cost-to-serve or competitor information; profits include discounts that may not repeat."

INTERVIEW CORNER

### Questions you may be asked

**Q: Walk me through how you use AI in your analysis workflow.** I frame the decision, give the AI a clear brief with schema and definitions, use it to draft and explain SQL or Python, run everything myself, reconcile totals and cross-check with a second method, use AI for chart and narrative drafts, then I validate each number and take responsibility for the recommendation.

**Q: What is an AI agent and how is it different from a chatbot?** A chatbot responds once to a prompt; an agent pursues a goal across multiple steps by calling tools, observing results and deciding what to do next, so it needs stronger guardrails and evaluation.

**Q: How do you stop an AI analytics tool from giving wrong numbers?** Governed semantic layer, read-only access to curated data, golden-question tests, execution-based evaluation, human review of important outputs, and logging.

**Q: A model says "contract type drives churn". What do you say?** The data shows an association; to claim causation we would need an experiment or a causal method, controlling for tenure and other factors.

TRACK 5

# Python for Data Science

Python from zero: data types, control flow, data structures, functions, OOP, files, NumPy, pandas, visualisation, dates, SQL connections, EDA and DSA for data roles.

1. Introduction to Python: Why, Install & Jupyter Notebooks
2. Variables, Data Types, Type Conversion, print(), f-strings & input()
3. Operators, Strings, Indexing, Slicing & String Methods
4. Control Flow: if, elif, else, Nested Conditions & match
5. Loops: while, for, range, break, continue & Patterns
6. Lists: Creating, Slicing, Methods, Sorting & Comprehensions
7. Tuples & Sets: Immutable Records, Unpacking and Set Algebra
8. Dictionaries: Key-Value Data, Counting and Grouping
9. Functions: Reusable Code, Arguments, Scope and Docstrings
10. Comprehensions, Lambda, map/filter and Generators
11. Object-Oriented Python: Classes, Objects, Inheritance and Dataclasses
12. Files, Exceptions, JSON and CSV
13. Modules, Packages, Virtual Environments and Git
14. Debugging, Testing, Regular Expressions and Code Quality
15. NumPy: Fast Arrays, Vectorisation and Broadcasting
16. pandas I: Series, DataFrames, Reading Data, Selecting and Filtering
17. pandas II: Data Cleaning: Missing Values, Types, Duplicates, Text and Outliers
18. pandas III: GroupBy, Pivot Tables, Merge/Join and Reshape (melt/unpivot)
19. pandas IV: Dates, Time Series, Resampling and Rolling Windows
20. Data Visualisation I: Matplotlib and Choosing the Right Chart
21. Data Visualisation II: Seaborn Statistical Charts and Interactive Plotly
22. Exploratory Data Analysis (EDA) Case Study: Telecom Customer Churn
23. Python and SQL: sqlite3, DuckDB, SQLAlchemy and pandas.read_sql
24. APIs, JSON and Web Scraping with requests and BeautifulSoup
25. Performance and Bigger Data: Vectorisation, Memory, Parquet, Polars and DuckDB
26. DSA I: Big-O, Arrays, Strings, Hashing, Two Pointers and Sliding Window
27. DSA II: Stacks, Queues, Linked Lists, Recursion, Sorting and Binary Search
28. DSA III: Trees, Heaps, Graphs (BFS/DFS) and Dynamic Programming
29. Python Capstone: End-to-End Retail Sales Analytics Project

PYTHON FOR DATA SCIENCE · LESSON 1

# Introduction to Python: Why, Install & Jupyter Notebooks

Learn what programming and Python are, how Python compares with other languages, how code runs (interpreters, compilers, bytecode), the properties of Python, and set up your environment (Anaconda, VS Code, Jupyter, Google Colab). Write and run your first cells, with markdown, magic commands and good notebook habits.

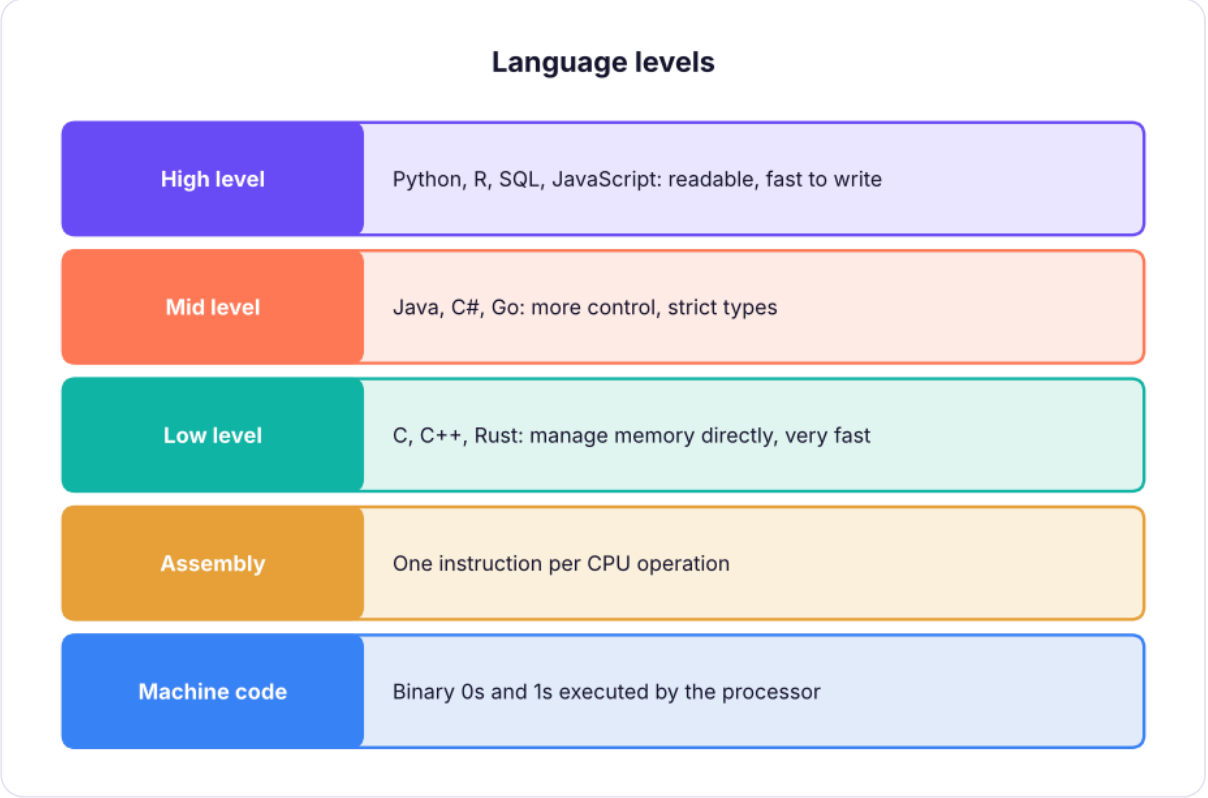
Python class 1 | Python 3.13, Jupyter, VS Code, Google Colab (2026) | Beginner | 90 min

**By the end of this lesson you can**

- Explain what a programming language is and why Python is the default language for data work
- Describe the difference between compiled and interpreted languages and how Python runs your code
- Install Python and Jupyter (or use Google Colab) and run code in cells
- Use Markdown cells, keyboard shortcuts, magic commands and restart-and-run-all
- Follow good habits: naming, comments, reproducibility and virtual environments

## What is programming?

A **program** is a set of precise instructions a computer follows. Computers only understand machine code (0s and 1s), so we write instructions in a **programming language** that humans can read, and a translator turns them into something the machine can run.



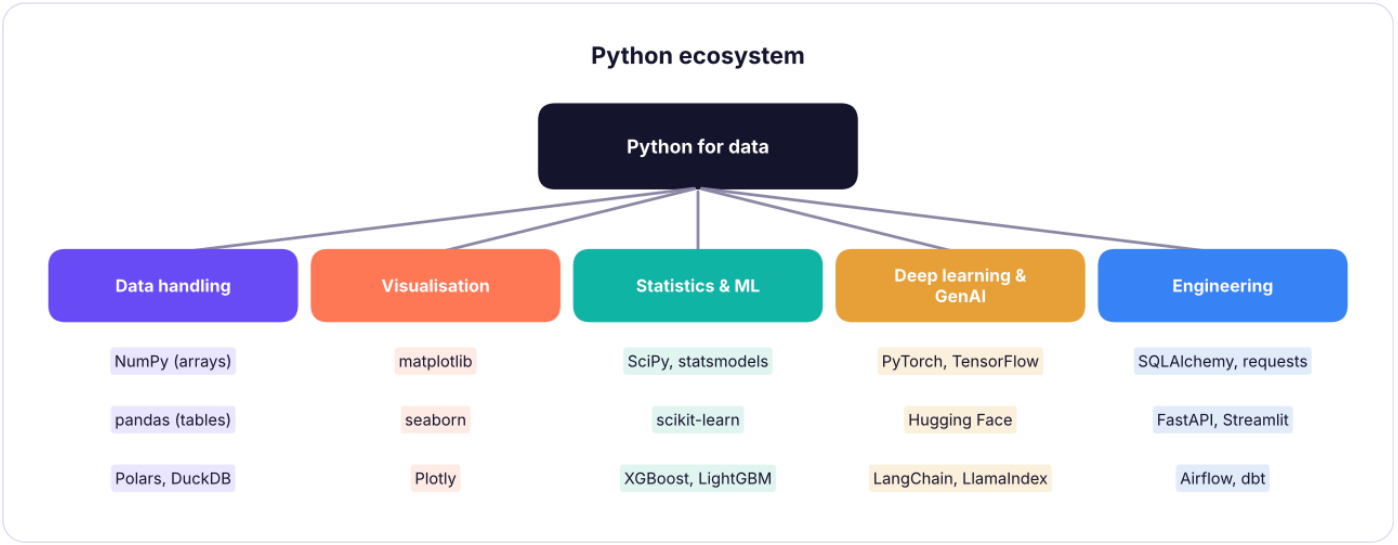

Languages sit on levels. High-level languages like Python are close to human language; low-level ones are close to the machine.

## Compiled vs interpreted

| | Compiled (C, C++, Rust, Go) | Interpreted (Python, JavaScript, R) |
|---|---|---|
| How it runs | Whole program translated to machine code **before** running | Translated and executed **line by line** while running |
| Speed | Usually faster at run time | Slower per operation, much faster to develop |
| Errors | Many caught before running | Some appear only when that line runs |
| Output | A separate executable file | Source file runs under an interpreter |

Python is usually called **interpreted**. In reality CPython (the standard version) first compiles your code to **bytecode** ( `.pyc` files) and then a virtual machine executes it. Python code that needs speed calls libraries written in C (NumPy, pandas), so data work is fast anyway.

# Why Python for data?




The Python data ecosystem. You will learn each of these libraries in this track.

| Property of Python | Meaning |
|---|---|
| **Readable** | Indentation and English-like keywords; fewer lines than most languages |
| **Interpreted, dynamically typed** | No compile step; variables take the type of the value they hold |
| **High level** | Memory is managed for you (garbage collection) |
| **Multi-paradigm** | Procedural, object-oriented and functional styles |
| **Huge ecosystem** | 500,000+ packages on PyPI; pandas, scikit-learn, PyTorch lead data and AI |
| **Open source and portable** | Free; runs on Windows, Mac, Linux, cloud notebooks |
| **Great glue** | Calls SQL, APIs, C/C++ code, Excel, Spark |

**NOTE** **Python versions**

Use Python **3.11, 3.12 or 3.13** (the current stable series in 2026). Python 2 ended in 2020 and must not be used. Check your version with `python --version`.

## Getting set up: three options

| Option | Best for | Steps |
|---|---|---|
| **Google Colab** (free, in the browser) | Starting today, no installation, GPU for ML | Go to colab.research.google.com → New notebook → sign in with Google |
| **Anaconda / Miniconda** | Full local data-science setup with packages and environments | Install Anaconda Distribution → open **Anaconda Navigator** or a terminal → `jupyter lab` |
| **Python + VS Code** | Professional workflow, notebooks and scripts in one editor | Install Python from python.org (tick "Add to PATH") → install **VS Code** → add the **Python** and **Jupyter** extensions |

STEP BY STEP **Local install with pip (any OS)**

1. Install Python 3.12+ from python.org (Windows: tick **Add python.exe to PATH**).
2. Open a terminal and create a **virtual environment** (an isolated set of packages for one project): `python -m venv .venv` then activate it: Windows `.venv\Scripts\activate`; Mac/Linux `source .venv/bin/activate`.
3. Install the course packages: `pip install jupyterlab numpy pandas matplotlib seaborn scikit-learn scipy statsmodels`
4. Start Jupyter: `jupyter lab` (opens in your browser).

## Jupyter notebooks

A **Jupyter notebook** ( `.ipynb` ) mixes **code cells**, **Markdown cells** (text, headings, images) and **outputs** (tables, charts) in one document. It is the standard tool for exploring data and sharing analysis. The name comes from Julia, Python and R.




Anatomy of a Jupyter notebook.

## The kernel

The **kernel** is the Python process behind the notebook. Variables you create stay in its memory until you restart it. Cells can be run in any order, which is flexible but dangerous: a notebook can look right while depending on a cell you deleted. **Before sharing, always Restart Kernel and Run All** to prove the notebook works top to bottom.

## Essential shortcuts (Jupyter / Colab)

| Action | Shortcut |
|---|---|
| Run cell, stay | `Ctrl + Enter` |
| Run cell, go to next | `Shift + Enter` |
| Insert cell above / below (command mode) | `A` / `B` |
| Delete cell (command mode) | `D, D` |
| Convert to Markdown / Code | `M` / `Y` |
| Command mode / edit mode | `Esc` / `Enter` |
| Comment / uncomment lines | `Ctrl + /` |
| Show function help | `Shift + Tab` (inside the brackets) |
| Autocomplete | `Tab` |
| Undo cell delete | `Z` (command mode) |

## Your first code

Python

```
print("Hello, data world!")
```

OUTPUT

```
Hello, data world!
```

Python is also a calculator:

Python

```
2 + 3 * 4
```

OUTPUT

```
14
```

Python

```
(2 + 3) * 4, 10 / 4, 10 // 4, 10 % 4, 2 ** 10
```

OUTPUT

```
(20, 2.5, 2, 2, 1024)
```

The **last expression** in a notebook cell is displayed automatically; to show several things use `print()`.

Python

```
name = "Varun"
learners = 25
print("Welcome to the course,", name)
print(f"This batch has {learners} learners")
```

OUTPUT

```
Welcome to the course, Varun
This batch has 25 learners
```

## Comments

```python
# This is a comment: Python ignores everything after the hash
price = 120          # you can also put a comment at the end of a line
"""A triple-quoted string on its own line is often used
as a multi-line comment or docstring"""
```

Write comments to explain **why**, not what. `x = x + 1 # add one` is noise; `# compensate for the header row` is useful.

## Errors are normal

Every programmer sees errors daily. Learn to read them from the **bottom line up**:

```python
print("Total:", total_amount)
```

OUTPUT

```
NameError: name 'total_amount' is not defined
```

`NameError: name 'total_amount' is not defined` says exactly what happened: we used a name we never created. Common beginner errors:

| Error | Usual cause |
|---|---|
| `NameError` | Typo, or the cell that defines the variable has not been run |
| `SyntaxError` | Missing bracket, quote or colon; wrong indentation |
| `TypeError` | Using a value of the wrong type ( `"5" + 3` ) |
| `ValueError` | Right type, wrong value ( `int("abc")` ) |
| `IndexError` , `KeyError` | Asking for a position or key that does not exist |
| `FileNotFoundError` | Wrong path or the file is not where you think |

> **TIP** **How to ask for help**
>
> Copy the **last line** of the error, search it, or paste the code and the error into an AI assistant (without secrets) and ask: *"What does this error mean and how do I fix it? Explain step by step."* Then fix it **yourself** and re-run: you learn by doing.

## Markdown cells

Change a cell to **Markdown** to write notes next to your code:

MARKDOWN

```
# Title
## Section
Some **bold** and *italic* text, a [link](https://example.com), and a list:
- item one
- item two

| Column | Value |
|---|---|
| A | 1 |
```

Good notebooks tell a story: a title, the question, the data, the steps with short explanations, and a conclusion.

## Magic commands and useful tools

Jupyter adds special commands beginning with `%`:

| Command | Purpose |
|---|---|
| `%timeit expr` | Time a short expression precisely |
| `%%time` | Time a whole cell |
| `%pwd`, `%ls` | Current folder, list files |
| `%matplotlib inline` | Show charts in the notebook (default now) |
| `%who` | List variables |
| `!pip install package` | Run a shell command (install a package) from a cell |

## Reproducibility habits

1. **Folder per project**: `data/`, `notebooks/`, `src/`, `README.md`.
2. **Virtual environment per project** and a `requirements.txt` (`pip freeze > requirements.txt`).
3. **Use relative paths** (`data/sales.csv`), not `C:\Users\you\Desktop\…`.
4. **Set random seeds** (`np.random.seed(42)`) so results repeat.
5. **Restart and Run All** before sharing; keep notebooks short and focused.
6. **Version control**: commit to Git; clear outputs of big notebooks.
7. **Do not commit** data with personal information or secrets (API keys); use environment variables.

DO IT WITH AI **Do it with AI**

Ask an assistant: *"I'm a beginner. Give me a 7-day practice plan for Python with 20 minutes a day, each day with three small exercises and answers I can check."* Use it to **practise**, not to skip the struggle: type the code, break it on purpose, and read the errors.

## Summary

KEY TAKEAWAYS

- A program is a set of instructions; Python is a **high-level, interpreted, readable** language with a huge data ecosystem (NumPy, pandas, scikit-learn, PyTorch).
- CPython compiles source to bytecode and runs it on a virtual machine; speed-critical libraries use C underneath.
- Start with **Google Colab** or install **Anaconda** or **Python + VS Code**; isolate projects with **virtual environments**.
- A Jupyter notebook mixes code, Markdown and output; the **kernel** holds state, so *Restart and Run All* before sharing.
- Read error messages from the bottom; learn the common error types.
- Use clear comments, relative paths, seeds and version control for reproducible work.

## Quick quiz

6 questions · tap an answer to check it

**1. Which best describes Python?**

A A low-level compiled language

B A high-level, interpreted, general-purpose language with a rich data ecosystem

C A database language

D A spreadsheet tool

**Answer: B.** Python is readable, dynamically typed and widely used for data.

**2. What does the kernel do in a notebook?**

A Formats Markdown

B Runs your code and keeps variables in memory

C Stores the file

D Draws charts only

**Answer: B.** The kernel is the Python process behind the notebook.

**3. Why run "Restart kernel and Run All" before sharing a notebook?**

A To make it smaller

B To confirm the notebook works from top to bottom without hidden state

C To delete output

D To change the Python version

**Answer: B.** Cells can be run out of order; this test removes that risk.

**4. What does `NameError: name 'x' is not defined` mean?**

A x is a bad name

B Python has no variable called x yet (typo or the defining cell was not run)

C x must be a number

D A syntax mistake

**Answer: B.** You used a name before creating it.

**5.** Which command installs a package from inside a notebook cell?

A `install pandas`

B `!pip install pandas`

C `%pip.exe pandas`

D `import install`

**Answer: B.** A leading `!` runs a shell command.

**6.** What is a virtual environment for?

A Faster internet

B Keeping the packages of one project separate from others

C Running Python 2

D Backing up data

**Answer: B.** Isolated environments avoid version conflicts.

EXERCISE

### Set up and run

(1) Open Colab or Jupyter and create a notebook called `day1.ipynb`. (2) Add a Markdown cell with a title and your goals. (3) In code cells, print your name, compute the area of a rectangle 12 × 7.5, and print the Python version ( `import sys; print(sys.version)` ). (4) Cause a `NameError` on purpose and fix it. (5) Restart and Run All.

**Show solution (worked answer)**

Python

```python
print("Asha Rao")
length, width = 12, 7.5
print("Area:", length * width)          # 90.0
import sys; print(sys.version)
print(undefined_name)                    # NameError: name 'undefined_name' is not defined
undefined_name = 5; print(undefined_name)
```

INTERVIEW CORNER

### Questions you may be asked

**Q: Why is Python popular for data science?** Readable syntax, a huge ecosystem (NumPy, pandas, scikit-learn, PyTorch), strong community, easy integration with SQL, APIs and cloud, and notebooks for exploration.

**Q: Is Python compiled or interpreted?** Both: CPython compiles source to bytecode, which the Python virtual machine interprets. It is called interpreted because there is no separate manual compile step.

**Q: What is a virtual environment?** An isolated Python installation with its own packages for one project (created with `venv` or conda), preventing dependency conflicts.

**Q: Notebook vs script?** Notebooks are interactive and great for exploration and communication; scripts ( `.py` ) are better for production code, testing and automation.

PYTHON FOR DATA SCIENCE · LESSON 2

# Variables, Data Types, Type Conversion, print(), f-strings & input()

Store and name values with variables, meet Python's core data types (int, float, str, bool, None, complex), convert between types, format output with print(), f-strings and .format(), and read user input with input() and sys.argv.

Python class 2 | Python 3.13 | Beginner | 110 min

**By the end of this lesson you can**

- Create variables and follow Python naming rules and conventions
- Identify and check types with type() and isinstance(); explain mutable vs immutable
- Convert between int, float, str and bool, and predict what int(), float(), bool() and str() do
- Format output with print() options, f-strings (including number formats) and str.format()
- Read and validate input with input(), and avoid float rounding surprises

## Variables

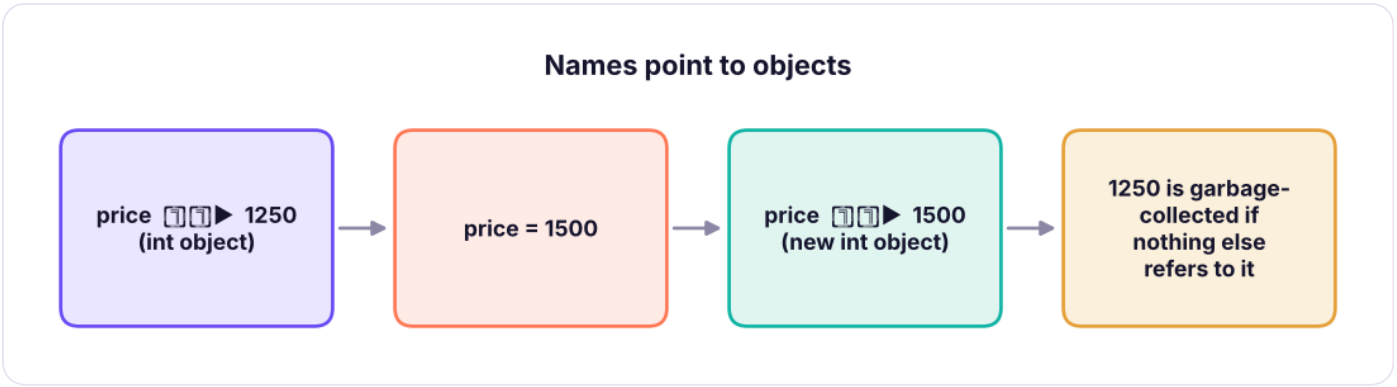
A **variable** is a name that refers to a value stored in memory. You create it by assigning with `=`: no declaration, no type needed.

Python

```
price = 1250          # an int
tax_rate = 0.18       # a float
product = "Laptop"    # a str
in_stock = True       # a bool
total = price * (1 + tax_rate)
print(product, "costs", total)
```

OUTPUT

```
Laptop costs 1475.0
```




A variable is a label attached to an object. Re-assigning moves the label; it does not change the old object.

## Naming rules and conventions

| Rule | Example |
|---|---|
| Letters, digits and `_` only; cannot start with a digit | `sales_2025` ✓, `2025_sales` × |
| Case-sensitive | `Total` and `total` are different |
| Cannot be a keyword (`if`, `for`, `class`, `None`, `True` ...) | `class` ×, `class_name` ✓ |
| **Convention**: `snake_case` for variables and functions, `PascalCase` for classes, `UPPER_CASE` for constants | `monthly_income`, `CustomerOrder`, `GST_RATE` |
| Use meaningful names | `avg_order_value` not `aov2` or `x` |
| Avoid shadowing built-ins | Do not name variables `list`, `sum`, `max`, `str`, `id`, `input` |

Python

```
a = b = c = 0                # chained assignment
x, y, z = 1, 2.5, "three"    # multiple assignment (tuple unpacking)
x, y = y, x                  # swap without a temp variable
print(a, b, c, x, y, z)
```

OUTPUT

```
0 0 0 2.5 1 three
```

## Data types

Everything in Python is an **object** with a **type**. Check it with `type()`.

| Type | Name | Example | Notes |
|---|---|---|---|
| `int` | Integer | `42`, `-7`, `1_000_000` | Unlimited size |
| `float` | Decimal number | `3.14`, `2e-3` | 64-bit binary floating point |
| `str` | Text | `"Pune"`, `'Pune'` | Immutable sequence of characters |
| `bool` | Boolean | `True`, `False` | Subclass of int ( `True == 1` ) |
| `NoneType` | "nothing" | `None` | Used for missing/absent values |
| `complex` | Complex number | `3 + 4j` | Rare in data work |
| `list`, `tuple`, `dict`, `set` | Collections | Lessons 8–10 | |

Python

```
values = [42, 3.14, "Pune", True, None, 3 + 4j, [1, 2], (1, 2), {"a": 1}, {1, 2}]
for v in values:
    print(f"{str(v):10s} → {type(v).__name__}")
```

OUTPUT

```
42         → int
3.14       → float
Pune       → str
True       → bool
None       → NoneType
(3+4j)     → complex
[1, 2]     → list
(1, 2)     → tuple
{'a': 1}   → dict
{1, 2}     → set
```

Python

```
print(isinstance(42, int), isinstance(True, int), isinstance(3.0, (int, float)))   # True True True
print(type(10 / 2), type(10 // 2))      # / always returns float; // with ints returns int
```

OUTPUT

```
True True True
<class 'float'> <class 'int'>
```

## Mutable vs immutable

- **Immutable**: cannot be changed after creation: `int`, `float`, `str`, `bool`, `tuple`, `frozenset`. "Changing" creates a new object.
- **Mutable**: can be changed in place: `list`, `dict`, `set`.

Python

```
s = "data"
try:
    s[0] = "D"                    # strings are immutable
except TypeError as e:
    print("Error:", e)
s2 = "D" + s[1:]                   # build a new string instead
print(s2, "| id changed:", id(s) != id(s2))
```

OUTPUT

```
Error: 'str' object does not support item assignment
Data | id changed: True
```

## Numbers: int and float

Python

```
big = 12_345_678_901_234_567_890 * 10      # ints never overflow
print(big)
print(0.1 + 0.2)                           # floats are approximate!
print(0.1 + 0.2 == 0.3, abs((0.1 + 0.2) - 0.3) < 1e-9)
print(round(2.675, 2), round(0.5), round(1.5), round(2.5))   # banker's rounding: halves go to
even
```

OUTPUT

```
123456789012345678900
0.30000000000000004
False True
2.67 0 2 2
```

**WATCH OUT** **Floating-point surprises**

Computers store decimals in binary, so `0.1 + 0.2` is `0.30000000000000004`. For **money** use `decimal.Decimal` (or integers of paise); for comparisons use `math.isclose(a, b)` or `abs(a - b) < tolerance`. Python's `round()` rounds halves to the nearest even number (`round(2.5) = 2`), unlike Excel's `ROUND`.

Python

```
from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2"))                    # exact decimal arithmetic
import math; print(math.isclose(0.1 + 0.2, 0.3))
```

OUTPUT

```
0.3
True
```

## Booleans and None

Python

```
print(True + True, 5 > 3, 5 == 5.0, "a" < "b", not False)
print(bool(0), bool(0.0), bool(""), bool([]), bool(None))      # all False: "falsy" values
print(bool(1), bool(-3), bool("0"), bool([0]))                 # all True: anything else is
"truthy"
print(None is None, type(None))
```

OUTPUT

```
2 True True True True
False False False False False
True True True True
True <class 'NoneType'>
```

**Falsy** values: `0`, `0.0`, `""`, `[]`, `{}`, `()`, `set()`, `None`, `False`. Everything else is truthy. `None` represents "no value" (like SQL NULL); test with `is None`, never `== None`.

# Type conversion

Convert with the type's name: `int()`, `float()`, `str()`, `bool()`, `list()`...

Python

```
print(int("42"), int(3.99), int(-3.99), int(True))          # int() truncates towards zero
print(float("3.5"), float(7), float("1e3"))
print(str(123), str(3.0), str([1, 2]))
print(int("1010", 2), int("ff", 16), bin(10), hex(255))     # other bases
print(round(3.7), round(3.14159, 2), int(round(3.7)))
```

OUTPUT

```
42 3 -3 1
3.5 7.0 1000.0
123 3.0 [1, 2]
```

```
10 255 0b1010 0xff
4 3.14 4
```

Python

```python
print("5" + 3)                    # TypeError: can only concatenate str to str
```

OUTPUT

```
TypeError: can only concatenate str (not "int") to str
```

Python

```python
print("5" + str(3), int("5") + 3, "5" * 3)      # explicit conversion; repetition for strings
```

OUTPUT

```
53 8 555
```

Python

```python
int("abc")                        # ValueError: invalid literal for int()
```

OUTPUT

```
ValueError: invalid literal for int() with base 10: 'abc'
```

| Conversion | Result | Note |
|---|---|---|
| `int("42")` | 42 | String must look like an integer |
| `int("3.7")` | ValueError | Convert to float first: `int(float("3.7"))` |
| `int(3.99)` | 3 | Truncates (not rounds) |
| `float("abc")` | ValueError | |
| `str(None)` | `'None'` | |
| `bool("False")` | True | Any non-empty string is True! |
| `list("abc")` | `['a','b','c']` | |

**Implicit conversion** happens in mixed arithmetic: `int + float → float`, `bool + int → int`.

Python

```
print(1 + 2.0, True + 1, type(1 + 2.0).__name__)
```

OUTPUT

```
3.0 2 float
```

## print(): showing output

Python

```
print("Order", 1001, "shipped")                    # items separated by a space
print("Order", 1001, "shipped", sep=" | ")          # change the separator
print("Loading", end="...")                         # change the line ending (no newline)
print("done")
print("Line one\nLine two\tTabbed")                 # escape sequences: \n newline, \t tab
print('He said "hi"', "It's fine", 'It\'s fine')    # quotes inside strings
print(r"C:\new\table")                              # raw string: backslashes kept
print("""A multi-line
string with triple quotes""")
```

OUTPUT

```
Order 1001 shipped
Order | 1001 | shipped
Loading...done
Line one
Line two        Tabbed
He said "hi" It's fine It's fine
C:\new\table
A multi-line
string with triple quotes
```

## f-strings: the modern way to format

Put `f` before the quotes and `{expression:format}` inside:

Python

```python
name, qty, price, share = "Asha", 3, 1234.5678, 0.2345
print(f"{name} bought {qty} items")
print(f"Price: {price:.2f}")                # 2 decimals
print(f"Price: {price:,.2f}")               # thousands separator
print(f"Share: {share:.1%}")                # percentage
print(f"ID: {qty:03d}")                      # zero-padded to 3 digits
print(f"[{name:<10}] [{name:>10}] [{name:^10}]")       # left, right, centre in width 10
print(f"{price:>12,.1f}|")                   # right-aligned width 12
print(f"{qty * price:,.2f}  {qty = }")       # expressions work; {x = } prints name and value
print(f"{1234567:,}  {255:#x}  {255:b}  {0.000123:.2e}")
```

OUTPUT

```
Asha bought 3 items
Price: 1234.57
Price: 1,234.57
Share: 23.4%
ID: 003
[Asha      ] [      Asha] [   Asha   ]
     1,234.6|
3,703.70  qty = 3
1,234,567  0xff  11111111  1.23e-04
```

| Format spec | Meaning |
|---|---|
| `.2f` | 2 decimal places (fixed) |
| `,` | Thousands separator |
| `.1%` | Percentage with 1 decimal |
| `05d` | Integer padded with zeros to width 5 |
| `<10`, `>10`, `^10` | Left / right / centre in width 10 |
| `e` | Scientific notation |
| `x`, `b`, `o` | Hex, binary, octal |

## .format() and % (older styles)

Python

```python
print("{} bought {} items at {:.1f}".format("Asha", 3, 12.345))
print("{1} before {0}".format("B", "A"))
print("{name} paid {amt:,.0f}".format(name="Ravi", amt=1234567))
print("%s bought %d items at %.1f" % ("Asha", 3, 12.345))       # legacy, still seen in old code
```

OUTPUT

```
Asha bought 3 items at 12.3
A before B
Ravi paid 1,234,567
Asha bought 3 items at 12.3
```

Prefer **f-strings** in new code; you will meet `.format()` and `%` in older code and logs.

## Reading input

`input()` reads **text** from the keyboard and always returns a `str`:

Python

```
name = input("Your name: ")                    # waits for the user
age = int(input("Your age: "))                 # convert to a number
height = float(input("Height in metres: "))
print(f"Hello {name}, you are {age} and {height:.2f} m tall.")
```

Because this handbook runs without a keyboard, here is the same logic with the typed values supplied in code:

Python

```
typed = ["Asha", "29", "1.62"]           # imagine the user typed these three answers
name = typed[0]; age = int(typed[1]); height = float(typed[2])
print(f"Hello {name}, you are {age} and {height:.2f} m tall.  In ten years: {age + 10}.")
```

OUTPUT

```
Hello Asha, you are 29 and 1.62 m tall.  In ten years: 39.
```

### Validate input

Python

```
while True:
    text = input("Enter quantity (1-100): ")
    if text.isdigit() and 1 ⩽ int(text) ⩽ 100:
        quantity = int(text); break
    print("Please type a whole number between 1 and 100.")
```

Other ways to get values into a program: **command-line arguments** ( `sys.argv` , `argparse` ), **files** (Lesson 13), **databases and APIs** (later lessons), and **environment variables** ( `os.environ["API_KEY"]` ) for secrets.

## Useful built-in functions

Python

```
nums = [4, 8, 15, 16, 23, 42]
print(len(nums), sum(nums), min(nums), max(nums), sorted(nums, reverse=True))
print(abs(-7), pow(2, 5), divmod(17, 5), round(sum(nums) / len(nums), 2))
print(type(nums).__name__, id(nums) > 0, dir(3)[:3])          # introspection helpers
help(len) if False else None                                   # help(len) prints documentation
```

OUTPUT

```
6 108 4 42 [42, 23, 16, 15, 8, 4]
7 32 (3, 2) 18.0
list True ['__abs__', '__add__', '__and__']
```

## Mini project: a bill calculator

Python

```
items = {"Notebook": (45.0, 10), "Pen pack": (60.0, 25), "Stapler": (180.0, 4)}     # name: (price, quantity)
GST = 0.18
subtotal = 0
print(f"{'Item':<12}{'Price':>8}{'Qty':>5}{'Amount':>10}")
for name, (price, qty) in items.items():
    amount = price * qty; subtotal += amount
    print(f"{name:<12}{price:>8.2f}{qty:>5}{amount:>10,.2f}")
print("-" * 35)
print(f"{'Subtotal':<25}{subtotal:>10,.2f}")
print(f"{'GST (18%)':<25}{subtotal * GST:>10,.2f}")
print(f"{'Total':<25}{subtotal * (1 + GST):>10,.2f}")
```

OUTPUT

```
Item           Price  Qty    Amount
Notebook       45.00   10    450.00
Pen pack       60.00   25  1,500.00
Stapler       180.00    4    720.00
-----------------------------------
Subtotal                   2,670.00
GST (18%)                    480.60
Total                      3,150.60
```

(The `for` loop and dictionary are explained in Lessons 6 and 9; notice how f-string alignment makes a tidy table.)

# Summary

**KEY TAKEAWAYS**

- A **variable** is a name pointing to an object; names are `snake_case`, case-sensitive, and must not be keywords or shadow built-ins.
- Core types: `int`, `float`, `str`, `bool`, `NoneType`; `list/tuple/dict/set` come later. Check with `type()` and `isinstance()`.
- Strings, numbers and tuples are **immutable**; lists, dicts and sets are **mutable**.
- Floats are approximate (`0.1 + 0.2`): use `Decimal` for money and `math.isclose` for comparisons. `round()` uses banker's rounding.
- Convert with `int()`, `float()`, `str()`, `bool()`; `int("3.7")` fails, `int(3.99)` truncates, `bool("False")` is True.
- Format with **f-strings** (`{x:,.2f}`, `{p:.1%}`, `{n:05d}`, `{s:<10}`); `print` has `sep` and `end`.
- `input()` returns a string: convert and validate it.

## Quick quiz

6 questions · tap an answer to check it

**1. What does `type(10 / 2)` return?**

A `int`

B `float`

C `str`

D `bool`

**Answer: B.** `/` always returns a float in Python 3 (5.0).

**2.** What is `int(-3.9)` ?

A −4

B −3

C 3

D −3.9

**Answer: B.** int() truncates towards zero.

**3.** Which is NOT falsy?

A `0`

B `""`

C `"0"`

D `None`

**Answer: C.** A non-empty string, including "0", is truthy.

**4.** What does `print(f"{0.2345:.1%}")` print?

A 0.2

B 23.4%

C 2.3%

D 0.23

**Answer: B.** `.1%` multiplies by 100 and shows one decimal. 0.2345 displays as 23.4% because 0.2345 is stored as a binary value just below 0.2345.

**5.** `x = input("Age: ")` and the user types 25. What is `type(x)` ?

A `int`

B `str`

C `float`

D `bool`

**Answer: B.** input() always returns text.

**6. Why is** `0.1 + 0.2 == 0.3` **False?**

A Python is buggy

B Decimal fractions cannot be represented exactly in binary floating point

C Because of integer division

D Because of rounding to even

**Answer: B.** Use math.isclose or Decimal.

EXERCISE **Practice**

(1) Ask for a price and quantity (simulate the input), compute the total with 18% GST and print it as `₹1,23,456.78`-style with thousands separators and 2 decimals. (2) Convert "1010" from binary to int and 255 to hex. (3) Explain the output of `print(round(2.5), round(3.5))`.

**Show solution (worked answer)**

Python

```
price, qty = float("1250.5"), int("3")
total = price * qty * 1.18
print(f"Total: ₹{total:,.2f}")          # ₹4,426.77
print(int("1010", 2), hex(255))          # 10 0xff
print(round(2.5), round(3.5))            # 2 4 : halves round to the nearest even number
```

INTERVIEW CORNER **Questions you may be asked**

**Q: What is the difference between mutable and immutable types? Give examples.** Immutable objects cannot change after creation (int, float, str, tuple); mutable ones can (list, dict, set). It matters for function arguments, dictionary keys (must be immutable/hashable) and copying.

**Q: What is the output of `bool("False")` and why?** True: any non-empty string is truthy, regardless of its content.

**Q: How do you compare floats safely?** `math.isclose(a, b)` or `abs(a - b) < tol`; use `Decimal` for exact decimal arithmetic such as money.

**Q: `is` vs `==`?** `==` compares values; `is` compares identity (same object). Use `is None` for None.

PYTHON FOR DATA SCIENCE · LESSON 3

# Operators, Strings, Indexing, Slicing & String Methods

Master Python's operators (arithmetic, comparison, logical, assignment, membership, identity, bitwise) and work with text: string indexing and slicing, immutability, every string method an analyst needs (split, join, strip, replace, find, startswith, case methods, isdigit...), escape sequences, and cleaning real messy text.

Python classes 3–4 | Python 3.13 | Beginner | 120 min

**By the end of this lesson you can**

- Use arithmetic, comparison, logical, membership and identity operators and explain operator precedence
- Index and slice strings forwards and backwards, including steps, and explain immutability
- Apply string methods to clean, split, search, join and format text
- Build small text-cleaning functions and check results on real data
- Recognise common string pitfalls (encoding, whitespace, case, escape characters)

## Operators

### Arithmetic

| Operator | Meaning | Example | Result |
|---|---|---|---|
| `+` `-` `*` | Add, subtract, multiply | `7 * 3` | 21 |
| `/` | True division (always float) | `7 / 2` | 3.5 |
| `//` | Floor division | `7 // 2`, `-7 // 2` | 3, **−4** |
| `%` | Remainder (modulo) | `7 % 2`, `-7 % 3` | 1, 2 |
| `**` | Power | `2 ** 10` | 1024 |

Python

```
print(7 / 2, 7 // 2, -7 // 2, 7 % 2, -7 % 3, 2 ** 10, 9 ** 0.5)
print(divmod(17, 5))                  # (quotient, remainder) in one call
hours = 135; print(f"{hours // 60} h {hours % 60} min")       # split minutes into hours and
minutes
print(10 % 3 == 1, 12 % 2 == 0)      # test odd/even and divisibility
```

OUTPUT

```
3.5 3 -4 1 2 1024 3.0
(3, 2)
2 h 15 min
True True
```

## Comparison and logical

Python

```
x = 15
print(x > 10, x == 15, x != 15, 10 < x <= 20)        # chained comparison: 10 < x and x <= 20
print(x > 10 and x < 20, x < 5 or x > 12, not x > 10)
print("apple" < "banana", "Zebra" < "apple")         # strings compare by character codes:
uppercase before lowercase
```

OUTPUT

```
True True False True
True True False
True True
```

**Short-circuit evaluation**: `A and B` stops at A if A is falsy; `A or B` stops at A if A is truthy. They return the **deciding operand**, not just True/False:

Python

```
print(0 or "default", "" or "n/a", "x" and "y", None and 1)
name = ""; print(name or "Anonymous")          # a common idiom for default values
print(5 > 3 and print("evaluated") is None)     # right side runs only when needed
```

OUTPUT

```
default n/a y None
Anonymous
evaluated
True
```

## Assignment, membership, identity

Python

```
count = 10; count += 5; count *= 2; count -= 1; count //= 4; count **= 2
print(count)
print(3 in [1, 2, 3], "py" in "python", "z" not in "python", "a" in {"a": 1})     # membership
a = [1, 2]; b = [1, 2]; c = a
print(a == b, a is b, a is c, None is None)                                        # equality vs identity
```

OUTPUT

```
49
True True True True
True False True True
```

## Bitwise (awareness)

`&` AND, `|` OR, `^` XOR, `~` NOT, `<<` `>>` shifts: used for flags and low-level work; in pandas/NumPy `&` and `|` combine boolean arrays (use brackets).

Python

```
print(5 & 3, 5 | 3, 5 ^ 3, 5 << 1, 5 >> 1, bin(5))
```

OUTPUT

```
1 7 6 10 2 0b101
```

## Precedence

From highest to lowest: `**` → unary `-x` → `* / // %` → `+ -` → comparisons ( `< == in is` ) → `not` → `and` → `or`. When in doubt, use brackets.

Python

```
print(2 + 3 * 4 ** 2, (2 + 3) * 4 ** 2, -2 ** 2, (-2) ** 2, not 1 == 2 or False)
```

OUTPUT

```
50 80 -4 4 True
```

## Walrus operator (3.8+)

`:=` assigns inside an expression: `if (n := len(items)) > 10: print(f"{n} items")`. Useful in loops and comprehensions; do not overuse.

# Strings

A **string** is an immutable sequence of Unicode characters. Create with single, double or triple quotes.

Python

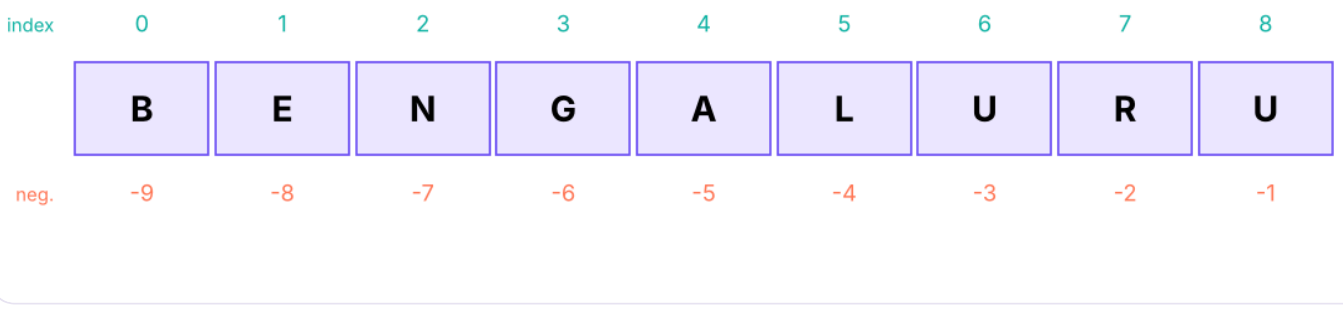
```
city = "Bengaluru"
quote = 'She said "hello"'
multi = """Line 1
Line 2"""
hindi = "नमस्ते"                    # Unicode works everywhere
print(len(city), len(hindi), multi.splitlines(), city.upper())
print("Ha" * 3, "data" + " " + "science", "-" * 20)
```

OUTPUT

```
9 6 ['Line 1', 'Line 2'] BENGALURU
HaHaHa data science --------------------
```

## Indexing

Characters are numbered from 0 on the left and from −1 on the right.




Indexing a string: positive indexes count from the left (starting at 0), negative from the right (starting at -1).

Python

```
w = "BENGALURU"
print(w[0], w[3], w[-1], w[-3])
try: print(w[20])
except IndexError as e: print("IndexError:", e)
```

OUTPUT

```
B G U U
IndexError: string index out of range
```

## Slicing: `s[start:stop:step]`

`start` is included, `stop` is **excluded**; omit a part to mean "from the beginning / to the end"; a negative step goes backwards.

Python

```
w = "BENGALURU"
print(w[0:3], w[:3], w[3:], w[3:6], w[-4:], w[:-1])
print(w[::2], w[::-1], w[1::3], w[8:2:-2])
print(w[2:100], repr(w[5:2]))            # out-of-range slices are safe; an empty slice is ''
```

OUTPUT

```
BEN BEN GALURU GAL LURU BENGALUR
BNAUU URULAGNEB EAR UUA
NGALURU ''
```

| Slice | Meaning |
|---|---|
| `s[:n]` | First n characters |
| `s[-n:]` | Last n characters |
| `s[a:b]` | From a up to (not including) b; length b − a |
| `s[::-1]` | Reversed |
| `s[::2]` | Every second character |

Python

```
order_id = "IN-2025-00042"
country, year, number = order_id[:2], int(order_id[3:7]), int(order_id[-5:])
print(country, year, number)                    # same as LEFT/MID/RIGHT in Excel and SQL
```

OUTPUT

```
IN 2025 42
```

# String methods

Strings are immutable, so every method **returns a new string**; the original stays unchanged.

## Case and cleaning

```python
s = "  new DELHI  "
print(repr(s.strip()), repr(s.lstrip()), repr(s.rstrip()))
print(s.strip().lower(), s.strip().upper(), s.strip().title(), s.strip().capitalize(),
s.strip().swapcase())
print("xxHixx".strip("x"), "2025-04-27".replace("-", "/"), "a,b,,c".replace(",,", ","))
print("Bengaluru".casefold() == "BENGALURU".casefold())        # robust case-insensitive
comparison
```

OUTPUT

```
'new DELHI' 'new DELHI  ' '  new DELHI'
new delhi NEW DELHI New Delhi New delhi NEW delhi
Hi 2025/04/27 a,b,c
True
```

## Searching and testing

```python
t = "Delhi, Mumbai, Delhi, Pune"
print(t.find("Delhi"), t.rfind("Delhi"), t.find("Goa"), t.count("Delhi"), t.index("Mumbai"))
print(t.startswith("Delhi"), t.endswith("Pune"), "Mumbai" in t)
print("12345".isdigit(), "12.5".isdigit(), "abc".isalpha(), "abc123".isalnum(), "  ".isspace(),
"Abc".istitle(), "abc".islower())
```

OUTPUT

```
0 15 -1 2 7
True True True
True False True True True True True
```

`find` returns −1 when not found; `index` raises `ValueError`. `isdigit` is False for decimals and negatives, so use `try: float(s)` to test numbers.

## Splitting and joining

Python

```
csv_line = "E101,Kavya Iyer,HR,97000"
parts = csv_line.split(",")
print(parts, len(parts))
print("one  two   three".split(), "a-b-c".split("-", 1), "line1\nline2".splitlines(),
"path/to/file.csv".rsplit("/", 1))
print(", ".join(["Pune", "Delhi", "Goa"]), "".join(["a", "b"]), " | ".join(str(n) for n in [1,
2, 3]))
print("abc".partition("b"), "IN-2025-00042".partition("-"))
```

OUTPUT

```
['E101', 'Kavya Iyer', 'HR', '97000'] 4
['one', 'two', 'three'] ['a', 'b-c'] ['line1', 'line2'] ['path/to', 'file.csv']
Pune, Delhi, Goa ab 1 | 2 | 3
('a', 'b', 'c') ('IN', '-', '2025-00042')
```

`split()` with no argument splits on any whitespace and ignores repeats: the best way to tokenise text. `join` is called on the **separator**, with a list of strings.

## Alignment, padding and formatting

Python

```
print("42".zfill(6), "7".rjust(4, "0"), "ab".ljust(5, ".") + "|", "ab".center(8, "*"))
print("Total".ljust(10) + "1,500".rjust(10))
print("{:>8} {:<8} {:^8}".format("right", "left", "mid"))
print("tab\tseparated", "new\\nline stays literal", "unicode: \u20b9 \u2713")
```

OUTPUT

```
000042 0007 ab...| ***ab***
Total          1,500
   right left       mid
tab     separated new\nline stays literal unicode: ₹ ✓
```

## Translating and encoding

Python

```
table = str.maketrans({"₹": "Rs ", ",": ""})
print("₹1,25,000".translate(table))
print("naïve café".encode("utf-8"), "abc".encode("ascii"), b"abc".decode())
print(ord("A"), chr(97), "é".encode("utf-8"), len("é".encode("utf-8")))
```

OUTPUT

```
Rs 125000
b'na\xc3\xafve caf\xc3\xa9' b'abc' abc
65 a b'\xc3\xa9' 2
```

NOTE **Encoding**

Text on disk is **bytes**. Most files today are **UTF-8**; if you see `Ã©` or `�`, the file was read with the wrong encoding. In pandas use `pd.read_csv(path, encoding="utf-8")` or `"latin-1"` / `"cp1252"` for old Windows exports.

## Method cheat sheet

| Goal | Methods |
|---|---|
| Change case | `lower()`, `upper()`, `title()`, `capitalize()`, `casefold()` |
| Trim | `strip()`, `lstrip()`, `rstrip()`, `removeprefix()`, `removesuffix()` |
| Replace | `replace(old, new, count)`, `translate()` |
| Search | `find()`, `rfind()`, `index()`, `count()`, `in`, `startswith()`, `endswith()` |
| Test | `isdigit()`, `isalpha()`, `isalnum()`, `isspace()`, `isupper()`, `islower()` |
| Split / join | `split()`, `rsplit()`, `splitlines()`, `partition()`, `join()` |
| Pad / align | `zfill()`, `ljust()`, `rjust()`, `center()` |
| Format | f-strings, `format()`, `format_map()` |

# Cleaning real text

Customer names from the course's `customers.csv`, cleaned with the same rules you used in Excel (`TRIM`, `PROPER`) and SQL:

Python

```
import pandas as pd
cust = pd.read_csv("customers.csv")
raw = ["  asha RAO ", "ravi   verma", "MEERA\tIYER", "kabir  singh "]          # messy examples
def clean_name(name):
    parts = name.split()                          # split() collapses all whitespace
    return " ".join(p.capitalize() for p in parts)
print([clean_name(n) for n in raw])
print(cust.customer_name.head(5).tolist())
first_names = [n.split()[0] for n in cust.customer_name]
print(len(first_names), "names; most common first names:",
pd.Series(first_names).value_counts().head(5).to_dict())
```

OUTPUT

```
['Asha Rao', 'Ravi Verma', 'Meera Iyer', 'Kabir Singh']
['Saanvi Khan', 'Meera Bose', 'Riya Shah', 'Diya Fernandes', 'Neha Sharma']
800 names; most common first names: {'Arjun': 36, 'Vikram': 31, 'Aarav': 31, 'Sneha': 30,
'Priya': 30}
```

Python

```
def initials(name): return "".join(w[0].upper() for w in name.split())
def mask_email(email):
    user, _, domain = email.partition("@")
    return user[0] + "*" * (len(user) - 2) + user[-1] + "@" + domain
print(initials("kavya iyer"), mask_email("kavya.iyer@abc.in"))
print("palindrome" if (s := "Level").lower() == s.lower()[::-1] else "not")
```

OUTPUT

```
KI k*********r@abc.in
palindrome
```

## Counting characters and words

Python

```
text = "data science is fun and data engineering is fun too"
words = text.split()
freq = {}
for w in words: freq[w] = freq.get(w, 0) + 1
print(freq)
print(max(freq, key=freq.get), len(words), len(set(words)))
```

OUTPUT

```
{'data': 2, 'science': 1, 'is': 2, 'fun': 2, 'and': 1, 'engineering': 1, 'too': 1}
data 10 7
```

(`collections.Counter(words)` does this in one line: Lesson 8.)

## Common pitfalls

| Pitfall | Example | Fix |
|---|---|---|
| Forgetting strings are immutable | `s.upper()` without assigning | `s = s.upper()` |
| Whitespace/case mismatches | `"Delhi " ≠ "Delhi"` | `strip()`, `lower()` before comparing |
| Using `+` with numbers | `"Total: " + 5` | f-string or `str(5)` |
| `isdigit` for decimals | `"3.5".isdigit()` → False | `try: float(x)` |
| Off-by-one in slices | `s[0:3]` has 3 characters | stop is excluded |
| Backslashes in Windows paths | `"C:\new"` → newline! | raw string `r"C:\new"` or forward slashes |
| Huge strings in loops | `s += x` in a loop | build a list and `"".join(list)` |
| Wrong encoding | `Ã©` characters | `encoding="utf-8"` |

DO IT WITH AI

### Do it with AI

Ask an assistant to produce 10 messy phone numbers (fake) and then write a function to standardise them to `+91-XXXXX-XXXXX`. Test it yourself on edge cases (spaces, dashes, leading 0, +91 already present) and compare approaches: string methods first, then regular expressions (Lesson 14).

## Summary

KEY TAKEAWAYS

- Operators: `/` true division, `//` floor division, `%` remainder, `**` power; comparisons chain (`10 < x ⩽ 20`); `and` / `or` short-circuit and return an operand; use `in`, `not in`, `is`.
- Strings are **immutable** sequences: index from 0 or −1, slice `[start:stop:step]` (stop excluded).
- Methods return new strings: `strip`, `lower/upper/title`, `replace`, `split` / `join`, `find` / `count`, `startswith`, `zfill`, `partition`.
- Clean text by trimming, normalising case/whitespace and replacing variants before comparing or counting.
- Watch pitfalls: forgetting to assign, off-by-one slices, wrong encodings, Windows backslashes.

## Quick quiz

6 questions · tap an answer to check it

**1.** What does `"BENGALURU"[2:5]` return?

- **A** NGA
- **B** ENG
- **C** NGAL
- **D** GAL

**Answer: A.** Index 2 is N, 3 is G and 4 is A; the stop index 5 is excluded.

**2.** What is `-7 // 2` in Python?

- **A** −3
- **B** −4
- **C** 3
- **D** −3.5

**Answer: B.** Floor division rounds down towards minus infinity.

**3.** What does `0 or "default"` return?

- **A** 0
- **B** True
- **C** "default"
- **D** None

**Answer: C.** `or` returns the first truthy operand (or the last operand).

**4.** Which statement is true about strings?

A `s[0] = "X"` changes the first character

B Strings are immutable, so methods like `upper()` return a new string

C `split()` changes the original string

D `strip()` removes only spaces inside the text

**Answer: B.** Assign the result to keep it.

**5.** How do you split text on any whitespace and ignore repeated spaces?

A `text.split(" ")`

B `text.split()`

C `text.partition()`

D `text.strip()`

**Answer: B.** `split()` without an argument splits on runs of whitespace.

**6.** `"3.5".isdigit()` returns:

A True

B False

C 3.5

D An error

**Answer: B.** isdigit() is True only for strings of digits; use float() inside try/except for decimals.

EXERCISE

### String practice

(1) From `"IN-2025-00042"` extract the country, year (int) and sequence number (int) with slicing. (2) Write `is_valid_pin(text)` that returns True for six-digit PIN codes not starting with 0. (3) Reverse the words in `"data science is fun"` (word order, not letters). (4) Turn `" rAHul VERMA "` into `"Rahul Verma"`.

**Show solution (worked answer)**

Python

```
oid = "IN-2025-00042"; print(oid[:2], int(oid[3:7]), int(oid[-5:]))
def is_valid_pin(t): return len(t) == 6 and t.isdigit() and t[0] != "0"
print(is_valid_pin("411001"), is_valid_pin("041100"), is_valid_pin("4110a1"))
print(" ".join("data science is fun".split()[::-1]))           # fun is science data
print(" ".join(w.capitalize() for w in "  rAHul   VERMA ".split()))
```

INTERVIEW CORNER

### Questions you may be asked

**Q: How do you reverse a string in Python?** `s[::-1]` (slice with step −1) or `"".join(reversed(s))`.

**Q: Why use `"".join(list)` instead of `+=` in a loop?** Strings are immutable; each `+=` creates a new string (quadratic time). Building a list and joining once is linear.

**Q: Difference between `find` and `index`?** `find` returns −1 if the substring is missing; `index` raises `ValueError`.

**Q: What does `//` do with negative numbers?** Floors toward minus infinity: `-7 // 2 = -4`, and `-7 % 3 = 2`, so `(a // b) * b + a % b == a` always holds.

PYTHON FOR DATA SCIENCE · LESSON 4

# Control Flow: if, elif, else, Nested Conditions & match

Make programs decide. Learn indentation, if/elif/else, comparison and logical conditions, truthiness, conditional expressions (ternary), nested decisions, the match statement, and how Python's if maps to Excel's IF/IFS and SQL's CASE WHEN, with data-flavoured examples.

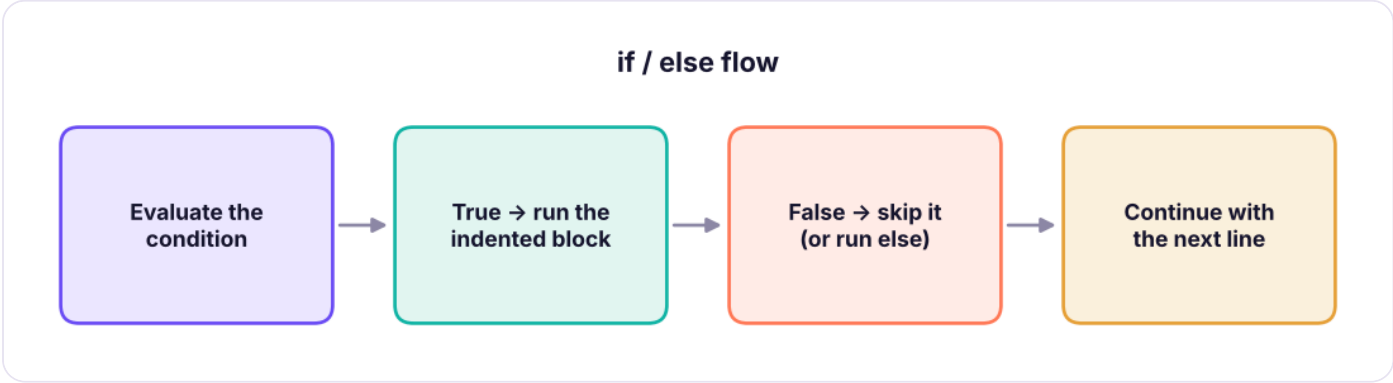
Python class 3 · Python 3.13 · Beginner · 90 min

**By the end of this lesson you can**

- Write correct if / elif / else blocks and explain why indentation matters
- Combine conditions with and, or, not, in, chained comparisons and truthiness
- Use conditional expressions and the match statement
- Order conditions correctly and avoid common branching bugs
- Translate a spreadsheet or SQL rule into Python

## Decisions and indentation

A program normally runs line by line. **Control flow** lets it choose between paths. In Python, a **colon** `:` starts a block and **indentation** (4 spaces by convention) says which lines belong to it.




An if statement tests a condition. If it is True the indented block runs; otherwise Python moves on (or runs else).

```python
marks = 82
if marks ⩾ 40:
    print("Pass")
    print("Well done!")
print("This line always runs")
```

OUTPUT

```
Pass
Well done!
This line always runs
```

```python
if True:
print("not indented")        # IndentationError: expected an indented block after 'if' statement
```

COMMON MISTAKES **Indentation rules**

Use **4 spaces** (never mix tabs and spaces). Everything in the same block has the **same indentation**. A missing colon or wrong indentation causes `SyntaxError` / `IndentationError`.

## if / elif / else

```python
marks = 82
if marks ⩾ 90:
    grade = "A+"
elif marks ⩾ 75:
    grade = "A"
elif marks ⩾ 60:
    grade = "B"
elif marks ⩾ 40:
    grade = "C"
else:
    grade = "Fail"
print(marks, "→", grade)
```

OUTPUT

```
82 → A
```

Python checks the conditions **top to bottom** and runs the **first** one that is true, then skips the rest. This is Excel's nested `IF` / `IFS` and SQL's `CASE WHEN` (Excel Lesson 5, SQL Lesson 7).

COMMON MISTAKES **Order matters**

If you test `marks ⩾ 40` first, an 82 gets "C" because the first true branch wins. Put the most specific / strictest condition first.

Python

```
for m in (95, 82, 61, 45, 12):
    g = "A+" if m ⩾ 90 else "A" if m ⩾ 75 else "B" if m ⩾ 60 else "C" if m ⩾ 40 else "Fail"
# chained conditional expression
    print(m, g)
```

OUTPUT

```
95 A+
82 A
61 B
45 C
12 Fail
```

## Conditional expression (ternary)

`value_if_true if condition else value_if_false` is Python's one-line `IF()`:

Python

```
salary = 97000
band = "High" if salary ⩾ 100000 else "Standard"
bonus = salary * 0.1 if band == "High" else 0
print(band, bonus)
```

OUTPUT

```
Standard 0
```

# Conditions

## Comparison and logical operators

Python

```python
age, city, income = 29, "Pune", 85000
print(age ⩾ 18 and city == "Pune")                    # both must be true
print(city in ("Pune", "Mumbai") or income > 100000)   # at least one
print(not (age < 18))
print(18 ⩽ age < 60)                                   # chained comparison
if 18 ⩽ age < 60 and income ⩾ 50000 and city in ("Pune", "Mumbai", "Delhi"):
    print("Eligible for the metro premium plan")
```

OUTPUT

```
True
True
True
True
Eligible for the metro premium plan
```

## Truthiness

Python treats "empty" and zero-like values as False (Lesson 2), so you can test them directly:

Python

```python
name = ""
items = [3, 5]
note = None
if not name: print("name is empty")
if items: print("items has", len(items), "elements")
if note is None: print("note is missing")                # use `is None`, not `== None`
if 0 < len(items) ⩽ 5: print("small list")
```

OUTPUT

```
name is empty
items has 2 elements
note is missing
small list
```

## Membership and string tests

Python

```
status = "Shipped"
if status in ("Shipped", "Delivered"): print("order has left the warehouse")
email = "kavya.iyer@abc.in"
if "@" in email and email.endswith(".in"): print("Indian email address")
```

OUTPUT

```
order has left the warehouse
Indian email address
```

## Nested conditions

Python

```
is_member, cart_value = True, 3200
if cart_value > 0:
    if is_member:
        discount = 0.1 if cart_value ⩾ 3000 else 0.05
    else:
        discount = 0.02 if cart_value ⩾ 3000 else 0
    print(f"Discount: {discount:.0%}")
else:
    print("Cart is empty")
```

OUTPUT

```
Discount: 10%
```

Prefer **flat** conditions with `and` to deeply nested ones when they read clearly:

Python

```
def shipping_fee(value, is_member):
    if value ⩽ 0: return 0
    if is_member and value ⩾ 500: return 0
    if value ⩾ 1500: return 0
    return 49
print([shipping_fee(v, m) for v, m in [(0, True), (600, True), (600, False), (1600, False)]])
```

OUTPUT

```
[0, 0, 49, 0]
```

Returning early (**guard clauses**) keeps code flat and readable.

# match: structural pattern matching (3.10+)

`match` compares one value with several patterns: a clean alternative to a long chain of `==` tests.

Python

```
def describe_status(code):
    match code:
        case 200 | 201:
            return "success"
        case 301 | 302:
            return "redirect"
        case 404:
            return "not found"
        case int(c) if 500 ⩽ c < 600:
            return "server error"
        case _:
            return "unknown"
print([describe_status(c) for c in (200, 404, 503, 999)])
```

OUTPUT

```
['success', 'not found', 'server error', 'unknown']
```

Python

```
def handle(command):
    match command.split():
        case ["load", filename]:  return f"loading {filename}"
        case ["save", filename]:  return f"saving {filename}"
        case ["quit"]:            return "bye"
        case _:                   return "unknown command"
print(handle("load sales.csv"), "|", handle("quit"), "|", handle("fly away"))
```

OUTPUT

```
loading sales.csv | bye | unknown command
```

# Data examples

## Rule-based bonus (the same as an Excel IF + AND)

Python

```
import pandas as pd
emp = pd.read_csv("employees_small.csv")
def bonus(salary, rating, dept):
    if rating ⩾ 5: rate = 0.15
    elif rating ⩾ 4: rate = 0.10
    elif rating ⩾ 3: rate = 0.05
    else: rate = 0
    if dept == "Sales": rate += 0.02            # extra for Sales
    return round(salary * rate)
emp["bonus"] = [bonus(s, r, d) for s, r, d in zip(emp.salary, emp.rating, emp.department)]
emp[["name", "department", "salary", "rating", "bonus"]].head(8)
```

OUTPUT

| | name | department | salary | rating | bonus |
|---|---|---|---|---|---|
| 0 | Kavya Iyer | HR | 97000 | 4 | 9700 |
| 1 | Imran Reddy | Finance | 157000 | 3 | 7850 |
| 2 | Harpreet Kulkarni | Finance | 131000 | 4 | 13100 |
| 3 | Diya Iyer | HR | 90000 | 4 | 9000 |
| 4 | Simran Iyer | HR | 139000 | 4 | 13900 |
| 5 | Saanvi Rao | HR | 83000 | 4 | 8300 |
| 6 | Isha Mehta | Sales | 57000 | 3 | 3990 |
| 7 | Aarav Kulkarni | HR | 101000 | 2 | 0 |

8 rows × 5 columns

(In pandas you would use `np.where`, `np.select` or `.apply`: Lesson 19. Writing the rule once as a function makes it testable.)

## A small decision tree: loan pre-check

Python

```python
def loan_precheck(age, income, existing_emi, credit_score):
    reasons = []
    if age < 21 or age > 60: reasons.append("age outside 21-60")
    if income < 25000: reasons.append("income below ₹25,000")
    if existing_emi > 0.5 * income: reasons.append("EMIs exceed 50% of income")
    if credit_score < 650: reasons.append("credit score below 650")
    return ("Eligible", []) if not reasons else ("Review", reasons)
print(loan_precheck(34, 70000, 20000, 730))
print(loan_precheck(19, 22000, 15000, 610))
```

OUTPUT

```
('Eligible', [])
('Review', ['age outside 21-60', 'income below ₹25,000', 'EMIs exceed 50% of income',
'credit score below 650'])
```

Collecting **all** failing reasons is more useful than stopping at the first, and shows how a series of independent `if` statements (not `elif`) is sometimes right.

## `if`/`elif` vs several `if`s

| Pattern | Use when |
|---|---|
| `if … elif … else` | Choices are **mutually exclusive**: only one branch should run |
| Several separate `if` | Each condition is **independent**: more than one may apply (collecting reasons, applying several rules) |

## Common mistakes

| Mistake | Fix |
|---|---|
| `if x = 5:` (assignment) | `if x == 5:` |
| `if x == 3 or 4:` (always true: 4 is truthy) | `if x in (3, 4):` |
| `if x == None:` | `if x is None:` |
| Comparing floats with `==` | `math.isclose` |
| Forgetting `elif` ordering | strictest first |
| `if len(items) > 0:` (verbose) | `if items:` |
| Deep nesting | guard clauses / `and` / helper functions |

DO IT WITH AI **Do it with AI**

Take an Excel rule from your own work, such as a nested IF for commission, and ask an assistant to convert it to Python with test cases. Then write five edge-case tests yourself (boundaries like exactly 50,000, blanks, negative values) and see if the code survives.

## Summary

KEY TAKEAWAYS

- A colon starts a block; **indentation** (4 spaces) defines it. `if / elif / else` runs the first true branch.
- Combine conditions with `and`, `or`, `not`, `in`, chained comparisons; empty/zero/None are falsy; test `None` with `is None`.
- Use the **ternary** `a if cond else b` for simple choices and the `match` statement for many exact cases and patterns.
- Order conditions from strictest to loosest; prefer flat code with guard clauses; use separate `if`s when several rules can apply.
- Wrap rules in functions so they can be tested and reused.

## Quick quiz

6 questions · tap an answer to check it

**1.** What does `x if cond else y` do?

A Runs a loop

B Evaluates to x when cond is True, otherwise y

C Defines a function

D Compares x and y

**Answer: B.** It is Python's conditional (ternary) expression.

**2.** Which condition is correct for "x is 3 or 4"?

**A** `x == 3 or 4`

**B** `x in (3, 4)`

**C** `x = 3 or 4`

**D** `x is (3, 4)`

**Answer: B.** `x == 3 or 4` is always truthy because 4 is truthy.

**3.** With `marks = 82`, which grade does `if marks >= 40: "C" elif marks >= 75: "A"` give?

**A** C

**B** A

**C** Fail

**D** Nothing

**Answer: A.** The first true branch wins, so the order is wrong.

**4.** Which is the best way to test for None?

**A** `x == None`

**B** `x is None`

**C** `not x`

**D** `x = None`

**Answer: B.** `is None` tests identity and avoids surprises with falsy values like 0.

**5.** What is a guard clause?

A A loop that never ends

B An early return that handles a special case so the rest of the function is not nested

C A try/except block

D A decorator

**Answer: B.** Guard clauses keep code flat and readable.

**6.** What does `match` do?

A Matches regular expressions only

B Compares a value against patterns and runs the first matching case

C Loops over a list

D Imports modules

**Answer: B.** Structural pattern matching arrived in Python 3.10.

EXERCISE

### Rules in code

Write `delivery_charge(order_value, city, express)`: free if order ≥ ₹1,500; otherwise ₹49 in metros (Delhi, Mumbai, Bengaluru, Chennai, Kolkata, Hyderabad) and ₹79 elsewhere; express adds ₹100 unless the value is ≥ ₹5,000. Test five cases including boundaries.

**Show solution (worked answer)**

Python

```
METROS = {"Delhi", "Mumbai", "Bengaluru", "Chennai", "Kolkata", "Hyderabad"}
def delivery_charge(value, city, express=False):
    fee = 0 if value ⩾ 1500 else (49 if city in METROS else 79)
    if express and value < 5000: fee += 100
    return fee
tests = [(1500, "Pune", False), (1499, "Delhi", False), (1499, "Pune", True), (5000,
"Pune", True), (100, "Mumbai", True)]
print([delivery_charge(*t) for t in tests])      # [0, 49, 179, 0, 149]
```

INTERVIEW CORNER

### Questions you may be asked

**Q: What is the difference between `elif` and multiple `if` statements?** `elif` branches are mutually exclusive (only the first true runs); separate `if`s are all evaluated, so several can run.

**Q: How do you write a switch-case in Python?** Use `match`/`case` (3.10+), or a dictionary mapping values to functions or results.

**Q: What are falsy values in Python?** `False`, `None`, `0`, `0.0`, `""`, `[]`, `{}`, `()`, `set()`, and objects whose `__bool__` returns False or `__len__` returns 0.

**Q: How do you replace nested if/else cleanly?** Guard clauses, combined conditions, lookup tables (dicts), or polymorphism.

PYTHON FOR DATA SCIENCE · LESSON 5

# Loops: while, for, range, break, continue & Patterns

Repeat work without repeating code. Learn while and for loops, range(), enumerate(), zip(), break, continue, the loop else clause, nested loops, accumulation, search and counting patterns, and common loop bugs; with an eye on when to replace loops with vectorised pandas/NumPy.

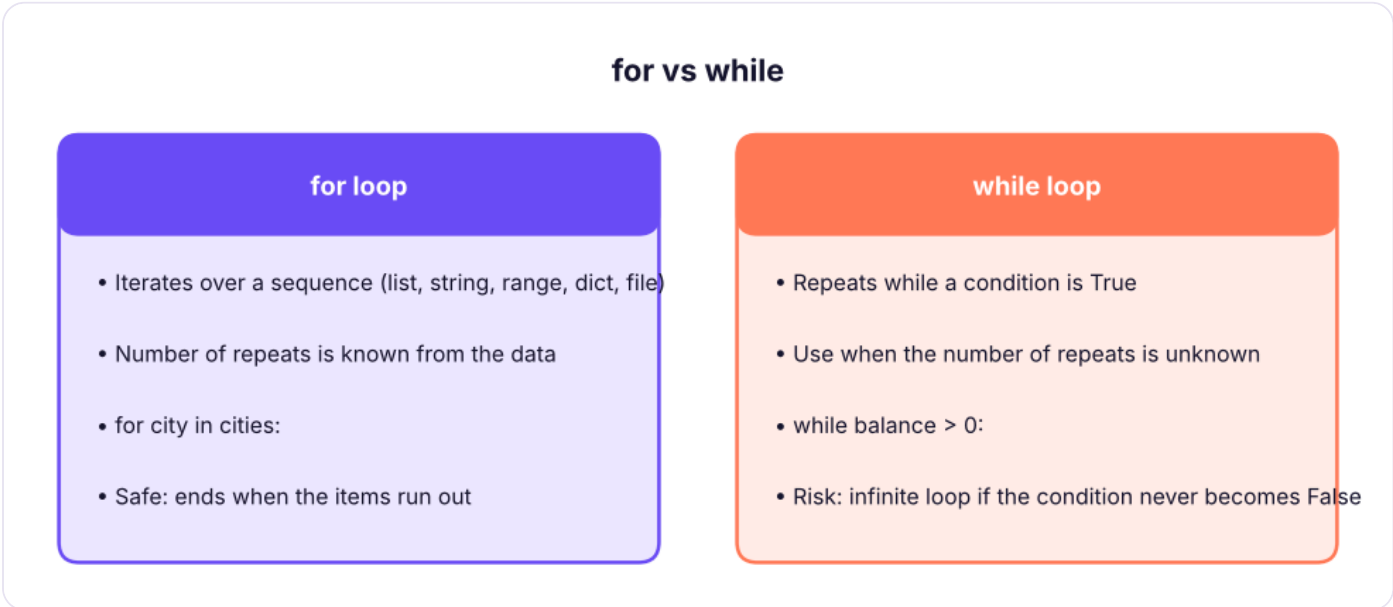
Python classes 3–4 | Python 3.13 | Beginner | 110 min

**By the end of this lesson you can**

- Write while and for loops and choose between them
- Use range(), enumerate() and zip() for index, value and parallel iteration
- Control loops with break, continue and the loop-else clause; avoid infinite loops
- Apply accumulator, counter, search, nested-loop and pattern-printing recipes
- Recognise when a loop should be replaced by a built-in, comprehension or vectorised operation

## Why loops?

A **loop** repeats a block of code. Processing 10,000 rows, trying each file in a folder, retrying a failed request, or checking each number in a list are all loops.




Two kinds of loop. for repeats over a known sequence; while repeats as long as a condition stays true.

## The while loop

Python

```
count = 1
while count ⩽ 5:
    print("Count:", count)
    count += 1               # without this line the loop would never end!
print("Done")
```

OUTPUT

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Done
```

Every while loop needs: **(1)** a starting value, **(2)** a condition, **(3)** a change inside the loop that eventually makes the condition False.

Python

```
# Compound growth: how many years until 1,00,000 doubles at 8%?
amount, years = 100000, 0
while amount < 200000:
    amount *= 1.08
    years += 1
print(f"{years} years → {amount:,.0f}")          # the 'rule of 72': 72 / 8 = 9 years
```

OUTPUT

```
10 years → 215,892
```

## while True with break

Use an "infinite" loop that you leave deliberately when the input is right:

Python

```
attempts = ["abc", "-4", "12"]        # simulated user answers
i = 0
while True:
    answer = attempts[i]; i += 1
    if answer.isdigit() and int(answer) > 0:
        print("Accepted:", answer); break
    print("Invalid:", answer)
```

OUTPUT

```
Invalid: abc
Invalid: -4
Accepted: 12
```

## The for loop

`for item in sequence:` takes each item in turn.

Python

```python
for city in ["Pune", "Delhi", "Kochi"]:
    print(city.upper())
for ch in "SQL":
    print(ch, end=" ")
print()
```

OUTPUT

```
PUNE
DELHI
KOCHI
S Q L
```

### range()

`range(stop)`, `range(start, stop)`, `range(start, stop, step)` produce integers (stop excluded). It is lazy: `range(1_000_000)` does not store a million numbers.

Python

```python
print(list(range(5)), list(range(2, 8)), list(range(10, 0, -3)), list(range(0, 1)))
total = 0
for n in range(1, 101): total += n
print("Sum 1..100 =", total, "| check:", sum(range(1, 101)), "| formula:", 100 * 101 // 2)
```

OUTPUT

```
[0, 1, 2, 3, 4] [2, 3, 4, 5, 6, 7] [10, 7, 4, 1] [0]
Sum 1..100 = 5050 | check: 5050 | formula: 5050
```

## enumerate() and zip()

Python

```
products = ["Pen", "Notebook", "Stapler"]
prices = [10, 45, 180]
for i, p in enumerate(products, start=1):         # index and value together
    print(i, p)
for p, price in zip(products, prices):            # walk two lists in parallel
    print(f"{p:<10}{price:>5}")
print(dict(zip(products, prices)))                 # build a dictionary from two lists
```

OUTPUT

```
1 Pen
2 Notebook
3 Stapler
Pen          10
Notebook     45
Stapler     180
{'Pen': 10, 'Notebook': 45, 'Stapler': 180}
```

Avoid `for i in range(len(x))` when you only need the items; use `for item in x` or `enumerate`.

## break, continue, else

Python

```
for n in range(1, 20):
    if n % 3 != 0: continue            # skip numbers not divisible by 3
    if n > 12: break                    # leave the loop completely
    print(n, end=" ")
print()
for n in range(2, 6):
    if n == 9: break
else:                                    # runs only if the loop finished WITHOUT break
    print("loop completed with no break")
```

OUTPUT

```
3 6 9 12
loop completed with no break
```

| Keyword | Effect |
| --- | --- |
| `break` | Exit the nearest loop immediately |
| `continue` | Skip to the next iteration |
| `else` on a loop | Runs if the loop ended normally (no `break`): handy for "search and report not found" |
| `pass` | Do nothing (placeholder) |

# Loop patterns

## Accumulate (sum / product / build a list)

Python

```python
orders = [1200, 450, 3000, 780, 2200]
total, large = 0, []
for amount in orders:
    total += amount
    if amount ⩾ 1000: large.append(amount)
print(total, large, total / len(orders))
```

OUTPUT

```
7630 [1200, 3000, 2200] 1526.0
```

## Count and find the maximum

Python

```python
sales = [("North", 120), ("South", 95), ("East", 140), ("West", 88)]
best_region, best_value = None, float("-inf")
count_above_100 = 0
for region, value in sales:
    if value > best_value: best_region, best_value = region, value
    if value > 100: count_above_100 += 1
print(best_region, best_value, count_above_100)
```

OUTPUT

```
East 140 2
```

(`max(sales, key=lambda x: x[1])` does the same in one line, Lesson 10.)

## Search with early exit

Python

```python
def find_first(items, target):
    for i, x in enumerate(items):
        if x == target: return i          # stop at the first match
    return -1
print(find_first(["a", "b", "c"], "c"), find_first(["a", "b"], "z"))
```

OUTPUT

```
2 -1
```

## Nested loops

Python

```python
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end="\t")
    print()
```

OUTPUT

```
1	2	3
2	4	6
3	6	9
```

Python

```python
# Pattern: right triangle of stars
for row in range(1, 6): print("*" * row)
```

OUTPUT

```
*
**
***
****
*****
```

Nested loops multiply the work: 1,000 × 1,000 = 1,000,000 iterations. Be careful with big data.

### Reading until a sentinel

```python
readings = iter([12, 15, 11, -1, 99])          # -1 marks the end
total = 0
for r in readings:
    if r == -1: break
    total += r
print("Total before sentinel:", total)
```

OUTPUT

```
Total before sentinel: 38
```

## Data examples

### Looping over rows (and why it is slow)

```python
import pandas as pd, time
s = pd.read_csv("superstore_sales.csv")
t0 = time.perf_counter()
loss_lines = 0
for profit in s["profit"]:                    # plain loop over a column (6,323 values)
    if profit < 0: loss_lines += 1
t1 = time.perf_counter()
vec = (s["profit"] < 0).sum()                  # vectorised: one operation on the whole column
t2 = time.perf_counter()
print(f"loop: {loss_lines} loss-making lines in {1000 * (t1 - t0):.2f} ms | vectorised: {vec} in
{1000 * (t2 - t1):.2f} ms")
```

OUTPUT

```
loop: 1682 loss-making lines in 0.84 ms | vectorised: 1682 in 0.34 ms
```

Same answer; the vectorised version is usually many times faster and shorter. **Rule:** in pandas/NumPy avoid Python loops over rows; use column operations, `groupby`, `np.where`. Use loops for files, APIs, orchestration, and small collections.

## Summarising with a loop

Python

```python
totals = {}
for region, category, sales in zip(s["region"][:2000], s["category"][:2000], s["sales"][:2000]):
    key = (region, category)
    totals[key] = totals.get(key, 0) + sales
for k in sorted(totals)[:6]: print(k, f"{totals[k]:,.0f}")
```

OUTPUT

```
('East', 'Furniture') 2,148,878
('East', 'Office Supplies') 717,588
('East', 'Technology') 10,862,196
('North', 'Furniture') 2,782,383
('North', 'Office Supplies') 645,662
('North', 'Technology') 13,766,572
```

This is a hand-made `GROUP BY`: you now see what `groupby` does behind the scenes.

# Common loop bugs

| Bug | Symptom | Fix |
|---|---|---|
| Missing update in `while` | Runs forever (press ■ to interrupt) | Change the condition variable inside the loop |
| Off-by-one with `range` | Misses the last item | Remember stop is excluded: `range(1, n + 1)` |
| Modifying a list while iterating over it | Skipped items | Iterate over a copy `for x in items[:]` or build a new list |
| `for i in range(len(a))` then using a wrong index | `IndexError` | Use `enumerate` / `zip` |
| Re-using a loop variable after the loop | Holds the last value | Avoid relying on it |
| Resetting the accumulator inside the loop | Wrong totals | Initialise **before** the loop |
| String `+=` in a loop | Slow for big text | `"".join(parts)` |

**TIP** **Debug with print or a debugger**

Print the loop variable and key values each iteration ( `print(i, value, total)` ), or use the debugger in VS Code/Jupyter ( `%debug` ). Test the loop on a tiny example by hand first.

## Summary

**KEY TAKEAWAYS**

- `while` repeats while a condition holds (make sure something changes!); `for` iterates over any sequence.
- `range(start, stop, step)` generates integers, stop excluded; `enumerate` gives index and value; `zip` iterates in parallel.
- `break` exits, `continue` skips to the next item, loop `else` runs when no `break` happened.
- Patterns: accumulator, counter, max/min tracking, search with early exit, nested loops, sentinel.
- For tables of data prefer **vectorised** pandas/NumPy over row loops; use comprehensions and built-ins for small collections.

### Quick quiz

6 questions · tap an answer to check it

**1.** What does `list(range(2, 10, 3))` give?

A [2, 5, 8, 11]

B [2, 5, 8]

C [3, 6, 9]

D [2, 10]

**Answer: B.** Start 2, step 3, stop 10 excluded.

**2. What does `continue` do inside a loop?**

A Ends the loop

B Skips the rest of this iteration and moves to the next

C Restarts the program

D Pauses the loop

**Answer: B.** `break` ends the loop; `continue` skips one iteration.

**3. When does a loop's `else` block run?**

A When the loop has an error

B When the loop finishes without hitting `break`

C Before the loop

D Every iteration

**Answer: B.** It is useful for "not found" logic after a search loop.

**4. Why is this loop infinite? `n = 5` then `while n > 0: print(n)`**

A print is wrong

B n is never changed inside the loop

C 5 is too large

D range is missing

**Answer: B.** The condition never becomes False.

**5.** What is a better way to count rows with negative profit in pandas?

A A for loop over rows

B `(df["profit"] < 0).sum()`

C A while loop

D Nested loops

**Answer: B.** Vectorised operations are shorter and faster.

**6.** Which function walks two lists at the same time?

A `range`

B `zip`

C `sorted`

D `len`

**Answer: B.** `zip(a, b)` pairs items by position.

EXERCISE

### Loop problems

(1) Print the multiplication table of 7 up to 7 × 10. (2) Given a list of order values, print the first one above 5,000 and its position, or "none". (3) Count vowels in a sentence. (4) Print the first 10 Fibonacci numbers. (5) FizzBuzz for 1..30.

**Show solution (worked answer)**

Python

```
for i in range(1, 11): print(f"7 x {i} = {7 * i}")
vals = [900, 4200, 6100, 800]
for i, v in enumerate(vals):
    if v > 5000: print(v, "at", i); break
else: print("none")
print(sum(ch in "aeiou" for ch in "data engineering is fun".lower()))
a, b = 0, 1
for _ in range(10): print(a, end=" "); a, b = b, a + b
print()
for n in range(1, 31): print("FizzBuzz" if n % 15 == 0 else "Fizz" if n % 3 == 0 else
"Buzz" if n % 5 == 0 else n, end=" ")
```

INTERVIEW CORNER

### Questions you may be asked

**Q: FizzBuzz, reverse a string, print Fibonacci: why do interviewers ask these?** They test loops, conditions, modulo, variable swapping and clear thinking, and whether you handle edge cases.

**Q: for vs while?** `for` iterates over a known sequence; `while` loops until a condition changes, for unknown counts such as retries or reading until a sentinel.

**Q: How would you speed up a slow loop over a DataFrame?** Vectorise (column operations, `np.where`, `groupby`), use built-ins or `apply` only when needed, or push the work into SQL.

**Q: What does `for i in range(len(lst))` signal?** Often a less Pythonic style; prefer iterating over items, `enumerate` or `zip` unless you need indexes to modify items.

PYTHON FOR DATA SCIENCE · LESSON 6

# Lists: Creating, Slicing, Methods, Sorting & Comprehensions

Store and process ordered collections with lists. Index and slice them, add, remove and sort items, understand mutability, aliasing and copying (shallow vs deep), use nested lists, list comprehensions, and common patterns for cleaning and summarising data.

Python class 5 | Python 3.13 | Beginner | 120 min

**By the end of this lesson you can**

- Create lists and use indexing, slicing, concatenation, repetition and membership tests
- Apply list methods (append, extend, insert, remove, pop, sort, reverse, index, count) and built-ins (len, sum, min, max, sorted)
- Explain mutability, aliasing and the difference between assignment, shallow copy and deep copy
- Build lists with comprehensions, including conditions and nested loops
- Use lists for typical data tasks: filtering, ranking, de-duplicating while keeping order, flattening and unzipping

## What is a list?

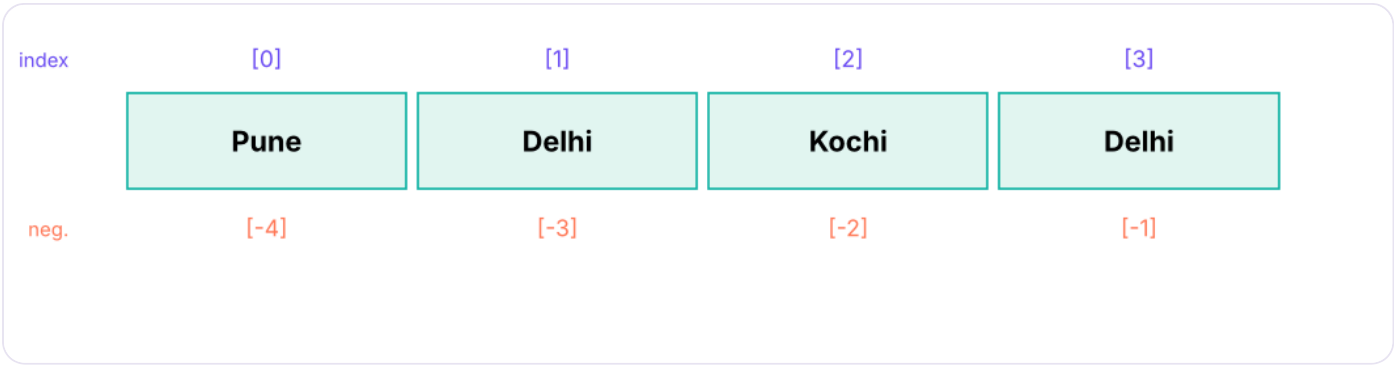
A **list** is an ordered, **mutable** collection of items written in square brackets. Items can be of any type, and may repeat.

Python

```
cities = ["Pune", "Delhi", "Kochi", "Delhi"]
mixed = [101, "Kavya", 97000.5, True, None]
empty = []
print(cities, len(cities), type(cities).__name__)
print(list("data"), list(range(5)), list((1, 2, 3)), [0] * 5)
```

OUTPUT

```
['Pune', 'Delhi', 'Kochi', 'Delhi'] 4 list
['d', 'a', 't', 'a'] [0, 1, 2, 3, 4] [1, 2, 3] [0, 0, 0, 0, 0]
```




A list is a row of numbered boxes. Items are referenced by position; the list itself can grow, shrink and change.

## Indexing and slicing

Works exactly like strings (Lesson 3), but lists can also be **changed**.

Python

```
nums = [10, 20, 30, 40, 50, 60]
print(nums[0], nums[-1], nums[1:4], nums[:2], nums[-2:], nums[::2], nums[::-1])
nums[0] = 99                  # change one item
nums[1:3] = [21, 31, 32]      # replace a slice (the length may change)
print(nums)
print(3 in nums, 99 in nums, nums.count(40), nums.index(40))
```

OUTPUT

```
10 60 [20, 30, 40] [10, 20] [50, 60] [10, 30, 50] [60, 50, 40, 30, 20, 10]
[99, 21, 31, 32, 40, 50, 60]
False True 1 4
```

## Adding and removing items

| Method | What it does | Example | Returns |
|---|---|---|---|
| `append(x)` | Add one item at the end | `a.append(5)` | None |
| `extend(iterable)` or `+=` | Add all items from another sequence | `a.extend([6, 7])` | None |
| `insert(i, x)` | Insert before position i | `a.insert(0, "first")` | None |
| `remove(x)` | Delete the first item equal to x | `a.remove(5)` | None (error if missing) |
| `pop(i=-1)` | Remove and return item at i | `last = a.pop()` | the item |
| `del a[i]` / `del a[i:j]` | Delete by position or slice | `del a[0]` | |
| `clear()` | Empty the list | `a.clear()` | None |

Python

```
a = [1, 2, 3]
a.append(4); a.extend([5, 6]); a.insert(0, 0); a += [7]
print(a)
print(a.pop(), a.pop(0), a)          # removes 7, then 0
a.remove(3); del a[0]; print(a)
print([1, 2] + [3], [1, 2] * 2)       # concatenation creates new lists
print(a.append(9))                    # None! methods that change a list return None
```

OUTPUT

```
[0, 1, 2, 3, 4, 5, 6, 7]
7 0 [1, 2, 3, 4, 5, 6]
[2, 4, 5, 6]
[1, 2, 3] [1, 2, 1, 2]
None
```

COMMON MISTAKES

### `x = x.append(...)`

`append`, `sort`, `reverse` and `extend` change the list **in place** and return `None`. Writing `nums = nums.sort()` leaves `nums` as `None`. Use `sorted(nums)` when you want a new list.

## Sorting and reversing

Python

```
scores = [72, 95, 61, 88, 95]
print(sorted(scores), sorted(scores, reverse=True), scores)        # sorted() returns a new
list
scores.sort(); print(scores)                                        # .sort() changes the list
in place
words = ["banana", "Apple", "cherry", "apple"]
print(sorted(words), sorted(words, key=str.lower), sorted(words, key=len))
staff = [("Asha", 29, 61000), ("Ravi", 34, 88000), ("Meera", 41, 72000)]
print(sorted(staff, key=lambda p: p[2], reverse=True))              # sort records by the 3rd
field
print(list(reversed([1, 2, 3])), min(staff, key=lambda p: p[1])[0])
```

OUTPUT

```
[61, 72, 88, 95, 95] [95, 95, 88, 72, 61] [72, 95, 61, 88, 95]
[61, 72, 88, 95, 95]
['Apple', 'apple', 'banana', 'cherry'] ['Apple', 'apple', 'banana', 'cherry'] ['Apple',
'apple', 'banana', 'cherry']
[('Ravi', 34, 88000), ('Meera', 41, 72000), ('Asha', 29, 61000)]
[3, 2, 1] Asha
```

`sorted()` and `.sort()` are **stable**: equal items keep their original order, so you can sort by several keys by sorting repeatedly from the least to the most important key, or use a tuple key: `key=lambda p: (p[1], -p[2])`.

## Built-in helpers

Python

```
data = [4, 8, 15, 16, 23, 42]
print(len(data), sum(data), min(data), max(data), sum(data) / len(data))
print(any(x > 40 for x in data), all(x > 0 for x in data))
print(list(enumerate(["a", "b"], 1)), list(zip([1, 2, 3], "abc")))
a, b, *rest = data; print(a, b, rest)                               # unpacking with *
first, *middle, last = data; print(first, middle, last)
```

OUTPUT

```
6 108 4 42 18.0
True True
[(1, 'a'), (2, 'b')] [(1, 'a'), (2, 'b'), (3, 'c')]
4 8 [15, 16, 23, 42]
4 [8, 15, 16, 23] 42
```

# Mutability, aliasing and copying

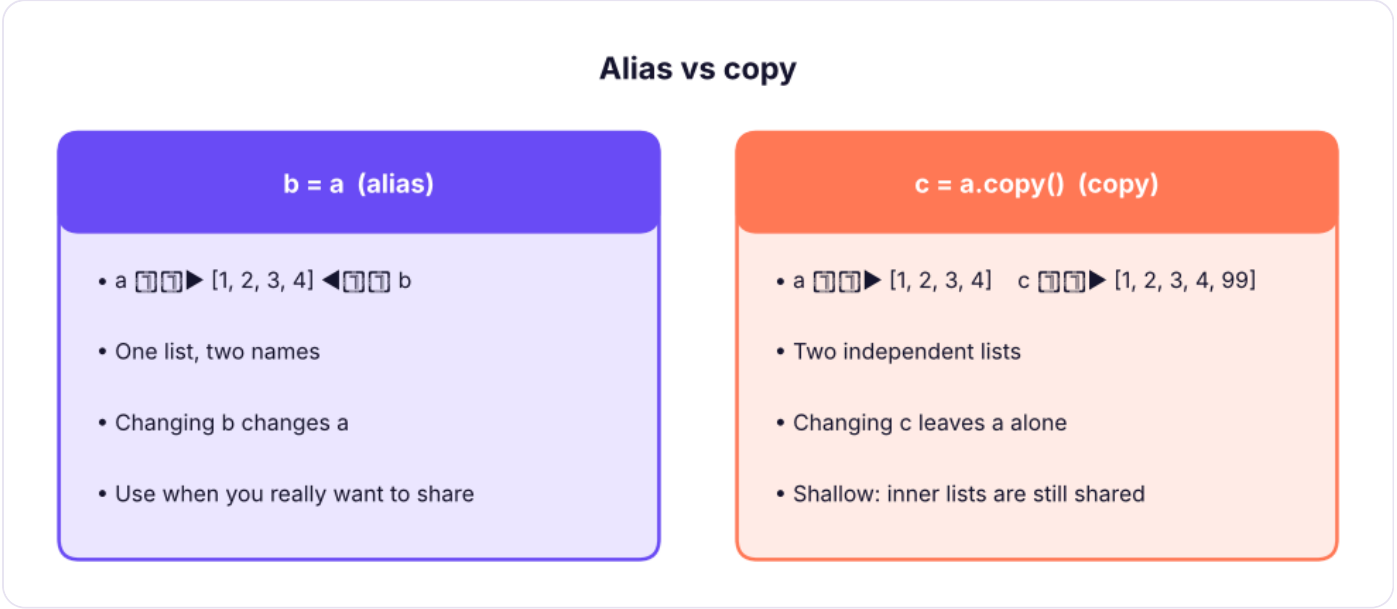
A variable holds a **reference** to a list. Assigning a list to another name does **not** copy it.

Python

```
a = [1, 2, 3]
b = a                      # b is another name for the SAME list
b.append(4)
print(a, b, a is b)
c = a.copy()               # a real (shallow) copy; also list(a) or a[:]
c.append(99)
print(a, c, a is c)
```

OUTPUT

```
[1, 2, 3, 4] [1, 2, 3, 4] True
[1, 2, 3, 4] [1, 2, 3, 4, 99] False
```




Assignment gives two names for one list; copy() makes a second list. A shallow copy still shares inner lists.

Python

```
import copy
grid = [[1, 2], [3, 4]]
shallow = grid.copy(); deep = copy.deepcopy(grid)
grid[0][0] = 100
print(shallow, deep)       # shallow copy sees the change in the inner list; deep copy does not
```

OUTPUT

```
[[100, 2], [3, 4]] [[1, 2], [3, 4]]
```

**WATCH OUT** **Mutable default arguments and `[[0] * 3] * 3`**

`[[0] * 3] * 3` creates **three references to the same inner list**; changing one row changes all. Build grids with a comprehension: `[[0] * 3 for _ in range(3)]`. Never use a list as a default function argument ( `def f(x, items=[])` ): use `None` and create the list inside (Lesson 9).

## Nested lists

Python

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(matrix[1][2], [row[0] for row in matrix], [sum(row) for row in matrix])
print([list(col) for col in zip(*matrix)])                # transpose
flat = [x for row in matrix for x in row]; print(flat)    # flatten
```

OUTPUT

```
6 [1, 4, 7] [6, 15, 24]
[[1, 4, 7], [2, 5, 8], [3, 6, 9]]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

## List comprehensions

A comprehension builds a list in one readable line: `[expression for item in iterable if condition]`.

Python

```
squares = [n ** 2 for n in range(1, 8)]
evens = [n for n in range(20) if n % 2 == 0]
upper = [c.upper() for c in ["pune", "delhi"]]
labels = ["High" if s ⩾ 80 else "Low" for s in [95, 60, 82]]
pairs = [(i, j) for i in range(1, 4) for j in range(i, 4)]
print(squares, evens, upper, labels, pairs, sep="\n")
```

OUTPUT

```
[1, 4, 9, 16, 25, 36, 49]
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
['PUNE', 'DELHI']
['High', 'Low', 'High']
[(1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)]
```

Compare with the loop version:

Python

```
result = []
for n in range(20):
    if n % 2 == 0:
        result.append(n)
print(result == [n for n in range(20) if n % 2 == 0])
```

OUTPUT

```
True
```

Use comprehensions for **simple** transformations and filters; switch to a loop when the logic needs several statements. For very large data use generators (Lesson 10) or pandas.

## Data tasks with lists

Python

```
import pandas as pd
emp = pd.read_csv("employees_small.csv")
salaries = emp.salary.tolist(); names = emp.name.tolist(); depts = emp.department.tolist()
top3 = sorted(zip(salaries, names), reverse=True)[:3]
print("Top 3:", [(n, s) for s, n in top3])
fin = [s for s, d in zip(salaries, depts) if d == "Finance"]
print("Finance:", len(fin), "people, average", round(sum(fin) / len(fin)))
above_avg = [n for n, s in zip(names, salaries) if s > sum(salaries) / len(salaries)]
print(len(above_avg), "people earn above the average;", above_avg[:3], "…")
```

OUTPUT

```
Top 3: [('Vivaan Gupta', 157000), ('Imran Reddy', 157000), ('Krishna Khan', 156000)]
Finance: 7 people, average 95714
9 people earn above the average; ['Imran Reddy', 'Harpreet Kulkarni', 'Simran Iyer'] …
```

Python

```
# Remove duplicates but keep the original order (set() would lose the order)
raw = ["Pune", "Delhi", "Pune", "Kochi", "Delhi", "Goa"]
seen, unique = set(), []
for c in raw:
    if c not in seen: seen.add(c); unique.append(c)
print(unique, "| shortcut:", list(dict.fromkeys(raw)))
# Chunk a list into groups of 4
ids = list(range(1, 11)); print([ids[i:i + 4] for i in range(0, len(ids), 4)])
# Moving average of 3
x = [10, 12, 9, 14, 15, 13]; print([round(sum(x[i:i + 3]) / 3, 1) for i in range(len(x) - 2)])
```

OUTPUT

```
['Pune', 'Delhi', 'Kochi', 'Goa'] | shortcut: ['Pune', 'Delhi', 'Kochi', 'Goa']
[[1, 2, 3, 4], [5, 6, 7, 8], [9, 10]]
[10.3, 11.7, 12.7, 14.0]
```

## Performance notes

| Operation | Cost |
|---|---|
| `a[i]`, `a.append(x)`, `a.pop()` | O(1) |
| `x in a`, `a.index(x)`, `a.remove(x)`, `a.insert(0, x)` | O(n) |
| `a.sort()` | O(n log n) |

For many membership tests use a **set** (O(1)); for a queue use `collections.deque`. (More in the DSA lessons.)

## Summary

**KEY TAKEAWAYS**

- A list is an ordered, **mutable** collection; index and slice like strings; negative indexes count from the end.
- `append`, `extend`, `insert`, `remove`, `pop`, `sort`, `reverse` change the list in place and return `None`; `sorted()` and `reversed()` give new objects.
- `b = a` is an **alias**; use `a.copy()` (shallow) or `copy.deepcopy(a)` for independent copies; build grids with comprehensions.
- Sort records with `key=` (and tuple keys); sorting is stable.
- Comprehensions `[f(x) for x in xs if cond]` create lists concisely; unpack with `a, *rest = items`.
- Membership tests on lists are O(n): use sets for lookups.

### Quick quiz

6 questions · tap an answer to check it

**1.** What does `[1, 2, 3].append(4)` return?

**A** [1, 2, 3, 4]

**B** None (the list is changed in place)

**C** 4

**D** A new list

**Answer: B.** Mutating methods return None.

**2.** After `b = a; b.append(1)`, what is true?

**A** a is unchanged

**B** a and b are the same list, so a also contains the new item

**C** b is a copy

**D** An error occurs

**Answer: B.** Assignment creates an alias, not a copy.

**3.** What does `[x * 2 for x in range(5) if x % 2 == 0]` return?

**A** [0, 2, 4, 6, 8]

**B** [0, 4, 8]

**C** [0, 2, 4]

**D** [2, 6]

**Answer: B.** Only x = 0, 2, 4 pass the filter and are doubled.

**4.** How do you sort a list of tuples by the second element, descending?

A `sorted(t)`

B `sorted(t, key=lambda p: p[1], reverse=True)`

C `t.sort(1)`

D `sort(t, 1)`

**Answer: B.** The key function selects the field to compare.

**5.** Why does `grid = [[0] * 3] * 3; grid[0][0] = 1` change every row?

A A bug in Python

B The outer multiplication repeats references to the same inner list

C Lists are immutable

D Comprehensions are required for all lists

**Answer: B.** Use `[[0] * 3 for _ in range(3)]`.

**6.** Which is the fastest way to test if an item is in a large collection many times?

A A list

B A set

C A string

D A tuple

**Answer: B.** Sets use hashing for O(1) average lookups.

EXERCISE

### List drills

(1) From `[3, 8, 1, 9, 4, 8]` return the second-largest distinct value. (2) Remove duplicates from `["b","a","b","c","a"]` keeping order. (3) Flatten `[[1,2],[3],[4,5,6]]`. (4) Given names and salaries lists, print "name: salary" sorted by salary descending. (5) Rotate `[1,2,3,4,5]` by 2 positions to the right.

**Show solution (worked answer)**

Python

```python
print(sorted(set([3, 8, 1, 9, 4, 8]))[-2])                      # 8
print(list(dict.fromkeys(["b", "a", "b", "c", "a"])))           # ['b', 'a', 'c']
print([x for sub in [[1, 2], [3], [4, 5, 6]] for x in sub])     # [1, 2, 3, 4, 5, 6]
names, sal = ["A", "B", "C"], [50, 90, 70]
for n, s in sorted(zip(names, sal), key=lambda p: -p[1]): print(f"{n}: {s}")
r = [1, 2, 3, 4, 5]; print(r[-2:] + r[:-2])                     # [4, 5, 1, 2, 3]
```

INTERVIEW CORNER

### Questions you may be asked

**Q: Difference between a list and a tuple?** Lists are mutable, tuples immutable (hashable if their items are); tuples are used for fixed records and as dictionary keys, lists for collections that change.

**Q: Shallow vs deep copy?** A shallow copy duplicates the outer list but shares inner objects; a deep copy recursively copies everything.

**Q: What is the time complexity of `x in list` vs `x in set`?** O(n) vs O(1) average.

**Q: How do you reverse a list in place?** `lst.reverse()` or `lst[:] = lst[::-1]`; `lst[::-1]` creates a reversed copy.

PYTHON FOR DATA SCIENCE · LESSON 7

# Tuples & Sets: Immutable Records, Unpacking and Set Algebra

Use tuples for fixed records and multiple return values, master unpacking and named tuples, and use sets for uniqueness, fast membership tests and set algebra (union, intersection, difference) on real customer data.

Python class 5 | Python 3.13 | Beginner | 90 min

**By the end of this lesson you can**

- Create tuples, explain immutability and hashability, and unpack tuples including starred unpacking
- Use namedtuple and dataclass-style records for readable fixed structures
- Create sets and frozensets and use add, discard, remove, update and set operators
- Apply union, intersection, difference and symmetric difference to compare groups
- Choose between list, tuple, set and dict for a given task

## Tuples

A **tuple** is an ordered, **immutable** sequence written with commas (brackets are usually optional).

Python

```
point = (28.61, 77.20)
single = (5,)                    # the trailing comma makes a one-item tuple
empty = ()
rec = "E101", "Kavya Iyer", "HR", 97000        # packing: no brackets needed
print(point, type(single).__name__, type((5)).__name__, rec[1], rec[-1], rec[1:3], len(rec))
print(rec.count("HR"), rec.index("HR"), (1, 2) + (3,), (0,) * 3, 5 in (1, 5, 9))
try: rec[0] = "E102"
except TypeError as e: print("Error:", e)
```

OUTPUT

```
(28.61, 77.2) tuple int Kavya Iyer 97000 ('Kavya Iyer', 'HR') 4
1 2 (1, 2, 3) (0, 0, 0) True
Error: 'tuple' object does not support item assignment
```

## Unpacking

Python

```
emp_id, name, dept, salary = rec                # one name per item
a, b = 1, 2; a, b = b, a                          # swap
first, *others = (10, 20, 30, 40)                 # * collects the rest into a list
*head, last = (10, 20, 30, 40)
x, (y, z) = 1, (2, 3)                             # nested unpacking
_, dept_only, *_ = rec                            # _ means "ignore"
print(emp_id, name, a, b, first, others, head, last, x, y, z, dept_only)
for city, sales in [("Pune", 120), ("Delhi", 95)]:     # unpack in loops
    print(city, sales)
```

OUTPUT

```
E101 Kavya Iyer 2 1 10 [20, 30, 40] [10, 20, 30] 40 1 2 3 Kavya Iyer
Pune 120
Delhi 95
```

## Why tuples?

| Use | Example |
|---|---|
| **Fixed records** | `(lat, lon)`, `(year, month)`, a row from a database |
| **Multiple return values** | `def stats(x): return min(x), max(x)` → `lo, hi = stats(data)` |
| **Dictionary keys and set members** (hashable) | `sales[("North", "Technology")]` |
| **Protection** from accidental change | Constants like `METROS = ("Delhi", "Mumbai")` |
| **Slightly less memory and faster** than lists | |

Python

```
def describe(values):
    return min(values), max(values), sum(values) / len(values)
lo, hi, avg = describe([4, 8, 15, 16, 23, 42])
print(lo, hi, round(avg, 2), describe([1, 2]))
print(hash((1, 2)), "| a tuple containing a list is not hashable:", end=" ")
try: hash((1, [2]))
except TypeError as e: print(e)
```

OUTPUT

```
4 42 18.0 (1, 2, 1.5)
-3550055125485641917 | a tuple containing a list is not hashable: unhashable type: 'list'
```

NOTE **Tuples are immutable, but their contents might not be**

`t = ([1, 2], 3)`; `t[0].append(9)` works, because the tuple holds a **reference** to a mutable list. The tuple itself still points to the same list.

## Named tuples and data classes

When a record has many fields, positions like `rec[3]` are hard to read. Name the fields:

Python

```
from collections import namedtuple
from dataclasses import dataclass
Employee = namedtuple("Employee", ["emp_id", "name", "dept", "salary"])
e = Employee("E101", "Kavya Iyer", "HR", 97000)
print(e, e.name, e[3], e._asdict(), e._replace(salary=100000).salary)

@dataclass(frozen=True)                # frozen = immutable, like a tuple, but with types and
methods
class Product:
    sku: str
    price: float
    def with_gst(self, rate=0.18): return round(self.price * (1 + rate), 2)
p = Product("P101", 4500.0)
print(p, p.with_gst())
```

OUTPUT

```
Employee(emp_id='E101', name='Kavya Iyer', dept='HR', salary=97000) Kavya Iyer 97000
{'emp_id': 'E101', 'name': 'Kavya Iyer', 'dept': 'HR', 'salary': 97000} 100000
Product(sku='P101', price=4500.0) 5310.0
```

# Sets

A **set** is an **unordered** collection of **unique**, hashable items, written with braces. Sets support fast membership tests and mathematical set operations.

Python

```
cities = {"Pune", "Delhi", "Kochi", "Delhi"}          # duplicates vanish
print(cities, len(cities), "Pune" in cities)
empty = set()                                          # {} is an empty DICT, not a set!
print(set("mississippi"), set([3, 1, 2, 3, 1]), sorted({5, 2, 9}), type({}).__name__,
type(empty).__name__)
```

OUTPUT

```
{'Delhi', 'Pune', 'Kochi'} 3 True
{'m', 'p', 's', 'i'} {1, 2, 3} [2, 5, 9] dict set
```

| Method / operator | Effect |
|---|---|
| `s.add(x)` | Add an item |
| `s.update(iterable)` or `s \|= other` | Add many items |
| `s.discard(x)` | Remove x if present (no error) |
| `s.remove(x)` | Remove x; `KeyError` if missing |
| `s.pop()` | Remove and return an arbitrary item |
| `s.clear()` | Empty the set |
| `x in s` | Membership, **O(1)** on average |
| `len(s)` , `min(s)` , `max(s)` , `sum(s)` , `sorted(s)` | Usual helpers (sorted returns a list) |

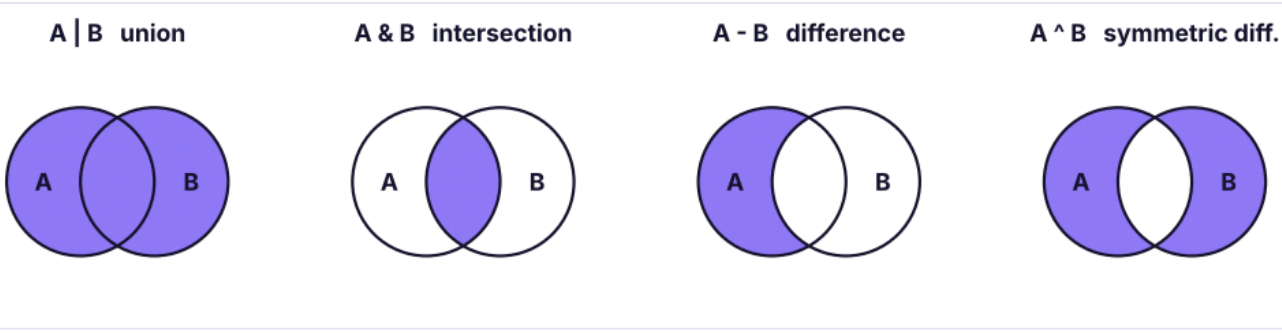
Python

```python
s = {1, 2, 3}
s.add(4); s.update([5, 6]); s.discard(10); s.remove(1)
print(s, s.pop() in {2, 3, 4, 5, 6})
```

OUTPUT

```
{3, 4, 5, 6} True
```

## Set algebra




The four set operations as Venn diagrams (A and B are sets of customers).

Python

```
a = {"Asha", "Ravi", "Meera", "Kabir"}
b = {"Meera", "Kabir", "Zoya", "Neha"}
print("union       :", sorted(a | b), "  also a.union(b)")
print("intersection:", sorted(a & b), "  also a.intersection(b)")
print("A - B       :", sorted(a - b), "  also a.difference(b)")
print("B - A       :", sorted(b - a))
print("A ^ B       :", sorted(a ^ b), "  (in exactly one)")
print(a ⩽ {"Asha", "Ravi", "Meera", "Kabir", "X"}, {"Asha"} < a, a.isdisjoint({"Zoya"}))     # subset, proper subset, disjoint
```

OUTPUT

```
union       : ['Asha', 'Kabir', 'Meera', 'Neha', 'Ravi', 'Zoya']   also a.union(b)
intersection: ['Kabir', 'Meera']   also a.intersection(b)
A - B       : ['Asha', 'Ravi']   also a.difference(b)
B - A       : ['Neha', 'Zoya']
A ^ B       : ['Asha', 'Neha', 'Ravi', 'Zoya']   (in exactly one)
True True True
```

These are the same operations as SQL's `UNION`, `INTERSECT` and `EXCEPT` (SQL Lesson 9).

## Frozen sets

`frozenset` is an immutable set: it can be a dictionary key or a member of another set.

Python

```
fs = frozenset({"North", "South"})
print(fs, {fs: "regions"}, fs | {"East"})
```

OUTPUT

```
frozenset({'North', 'South'}) {frozenset({'North', 'South'}): 'regions'}
frozenset({'North', 'South', 'East'})
```

# Real data: customer movement with sets

The Superstore data lets us rebuild the retention analysis from SQL Lesson 9 with sets:

Python

```
import pandas as pd
s = pd.read_csv("superstore_sales.csv", parse_dates=["order_date"])
cust = {y: set(s.loc[s.order_date.dt.year == y, "customer_id"]) for y in (2023, 2024, 2025)}
print({y: len(c) for y, c in cust.items()})
retained = cust[2024] & cust[2025]
lost = cust[2024] - cust[2025]
new_in_2025 = cust[2025] - (cust[2023] | cust[2024])
print(f"Retained 2024→2025: {len(retained)} | lost: {len(lost)} | brand new in 2025:
{len(new_in_2025)}")
print(f"Retention rate: {len(retained) / len(cust[2024]):.1%}  (matches the SQL result: 467 of
590)")
print("Bought in all 3 years:", len(cust[2023] & cust[2024] & cust[2025]), "| bought in exactly
one year:", len((cust[2023] ^ cust[2024] ^ cust[2025]) - (cust[2023] & cust[2024] &
cust[2025])))
```

OUTPUT

```
{2023: 540, 2024: 590, 2025: 623}
Retained 2024→2025: 467 | lost: 123 | brand new in 2025: 52
Retention rate: 79.2%  (matches the SQL result: 467 of 590)
Bought in all 3 years: 311 | bought in exactly one year: 128
```

Same answers as the SQL queries, in three lines. Sets are also the fastest way to **de-duplicate** and to **check membership** in big collections:

Python

```
import time, random
big_list = list(range(200_000)); big_set = set(big_list); probes = [random.randrange(400_000)
for _ in range(300)]
t0 = time.perf_counter(); [p in big_list for p in probes]; t1 = time.perf_counter(); [p in
big_set for p in probes]; t2 = time.perf_counter()
print(f"300 lookups: list {1000 * (t1 - t0):.1f} ms | set {1000 * (t2 - t1):.3f} ms")
```

OUTPUT

```
300 lookups: list 329.5 ms | set 0.244 ms
```

## Choosing a collection

| Need | Use |
|---|---|
| Ordered, changeable sequence | `list` |
| Ordered, fixed record or dict key | `tuple` |
| Unique items, fast "is it in there?", set algebra | `set` |
| Key → value lookup | `dict` (next lesson) |
| Unique **and** ordered | `dict.fromkeys(items)` or a list + set |
| FIFO queue | `collections.deque` |
| Counting | `collections.Counter` |

## Summary

**KEY TAKEAWAYS**

- **Tuples** are ordered and immutable: use them for fixed records, multiple return values and dictionary keys; unpack with `a, b = t` and `first, *rest = t`.
- A one-item tuple needs a comma: `(5,)`. Use **namedtuple** or a frozen **dataclass** for readable records.
- **Sets** hold unique, hashable items with O(1) membership; `{}` is an empty dict, so use `set()`.
- Set algebra: `|` union, `&` intersection, `-` difference, `^` symmetric difference; subset tests with `⩽`.
- Sets turn retention/overlap analyses into one-liners and are far faster than lists for membership tests.

### Quick quiz

6 questions · tap an answer to check it

**1.** What is `type((5))` ?

**A** tuple

**B** int

**C** list

**D** set

**Answer: B.** Brackets alone do not make a tuple; `(5,)` does.

**2.** What does `{1, 2, 3} & {2, 3, 4}` return?

**A** {1, 2, 3, 4}

**B** {2, 3}

**C** {1, 4}

**D** {1}

**Answer: B.** `&` is the intersection.

**3.** Which can be used as a dictionary key?

**A** A list

**B** A tuple of strings

**C** A set

**D** A dict

**Answer: B.** Keys must be hashable; lists, sets and dicts are not.

**4.** How do you create an empty set?

A `{}`

B `set()`

C `[]`

D `()`

**Answer: B.** `{}` creates an empty dictionary.

**5.** What does `first, *rest = (1, 2, 3, 4)` assign to `rest`?

A (2, 3, 4)

B [2, 3, 4]

C 4

D 2

**Answer: B.** Starred unpacking collects the remaining items into a list.

**6.** Why are set lookups faster than list lookups?

A Sets are sorted

B Sets use hashing, giving O(1) average membership tests

C Sets store fewer items

D They are not faster

**Answer: B.** A list scans item by item (O(n)).

EXERCISE

### Set problems

(1) Find the common cities between `{"Pune","Delhi","Goa"}` and `{"Goa","Kochi","Pune"}`. (2) Customers who bought in 2023 and 2024 but not in 2025 (use the Superstore sets). (3) Write `min_max_mean(values)` returning a tuple, and unpack it. (4) Check whether two words are anagrams using `sorted` and also using `collections.Counter`.

**Show solution (worked answer)**

Python

```
print({"Pune","Delhi","Goa"} & {"Goa","Kochi","Pune"})
print(len((cust[2023] & cust[2024]) - cust[2025]))
def min_max_mean(v): return min(v), max(v), sum(v) / len(v)
lo, hi, m = min_max_mean([3, 9, 6])
from collections import Counter
print(sorted("listen") == sorted("silent"), Counter("listen") == Counter("silent"))
```

INTERVIEW CORNER

### Questions you may be asked

**Q: Tuple vs list: when would you choose each?** Tuple for fixed, heterogeneous records, hashable keys and returning several values; list for homogeneous collections that change.

**Q: How do you remove duplicates from a list while preserving order?** `list(dict.fromkeys(items))`; `list(set(items))` does not preserve order.

**Q: What is the time complexity of set membership and why?** Average O(1) because sets are hash tables (worst case O(n) with many collisions).

**Q: How do you find items in A but not in B?** `A - B` for sets; for lists use a set for B: `[x for x in A if x not in set_b]`.

PYTHON FOR DATA SCIENCE · LESSON 8

# Dictionaries: Key-Value Data, Counting and Grouping

Store and look up data by key. Create and update dictionaries, use get, setdefault, items, Counter and defaultdict, nest dictionaries, and convert between dictionaries, JSON-like records and DataFrames using the Superstore data.

Python class 6 | Python 3.13 | Beginner | 100 min

**By the end of this lesson you can**

- Create dictionaries and read, add, update and delete keys safely
- Loop over keys, values and items and merge dictionaries
- Count and group with get, setdefault, Counter and defaultdict
- Work with nested dictionaries and lists of dictionaries (JSON-like records)
- Convert between dictionaries and pandas DataFrames

## What is a dictionary?

A **dictionary** ( `dict` ) maps **keys** to **values**: look up by name instead of by position. It keeps insertion order (Python 3.7+), and lookups are O(1) on average.




A dictionary is a set of key → value pairs. The key is hashed, so the lookup does not scan the data.

Python

```
emp = {"name": "Kavya Iyer", "dept": "HR", "salary": 97000}
print(emp["name"], emp.get("dept"), emp.get("city"), emp.get("city", "unknown"), len(emp),
"salary" in emp)
try: emp["city"]
except KeyError as e: print("KeyError:", e)
```

OUTPUT

```
Kavya Iyer HR None unknown 3 True
KeyError: 'city'
```

`d[key]` raises `KeyError` when missing; `d.get(key, default)` never does.

## Changing a dictionary

Python

```
emp["city"] = "Pune"                # add
emp["salary"] += 3000                # update
emp.update({"dept": "Finance", "grade": "M2"})
removed = emp.pop("grade")          # remove and return
del emp["city"]
print(emp, removed)
print(emp.setdefault("level", "L3"), emp.setdefault("dept", "X"), emp)    # set only if missing
```

OUTPUT

```
{'name': 'Kavya Iyer', 'dept': 'Finance', 'salary': 100000} M2
L3 Finance {'name': 'Kavya Iyer', 'dept': 'Finance', 'salary': 100000, 'level': 'L3'}
```

| Method | Effect |
| --- | --- |
| `d[k] = v` | Add or overwrite |
| `d.get(k, default)` | Safe read |
| `d.setdefault(k, default)` | Read; insert default if missing |
| `d.update(other)` or `d \| other` | Merge (right side wins) |
| `d.pop(k[, default])` | Remove and return |
| `d.popitem()` | Remove the last pair |
| `del d[k]` | Delete (KeyError if missing) |
| `d.keys()` , `d.values()` , `d.items()` | Views |
| `d.copy()` | Shallow copy |
| `dict.fromkeys(keys, value)` | Create with the same value |

## Looping

Python

```python
prices = {"Pen": 10, "Notebook": 45, "Stapler": 180}
for k in prices: print(k, end=" ")
print()
for k, v in prices.items(): print(f"{k:<10}{v:>5}")
print(list(prices.values()), sum(prices.values()), max(prices, key=prices.get),
sorted(prices.items(), key=lambda kv: -kv[1]))
merged = prices | {"Pen": 12, "Folder": 30}              # Python 3.9+
print(merged)
print({v: k for k, v in prices.items()})                 # invert (values must be unique and
hashable)
```

OUTPUT

```
Pen Notebook Stapler
Pen          10
Notebook     45
Stapler     180
[10, 45, 180] 235 Stapler [('Stapler', 180), ('Notebook', 45), ('Pen', 10)]
{'Pen': 12, 'Notebook': 45, 'Stapler': 180, 'Folder': 30}
{10: 'Pen', 45: 'Notebook', 180: 'Stapler'}
```

## Counting and grouping

The most common dictionary jobs are **count** and **group**:

Python

```python
from collections import Counter, defaultdict
words = "the cat and the hat and the bat".split()
manual = {}
for w in words: manual[w] = manual.get(w, 0) + 1            # classic pattern
print(manual)
c = Counter(words)
print(c, c.most_common(2), c["the"], c["dog"])
groups = defaultdict(list)                                   # missing key starts as an empty list
for w in words: groups[len(w)].append(w)
print(dict(groups))
```

OUTPUT

```
{'the': 3, 'cat': 1, 'and': 2, 'hat': 1, 'bat': 1}
Counter({'the': 3, 'and': 2, 'cat': 1, 'hat': 1, 'bat': 1}) [('the', 3), ('and', 2)] 3 0
{3: ['the', 'cat', 'and', 'the', 'hat', 'and', 'the', 'bat']}
```

Python

```python
import pandas as pd
s = pd.read_csv("superstore_sales.csv")
by_region = Counter(s["region"])                             # count rows
revenue = defaultdict(float)
for r, amount in zip(s["region"], s["sales"]): revenue[r] += amount
print(by_region.most_common())
print({k: round(v / 1e6, 1) for k, v in sorted(revenue.items(), key=lambda kv: -kv[1])}, "(sales in millions)")
print(s.groupby("region")["sales"].sum().div(1e6).round(1).sort_values(ascending=False).to_dict())   # same in pandas
```

OUTPUT

```
[('South', 1656), ('West', 1614), ('North', 1530), ('East', 1523)]
{'South': 54.0, 'West': 52.0, 'North': 49.0, 'East': 46.5} (sales in millions)
{'South': 54.0, 'West': 52.0, 'North': 49.0, 'East': 46.5}
```

## Nested dictionaries and JSON-like records

Real data (API responses, configs, documents) nests dictionaries and lists. Python's `json` module reads and writes exactly this structure.

Python

```
import json
order = {"order_id": "O-1001", "customer": {"id": "C7", "name": "Asha", "city": "Pune"},
         "items": [{"sku": "P1", "qty": 2, "price": 450.0}, {"sku": "P2", "qty": 1, "price":
1200.0}]}
total = sum(i["qty"] * i["price"] for i in order["items"])
print(order["customer"]["city"], len(order["items"]), total, order.get("shipping",
{}).get("city", "n/a"))
text = json.dumps(order, indent=2)
print(text[:90].replace("\n", " "), "...", json.loads(text) == order)
```

OUTPUT

```
Pune 2 2100.0 n/a
{   "order_id": "O-1001",   "customer": {     "id": "C7",     "name": "Asha",     "city":
... True
```

A **list of dictionaries** is the standard row format, and it converts straight to a DataFrame:

Python

```
rows = [{"sku": "P1", "qty": 2, "price": 450.0}, {"sku": "P2", "qty": 1, "price": 1200.0},
{"sku": "P3", "qty": 5, "price": 80.0}]
df = pd.DataFrame(rows)
df["value"] = df.qty * df.price
print(df)
print(df.set_index("sku")["value"].to_dict(), df.to_dict("records")[0])
```

OUTPUT

```
  sku  qty   price   value
0  P1    2   450.0   900.0
1  P2    1  1200.0  1200.0
2  P3    5    80.0   400.0
{'P1': 900.0, 'P2': 1200.0, 'P3': 400.0} {'sku': 'P1', 'qty': 2, 'price': 450.0, 'value':
900.0}
```

## Lookup tables and dispatch

Python

```
tier = {"North": "A", "South": "A", "East": "B", "West": "C"}                # mapping/lookup
print(s["region"].map(tier).value_counts().to_dict())
ops = {"add": lambda a, b: a + b, "mul": lambda a, b: a * b}                  # dictionary of
functions
print(ops["add"](3, 4), ops["mul"](3, 4))
```

OUTPUT

```
{'A': 3186, 'C': 1614, 'B': 1523}
7 12
```

**NOTE** **Keys must be hashable**

Strings, numbers and tuples work as keys; lists, sets and dicts do not. Use a tuple `("North", 2025)` for composite keys. Copying: `d.copy()` is shallow; use `copy.deepcopy` for nested data (Lesson 6).

## Common mistakes

| Mistake | Fix |
|---|---|
| `d["missing"]` raises KeyError | `d.get(k, default)` or `if k in d` |
| Changing a dictionary's size while looping over it | Loop over `list(d)` or build a new dict |
| `{}` for an empty set | `set()`; `{}` is an empty dict |
| Duplicate keys in a literal | The last one silently wins |
| Assuming a shallow copy protects nested data | `copy.deepcopy` |

## Summary

**KEY TAKEAWAYS**

- A dict maps hashable keys to values with O(1) average lookup and keeps insertion order.
- Use `get` for safe reads, `setdefault` / `defaultdict` for grouping, `Counter` for counting, `d | other` to merge.
- Loop with `items()`; build with dict comprehensions `{k: f(v) for k, v in ...}`.
- Nested dict/list structures are JSON; a list of dicts converts to a DataFrame with `pd.DataFrame(rows)`.
- Dictionaries are lookup tables: `series.map(dict)` is a fast way to recode a column.

### Quick quiz

6 questions · tap an answer to check it

**1.** What does `d.get("x", 0)` do when "x" is missing?

A Raises KeyError

B Returns 0

C Adds the key

D Returns None

**Answer: B.** `get` never raises; it returns the default.

**2.** Which is a valid dictionary key?

A `[1, 2]`

B `(1, 2)`

C `{1, 2}`

D `{"a": 1}`

**Answer: B.** Keys must be hashable (immutable).

**3.** Which tool counts occurrences most directly?

A `defaultdict(list)`

B `collections.Counter`

C `set`

D `zip`

**Answer: B.** `Counter(items).most_common()`.

**4.** What does `{"a": 1} | {"a": 2, "b": 3}` give?

A {"a": 1, "b": 3}

B {"a": 2, "b": 3}

C Error

D {"a": 3}

**Answer: B.** The right-hand dictionary wins on conflicts.

**5.** How do you loop over keys and values together?

A `for k in d.values()`

B `for k, v in d.items()`

C `for k, v in d`

D `for v in d.keys()`

**Answer: B.** `items()` yields (key, value) pairs.

**6.** What does `pd.DataFrame(list_of_dicts)` produce?

A A Series

B A table with one row per dictionary and columns from the keys

C A list

D An error

**Answer: B.** This is the standard way to turn JSON-like records into a table.

EXERCISE

### Dictionary problems

(1) Count the frequency of each character in `"banana"`. (2) Group Superstore product `category` into a dictionary of sets of `sub_category`. (3) Invert `{"a": 1, "b": 2}`. (4) Given a list of orders, compute the total per customer using `defaultdict`. (5) Merge two dictionaries where the second overrides the first.

**Show solution (worked answer)**

Python

```
from collections import Counter, defaultdict
print(Counter("banana"))
g = defaultdict(set)
for c, sc in zip(s["category"], s["sub_category"]): g[c].add(sc)
print({k: sorted(v) for k, v in g.items()})
print({v: k for k, v in {"a": 1, "b": 2}.items()})
orders = [("C1", 100), ("C2", 50), ("C1", 25)]
t = defaultdict(int)
for cid, amt in orders: t[cid] += amt
print(dict(t), {"x": 1} | {"x": 9, "y": 2})
```

INTERVIEW CORNER

### Questions you may be asked

**Q: How does a dictionary achieve O(1) lookup?** It hashes the key to find a slot in a hash table; collisions are resolved by probing, so the average cost is constant.

**Q: `d[k]` vs `d.get(k)`?** `d[k]` raises KeyError for a missing key; `get` returns None (or a default).

**Q: Are dictionaries ordered?** Yes, they preserve insertion order from Python 3.7 onwards.

**Q: Count word frequencies in a text.** `Counter(text.lower().split()).most_common(10)`.

PYTHON FOR DATA SCIENCE · LESSON 9

# Functions: Reusable Code, Arguments, Scope and Docstrings

Package logic into reusable functions. Learn def, parameters and arguments (positional, keyword, default, *args, **kwargs), return values, scope, docstrings and type hints, plus testing small functions on real data.

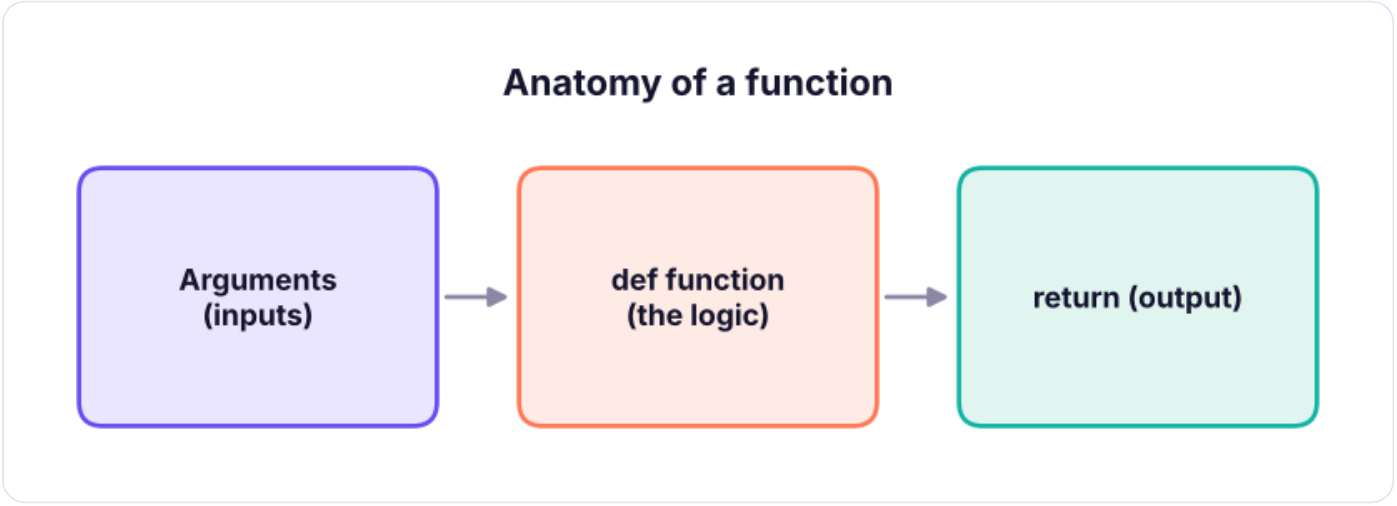
Python class 7 | Python 3.13 | Beginner | 100 min

**By the end of this lesson you can**

- Define and call functions with positional, keyword and default arguments
- Use *args and **kwargs and return multiple values
- Explain local vs global scope and avoid the mutable-default trap
- Write docstrings and type hints that make functions self-explaining
- Build small, testable functions for data cleaning and metrics

## Why functions?

A **function** is a named, reusable block of code. It removes copy-paste, makes code testable, and lets you give a name to an idea ( `clean_phone` , `margin_pct` ).




A function takes inputs (arguments), runs its body, and gives back an output (return value).

Python

```
def greet(name):
    return f"Hello, {name}!"
print(greet("Asha"), greet("Ravi"))
```

OUTPUT

```
Hello, Asha! Hello, Ravi!
```

## Parameters and arguments

Python

```
def margin_pct(profit, sales, decimals=1):
    """Profit as a percentage of sales."""
    return round(100 * profit / sales, decimals)

print(margin_pct(25, 200))                    # positional
print(margin_pct(sales=200, profit=25))       # keyword: order does not matter
print(margin_pct(25, 200, decimals=3))        # default overridden
```

OUTPUT

```
12.5
12.5
12.5
```

| Kind | Example | Notes |
|---|---|---|
| Positional | `f(1, 2)` | Matched by order |
| Keyword | `f(b=2, a=1)` | Matched by name; clearer |
| Default | `def f(a, b=10)` | Optional argument; defaults go last |
| `*args` | `def f(*args)` | Extra positional arguments as a tuple |
| `**kwargs` | `def f(**kwargs)` | Extra keyword arguments as a dict |
| Keyword-only | `def f(a, *, b)` | `b` must be named |

Python

```
def total(*nums): return sum(nums)
def describe(**info): return ", ".join(f"{k}={v}" for k, v in info.items())
def report(title, *values, sep=" | ", **opts): return title + ": " + sep.join(map(str, values))
+ f" {opts}"
print(total(1, 2, 3), total(), describe(city="Pune", year=2025))
print(report("Sales", 10, 20, 30, sep=",", currency="INR"))
nums = [4, 5, 6]; opts = {"decimals": 2}
print(total(*nums), margin_pct(7, 30, **opts))      # unpack a list / dict into arguments
```

OUTPUT

```
6 0 city=Pune, year=2025
Sales: 10,20,30 {'currency': 'INR'}
15 23.33
```

## Return values

Python

```
def stats(values):
    n = len(values)
    mean = sum(values) / n
    var = sum((v - mean) ** 2 for v in values) / (n - 1)
    return min(values), max(values), round(mean, 2), round(var ** 0.5, 2)     # a tuple
lo, hi, mean, sd = stats([12, 15, 11, 18, 14])
print(lo, hi, mean, sd)

def say(): print("no return")
print(say())                                  # a function with no return gives None
```

OUTPUT

```
11 18 14.0 2.74
no return
None
```

TIP **Return, do not print**

A function that **prints** can only be read by humans. One that **returns** can be tested, combined and reused. Print at the very end, outside the function.

## Scope: local vs global

Python

```
rate = 0.18                       # global
def price_with_gst(price):
    gst = price * rate            # reads the global
    return price + gst            # gst is local: gone after the call
print(price_with_gst(1000))
try: print(gst)
except NameError as e: print("NameError:", e)

counter = 0
def bump():
    global counter                # needed to *rebind* a global (rarely a good idea)
    counter += 1
bump(); bump(); print(counter)
```

OUTPUT

```
1180.0
NameError: name 'gst' is not defined
2
```

Rule of thumb (LEGB): Python looks in **L**ocal, then **E**nclosing, then **G**lobal, then **B**uilt-in scope. Prefer passing values in and returning values out.

## The mutable-default trap

Python

```
def add_item(x, bucket=[]):          # BAD: one list shared by every call
    bucket.append(x); return bucket
print(add_item(1), add_item(2))

def add_item_ok(x, bucket=None):     # GOOD
    if bucket is None: bucket = []
    bucket.append(x); return bucket
print(add_item_ok(1), add_item_ok(2))
```

OUTPUT

```
[1, 2] [1, 2]
[1] [2]
```

Lists, dicts and sets as default values are created **once**, when `def` runs. Use `None` and create the object inside.

## Docstrings and type hints

Python

```
def clean_phone(raw: str, country_code: str = "+91") -> str:
    """Return a phone number as '+91 XXXXX XXXXX', or '' if it is not 10 digits.

    >>> clean_phone("98765-43210")
    '+91 98765 43210'
    """
    digits = "".join(ch for ch in raw if ch.isdigit())[-10:]
    if len(digits) != 10: return ""
    return f"{country_code} {digits[:5]} {digits[5:]}"

print(clean_phone("98765-43210"), repr(clean_phone("12345")), clean_phone("+91 (98765) 43210"))
help(clean_phone)
```

OUTPUT

```
+91 98765 43210 '' +91 98765 43210
Help on function clean_phone in module __main__:

clean_phone(raw: str, country_code: str = '+91') -> str
    Return a phone number as '+91 XXXXX XXXXX', or '' if it is not 10 digits.

    >>> clean_phone("98765-43210")
    '+91 98765 43210'
```

Type hints do not change how the code runs, but editors and tools such as `mypy` use them, and they document intent. Write the docstring first line as a short summary; tools like `help()` and IDE hovers display it.

## Functions are objects

Python

```
def square(x): return x * x
fn = square                                  # assign, store in a list, pass as an argument
def apply_twice(f, x): return f(f(x))
print(fn(4), apply_twice(square, 3), [f.__name__ for f in (square, len, str)])

def make_multiplier(n):                      # a function that returns a function (a closure)
    def times(x): return x * n
    return times
double, triple = make_multiplier(2), make_multiplier(3)
print(double(7), triple(7))
```

OUTPUT

```
16 81 ['square', 'len', 'str']
14 21
```

## Functions on real data

Small, pure functions are easy to test and to reuse across `apply`, comprehensions and pipelines.

Python

```python
import pandas as pd
s = pd.read_csv("superstore_sales.csv", parse_dates=["order_date"])

def discount_band(d: float) -> str:
    if d == 0: return "none"
    if d <= 0.10: return "low"
    if d <= 0.20: return "medium"
    return "high"

def summarise(df: pd.DataFrame, by: str) -> pd.DataFrame:
    """Sales, profit and margin % per group."""
    g = df.groupby(by).agg(sales=("sales", "sum"), profit=("profit", "sum"), lines=("sales",
"size"))
    g["margin_pct"] = (100 * g.profit / g.sales).round(1)
    return g.round(0).astype({"sales": int, "profit": int})

s["band"] = s["discount"].map(discount_band)
print(summarise(s, "band").sort_values("margin_pct", ascending=False))
print(summarise(s, "region").sort_values("sales", ascending=False))
```

OUTPUT

```
            sales    profit  lines  margin_pct
band
none    123936290  21718963   3480        18.0
low      21864636   1286023    726         6.0
medium   35864461   -588882   1217        -2.0
high     19748182  -4063940    900       -21.0
           sales   profit  lines  margin_pct
region
South   53952768  4702012   1656         9.0
West    51958520  4719833   1614         9.0
North   49018161  4555830   1530         9.0
East    46484119  4374490   1523         9.0
```

The same `summarise` works for any column. That reuse is the point: write once, call with `by="category"`, `by="segment"`, and so on.

## Testing a function

Python

```
assert margin_pct(25, 200) == 12.5
assert clean_phone("12345") == ""
assert discount_band(0.15) == "medium"
print("all checks passed")
```

OUTPUT

```
all checks passed
```

`assert` is the quickest test. Larger projects use `pytest` (Lesson 14).

# Recursion in one paragraph

A function can call itself; it needs a **base case** that stops the calls.

Python

```
def factorial(n):
    return 1 if n <= 1 else n * factorial(n - 1)
def fib(n, memo={}):                    # deliberate cache; see the lesson on
functools.lru_cache
    if n < 2: return n
    if n not in memo: memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]
print(factorial(6), [fib(i) for i in range(12)], fib(60))
```

OUTPUT

```
720 [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89] 1548008755920
```

Python limits recursion depth (about 1,000 calls), so prefer loops for deep or large problems.

## Common mistakes

| Mistake | Fix |
|---|---|
| Forgetting `return` (function gives `None`) | Check the value you expect back |
| Mutable default arguments | Use `None` and create inside |
| Calling `f` instead of `f()` | `f` is the function object; `f()` runs it |
| Modifying a global inside a function without `global` | `UnboundLocalError`; pass and return values instead |
| Function does too many things | One job per function; compose small ones |
| Positional order confusion | Use keyword arguments for clarity |

## Summary

**KEY TAKEAWAYS**

- `def name(params): ... return value` defines a function; no `return` means it returns `None`.
- Arguments can be positional, keyword, default, `*args` (tuple) and `**kwargs` (dict); put defaults last.
- Variables created inside a function are local; read globals freely, but prefer parameters and return values.
- Never use a mutable object as a default; use `None`.
- Docstrings and type hints document intent; `assert` gives quick tests.
- Functions are objects: you can pass them around, store them and return them (closures).

### Quick quiz

6 questions · tap an answer to check it

**1. What does a function without a `return` statement return?**

A 0

B None

C An empty string

D An error

**Answer: B.** Python returns `None` implicitly.

**2.** What is wrong with `def f(x, items=[]):` ?

A Nothing

B The default list is created once and shared across calls

C Lists cannot be arguments

D It must be a tuple

**Answer: B.** Use `items=None` and create the list inside.

**3.** What does `*args` collect?

A Keyword arguments in a dict

B Extra positional arguments in a tuple

C Only the first argument

D Nothing

**Answer: B.** `**kwargs` collects the keyword ones.

**4.** Which scope does Python search first?

A Global

B Built-in

C Local

D Enclosing only

**Answer: C.** LEGB: Local, Enclosing, Global, Built-in.

**5.** How do you call `f(a, b)` with a list `pair = [1, 2]`?

A `f(pair)`

B `f(*pair)`

C `f(**pair)`

D `f[pair]`

**Answer: B.** `*` unpacks the list into positional arguments.

**6.** Why return values instead of printing them?

A Printing is slower

B Returned values can be tested, reused and combined

C Print is not allowed in functions

D Python requires it

**Answer: B.** Printing is for the final display only.

EXERCISE

### Function problems

(1) Write `is_prime(n)`. (2) Write `percent_change(old, new)` returning a rounded percentage and guard against `old == 0`. (3) Write `top_n(df, col, n=5)` that returns the top-n Superstore rows by a column. (4) Write a function that accepts any number of numbers and returns their average, or `None` if none are given. (5) Add `assert` tests for each.

**Show solution (worked answer)**

Python

```
def is_prime(n):
    return n > 1 and all(n % d for d in range(2, int(n ** 0.5) + 1))
def percent_change(old, new): return None if old == 0 else round(100 * (new - old) / old,
1)
def top_n(df, col, n=5): return df.nlargest(n, col)
def average(*nums): return sum(nums) / len(nums) if nums else None
assert [x for x in range(20) if is_prime(x)] == [2, 3, 5, 7, 11, 13, 17, 19]
assert percent_change(0, 5) is None and percent_change(200, 250) == 25.0
assert average() is None and average(2, 4) == 3
print(top_n(s, "sales", 3)[["order_id", "sales"]])
```

INTERVIEW CORNER

### Questions you may be asked

**Q: Difference between an argument and a parameter?** Parameters are the names in the `def`; arguments are the values you pass when calling.

**Q: What are `*args` and `**kwargs`?** They let a function accept any number of extra positional and keyword arguments; the first arrives as a tuple, the second as a dict.

**Q: What is a closure?** A function that remembers variables from the scope in which it was created, even after that scope has finished.

**Q: Why is a mutable default argument dangerous?** It is evaluated once at definition time, so state leaks between calls.

PYTHON FOR DATA SCIENCE · LESSON 10

# Comprehensions, Lambda, map/filter and Generators

Write compact, readable Python. Learn list, set and dict comprehensions, lambda with sorted/min/max, map and filter, any and all, generators and itertools, and when plain loops or pandas are the better choice.

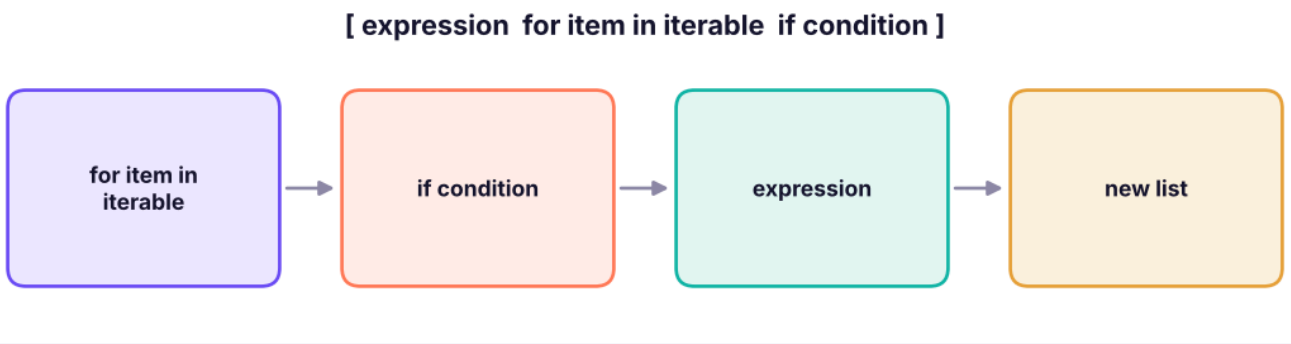
Python class 8 | Python 3.13 | Beginner | 100 min

**By the end of this lesson you can**

- Write list, set and dictionary comprehensions with conditions and nesting
- Use lambda functions with sorted, min, max, map and filter
- Use any, all, zip, enumerate and sum with generator expressions
- Explain lazy evaluation and write generator functions with yield
- Use itertools and functools for common patterns and know when not to be clever

## List comprehensions

A **comprehension** builds a collection from another in one expression: `[expression for item in iterable if condition]`.




A comprehension reads like the sentence "give me X for each item, if the condition holds".

Python

```
nums = [1, 2, 3, 4, 5, 6]
squares = [n * n for n in nums]
evens = [n for n in nums if n % 2 == 0]
labels = ["even" if n % 2 == 0 else "odd" for n in nums]          # if/else goes BEFORE the for
loop_version = []
for n in nums:
    if n % 2 == 0: loop_version.append(n * n)
print(squares, evens, labels, [n * n for n in nums if n % 2 == 0] == loop_version)
```

OUTPUT

```
[1, 4, 9, 16, 25, 36] [2, 4, 6] ['odd', 'even', 'odd', 'even', 'odd', 'even'] True
```

Python

```
words = ["  Pune ", "delhi", " KOCHI", "goa  "]
print([w.strip().title() for w in words])
print([(x, y) for x in (1, 2) for y in ("a", "b")])            # two loops
matrix = [[1, 2, 3], [4, 5, 6]]
print([v for row in matrix for v in row], [list(col) for col in zip(*matrix)])    # flatten, transpose
```

OUTPUT

```
['Pune', 'Delhi', 'Kochi', 'Goa']
[(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
[1, 2, 3, 4, 5, 6] [[1, 4], [2, 5], [3, 6]]
```

## Set and dictionary comprehensions

Python

```
print({len(w) for w in ["tea", "cup", "mug", "plate"]})                    # set
prices = {"Pen": 10, "Notebook": 45, "Stapler": 180}
print({k: round(v * 1.18, 2) for k, v in prices.items()})                  # dict
print({k: v for k, v in prices.items() if v > 20}, {c: ord(c) for c in "abc"})
```

OUTPUT

```
{3, 5}
{'Pen': 11.8, 'Notebook': 53.1, 'Stapler': 212.4}
{'Notebook': 45, 'Stapler': 180} {'a': 97, 'b': 98, 'c': 99}
```

> **TIP** **Keep it readable**
>
> If a comprehension needs more than one condition or nested level, or does not fit comfortably on one or two lines, use a normal loop or a helper function. Clever is not the goal; clear is.

## Lambda functions

A **lambda** is a small anonymous function: `lambda args: expression`.

Python

```
sq = lambda x: x * x                          # fine to learn with, but def is better for named functions
print(sq(5), (lambda a, b: a + b)(2, 3))
people = [("Asha", 29, 62000), ("Ravi", 35, 88000), ("Meera", 31, 75000)]
print(sorted(people, key=lambda p: p[2], reverse=True))        # sort by salary
print(min(people, key=lambda p: p[1]), max(people, key=lambda p: p[2])[0])
print(sorted(["banana", "kiwi", "apple", "fig"], key=lambda w: (len(w), w)))    # sort by length, then alphabet
```

OUTPUT

```
25 5
[('Ravi', 35, 88000), ('Meera', 31, 75000), ('Asha', 29, 62000)]
('Asha', 29, 62000) Ravi
['fig', 'kiwi', 'apple', 'banana']
```

Lambdas shine as the `key=` for `sorted`, `min`, `max`, and in `df.apply`.

## map, filter, any, all

Python

```
nums = [3, 8, 12, 5, 20]
print(list(map(lambda x: x * 2, nums)), list(map(str, nums)), list(filter(lambda x: x > 6, nums)))
print([x * 2 for x in nums], [x for x in nums if x > 6])        # comprehensions say the same thing
print(any(n > 15 for n in nums), all(n > 0 for n in nums), sum(1 for n in nums if n % 2 == 0))
print(list(zip(["a", "b", "c"], [1, 2, 3])), dict(zip("abc", range(3))), list(enumerate("xyz", 1)))
```

OUTPUT

```
[6, 16, 24, 10, 40] ['3', '8', '12', '5', '20'] [8, 12, 20]
[6, 16, 24, 10, 40] [8, 12, 20]
```

```
True True 3
[('a', 1), ('b', 2), ('c', 3)] {'a': 0, 'b': 1, 'c': 2} [(1, 'x'), (2, 'y'), (3, 'z')]
```

`map` and `filter` return lazy iterators; wrap with `list(...)`. Most Python programmers prefer comprehensions, but you will see both.

## Generators: lazy sequences

A **generator expression** uses parentheses and produces items one at a time, without building the whole list in memory.

Python

```python
import sys
big_list = [n * n for n in range(1_000_000)]
big_gen = (n * n for n in range(1_000_000))
print(sys.getsizeof(big_list) // 1000, "KB list vs", sys.getsizeof(big_gen), "bytes generator")
print(sum(n * n for n in range(1_000_000)), next(iter(big_gen)), next(big_gen))
```

OUTPUT

```
8448 KB list vs 200 bytes generator
333332833333500000 0 1
```

A **generator function** uses `yield`. It pauses at each `yield` and resumes on the next request:

Python

```python
def countdown(n):
    while n > 0:
        yield n
        n -= 1
print(list(countdown(5)))
def read_in_batches(items, size):
    for i in range(0, len(items), size):
        yield items[i:i + size]
for batch in read_in_batches(list(range(10)), 4): print(batch)
def fib():
    a, b = 0, 1
    while True:                  # infinite sequence, safe because it is lazy
        yield a
        a, b = b, a + b
import itertools
print(list(itertools.islice(fib(), 12)))
```

OUTPUT

```
[5, 4, 3, 2, 1]
[0, 1, 2, 3]
```

```
[4, 5, 6, 7]
[8, 9]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Generators are how Python reads large files line by line, streams API pages and chains pipeline steps with constant memory. Once consumed, a generator is empty:

Python

```python
g = (x for x in range(3))
print(list(g), list(g))
```

OUTPUT

```
[0, 1, 2] []
```

## itertools and functools highlights

Python

```python
import itertools, functools, operator
from collections import Counter
print(list(itertools.chain([1, 2], [3], [4, 5])), list(itertools.combinations("ABC", 2)),
list(itertools.product("AB", [0, 1])))
print(list(itertools.accumulate([10, 20, 5, 15])), functools.reduce(operator.mul, [1, 2, 3, 4,
5]))
for key, grp in itertools.groupby(sorted(["ant", "bee", "ape", "bat"]), key=lambda w: w[0]):
print(key, list(grp))
@functools.lru_cache(maxsize=None)              # memoise a function: remember results of
earlier calls
def slow_square(n): return n * n
print([slow_square(n) for n in (4, 4, 5, 4)], slow_square.cache_info())
```

OUTPUT

```
[1, 2, 3, 4, 5] [('A', 'B'), ('A', 'C'), ('B', 'C')] [('A', 0), ('A', 1), ('B', 0), ('B',
1)]
[10, 30, 35, 50] 120
a ['ant', 'ape']
b ['bat', 'bee']
[16, 16, 25, 16] CacheInfo(hits=2, misses=2, maxsize=None, currsize=2)
```

## Real data examples

Python

```python
import pandas as pd
s = pd.read_csv("superstore_sales.csv", parse_dates=["order_date"])
regions = sorted(s["region"].unique())
print({r: round(float(s.loc[s.region == r, "sales"].sum()) / 1e6, 1) for r in regions})
# dict comprehension
top_customers = [c for c, n in s["customer_id"].value_counts().items() if n >= 18][:5]
print(top_customers, len([c for c, n in s["customer_id"].value_counts().items() if n >= 18]))
rows = s.head(4)[["order_id", "sales", "profit"]].to_dict("records")
print([{**r, "margin": round(100 * r["profit"] / r["sales"], 1)} for r in rows])
# enrich records
print(sorted(s["category"].unique(), key=lambda c: -s.loc[s.category == c, "sales"].sum()))
# sort by a computed key
```

OUTPUT

```
{'East': 46.5, 'North': 49.0, 'South': 54.0, 'West': 52.0}
['C0719', 'C0149', 'C0126', 'C0384', 'C0257'] 21
[{'order_id': 'IN-2023-00001', 'sales': 36000.0, 'profit': 3472.55, 'margin': 9.6},
{'order_id': 'IN-2023-00001', 'sales': 3850.0, 'profit': 1169.54, 'margin': 30.4},
{'order_id': 'IN-2023-00001', 'sales': 62080.0, 'profit': 6044.47, 'margin': 9.7},
{'order_id': 'IN-2023-00002', 'sales': 5216.0, 'profit': 450.16, 'margin': 8.6}]
['Technology', 'Furniture', 'Office Supplies']
```

## Which tool when?

| Situation | Prefer |
|---|---|
| Transform or filter a small list | List comprehension |
| Sort by a rule | `sorted(..., key=lambda ...)` |
| Huge or streaming data | Generator (expression or function) |
| Table columns | Vectorised pandas ( `df["a"] * 2` , `np.where` ), not comprehensions |
| Multi-step logic with side effects | Plain `for` loop |
| Reuse the same transformation | Named `def` function |

# Summary

**KEY TAKEAWAYS**

- `[expr for x in it if cond]` builds a list; `{…}` builds sets and dicts; the conditional expression ( `a if c else b` ) comes before `for` .
- `lambda` is an anonymous one-expression function, mostly used as `key=` and in `apply` .
- `map` / `filter` are lazy; comprehensions are usually clearer. `any` , `all` , `sum` work well with generator expressions.
- Generators ( `( … )` or `yield` ) produce values lazily, use almost no memory and can be infinite; they can be consumed only once.
- `itertools` and `functools` ( `chain` , `combinations` , `groupby` , `lru_cache` , `reduce` ) cover many patterns; keep code readable and vectorise table work.

## Quick quiz

6 questions · tap an answer to check it

**1. What does `[x * 2 for x in range(4) if x != 2]` give?**

**A** [0, 2, 4, 6]

**B** [0, 2, 6]

**C** [0, 4, 6]

**D** [2, 6]

**Answer: B.** Item 2 is filtered out before doubling.

**2. Where does an `if ... else` go in a comprehension?**

**A** After the for

**B** Before the for, as part of the expression

**C** It is not allowed

**D** At the end

**Answer: B.** `[a if cond else b for x in it]` .

**3.** What is the main advantage of a generator expression over a list comprehension?

A It is faster to type

B It is lazy and uses very little memory

C It supports indexing

D It can be reused

**Answer: B.** Values are produced one at a time.

**4.** What does `sorted(people, key=lambda p: p[2])` do?

A Sorts by the first item

B Sorts by the third item of each tuple

C Reverses the list

D Raises an error

**Answer: B.** `key` supplies the sort value.

**5.** What happens when you iterate a generator twice?

A It restarts

B The second pass is empty because it was consumed

C It raises an error

D It doubles the values

**Answer: B.** Create a new generator or use a list.

**6.** Which is usually best for adding a column to a DataFrame?

- **A** A for loop over rows
- **B** A vectorised column operation
- **C** A generator
- **D** A lambda inside a nested loop

**Answer: B.** Vectorised code is shorter and much faster.

EXERCISE **Comprehension problems**

(1) List the squares of odd numbers from 1 to 20. (2) Build a dictionary from words to their lengths for words longer than 3 letters. (3) Flatten `[[1,2],[3],[4,5,6]]`. (4) Sort `["pune","Delhi","goa","Kochi"]` case-insensitively. (5) Write a generator that yields only the primes below 50. (6) From the Superstore data, list sub-categories with total profit below zero.

**Show solution (worked answer)**

Python

```
print([n * n for n in range(1, 21) if n % 2])
print({w: len(w) for w in ["tea", "plate", "mug", "bottle"] if len(w) > 3})
print([v for part in [[1, 2], [3], [4, 5, 6]] for v in part])
print(sorted(["pune", "Delhi", "goa", "Kochi"], key=str.lower))
def primes(limit):
    for n in range(2, limit):
        if all(n % d for d in range(2, int(n ** 0.5) + 1)): yield n
print(list(primes(50)))
loss = s.groupby("sub_category")["profit"].sum()
print(list(loss[loss < 0].index))
```

INTERVIEW CORNER **Questions you may be asked**

**Q: List comprehension vs generator expression?** Both iterate with the same syntax; the list builds everything now in memory, the generator yields lazily and is single-use.

**Q: When should you avoid a comprehension?** When logic needs several statements, side effects, or deep nesting; readability matters more than brevity.

**Q: What does `yield` do?** It turns a function into a generator: the function pauses, hands back a value, and resumes where it left off on the next call.

**Q: How do you flatten a list of lists?** `[x for sub in lists for x in sub]` or `itertools.chain.from_iterable(lists)`.

PYTHON FOR DATA SCIENCE · LESSON 11

# Object-Oriented Python: Classes, Objects, Inheritance and Dataclasses

Model real things as objects. Learn classes, **init**, attributes and methods, encapsulation, inheritance, polymorphism, special (dunder) methods, class and static methods, properties and dataclasses, with a small data-pipeline example.

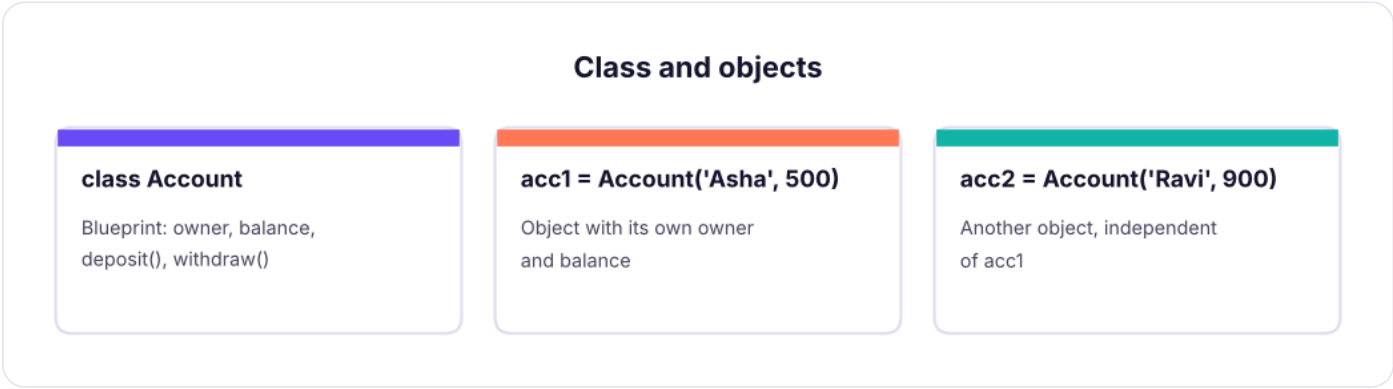
Python class 9 | Python 3.13 | Intermediate | 110 min

**By the end of this lesson you can**

- Define classes with attributes and methods and create objects
- Use inheritance, super() and polymorphism to share and customise behaviour
- Implement **str**, **repr**, **eq** and **len** to make objects behave like built-ins
- Use @property, @classmethod and @staticmethod appropriately
- Choose between a plain class, a dataclass and a dictionary

## Why classes?

A **class** bundles **data** (attributes) and **behaviour** (methods) into one unit. An **object** is one instance of a class. Libraries you already use are classes: a pandas `DataFrame` has data (rows) and methods ( `.groupby()` ).




A class is the blueprint; objects are the things built from it, each with its own data.

Python

```python
class Account:
    bank = "SkillBank"                        # class attribute: shared by all objects

    def __init__(self, owner, balance=0):     # constructor: runs when you create an object
        self.owner = owner                     # instance attributes: belong to this object
        self.balance = balance

    def deposit(self, amount):
        if amount ⩽ 0: raise ValueError("amount must be positive")
        self.balance += amount
        return self.balance

    def withdraw(self, amount):
        if amount > self.balance: raise ValueError("insufficient funds")
        self.balance -= amount
        return self.balance

a, b = Account("Asha", 500), Account("Ravi", 900)
a.deposit(250); b.withdraw(100)
print(a.owner, a.balance, b.owner, b.balance, Account.bank, a.bank)
try: a.withdraw(5000)
except ValueError as e: print("Error:", e)
```

OUTPUT

```
Asha 750 Ravi 800 SkillBank SkillBank
Error: insufficient funds
```

`self` is the object the method was called on. Python passes it automatically: `a.deposit(250)` is `Account.deposit(a, 250)`.

## Special (dunder) methods

Methods with double underscores customise how objects behave with `print`, `==`, `len`, `+` and more.

Python

```python
class Account:
    def __init__(self, owner, balance=0): self.owner, self.balance = owner, balance
    def __repr__(self): return f"Account({self.owner!r}, {self.balance})"      # for developers
    def __str__(self): return f"{self.owner}: ₹{self.balance:,}"                # for print()
    def __eq__(self, other): return (self.owner, self.balance) == (other.owner, other.balance)
    def __lt__(self, other): return self.balance < other.balance                # enables
sorting
    def __add__(self, other): return Account(self.owner + "&" + other.owner, self.balance +
other.balance)

x, y = Account("Asha", 5000), Account("Ravi", 9000)
print(x, "|", repr(x), "|", x == Account("Asha", 5000), "|", sorted([y, x]), "|", x + y)
```

OUTPUT

```
Asha: ₹5,000 | Account('Asha', 5000) | True | [Account('Asha', 5000), Account('Ravi', 9000)] | Asha&Ravi: ₹14,000
```

| Method | Used by |
|---|---|
| `__init__` | Creating the object |
| `__repr__` , `__str__` | `repr()` , `print()` |
| `__eq__` , `__lt__` | `==` , `<` , sorting |
| `__len__` , `__getitem__` | `len(obj)` , `obj[i]` |
| `__iter__` , `__next__` | `for` loops |
| `__enter__` , `__exit__` | `with` blocks |
| `__call__` | Calling the object like a function |

## Encapsulation and properties

Python has no strict private members; a leading underscore ( `_balance` ) means "internal, please do not touch". A **property** exposes an attribute while keeping control over it.

Python

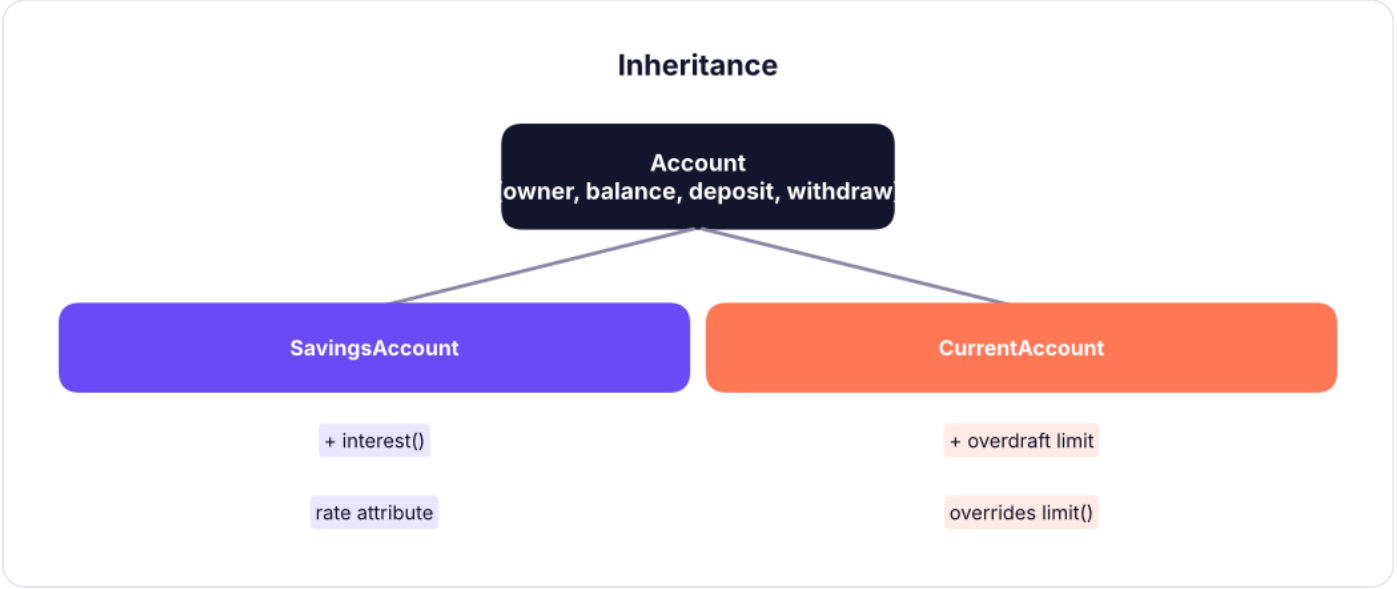
```python
class Temperature:
    def __init__(self, celsius): self._c = celsius
    @property
    def celsius(self): return self._c
    @celsius.setter
    def celsius(self, v):
        if v < -273.15: raise ValueError("below absolute zero")
        self._c = v
    @property
    def fahrenheit(self): return self._c * 9 / 5 + 32          # computed, read-only
t = Temperature(25)
t.celsius = 30
print(t.celsius, t.fahrenheit)
try: t.celsius = -500
except ValueError as e: print("Error:", e)
```

OUTPUT

```
30 86.0
Error: below absolute zero
```

# Inheritance and polymorphism

A **child** class reuses a **parent** and adds or overrides behaviour. `super()` calls the parent's version.




Savings and Current accounts inherit everything from Account and add their own rules.

Python

```
class Account:
    def __init__(self, owner, balance=0): self.owner, self.balance = owner, balance
    def withdraw(self, amount):
        if amount > self.limit(): raise ValueError("limit exceeded")
        self.balance -= amount
    def limit(self): return self.balance
    def __str__(self): return f"{type(self).__name__}({self.owner}, {self.balance})"

class Savings(Account):
    def __init__(self, owner, balance=0, rate=0.04):
        super().__init__(owner, balance)            # reuse the parent's setup
        self.rate = rate
    def interest(self): return round(self.balance * self.rate, 2)

class Current(Account):
    def __init__(self, owner, balance=0, overdraft=10000):
        super().__init__(owner, balance); self.overdraft = overdraft
    def limit(self): return self.balance + self.overdraft          # override

accounts = [Savings("Asha", 50000), Current("Ravi", 2000)]
for acc in accounts:                                               # polymorphism: same call,
different behaviour
    acc.withdraw(5000)
    print(acc, "| limit now", acc.limit())
print(accounts[0].interest(), isinstance(accounts[0], Account), issubclass(Current, Account),
[c.__name__ for c in Current.__mro__])
```

OUTPUT

```
Savings(Asha, 45000) | limit now 45000
Current(Ravi, -3000) | limit now 7000
1800.0 True True ['Current', 'Account', 'object']
```

**Polymorphism** means different classes respond to the same method name in their own way, so code like the loop above works with any account type.

> **NOTE** **Composition often beats inheritance**
>
> "Is-a" relationships (a Savings **is an** Account) suit inheritance. "Has-a" relationships (a Pipeline **has** steps) suit composition: store other objects as attributes. Prefer composition when in doubt.

## Class and static methods

Python

```python
class Order:
    GST = 0.18
    def __init__(self, amount): self.amount = amount
    @classmethod
    def from_string(cls, text):                    # alternative constructor: "₹1,200"
        return cls(float(text.replace("₹", "").replace(",", "")))
    @staticmethod
    def is_valid(amount): return amount > 0        # no self/cls: just a function in the class namespace
    def total(self): return round(self.amount * (1 + self.GST), 2)
o = Order.from_string("₹1,200")
print(o.amount, o.total(), Order.is_valid(-5))
```

OUTPUT

```
1200.0 1416.0 False
```

## Dataclasses: classes for data

For classes that mainly store data, `@dataclass` generates `__init__`, `__repr__` and `__eq__` for you.

Python

```python
from dataclasses import dataclass, field, asdict

@dataclass(order=True)
class Employee:
    name: str
    dept: str
    salary: float = 0.0
    skills: list = field(default_factory=list)       # mutable default done safely
    def raise_salary(self, pct): self.salary = round(self.salary * (1 + pct / 100), 2)

staff = [Employee("Kavya", "HR", 97000, ["Excel"]), Employee("Rohan", "IT", 120000),
Employee("Asha", "HR", 88000)]
staff[0].raise_salary(10)
print(staff[0], "|", staff[0] == Employee("Kavya", "HR", 106700.0, ["Excel"]), "|", max(staff,
key=lambda e: e.salary).name)
print(asdict(staff[1]), sorted(staff)[0].name)
import pandas as pd
print(pd.DataFrame([asdict(e) for e in staff]).groupby("dept")["salary"].mean())
```

OUTPUT

```
Employee(name='Kavya', dept='HR', salary=106700.0, skills=['Excel']) | True | Rohan
{'name': 'Rohan', 'dept': 'IT', 'salary': 120000, 'skills': []} Asha
dept
HR     97350.0
IT    120000.0
Name: salary, dtype: float64
```

## A small data-pipeline class

Python

```
import pandas as pd

class CleaningPipeline:
    """Holds a list of cleaning steps and applies them in order."""
    def __init__(self): self.steps = []
    def add(self, name, func): self.steps.append((name, func)); return self      # return self
allows chaining
    def run(self, df):
        for name, func in self.steps:
            before = len(df); df = func(df)
            print(f"  {name:<18} rows {before} → {len(df)}")
        return df

raw = pd.DataFrame({"city": [" Pune", "pune ", "Delhi", None, "Delhi"], "sales": [100, 100, 250,
80, -5]})
pipe = (CleaningPipeline()
        .add("drop missing city", lambda d: d.dropna(subset=["city"]))
        .add("tidy text", lambda d: d.assign(city=d.city.str.strip().str.title()))
        .add("remove negatives", lambda d: d[d.sales ⩾ 0])
        .add("drop duplicates", lambda d: d.drop_duplicates()))
print(pipe.run(raw))
```

OUTPUT

```
  drop missing city  rows 5 → 4
  tidy text          rows 4 → 4
  remove negatives   rows 4 → 3
  drop duplicates    rows 3 → 2
    city  sales
0   Pune    100
2  Delhi    250
```

This is the idea behind scikit-learn pipelines (ML track): objects with `fit` / `transform` methods that you chain.